

Tokenization

A Survey for Modern NLP

Marco Cognetta^{1,†} Christopher Akiki² Pawan Sasanka Ammanamanchi³ Catherine Arnett⁴
Thomas Bauwens⁵ Pavel Chizhov^{6,7} Pieter Delobelle^{5,7} Konstantin Dobler⁸ Desmond Elliott⁹
Omer Goldman¹⁰ Tatsuya Hiraoka¹¹ Valentin Hofmann^{12,13} Eugene Jang¹⁴ Julie Kallini¹⁵
Ilker Kesen⁹ Sachin Kumar¹⁶ Sander Land¹⁷ Jindřich Libovický¹⁸ Tomasz Limisiewicz¹⁹ Alisa Liu¹⁹
Clara Meister²⁰ Benjamin Minixhofer¹⁰ Sangwhan Moon¹ Tiago Pimentel²¹ David Pohl²²
Suchir Salhan¹⁰ Tomohiro Sawada²³ Pavel Stepachev³ Gianluca Vico¹⁸ Philip Whittington²¹
Xiulin Yang²⁴ Yuval Pinter²⁵

¹Google ²ScaDS.AI Leipzig ³Independent ⁴EleutherAI ⁵KU Leuven ⁶CAIRO THWS Würzburg ⁷Pleias ⁸Hasso Plattner Institute ⁹University of Copenhagen ¹⁰University of Cambridge ¹¹Mohamed bin Zayed University of Artificial Intelligence ¹²LMU Munich ¹³Munich Center for Machine Learning ¹⁴Northeastern University ¹⁵Stanford University ¹⁶Ohio State University ¹⁷Neko Health ¹⁸Charles University ¹⁹University of Washington ²⁰EPFL ²¹ETH Zurich ²²Institute of Science Tokyo ²³Georgia Institute of Technology ²⁴Georgetown University ²⁵Ben-Gurion University of the Negev

Abstract

While modern language models take raw text as their input and produce raw text as output, they do not operate over text directly. Hidden in the very first step of language model pipelines is *tokenization*, where raw text is converted to a sequence of *tokens* which the model is designed to process.

The dominant approach to tokenization is *subword tokenization*, which uses tokens whose granularity lies between that of characters (or bytes) and words. Subword tokenization is ubiquitous in modern natural language processing (NLP) due to its advantages: compared to word-level models, it solves the out-of-vocabulary problem while constraining the vocabulary size, reducing the number of embedding parameters and logits needed. Compared to character-level models, it improves the inductive bias of models by packing more information into a token while producing significantly shorter sequences.

While this technique is powerful, it is not without flaws. To name a few: it imposes a rigid system for how tokens are built, and one system may not generalize across languages; segmentation ambiguities create room for adversarial attacks; in multilingual settings, balancing the token allocation between high- and low-resource languages must be done explicitly; and the reliance on a static, pre-computed vocabulary means the tokenization process is disjoint from the model’s training, preventing true end-to-end optimization. There is substantial room for innovation in this domain, yet progress is contingent upon a clear understanding of existing methodologies and current bottlenecks. This survey aims to serve that purpose.

We present tokenization in the context of the modern large language model (LLM) era and cover the breadth of areas that tokenization directly influences. We discuss algorithms, techniques, pitfalls, theory, and practice, along with jumping-off points for prospective researchers and practitioners. We hope that this survey brings more attention to the field of tokenization as an area in which much progress is still to be made.

[†]Corresponding Author: cognetta.marco@gmail.com;  The Token #ization Discord

1 Introduction

A disconnect exists in the modern language modeling pipeline. The internals of a language model are real-valued vectors, but the inputs and outputs are text. This gap is bridged by **tokenization**, the interface by which text is converted to a set of discrete **tokens** that are explicitly assigned real-valued *embedding vectors*, the representation that a model can handle directly (see Figure 1.1).

This interface is relatively understudied, as it is often seen as a marginal concern compared to, for example, the modeling gains enabled by better, larger, or faster model-architectural components such as the attention mechanism. However, tokenization is a fundamental step in language modeling that determines precisely what the model is given to process, and therefore it has both profound and subtle effects on the final performance of a model. In addition, no systematic theory yet provides clear guidance for constructing an optimal tokenizer, and so many variables must be carefully balanced to arrive at an acceptable one.

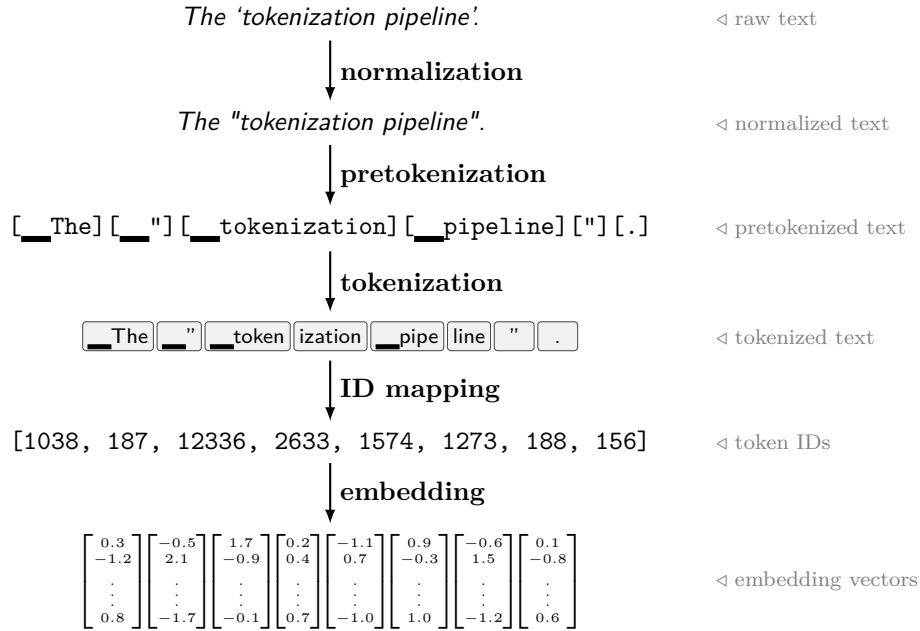


Figure 1.1 – *Tokenization* is the interface between written text and a language model’s internal representation. A text input is *normalized* (possibly lossily) and *pretokenized* before being converted into a list of *tokens* by a *tokenizer*, after which the tokens are mapped to integer *IDs*, which are associated with real-valued *embedding vectors* that form the model’s representation of the input. A corresponding *detokenization* step exists to convert the token id list that is generated by the model to tokens and then back into raw text.

Subword tokenization, where the tokens inhabit a granularity lying between individual characters and full words, is used in nearly all modern NLP modeling pipelines. With the rise of LLMs, the field quickly coalesced around a small number of subword tokenization schemes and has not iterated on them with the same intensity as with other parts of the language modeling pipeline. This is perhaps because tokenization ablation studies are costly and noisy, or perhaps because of the thought that the shortcomings of a particular tokenization algorithm can be made up by improvements to other parts of the model. However, even as language models have improved dramatically with both the scale of their parameters and their training data, we can still see the negative effects of poor tokenization, as well as areas where improvements to tokenization drive improvements in model quality (Tao et al., 2024; Huang et al., 2025; Limisiewicz et al., 2026).

Tokenization is intimately linked to some of the most visible failures in modern language modeling. These range from brittleness in reasoning capabilities to systemic risks: e.g., failing to perform simple arithmetic (Wallace et al., 2019; Thawani et al., 2021; Singh & Strouse, 2024) or word puzzles (Edman et al., 2024b; Uzan & Pinter, 2026), not being able to count the *r*’s in “strawberry” (Silberling, 2024; Cosma et al., 2025), and suffering from adversarial safety vulnerabilities (Geh et al., 2025). Further, tokenization drives

significant inequities, causing large performance variance across languages (Ali et al., 2024) and inflating the monetary cost of inference for non-English users (Ahia et al., 2023; Petrov et al., 2023).

Given these downstream consequences, tokenization cannot be treated as a mere implementation detail, but rather as a central design choice that compels us to ask: *What is the best way to produce a tokenizer? How should tokenizers be evaluated? How should we handle cases where token sequences do not align with linguistic and morphological units? Can we train a tokenizer jointly with a language model? How does the choice of tokenizer interact with the selection of intended downstream tasks?* Each of these, and more, are important questions in modern tokenization and language modeling research, and have spawned numerous subfields investigating each part of the tokenization process as well as attempts to eliminate tokenizers altogether.

We present a survey of the tokenization landscape focused on the foundations of tokenization and its use across all aspects of NLP. We cover algorithms, techniques, considerations, pitfalls, areas where improved tokenization can benefit practitioners, and more, to provide the reader with inspiration for new research directions and to justify experimenting with tokenizer ablation and analysis in their modeling stack.

In Section 2, we cover the basics of tokenization and its historical motivations. In Section 3, we provide an overview of the foundational algorithms used in subword tokenization and their modern variants, highlighting the main innovations that have been made since their inception. In Section 4, we cover the *pretokenization* step, an important aspect of the tokenization pipeline that is both widespread and empirically motivated, but brittle. We move on to aspects of multilinguality by surveying language-specific tokenization techniques in Section 5 and tokenization in a multilingual setting in Section 6. We then turn to the evaluation (both intrinsic and extrinsic) of tokenizers in Section 7. The use of non-canonical tokenizations in both training and inference is covered in Section 8. We then consider methods that eschew subword tokenization by using byte- or character-level inputs, latent or hierarchical subword representations, and dynamic segmentation in Section 9, or by abandoning text entirely and using visual features in Section 10. In Section 11, we survey the types and characteristics of tokenizers that are used in today’s LLMs, and in Section 12 we cover the ways in which tokenizers can be modified or replaced in modern models. Section 13 examines tokenization as a computational device through a theoretical lens. Finally, in Section 14, we briefly survey topics that are adjacent, but still closely related, to tokenization, such as constrained generation, tokenization in non-linguistic domains, tokenizer security, and token embeddings.

Contents

1 Introduction	2	7.3 Questions	54
2 Tokenization	5	8 Non-Canonical Tokenizations	55
2.1 History and Evolution	5	8.1 In Language Model Training	55
2.2 Tokenization and Language Modeling	8	8.2 In Language Model Inference	57
2.3 The Current State of Tokenization (Research)	9	8.3 Questions	58
3 Tokenization Algorithms	13	9 Byte and Character Representations	59
3.1 Byte-pair Encoding (BPE)	13	9.1 Pure Byte- and Character-Level Representations	59
3.2 Unigram Language Modeling Tokenization	20	9.2 Hierarchical Models and Latent Tokenization	60
3.3 WordPiece	20	9.3 Questions	64
3.4 Contextual Vocabularies	23	10 Visual Text Representations	65
3.5 Globally Optimized Vocabulary Construction	23	10.1 Rendering Text for Visual Language Modeling	65
3.6 General Inference Algorithms	23	10.2 Purely Visual Text Processing	67
3.7 Stochastic Tokenization	26	10.3 Combining Visual Representations with Tokenizers	68
3.8 Joint Tokenizer-Task Training	27	10.4 Open Challenges with Visual Text Representations	69
3.9 Questions	27	10.5 Questions	70
4 Pretokenization	28	11 Tokenizers in Modern LLMs	72
4.1 Normalization	28	11.1 State-of-the-Art LLMs	72
4.2 Whitespace Delimitation	29	11.2 Special Uses of Tokenizers in LLMs	75
4.3 Rule-Based Pretokenization	30	11.3 Tokenization and Language Model Inference Cost	76
4.4 Morphological Pretokenization	30	11.4 Tokenization and Scaling Laws	77
4.5 Pretokenizing Numbers	31	11.5 Questions	77
4.6 Questions	32	12 Tokenizer Modification and Cross-Tokenizer Knowledge Transfer	79
5 Language- and Script-Specific Considerations	33	12.1 Vocabulary Modification	79
5.1 Writing System Typology	33	12.2 Embedding (Re-)Initialization	80
5.2 Script-Specific Tokenizers	35	12.3 Knowledge Distillation Across Tokenizers	82
5.3 Transliterating to a Different Script	37	12.4 Questions	83
5.4 Morphological Typology	38	13 Theoretical Aspects of Tokenization	84
5.5 Syntactic Considerations	40	13.1 Tokenization and Automata Theory	84
5.6 Questions	41	13.2 Tokenizer Complexity and Hardness Results	87
6 Multilingual Tokenization	42	13.3 Questions	89
6.1 Allocation Bias	42	14 Additional Topics	90
6.2 Vocabulary Overlap and Transfer Learning	43	14.1 Tokenization in Non-Linguistic/Text Domains	90
6.3 Code Switching and Language Variation	43	14.2 Embedding Layers	91
6.4 Mitigation Strategies	44	14.3 Tokenizer Security	93
6.5 Questions	46	14.4 Constrained Generation	94
7 Evaluation	48	14.5 Token Healing	95
7.1 Extrinsic Evaluation	48	14.6 Questions	96
7.2 Intrinsic Evaluation	50	15 Conclusion	97

2 Tokenization

Tokenization is, at its core, the process of converting **raw text** into the sequence of tokens which are actually processed by a language model. How should we perform this conversion? Tokenization research tries to answer this question by investigating what properties a sequence of tokens should have to be amenable to training a performant language model, and how to measure and achieve those properties.

To give a sense of the range of choices available, let us consider: what is the right way to tokenize the word *disjointed*? A human might say `dis joint ed`, aligning with the component morphemes; however, morphemes might be hard to obtain automatically, and may not be the optimal choice for all words. Alternatively, it might be better to tokenize it as a sequence of characters, `d i s j o i n t e d`, giving just the raw information to the computer and letting it handle the interpretation. This option, however, might be too granular, with each piece encoding no real meaning. Yet another option would be to tokenize it as a single token, `disjointed`, preserving the word as a single semantic unit. This works for a specific example, but what about *disjointedly*, *disjointedness*, *disjoint*, *jointly*, etc.? Representing each of these with its own token, keeping in mind that the token inventory itself is nothing but a mapping to arbitrary numerical IDs, would ignore their overlapping semantics. Moreover, there exist infinitely many orthographic words, making it computationally infeasible to explicitly represent all of them as single tokens.

The conversion from raw text to the tokenized form is done by a **tokenizer**: an object consisting of a **vocabulary** and a **tokenization inference algorithm**. Modern tokenization algorithms have coalesced around data-driven approaches based mostly on compression algorithms (introduced in Section 3), but tokenization has taken on several forms historically. We cover them briefly here to contextualize the onset of subword tokenization, as well as modern takes on character-level (or byte-level) tokenization, but they are not the focus of our survey.

A glossary of core terms used throughout the survey is presented in Table 2.2.

2.1 History and Evolution

In this section, we aim to outline the goals of tokenization and how different approaches balance them. We do this roughly in chronological order as these approaches were proposed or adopted. For a more detailed history of tokenization until 2021, see Mielke et al. (2021).

Since the earliest era of NLP, researchers have assumed tokenized text as input, mostly in the form of orthographic words (e.g., Richens & Booth, 1952; Winograd, 1971; Jurafsky & Martin, 2026). Words to be included in the vocabulary were selected manually or with heuristics such as by whitespace and punctuation, a convenient choice for English and many European languages. This approach has two main drawbacks. First, even with very large vocabularies, there can be **out-of-vocabulary** tokens (OOVs). For example, even if a model has `draw` and `drawing` in its vocabulary, if it does not have `drawings`, then that word must be treated as an OOV token when observed, preventing any possibility of obtaining information related to its meaning. Consider the following example, where `America` is in the vocabulary, but `Americans` is not.

Example	
Raw Text:	<i>True or false: Americans are from America.</i>
Word Tokenized:	<code>True</code> <code>or</code> <code>false</code> <code>:</code> <code><OOV></code> <code>are</code> <code>from</code> <code>America</code> <code>.</code>

Since *Americans* is replaced with an ambiguous `<OOV>`¹ token, a model cannot possibly know what was originally in that position, and so this question becomes impossible to answer; the `<OOV>` token could just as easily have been *Germans* or *tokenizers* as it could have been *Americans*.

The second problem is that, in a modern neural language modeling setting, by increasing the vocabulary size to account for more words, one is also increasing the parameter count of the language model that will eventually use this vocabulary. By some counts, English has at least a million words (Merriam-Webster,

¹Some methods and models refer to the `<OOV>` token as `<UNK>`, for *unknown*.

n.d.), which would account for a huge percentage of the parameter budgets even for contemporary large language models (LLMs).² Additionally, even with a large vocabulary, such a tokenizer would still not be able to assign a token ID to any word outside its vocabulary, including misspelled words, novel words, and non-word components (e.g., code, URLs, and cardinal numbers).

The problems with word-level models are particularly severe in morphologically rich languages such as German, Turkish, and Korean, which can form arbitrarily many word-forms through morphological production (Hakkani-Tür et al., 2002), and for languages such as Chinese and Japanese, whose scripts do not use clear word boundaries, making even the definition of a “word” non-trivial (Mochihashi et al., 2009). The granularity at which models defined “tokenization” before the advent of subword models is now often reserved for *pretokenization*, with classic-age tokenizers often being used as pretokenizers (see Sections 4 and 5, *inter alia*).

On the other extreme, before the proposal of current subword tokenization methods, some language models operated over characters (Sutskever et al., 2011b; Graves, 2013). This has two benefits. First, it effectively eliminates the OOV problem: as long as the text is made up of characters from our base vocabulary (e.g., the English alphabet, or a subset of Unicode characters), we can always assign it a valid sequence of tokens without any OOVs. This scheme can seamlessly account for the minor variations in text that make word-level modeling so difficult, such as typos, inflections, gibberish, etc., all of which are able to be processed at the character level. Second, we require very few parameters to encode the vocabulary, as the number of characters in a script is often small, and certainly smaller than the number of words in the language.

This method has shown promise (and we cover it further in Section 9), but has three serious drawbacks. First, characters as individual units often carry little to no meaning, and so processing a long sequence of them often involves spending computation handling low-signal inputs. In the following example, the model requires 10 times as many inference calls to process the same text as the word-level tokenization, but the individual characters are all nearly meaningless in isolation.

Example	
Raw Text:	<i>California</i>
Character Tokenized:	C a l i f o r n i a
Word Tokenized:	California

Second, character-level modeling relies on knowing *a priori* what characters will possibly be processed, and on having enough memory to store representations of them. If a model is trained only on Latin-script languages, but unexpectedly sees a Chinese character, it will not know how to process it. Additionally, if the model expects to see a mix of scripts that has a large union of characters (in the extreme case, the entire Unicode character set), it may not be feasible to have a representation for each of them. One solution for this is to encode the text as the UTF-8 byte-level sequence, which fully resolves the OOV problem (for any Unicode script) with a vocabulary of only 256 bytes.

Example	
Raw Text:	<i>Japan</i>
Byte Tokenized:	<0x4a> <0x61> <0x70> <0x61> <0x6e>

Example	
Raw Text:	日本
Byte Tokenized:	<0xe6> <0x97> <0xa5> <0xe6> <0x9c> <0xac>

²E.g., for Llama 3, with an embedding dimension of $d = 4096$, a vocabulary size of 1 million would result in more than 4 billion parameters dedicated to the embedding layer. The smallest Llama 3 model has 8 billion parameters, so embeddings would consume most of the parameter budget.

However, this only exacerbates the third problem: language models struggle with long sequences. Character-level tokenization produces much longer sequences than word-level tokenization, creating both representational and computational difficulties. Representationally, information is spread across many more positions, making it harder for a model to compress and retrieve. Computationally, longer sequences require more work to process, especially in transformer models, where standard self-attention has quadratic complexity in the input length (Vaswani et al., 2017). Byte-level modeling can further increase sequence length, since UTF-8 can require up to four bytes for a single code point.

Despite these challenges, character- and byte-level modeling has seen a resurgence in modern NLP with advancements in latent embeddings, which we cover in Section 9.2, or language model architectures with better runtime complexity than the standard transformer (which we do not cover in this survey; Peng et al., 2023; Gu & Dao, 2024, *inter alia*).

As a middle ground between character- and word-level tokenization, embedding models such as FastText (Borjanowski et al., 2017) popularized character n -gram tokenization. In this scheme, words were broken down into a sequence of short, character-level n -grams. Each of these n -grams is assumed to be in-vocabulary, since there is a finite, enumerable number of them based on the size of the base alphabet and the n -gram length. In practice, n -grams were randomly assigned, based on a hashing function, to one of a fixed number of buckets for embeddings. Consequently, even if a previously unseen n -gram is observed, its hash still maps to a defined embedding, meaning no n -gram is considered OOV.

Example	
Raw Text:	<i>Kentucky</i>
Character Tokenized:	K e n t u c k y
3-gram Tokenized:	Ken ent ntu tuc uck cky

This approach uses larger units of text than just character-level tokenization, which should hold more semantic meaning, but still can tokenize any input word without OOVs (provided it uses the same base alphabet). However, it too has drawbacks. First, the tokenized sequence length is still linear in the length of the word,³ so there is no runtime improvement over the character-level model and it still suffers from information loss due to long sequences. Second, the number of n -grams grows exponentially with the size of the alphabet and the length of the n -gram, necessitating the aforementioned hashing trick to keep parameter costs manageable. This introduces hash collisions, where two distinct n -grams are mapped to the same embedding, which means the model is unable to distinguish between them, making it unsuitable for generative models.

Yet another approach is using linguistically informed, rule-based tokenization such as stemming or morphological segmentation. These approaches attempted to break down words into a more manageable set of linguistic units to reduce the vocabulary size while mitigating OOVs and still preserving the core semantic meaning. However, the processes of stemming, lemmatization, and morphological analysis tend to rely on language models or complicated heuristic algorithms that may not always be available for a given language, and may introduce errors. In addition, stemming and lemmatization inherently drop information when the conversion to the base form is performed, as shown in the following example:

Example	
Raw Text:	<i>The dogs were chasing the cat</i>
Lemma Tokenized:	The dog be chase the cat
Stemming Tokenized:	The dog were chas the cat
Morphological Tokenization:	The dog s were chas ing the cat

³Specifically, it is $O(|w| - n)$, where $|w|$ is the number of characters in the sequence, and n is the n -gram length. n is often treated as a constant, meaning the length reduces to $\Theta(|w|)$.

Finally, each of these morphological tokenization schemes still suffers from the same problem as word-level models: if the number of stems, lemmas, or morphemes is too large, we must either pay enormous parameter costs or allow OOVs. Nevertheless, morphological segmentation has continued to find use in modern NLP, not as a tokenization algorithm, but instead as a *pretokenization* algorithm (discussed in more detail in Section 4).

Each of the tokenization schemes discussed above have different characteristics: some work on the word level, some on the character level, and some in-between. However, they share a common feature in that they are all *not learned*. Character-level and n -gram tokenizers only require knowledge of the base alphabet, while word-level tokenizers just require a curated word list. Morphological tokenizers use complex algorithms for tokenization, but these are typically crafted by domain experts using linguistic knowledge.

By contrast, tokenization into *disjoint, concatenative subword units*, henceforth “subword tokenization” or even just “tokenization” when clear, uses a different paradigm of constructing a vocabulary and learning a tokenization function purely from data, but also takes some inspiration from these prior methods. Subword tokenization seeks to eliminate OOVs like character-level models do, while packing as much information into a single token as possible like word-level models. Operating in between characters and words, like the n -gram and morphological tokenizers, it learns which pieces of the words are meaningful as an optimization problem, rather than a static assignment or heuristic. We introduce many subword tokenization algorithms in Section 3.

Subword tokens, the items in a **subword tokenizer**’s vocabulary, have several notational conventions. In this survey, we adopt the “space-initial” notation and denote space characters with `▬` (U+2581), as popularized by the SentencePiece tokenizer library (Kudo & Richardson, 2018). For example, the sentence *Test sentence* could be tokenized as `Test▬sentence`. The **detokenization** function, denoted DETOKENIZE, can be used to recover the surface form from the token sequence by concatenating the tokens and applying the whitespace information:

$$\text{DETOKENIZE}(\text{Test▬sentence}) = \text{Test sentence}.$$

Other common conventions are “trailing-space” marking (e.g., `Test▬sentence`) or “internal” subword marking (e.g., `Test▬sentence`). Internal marking denotes which tokens *do not* have a space before them and was popularized by the BERT family’s tokenizer (Devlin et al., 2019). The GPT family’s tokenizer follows the space-initial marker convention, but uses `Ġ` (U+0120) to denote spaces (e.g., `TestĠsentence`). The choice of space marking is often implemented as a pretokenization step, as discussed in Section 4.2.

2.2 Tokenization and Language Modeling

The primary use case for tokenization is in **language modeling**: estimating a probability distribution over *strings*. Notably, a language model does not operate on strings directly; it operates on the token sequences a tokenizer produces. The tokenizer fixes the mapping between strings and token sequences, so it sits between the text and the distribution the model estimates. This mismatch is usually assumed to be inconsequential, but it introduces gaps that affect specific analyses that researchers and practitioners should be aware of.

Language models are commonly described as defining probability distributions over strings. For language models that use subword tokenizers, the distribution is different: a subword language model p defines a distribution over *token* sequences $\mathbf{t} \in \mathcal{V}^*$ drawn from a finite subword vocabulary $\mathcal{V}$. For subword tokenizers, DETOKENIZE is in general many-to-one: for a fixed subword vocabulary, a string $\mathbf{s}$ may have many distinct token sequence that detokenize to it. We denote the set of all tokenizations of a string $\mathcal{T}(\mathbf{s}) = \{\mathbf{t} \in \mathcal{V}^* : \text{DETOKENIZE}(\mathbf{t}) = \mathbf{s}\}$.

As a concrete example, consider the string *sample sentence*. A given tokenizer maps this string to the sequence $\text{TOKENIZE}(\text{sample sentence}) = \text{sample▬sentence}$, which we call its **canonical tokenization**. But other token sequences in the vocabulary can also detokenize to *sample sentence*, e.g., `sa m ple▬sent en ce`. Consequently, the theoretically sound way to recover a distribution over strings from a distribution over

token sequences is to **marginalize** over $\mathcal{T}(\mathbf{s})$:

$$p(\mathbf{s}) = \sum_{\mathbf{t} \in \mathcal{T}(\mathbf{s})} p(\mathbf{t}). \quad (1)$$

Following the example above, using just the canonical tokenization of a string for its probability gives the estimate:

$$p(\text{sample sentence}) \approx p(\text{sample} \text{▬} \text{sent} \text{ence}).$$

However, to get the true probability of the string, we should instead compute the marginal:

$$p(\text{sample sentence}) = p(\text{sample} \text{▬} \text{sent} \text{ence}) + p(\text{sa} \text{ m} \text{ ple} \text{▬} \text{sent} \text{ en} \text{ ce}) + p(\text{sample} \text{▬} \text{se} \text{ nt} \text{ en} \text{ ce}) + \dots$$

In general, the marginal is difficult to compute and is often dominated by the canonical tokenization, which affirms using the canonical tokenization as an approximation of a word’s probability (Geh et al., 2024). We discuss the marginal in more detail in Section 8.2.

The mismatch between modeling the probability of strings and tokenizations shows up in many areas of tokenization research that we cover in this survey. For example, it causes difficulty in comparing language models that use different tokenizers (Section 7) or with model distillation between models with different tokenizers (Section 12), and it motivates byte- or character-level language modeling (Section 9), which has a one-to-one correspondence between the string and the tokenization processed by the model.

Tokenization has another direct implication for modern neural language modeling: the input and output layers of the underlying neural network architecture. While the internal layers of a language model take on many forms, though often derived from the standard transformer architecture (Vaswani et al., 2017), a typical language model has an input **embedding layer** that holds a continuous vector representation for each token that lives in the model’s internal embedding space. Thus, if a language model has a vocabulary of size $|\mathcal{V}|$ and an internal dimension of d , then a standard embedding layer consumes $O(|\mathcal{V}|d)$ parameters. Likewise, a model has an **output layer** (sometimes called the *unembedding layer*, *decoding layer*, or *language modeling head*), which projects from the model’s internal hidden representation to a vector of size $|\mathcal{V}|$, typically representing **logits**—unnormalized log-probabilities corresponding to the likelihood of a given token in the vocabulary that can be passed through the *softmax* function to produce a probability distribution. As with the embedding layer, the output layer also sometimes requires an additional $O(|\mathcal{V}|d)$ parameters. For models with large vocabularies or embedding dimensions, these layers can collectively account for a huge percentage of a model’s parameters, as shown in Table 2.1, especially for smaller models. Thus, the choice of tokenizer, or more specifically the vocabulary, affects the downstream model by steering its design and parameter allocation as well as by determining what computational requirements there are to host the model, which affects what it can be used for and by whom.

2.3 The Current State of Tokenization (Research)

Tokenization has been an understudied area of research in the context of modern LLMs. Whereas in the past, task-specific models were trained from scratch and ablation studies could be performed cheaply and easily on account of the model and dataset sizes being relatively small (Gowda & May, 2020), with hundred-billion-parameter-scale models and web-scale data, it is no longer cost-effective or feasible to perform wide hyperparameter sweeps. It is also difficult to compare intrinsic metrics across models with different tokenizers, since perplexity is not directly comparable if the tokenization is different (Mielke, 2019), and so it is hard to know which model is better after the pretraining phase with different tokenizers. Further, many LLMs come as a family of models, where smaller versions are trained from a primary version via distillation (Dubey et al., 2024; Gemma Team et al., 2025). The classic knowledge distillation setup assumes that the output space—in this case, the vocabulary of the model—is the same between the teacher and student models, so the loss function can be computed from the logits directly (Hinton et al., 2015). By changing the tokenizer, this assumption is violated, and knowledge distillation cannot be directly applied (though distilling models across tokenizers is an active area of research, which we introduce in Section 12).

Model	$ \mathcal{V} $	d	Emb Params (B)	Total Params (B)	Embedding %
Gemma 4 E2B [†]	262k	1536	2.78	4.65	59.80
Gemma 4 E4B [†]	262k	2560	3.54	7.52	47.15
Gemma 4 12B [†]	262k	3840	1.01	11.94	8.43
Gemma 4 31B [†]	262k	5376	1.41	30.70	4.59
gpt-oss-20b	201k	2880	1.16	20.91	5.54
gpt-oss-120b	201k	2880	1.16	116.83	0.99
Ministral 3 3B [†]	131k	3072	0.40	3.43	11.74
Ministral 3 8B	131k	4096	1.07	8.49	12.65
Ministral 3 14B	131k	5120	1.34	13.51	9.94
Mistral Medium 3.5	131k	12 288	3.22	125.03	2.58
Qwen3.5 0.8B [†]	248k	1024	0.25	0.77	32.90
Qwen3.5 2B [†]	248k	2048	0.51	1.94	26.18
Qwen3.5 4B [†]	248k	2560	0.64	4.33	14.69
Qwen3.5 9B	248k	4096	2.03	9.20	22.12
Qwen3.5 27B	248k	5120	2.54	27.32	9.31

Table 2.1 – Vocabulary size $|\mathcal{V}|$, model dimension d , the total parameters held by the embedding and output layers, the total parameters in the model, and the percentage of total parameters allocated to the embedding and output layers. For MoE models and the **Gemma 4 EXB** series, the parameter count refers to the full model, not just active or non-embedding parameters. For multimodal models, only text parameters are counted. [†]denotes *tied* embeddings, where the embedding and output layers share the same parameters.

On the other hand, scaling parameters elsewhere in the modern transformer-based LLM architecture is an attractive option compared to modifying the tokenizer. Scaling internal parameters rather than the vocabulary size still allows for direct comparison between different models and there are established scaling trends that practitioners can lean on to make informed decisions about how best to allocate their limited resources (Sutton, 2019; Kaplan et al., 2020; Hoffmann et al., 2022).

Additionally, while there are well-defined benchmarks for many language modeling tasks, these are typically downstream, extrinsic metrics that do not rely directly on tokenization and which are most often approached by taking a pretrained language model and fine-tuning or prompting it for the specific task, which does not modify the tokenizer (Devlin et al., 2019; Ruder et al., 2019; Brown et al., 2020; Wei et al., 2022).

Some intrinsic tasks exist that are designed specifically to encourage experimentation on architectures and hyperparameters in a consistent setting. For example, NanoGPT Speedrun optimizes for fastest wall-time to train a model to beat the GPT-2 perplexity on the FineWeb (Penedo et al., 2025) validation set (Jordan et al., 2024) given a fixed compute infrastructure, and has spurred numerous advancements in model design. A common requirement in this and similar benchmarks is that models must use a fixed tokenizer to make it easier to compare perplexity over a validation set, which means tokenization is not a valid component to optimize.⁴

Taken together, these factors have greatly limited experimentation with tokenization relative to other areas of the neural language modeling stack. This disparity is reflected in the number of tokenization-related papers published each year in premier venues, compared to the total number of NLP papers, as shown in Figure 2.1. In 2025, papers about tokenization (though not necessarily exclusively about tokenization) made up at most 2% of all papers.

Nevertheless, tokenization affects models in many ways. As models become more multilingual, they must be better suited to the languages that they eventually process (Fujii et al., 2024) (Sections 6 and 12); as they are used in more sensitive tasks, they must be reliable and robust to variations in tokenization that cause issues (Geh et al., 2025) (Sections 8 and 14.5); as they become more widely used, inference costs must be kept in check (Ahia et al., 2023) (Sections 7 and 11.3); to name a few. Each of these axes is directly

⁴However, an [unofficial NanoGPT Speedrun record](#) used the TokenMonster tokenizer (Forsythe, 2023).

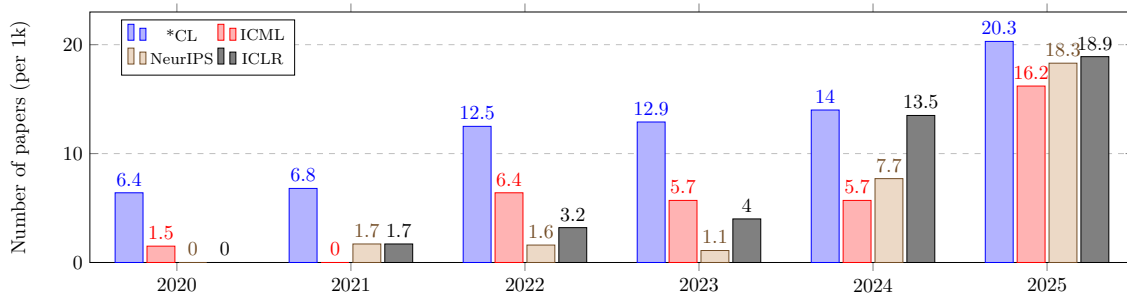


Figure 2.1 – Proportion of *CL and ML (NeurIPS, ICLR, ICML) abstracts mentioning “subword”, “tokenization” or “tokenizer” since 2020.

influenced by the tokenizer but lack the resources and metrics to properly study, compared to, for example parameter scaling.

Thus, in addition to this paper’s goal of surveying the field of tokenization, a secondary goal is to encourage more experimentation into tokenization for language models. In this survey, we highlight a large body of work that proposes tokenization methods that should improve language model performance; however, these methods are rarely adopted at scale (Section 11). This may be because these methods are only tested at smaller scales or do not make clear links to improvements in downstream performance. Additionally, small changes in the tokenizer may also have negative impacts on the model that will only become apparent after an expensive training run. Developing a better understanding of tokenization and creating well-defined benchmarks and metrics (see Section 7) allows tokenization to be more confidently treated as a tunable parameter and reduces the barrier for innovation in this space.

Term	Notation	Definition and Notes
Token	In text: <code>token_a</code> <code>token_b</code> In math: $\mathbf{t} = t_1 t_2 \dots t_n$	A discrete unit of text produced by a tokenizer. We denote individual tokens as <code>token</code> in this paper. For brevity, we omit artifacts like space markers, end-of-word markers, etc., where clear. In mathematical notation, we use the bold $\mathbf{t} = t_1 t_2 \dots t_n$ to refer to a token sequence.
Tokenizer	[Tokenizer name]	An object that transforms text into a list of tokens, comprised of an alphabet, a pretokenizer, a vocabulary, an inference algorithm, and possibly auxiliary elements.
Alphabet	Σ	The set of atomic elements in strings a tokenizer is equipped to process.
Vocabulary	$\mathcal{V}$	The fixed set of tokens that a tokenizer can output. Typically, the alphabet is a subset of the token vocabulary.
Surface form	In text: <i>surface form</i> In math: $\mathbf{s}$	A raw text string, often in the context of a string encoded by a token sequence.
Tokenization	$\text{TOKENIZE} : \Sigma^* \rightarrow \mathcal{V}^*$	The application of the tokenizer to transform text into a tokenized sequence from its vocabulary. Sometimes called <i>encoding</i> .
Detokenization	$\text{DETOKENIZE} : \mathcal{V}^* \rightarrow \Sigma^*$	The reversal process where a token sequence is converted back to its surface form.
Canonical (Tokenization)		The token sequence produced by a deterministic tokenizer given an input.
(Tokenizer) training		The process of learning the tokenizer elements (including the vocabulary), typically data-driven. Sometimes called <i>learning</i> or <i>vocabularization</i> .
Pretokenization		A typically non-data-driven process preceding the application of core tokenizer functions (training and inference), manipulating and dividing input text into more easily ingestible tokenizer inputs.
Pretokens	[pretoken_a] [pretoken_b]	The units resulting from pretokenization that are processed individually by the tokenizer during training and inference.
Language Model Inference	$p(\mathbf{t})$ $p(t \mid \mathbf{t})$	Computing the probability of a tokenized sequence or of predicting a token given a context using a probabilistic language model.
Embedding Vector	$\mathbf{e} \in \mathbb{R}^d$	A d -dimensional real vector corresponding to a token and used as the representation for the token in a language model's parameters.
All Possible Tokenizations	$\mathcal{T}(\mathbf{s})$	The set of all possible tokenizations of $\mathbf{s}$.

Table 2.2 – Vocabulary, definitions, and notation used throughout this paper.

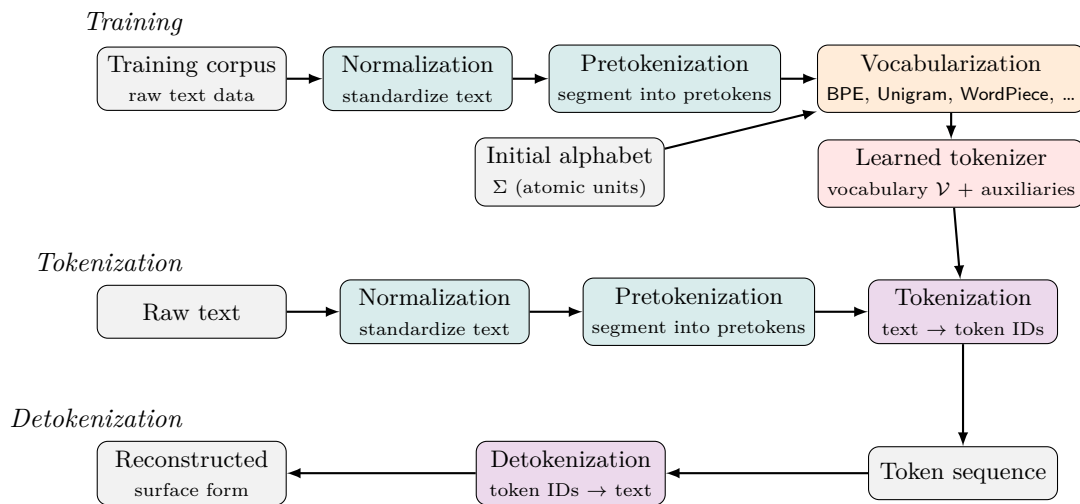


Figure 3.1 – Overview of tokenizer training (corpus + initial alphabet → normalization → pretokenization → training algorithm → learned tokenizer), tokenization (raw text → normalization → pretokenization → token sequence), and detokenization (token sequence → reconstructed surface form). The learned tokenizer parameterizes both tokenization and detokenization.

3 Tokenization Algorithms

We now turn to the basis of modern tokenization, which is dominated by *subword* tokenization methods. We introduce the core algorithms, along with their recent variants that are designed to mitigate some of the issues present in the original schemes.

Algorithmically, we distinguish between **tokenizer training**, which produces a tokenizer from a training corpus and initial **alphabet** (typically the set of bytes or Unicode **characters** appearing in the corpus), and **inference**, which applies the learned tokenizer to convert text to tokens.⁵ In this section, we do not cover the usual preprocessing steps that are often applied to put the text into a form that can be processed by the tokenizer: **normalization**, which standardizes the input text, and **pretokenization**, which segments text into coarse units. These steps, covered in Section 4, help enforce desired vocabulary properties, for example, by preventing tokens from spanning word boundaries or by ensuring that sequences of digits are tokenized digit-by-digit. Pretokenization can also substantially speed up training and inference by breaking the text into smaller chunks which can be processed in parallel. In this section, we assume that the text has already been normalized and pretokenized before being fed to the training or inference algorithms. The overall scheme is shown in Figure 3.1, and Table 3.1 provides a reference for delineating the terms and algorithms mentioned in this section.

3.1 Byte-pair Encoding (BPE)

Byte-pair encoding (BPE) is an algorithm originally developed for data compression (Gage, 1994) and modified for language subword tokenization by Sennrich et al. (2016). The intuition behind BPE is to identify pairs of adjacent tokens that co-occur frequently in text and merge them into a new, larger token. This has an obvious compression motivation: replacing a pair of tokens with a single token strictly decreases the sequence length, and biasing towards frequently occurring pairs improves the empirical compression rate. In addition, iteratively merging frequent pairs tends to surface linguistically meaningful units such as morphemes, which recur across many words and therefore appear as frequent character sequences.

BPE has separate training and inference stages, but they are closely coupled: the training algorithm is a greedy heuristic that, starting from the base alphabet, iteratively merges the most common pair of adjacent tokens to build a vocabulary of subwords, and inference replays these merges in order over the input text.

⁵Sometimes referred to as **encoding**, **decoding**, or **tokenization**.

	Vocabulary training (steps; criterion; auxiliary element)	Inference
Byte-pair encoding (BPE) (3.1)	Bottom-up (merges) Frequency-based	Merges
BPE variants (3.1.3)	Dropout	Merges (stochastic)
	Picky	Merges w/ removals
	Scaffold	Merges w/ extra rules
	Trimmed	Merges w/ extra rules
	Knockout	Merges w/ extra rules
	Boundless	Merges
	Super	Merges
	Parity-aware	Merges
	S-BPE	Merges
Unigram (3.2)	Top-down (ablations) Likelihood	Likelihood
MinGram (3.2)	Top-down (pruning) w/ BPE seed Token count	Shortest Path w/ likelihood tiebreak
WordPiece (3.3)	Bottom-up (merges) LM-based	MaxMatch
SaGe (3.4)	Top-down (ablations) Context-based	MaxMatch
VOLT (3.5)	Optimal Transport	Merges
ConvexTok (3.5)	Integer Linear Programming	Shortest Path
MaxMatch (3.6)	Any	Greedy (left to right, longest token)
PathPiece (3.6)	Any	Shortest Path
FLOTA (3.6)	Any	Greedy (longest token)
DP Inference (3.6)	Any	Dynamic Programming

Table 3.1 – Modern tokenizer variants and their vocabulary training and inference characteristics.

The original formulation operates on a character alphabet (Sennrich et al., 2016), but operating directly on UTF-8 bytes (*Byte* BPE, or BBPE; Wang et al., 2020a) to avoid out-of-vocabulary problems was popularized by GPT-2 (Radford et al., 2019) and has become the de facto standard (see Section 11).

3.1.1 BPE Training

We first cover BPE training, which produces a vocabulary and an ordered merge list from a given corpus. Code 3.1 provides a basic Python implementation. Most implementations of BPE take a pretokenized corpus as input, and the co-occurrence statistics are aggregated over the bigrams that appear in each pretoken. We include a simple pretokenization scheme in our implementation to make this concrete.

The algorithm repeatedly searches for the most frequently co-occurring token pair across all pretokens, merges it into a new token, and adds the new token to the vocabulary until the desired vocabulary size is reached or no candidate pairs remain (that is, every pretoken has been reduced to a single token). Adding a new token to $\mathcal{V}$ also entails merging it throughout the corpus and appending the merge information to the merge list μ .

Some common optimizations for BPE training are to represent the corpus with a pretoken-to-count dictionary to avoid having to iterate over all pretokens in the corpus at each step; using a priority queue to quickly find the most frequent pair; and incrementally updating pair counts after each merge (Sennrich et al., 2016; Zouhar et al., 2023b).

3.1.2 BPE Inference

After training is complete and we have a BPE tokenizer $(\mathcal{V}, \mu)$, we wish to apply it to raw text to obtain a token sequence. BPE inference works independently on each pretoken by applying an iterative encoding algorithm. The simplest implementation, given in Code 3.2, applies each merge in the merge list one by one to a pretoken. This mimics the training process, which formed longer and longer tokens with each merge. Merges are applied in the same order that the training algorithm added them to the merge list.

For a given pretoken, most merges in the list will have no effect since the component bigrams are not present in the pretoken’s surface form. As the size of the merge list grows (for example, to hundreds of thousands, as in modern LLM tokenizers; see Section 11), this results in significant wasted computation. An alternative implementation, shown in Code 3.3, avoids applying merges wastefully by first enumerating all token bigrams in a pretoken that correspond to a merge in the merge list, and then applying the one whose index in the ordered merge list is earliest. In practice, many optimizations to this basic implementation are done to speed up tokenization. In particular, instead of reconstructing the candidate bigram list from scratch at each iteration, we can update it incrementally by tracking the position of each candidate bigram in the pretoken. Then, a priority queue ordered by the merge’s index in μ can be used to quickly find the next merge to apply by popping the lowest-index merge and any other merges that share a component with it, and pushing the newly formed candidate merges. These optimizations reduce the time complexity of finding the lowest-index merge and updating the list from $O(|t|)$ to $O(\log |t|)$ at each iteration.

A complexity-theoretic analysis of BPE implied that inference could be performed in *linear* time to the length of the input (Berglund & van der Merwe, 2023; Berglund et al., 2024). Mamouras et al. (2026) and Jiang & Gong (2026) described streaming algorithms which match the linear-time bound, but also include terms derived from the size of the vocabulary.

Several libraries provide efficient implementations of BPE, including *tiktoken* (OpenAI, 2022), Hugging Face *tokenizers* (HuggingFace, 2019), and *SentencePiece* (Kudo & Richardson, 2018). For a particularly readable implementation of the basic BPE algorithm, see Karpathy (2024).

Byte-Pair Encoding Training

```
import collections, re

def pretokenize(sentence):
    """ A simple, example pretokenization function.
    "Hello, world!" -> [['H', 'e', 'l', 'l', 'o'], [','], [' ', 'w', 'o', 'r', 'l', 'd'], ['!']]
    """
    PRETOKEN_REGEX = re.compile(r" ?[^\s,.;'! ]+|[^\s,.;'! ]")
    return [list(pretoken) for pretoken in PRETOKEN_REGEX.findall(sentence)]

def apply_merge(sequence, pair):
    idx = 0
    a, b = pair

    while idx < len(sequence) - 1:
        if sequence[idx] == a and sequence[idx + 1] == b:
            sequence[idx : idx + 2] = [a + b]
            idx += 1

def bpe_training(corpus, target_vocabulary_size):
    pretokenized_corpus = [
        pretoken for sentence in corpus for pretoken in pretokenize(sentence)
    ]

    # Seed the vocabulary with all unique characters in the corpus.
    vocab = sorted({c for pretoken in pretokenized_corpus for c in pretoken})
    merges = []

    # Repeat until we reach the target vocabulary size.
    while len(vocab) < target_vocabulary_size:
        pair_counts = collections.Counter(
            (a, b)
            for pretoken in pretokenized_corpus
            for a, b in zip(pretoken, pretoken[1:])
        )

        # If there are no more possible pairs (each pretoken is a single token), stop early.
        if len(pair_counts) == 0: break

        # Add the most common pair information to the vocab and update the corpus.
        max_pair = max(pair_counts, key=pair_counts.get)
        merges.append(max_pair)
        vocab.append(max_pair[0] + max_pair[1])

        for pretoken in pretokenized_corpus:
            apply_merge(pretoken, max_pair)
    return vocab, merges
```

Code 3.1 – Python code for a BPE training loop, including pretokenization. This is the textbook algorithm, but many optimizations exist (for example, avoiding iterating over the entire corpus each iteration by using a better representation of the pretokens).

Byte-Pair Encoding Inference (Iterative Version)

```
def bpe_inference(sentence, merges):
    pretokenized_sequence = pretokenize(sentence)

    # Apply each merge in order to each pretoken independently.
    for merge in merges:
        for pretoken in pretokenized_sequence:
            apply_merge(pretoken, merge)

    # Return the final tokenized forms of all pretokens.
    return [
        token
        for pretoken in pretokenized_sequence
        for token in pretoken
    ]
```

Code 3.2 – Python code for the basic BPE inference loop. This version iterates over all merges in the merge list in order, and tries to apply them.

Byte-Pair Encoding Inference (Priority Version)

```
def bpe_inference_priority(sentence, merges):
    # A token pair -> index mapping for fast lookup (this can be precomputed).
    ranks = {pair: rank for rank, pair in enumerate(merges)}

    def encode(pretoken):
        while len(pretoken) > 1:
            # Among all token bigrams, keep the ones that are in the merge list.
            possible_merges = [
                pair for pair in zip(pretoken, pretoken[1:]) if pair in ranks
            ]

            # If there are no candidate merges, end early.
            if len(possible_merges) == 0: break

            # Find the available merge that is earliest in the merge list.
            merge = min(possible_merges, key=ranks.get)

            apply_merge(pretoken, merge)
        return pretoken

    # Return the final tokenized forms of all pretokens.
    return [
        token
        for pretoken in pretokenize(sentence)
        for token in encode(pretoken)
    ]
```

Code 3.3 – Python code for the basic BPE inference loop. This version iteratively finds the highest priority merges (the earliest in the merge list) and applies it, until there are no more possible merges. It is equivalent to the iterative BPE implementation (Algorithm 3.2), but avoids trying to apply merges that cannot exist in the input, allowing for some additional optimizations like the use of a priority queue.

3.1.3 BPE Variants

BPE is generally a good default choice as a simple algorithm with a clear objective. However, it has its inherent problems that can become critical in certain scenarios. Such issues are addressed in modified versions of BPE.

Vocabulary Trimming The default BPE implementation produces tokens that exist in the vocabulary but do not appear during inference, or are very rare in practice. These **rare tokens** (or **unreachable tokens**) are then passed to a model where their embedding vectors are undertrained, contributing little semantic benefit for the model, or even retaining their initialization values (perhaps dragged to zero by weight regularization). Undertrained tokens are potentially harmful, as they clutter the vocabulary, thus affecting the tokenizer’s efficiency, and are unknown to the model, which might lead to reduced performance (Sennrich et al., 2016; 2017) as well as hallucinations and safety risks (Geiping et al., 2024; Land & Bartolo, 2024). Sennrich (2015) suggested a best practice for using BPE tokenizers where underused tokens are removed from the vocabulary and replaced by their component merge sequence. For example, if a token `ab`, which is formed by the merge `a` `b`, is deemed rare, it is replaced (recursively) with its component sequence `a` `b`.

Rarely occurring tokens originate from three typical sources:

1. Overfitting to the tokenizer’s training sample, when some specific terms are frequent in the training sample but unlikely to appear in practice.
2. **Intermediate “junk” tokens** (Bostrom & Durrett, 2020) that are needed to form long tokens (e.g., `entucky`), which becomes redundant after merging with `K` into `Kentucky`).
3. Single bytes that are not used in UTF-8 (Land & Bartolo, 2024).

The straightforward way for mitigating the undertrained tokens problem is to remove them, a practice known as **vocabulary trimming**. Cognetta et al. (2024a) perform a breadth of ablation studies in a machine translation setting to determine under what conditions trimming the vocabulary helps. Their approach, **TrimmedBPE**, defines underused tokens as appearing below a certain frequency threshold on a given corpus, and splits them during tokenizer inference. Because decomposing tokens lengthens the resulting sequences, the study pays particular attention to the compression cost of trimming, and finds that trimmed models generally underperform the untrimmed baseline. Purason et al. (2026) develop the vocabulary trimming idea further and decouple this procedure from inference. They propose leaf-based pruning, which trims the vocabulary starting from the leaves of the BPE structure (tokens that do not appear as a component of another merge), thereby retaining the BPE logic and making it usable with general BPE inference implementations.

A related line of work is aimed at detecting the intermediate “junk” tokens *during* vocabulary building and removing them from the vocabulary. **PickyBPE** (Chizhov et al., 2024) uses the metric Intersection over Self (IoS), computed as the token-pair frequency divided by token frequency. As token-pair occurrences are always a subset of their constituent tokens’ occurrences, this metric lies within $[0, 1]$. The closer this value is to 1, the more likely the token is intermediate, as it is mostly used as a step on the way to form the next merge (like `entucky`). This method requires setting a relative threshold τ for token removal, with the recommended value being 0.9. A variant of PickyBPE reverts tokens to the original byte sequences upon removal (Schmidt et al., 2025).

Another approach by Lian et al. (2025), called **ScaffoldBPE**, tracks how much of each token’s frequency survives the merges it participates in. When a newly merged token is added to the vocabulary, if the remaining count of either component token falls below that of the newly merged token, the component is marked as a “scaffolding” token and removed from the final vocabulary.

These algorithms differ in their approach to inference. PickyBPE stores the vocabulary as a sequence of events: merges and removals, which all have unique IDs in the order of application during training. At the inference stage, the algorithm iteratively selects the event with the lowest ID from the currently possible events, thereby strictly adhering to the event sequence during training. ScaffoldBPE splits the excluded scaffold

tokens in the output sequence, similarly to TrimmedBPE. This introduces a trade-off between vocabulary creation simplicity and optimal corrective behavior, since splitting the token during training allows a new sequence of merges to form in its place, which is not followed during inference and leads to suboptimal segmentation. For example, if at a certain training step token `[bc]` is removed, `[a][bc][de]` will be split to `[a][b][c][de]`, which makes it possible for a token `[cde]` to appear. PickyBPE will follow this path during inference and segment this string to `[a][b][cde]`, whereas ScaffoldBPE will produce `[a][bc][de]` and then split `[bc]`, resulting in `[a][b][c][de]`, not allowing `[cde]` to appear (Chizhov et al., 2024).

An alternative response to the same problem is to keep rare tokens in the vocabulary but expose the model to them explicitly during training, e.g., via BPE-dropout (Provilkov et al., 2020).

Bauwens & Delobelle (2024) address a different problem with BPE merges, whereby early merges can precipitate morpheme boundary violations that cannot be undone given the forward-only aspect of the inference algorithm. These merges are not necessarily scaffold merges, as they may result in high-frequency tokens which are nevertheless detrimental to morphology. They introduce **BPE-knockout**, which uses supervised segmentation data (e.g., a corpus of morphological decompositions) to find problematic merges and remove them. Merges are removed from the BPE merge graph via **byte-tuple encoding**, which modifies merges containing tokens that can no longer be formed by letting them merge more than two tokens at once. The resulting merge list, with some entries removed and some modified, is used during inference, showing improvements for BERT-based models in several languages. **ReBPE** (Bauwens & de Lhoneux, 2026) applies BPE-knockout iteratively, interspersed with restoration phases that try to add better merges to the tokenizer. This procedure is shown to significantly improve morphological alignment over BPE-knockout.

Superword Tokens Whitespace and other syntactic delimiters normally serve as pretokenization boundaries, limiting models’ capacity for extracting semantics from common and useful multi-word expressions, instead forcing vocabularies to saturate by assigning single tokens to rare or niche pretokens (Reddy et al., 2025). Superword algorithms allow crossing pretoken boundaries by creating tokens which encompass multiple words or syntactic elements. Gee et al. (2023) add multi-word tokens to pretrained models, rather than incorporate them into the vocabulary learning process. **SuperBPE** (Liu et al., 2025) trains an ordinary BPE tokenizer up to a certain point (tunable as a hyperparameter), after which pretoken boundaries are removed and superword training begins. This increases average token length, boosts compression, and improves the model’s downstream performance. **BoundlessBPE** (Schmidt et al., 2025) has a different approach: superword merges are allowed from the very beginning of tokenizer training, but only in cases where the tokens taking part in the merge represent complete pretokens, thus avoiding partial cross-boundary tokens like the second one in `[fortr][ess][of][solitude]`. Schmidt et al. (2026) offer a significantly faster implementation for both superword models.

Modifying the Merge Ranking Objective Vilar & Federico (2021) propose **S-BPE**, a statistical extension of BPE. Apart from raw pair frequency, S-BPE’s training objective includes the *pointwise mutual information* score (PMI; Church & Hanks, 1990):

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x) \cdot P(y)}. \quad (2)$$

Here $P(x)$ and $P(y)$ are token probabilities, and $P(x, y)$ is a pair probability. Apart from this, S-BPE also has a stopping criterion based on likelihood decay and accounts for low-frequency false positives. Foroutan et al. (2026) suggest parity-aware BPE (**PA-BPE**) to improve the support for low-resource languages. At each merge selection, PA-BPE detects the language that is currently compressed the least well in the training sample and selects the most frequent merge using only the data in this language. The algorithm also optionally uses a sliding window to prevent long series of merges within a single language. This modification trades a negligible decrease in overall compression for major improvements in language compression parity and better support for low-resource languages, thus mitigating the issues discussed by Ahia et al. (2023). Chizhov et al. (2026) propose source-attributed BPE (**SA-BPE**) for code models, incorporating repository and language count into the BPE training objective. This approach reduces overfitting to single-repository variable names, thereby substantially decreasing the number of undertrained tokens.

3.2 Unigram Language Modeling Tokenization

The **Unigram** tokenization algorithm (Kudo, 2018), sometimes known as **UnigramLM** or **ULM**, takes a probabilistic modeling approach to tokenization. The main underlying assumption of **Unigram** is that text is the surface realization of samples from a latent-variable generative model. This generative model is parameterized by a unigram distribution; each token in a token sequence is drawn independently from a unigram language model, and the strings we see are the concatenations of them. Tokenization is then simply the recovery of this underlying segmentation. We provide a brief description of the algorithm here; for a more in-depth exposition, see Meister (2025).

Given the above probabilistic foundation, the first practical challenge lies in learning the specific parameters—namely, the vocabulary and unigram distribution probabilities. The **Unigram** algorithm provides a **top-down vocabulary construction** method for learning both of these parameters. The training algorithm uses *expectation-maximization* (EM) with iterative vocabulary pruning. Starting from a large initial seed vocabulary, the algorithm alternates between: (1) computing expected (E) token counts across all possible segmentations of the corpus, weighted by their probability under the current model; and (2) updating token probabilities via maximum (M) likelihood. After each EM iteration, a pruning step removes tokens whose absence causes the smallest decrease in corpus log-likelihood, continuing until the desired vocabulary size is reached. A reference implementation is given in Code 3.4.

At inference time, we must choose a segmentation according to this model. **Unigram** chooses the most probable segmentation under the model, which can be found efficiently via the Viterbi algorithm (Viterbi, 1967), as shown in Code 3.5.

In practice, the **Unigram** algorithm is primarily defined by its implementation in **SentencePiece** (Kudo & Richardson, 2018), which makes several engineering choices not fully specified in the original paper. These include vocabulary initialization via suffix arrays, a Bayesian variant of the M-step, and approximations to the pruning loss computation. Recent work has provided clearer documentation of these implementation details (Land & Pinter, 2026), finding that many expensive defaults have minimal impact on the final tokenizer. **MinGram** (Land, 2026b) takes this further, replacing the suffix-array initialization with a BPE-derived seed vocabulary, marginal EM with hard EM over a minimum-token path, and the iterative pruning loop with a single score-based pruning step. Sequence compression becomes the primary training objective, with the learned unigram score retained only as a tiebreak among equally short segmentations.

Unigram provides several advantages over BPE, creating seemingly more morphologically plausible tokens (Bostrom & Durrett, 2020; Vemula et al., 2025b). Despite these advantages, its adoption has been limited, perhaps in part due to the inference efficiency of BPE, which makes it an appealing choice for practitioners. Due to its probabilistic nature, **Unigram** naturally supports *subword regularization* (see Sections 3.7 and 8): sampling from the distribution over segmentations during training, which can improve model robustness (Kudo, 2018). BPE achieves a similar effect only through ad hoc modifications such as BPE-dropout (Provilkov et al., 2020).

3.3 WordPiece

WordPiece was originally introduced by Schuster & Nakajima (2012) under the name “WordPieceModel”. The training part of this algorithm iteratively merges pairs of tokens and adds them to the vocabulary. During the vocabulary initialization, **WordPiece** builds a probabilistic language model on the training data and, each time, chooses a pair of tokens that increases the overall likelihood the most. This idea was later adapted in tutorials and course materials⁶ as greedy BPE-style merging with an objective similar to PMI, usually without the logarithm:

$$\text{Score}(x, y) = \frac{P(x, y)}{P(x) \cdot P(y)}. \quad (3)$$

This objective essentially prioritizes merges that maximize the overall likelihood, which, for a simple probabilistic model, is the product of individual token probabilities. Although not mentioned in the original

⁶For example, in Hugging Face’s online course (<https://huggingface.co/learn/llm-course/chapter6/6>).

Unigram Training

```
import collections, math
from scipy.special import logsumexp

def expected_counts(pt_counts, logp):
    """ E-step: expected count of each token, over ALL segmentations of the corpus. """
    counts = collections.Counter()
    for w, f in pt_counts.items(): # pretoken -> frequency
        n = len(w)
        alpha, beta = [0.0] + [-math.inf] * n, [-math.inf] * n + [0.0]
        for i in range(1, n + 1): # Forward pass: log-probability of the prefix w[:i].
            alpha[i] = logsumexp([alpha[j] + logp[w[j:i]]
                                for j in range(i) if w[j:i] in logp])
        for i in range(n - 1, -1, -1): # Backward pass: of the suffix w[i:].
            beta[i] = logsumexp([logp[w[i:j]] + beta[j]
                                for j in range(i + 1, n + 1) if w[i:j] in logp])
        for i in range(n):
            for j in range(i + 1, n + 1):
                if (t := w[i:j]) in logp: # Posterior probability of t at this position.
                    counts[t] += f * math.exp(alpha[i] + logp[t] + beta[j] - alpha[n])
    return counts

def viterbi_score(string, logp, exclude=None):
    """ Log-probability of the single best segmentation, or -inf if none exists. """
    best = [0.0] + [-math.inf] * len(string)
    for i in range(1, len(string) + 1):
        best[i] = max((best[j] + logp[string[j:i]] for j in range(i)
                       if string[j:i] in logp and string[j:i] != exclude),
                      default=-math.inf)
    return best[-1]

def unigram_training(corpus, target_vocabulary_size, seed_vocabulary, em_iterations=2,
                    shrink_factor=0.75, overshoot=1.1):
    pt_counts = collections.Counter("".join(pt) for s in corpus for pt in pretokenize(s))
    characters = {c for w in pt_counts for c in w}
    logp = dict(seed_vocabulary)
    interim_size = int(overshoot * target_vocabulary_size)
    while True:
        for _ in range(em_iterations): # Re-estimate p(token) for a fixed vocabulary.
            counts = expected_counts(pt_counts, logp)
            total = sum(counts.values())
            logp = {t: math.log(c / total) for t, c in counts.items() if c > 0}
            if len(logp) <= interim_size: break
        # Loss of a token: likelihood decrease if its occurrences were re-segmented.
        # Infinite for single characters, which are therefore never pruned here.
        loss = {t: counts[t] * (lp - viterbi_score(t, logp, exclude=t))
                for t, lp in logp.items()}
        new_size = max(interim_size, int(shrink_factor * len(logp)))
        logp = {t: logp[t] for t in sorted(loss, key=loss.get, reverse=True)[:new_size]}
        # The final cut is made by probability, but every character must be retained.
        ranked = [t for t in sorted(logp, key=logp.get, reverse=True) if t not in characters]
        keep = characters | set(ranked[:target_vocabulary_size - len(characters)])
    return {t: logp[t] for t in keep}
```

Code 3.4 – Python code for Unigram training, following the SentencePiece implementation (Kudo & Richardson, 2018) of the algorithm of Kudo (2018), with minor omissions for clarity, and reusing the `pretokenize` function of Algorithm 3.1. Unlike BPE, the vocabulary starts large and is pruned down, and the artifact produced is a vocabulary with probabilities. The `seed_vocabulary` argument maps tokens to initial log-probabilities and holds every character of the corpus along with a large number of frequent substrings, which SentencePiece enumerates using a suffix array.

Unigram Inference

```
def unigram_inference(string, probs):
    """ `probs` holds token probabilities, e.g.: {"a": 0.1, "b": 0.1, "ab": 0.2, ...} """
    n = len(string)

    # `best[i]` holds the probability of the highest-probability segmentation of `string[:i]`.
    best = [0.0] * (n + 1)
    best[0] = 1.0

    # `trace[i]` holds the final token in the highest-probability segmentation of `string[:i]`.
    trace = [None] * (n + 1)

    for right in range(1, n + 1):
        for left in range(right):
            token = string[left:right]
            if token not in probs: continue

            # Probability the best segmentation of `string[:left]` followed by `token`.
            score = best[left] * probs[token]

            # If we have found a new best segmentation, update the dynamic programming tables.
            if score > best[right]:
                best[right] = score
                trace[right] = token

    # Reconstruct the highest-probability segmentation by following the trace backwards.
    tokens = []
    position = n

    while position > 0:
        token = trace[position]
        tokens.append(token)
        position -= len(token)

    # `tokens` was populated in reverse order.
    return tokens[::-1]
```

Code 3.5 – Python code for the basic Unigram inference algorithm. This finds the highest probability segmentation, but other variants can sample from the distribution over segmentations. In practice, inference is often performed in the log-probability domain for numerical stability. As with other tokenization algorithms, inference is often applied independently over each pretoken, but we omit that here for brevity.

work by [Schuster & Nakajima \(2012\)](#), the PMI criterion closely corresponds to the original likelihood-based training objective.

The original inference procedure of WordPiece was described similarly to that of BPE ([Schuster & Nakajima, 2012](#)), as deterministic decoding using the learned binary tree of merges and was used in Google’s neural machine translation system ([Wu et al., 2016](#)). BERT ([Devlin et al., 2019](#)), which used a WordPiece-trained vocabulary, coupled it with a greedy MaxMatch inference algorithm (see Section 3.6), eliminating the need for the merge list. This means the most popular implementation of WordPiece stored only the vocabulary artifact. As a result, “WordPiece” denotes a family of related algorithms rather than a single one, differing

even in the training stage: TensorFlow learns the vocabulary top-down,⁷ whereas Hugging Face’s `tokenizers` trains it with the BPE algorithm.⁸

3.4 Contextual Vocabularies

The training phases of BPE, Unigram, and most variants are all reducible to functions of pretoken-level unigram counts: vocabularies are optimized locally and the interaction between neighboring tokens beyond simple co-occurrence statistics do not factor into the overall objectives. This creates a discrepancy with the main use case of tokenizers, which is to provide language models with token lists that are helpful for *contextual* processing. **SaGe** (Yehezkel & Pinter, 2023) addresses this issue by incorporating a contextual objective into the training procedure. SaGe’s top-down algorithm (see Section 3.2) trains a SkipGram model (Mikolov et al., 2013) over intermediate vocabularies, iteratively pruning tokens based on their ability to predict neighboring tokens. The resulting vocabulary consists of tokens that appear in fewer contexts overall, providing a downstream model with a more focused set of tokens for which less noisy embeddings can be learned during pretraining.

One technical drawback of contextual vocabularies is that the need for context information does not allow for the pretoken-counter implementation afforded to BPE and Unigram. Instead the entire corpus must be processed at each iteration, greatly reducing training speed. SaGe 2.0⁹ improves the original naïve implementation by parallelizing the counting of affected contexts for candidate token removals over corpus lines, speeding up vocabulary training from three orders of magnitude slower than pretoken-aggregation to one.

3.5 Globally Optimized Vocabulary Construction

The tokenizer training algorithms we have discussed so far have been locally optimized and greedy: they add or remove tokens based on a local objective (like co-occurrence or frequency) while ignoring what could be a better, global solution.

One form of global vocabulary construction is through the lens of optimal transport theory. **VOLT** (*Vocabulary Learning via Optimal Transport*) treats both the final size of the vocabulary and the selection of the tokens as an optimal transport problem by determining the marginal utility, which it defines with an entropy measure, of adding an additional set of tokens to the vocabulary (Xu et al., 2021). The authors find that, relative to the typical vocabulary sizes used in machine translation, VOLT produces much smaller vocabularies but improves downstream performance.

Another form of tokenizer training is where the global objective that is desired is used in an integer linear program designed to optimize the tokenizer’s vocabulary construction. In **ConvexTok**, the vocabulary is recovered from an approximate solution to a linear program and a shortest path algorithm is used during tokenizer inference (Tempus et al., 2026). This formulation has the additional benefit of evaluating how far a provided tokenizer is from optimal, given the objective function.

3.6 General Inference Algorithms

Apart from the inference algorithms tied to the specific tokenizer training algorithm and its artifacts, such as BPE’s merge-directed inference and Unigram’s likelihood-based Viterbi decoding, there is a host of inference algorithms that can be applied on any text with any subword vocabulary and can allow a user to choose some inference characteristics that may not emerge from training. Decoupling training-time inference and application-time inference can increase performance on certain metrics (Uzan et al., 2024).

⁷As of May 2026, TensorFlow’s `wordpiece_vocab` learner performs no greedy merging at all: it starts from whole words and iteratively splits them, refining subword boundaries from substring frequencies and thresholds (https://github.com/tensorflow/text/blob/v2.20.1/tensorflow_text/tools/wordpiece_vocab/wordpiece_tokenizer_learner_lib.py).

⁸The WordPiece trainer in Hugging Face’s `tokenizers` runs BPE training internally rather than the PMI-greedy procedure, as confirmed by the maintainers (<https://github.com/huggingface/tokenizers/issues/144>).

⁹<https://github.com/MeLeLBGU/SaGe>.

MaxMatch Inference

```
import re

def pretokenize(sentence):
    """ A simple, example pretokenization function.
    "Hello, world!" -> ["Hello", ",", " world", "!"]
    """
    PRETOKEN_REGEX = re.compile(r" ?[^\s,.;'! ]+|[^\s,.;'! ]")
    return PRETOKEN_REGEX.findall(sentence)

def maxmatch_inference(sentence, vocabulary):
    max_len = max(len(token) for token in vocabulary)
    tokenized = []

    for pretoken in pretokenize(sentence):
        idx = 0
        while idx < len(pretoken):
            # Individual characters are assumed to be in the vocabulary.
            candidate = pretoken[idx]

            # Search backwards from the longest possible length and check
            # if we find a substring that matches a token in the vocabulary.
            for length in range(min(max_len, len(pretoken) - idx), 0, -1):
                if pretoken[idx:idx + length] in vocabulary:
                    candidate = pretoken[idx:idx + length]
                    break

            # Add the next token to the tokenized output and skip forward
            # in the input.
            tokenized.append(candidate)
            idx += len(candidate)

    return tokenized
```

Code 3.6 – Python code for the basic MaxMatch inference algorithm. At each step, we search for the longest possible token that matches the current input and use that as the next token.

A widely deployed training-agnostic subword inference algorithm is **MaxMatch** (Palmer, 2000), brought into mainstream subword tokenization as the inference procedure for WordPiece (Section 3.3), though the underlying greedy maximum-matching strategy predates subword segmentation by several decades in the lexicon-based word segmentation literature. MaxMatch scans the input left-to-right, at each step emitting the longest prefix of the remaining string that appears in the vocabulary, as shown in Code 3.6. MaxMatch has persisted as an inference algorithm even where the original WordPiece training procedure has not. As mentioned, contemporary “WordPiece” tokenizers in the Hugging Face ecosystem are trained with BPE, while still using MaxMatch at decoding time. Although developed in the context of masked language models, MaxMatch has an underappreciated property relative to its competitors in the context of generative modeling. Among the deterministic algorithms surveyed here, MaxMatch is the one whose left-to-right, lookahead-free structure aligns with how autoregressive language models actually *generate* text. Uzan et al. (2024) additionally evaluate a greedy right-to-left variant, finding that direction of scan has only a modest effect on standard intrinsic tokenizer metrics.

Song et al. (2021) describe a faster implementation of the MaxMatch WordPiece tokenization algorithm that utilizes Aho-Corasick automata (Aho & Corasick, 1975) in order to provide linear-time tokenization in the length of the input, compared to the naïve MaxMatch implementation that requires $O(nm)$ time, where n is the length of the input and m is the length of the longest token in the vocabulary.

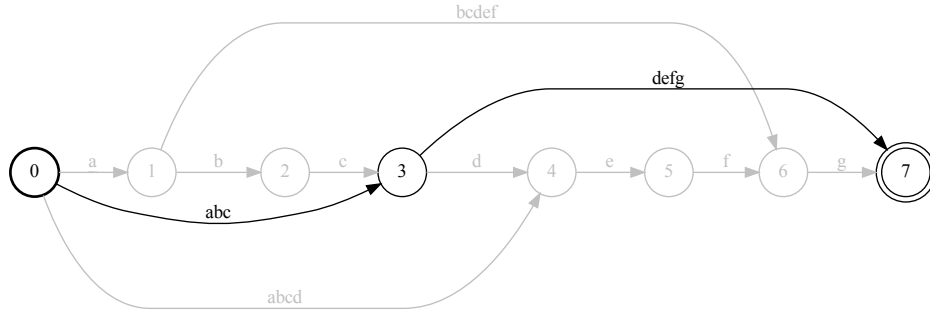


Figure 3.2 – PathPiece searches for the shortest path in the lattice of all tokenizations under a given vocabulary. In this case, the input is *abcdefg*, the vocabulary is $\{\boxed{a}, \boxed{b}, \boxed{c}, \boxed{d}, \boxed{e}, \boxed{f}, \boxed{g}, \boxed{abc}, \boxed{abcd}, \boxed{bcdef}, \boxed{defg}\}$, and the shortest path is $\boxed{abc}\boxed{defg}$. If multiple tokenizations have the same length, a tie-breaking rule must be applied.

A second family of inference algorithms targets the *minimum number* of tokens needed to cover the input. Schmidt et al. (2024) introduce **PathPiece**, a tokenizer whose segmentation algorithm achieves this objective by finding a shortest path through the lattice of valid segmentations. An example is given in Figure 3.2.

While shortest-path decoding over a segmentation lattice has a long history, PathPiece is the clearest recent formulation of minimum-token segmentation as a standalone inference objective for a fixed subword vocabulary. Sawada & Goyal (2025) apply the same shortest-path objective to BPE vocabularies under the name *maximal compression*, motivated by the observation that BPE merge lists expose an attack surface for training-data inference (Hayase et al., 2024) and may therefore be undesirable to ship with deployed models.

Example

Input	<i>abcdefg</i>
Vocabulary	$\{\boxed{a}, \boxed{b}, \boxed{c}, \boxed{d}, \boxed{e}, \boxed{f}, \boxed{g}, \boxed{abc}, \boxed{abcd}, \boxed{bcdef}, \boxed{defg}\}$
MaxMatch	$\boxed{abcd} \boxed{e} \boxed{f} \boxed{g}$
FLOTA	$\boxed{a} \boxed{bcdef} \boxed{g}$
PathPiece	$\boxed{abc} \boxed{defg}$

Example 3.3 – Examples of different generic tokenization strategies on the same input and given the same vocabulary. MaxMatch commits to the longest available prefix, $\boxed{abcd}$, and is then forced to fall back to single characters on the remaining suffix. FLOTA instead first selects the longest vocabulary token appearing anywhere in the string, $\boxed{bcdef}$, and then recurses on the remaining prefix and suffix, yielding $\boxed{a} \boxed{bcdef} \boxed{g}$. PathPiece finds the globally shortest segmentation, $\boxed{abc} \boxed{defg}$. The three algorithms thus disagree not because of tie-breaking details, but because they optimize different objectives: longest prefix token, longest token, and fewest tokens overall.

Another objective is to maximize the length of *individual* tokens rather than the prefix or the segmentation as a whole. **Few Longest Token Approximation (FLOTA)** (Hofmann et al., 2022) pursues this goal. Unlike MaxMatch, FLOTA is not constrained to consume the input left-to-right. It first identifies the longest vocabulary token appearing anywhere in the string, then recurses on the remaining prefix and suffix. The original FLOTA algorithm recurses to only a fixed maximum depth and drops all remaining untokenized spans, but, in theory, the recursive procedure could be run to full coverage, allowing exact reconstruction of the input. Another tokenization algorithm, **LBPE**, takes token length rather than merge index as the priority metric during inference (Lian et al., 2024), iteratively emitting the longest possible token anywhere in the yet untokenized string, making it equivalent to FLOTA with unbounded recursive depth.

One can treat segmentation as a *latent variable* to be marginalized during training rather than fixed as a preprocessing step. **Dynamic Programming Encoding (DPE)** (He et al., 2020) is a *mixed character-subword*

transformer whose decoder processes characters but emits a distribution over the subword vocabulary at each position. Because probabilities are conditioned on characters rather than on a specific segmentation of the context, the sum over all compatible segmentations decomposes into an efficient dynamic programming recursion analogous to the **Unigram** objective (Section 3.2). At inference, the same recurrence is run through Viterbi decoding, yielding the most probable segmentation. Meyer & Buys (2022) extend the architecture to autoregressive language modeling, showing improvements for low-resource agglutinative languages and more morpheme-like subwords. Meyer & Buys (2023) further unify segmentation and translation end-to-end, introducing *dynamic decoding* to adapt segmentations on-the-fly during generation, with the largest gains in very low-resource settings.

Multiplexing and Multiple Tokenizations The many different training-agnostic tokenization inference algorithms can be leveraged to form a **multiplexing tokenizer**. For a given input, one or several tokenization inference algorithms can be chosen to produce a (set of) tokenizations. One use of this is for subword regularization, i.e., to make the tokenizer’s output less stable. For example, Kambhatla et al. (2022) randomly select tokenizers for the input and output for a low-resource translation task and prepend a special tag that tells the model which tokenizer’s segmentation it should try to generate.

Multiple tokenizers can also be used simultaneously to exploit the advantages that different tokenization methods may have. Wu et al. (2020) use two different tokenizers to encode the same sequence by using self-attention and cross-tokenizer attention and find that the multi-view representation improves performance across several translation and summarization tasks. On the decoding side, Srinivasan et al. (2019) use a single input tokenizer with multiple independent heads corresponding to different target-sequence tokenizers for machine translation.

Lattice Tokenizations Rather than feeding a single tokenization in as input to a model, some works have explored **lattice tokenizations**, where a tokenization lattice (a directed, acyclic graph representing all possible tokenizations of an input given a subword vocabulary) is passed in as input in order to allow the model to learn from all segmentations of the input simultaneously and without needing to enumerate and process them individually (Buckman & Neubig, 2018). When used with transformers, a specialized positional encoding that encodes the incident and target states of the arc encoding a token in the lattice can be used to embed exactly how different tokens cover the input surface form (Xiao et al., 2019; Lai et al., 2021).

Subword lattices have other uses with respect to tokenization, which we cover more in Section 13.1.

3.7 Stochastic Tokenization

A common theme among the tokenization algorithms discussed in this section is that they are *deterministic*: for a given input, the output tokenization will always be the same. **Unigram** was the first to break this trend by introducing **stochastic tokenization** according to the unigram probability distribution produced during the tokenizer training process. Stochastic tokenization samples tokenizations for an input, rather than using the deterministic tokenization procedure in order to make the model more robust to minor variations in tokenization from new words, typos, capitalization, etc. It can also act as data augmentation since it exposes the model to a wider variety of contexts, even though the underlying text is the same. Stochastic variants of the tokenization schemes include **BPE-dropout** (Provilkov et al., 2020), which randomly forbids candidate merges during the merging procedure (Code 3.7 shows an implementation), and **MaxMatch-Dropout** (Hiraoka, 2022), which randomly rejects the longest token found in favor of a sequence of shorter tokens at each iteration.

Generic, tokenizer-agnostic stochastic tokenizers have also been used, which generate tokenizations uniformly at random from the set of all tokenizations, possibly with additional constraints like a maximum sequence length (Cognetta et al., 2024c; Bauwens et al., 2025). We cover stochastic tokenization more in Section 8.

BPE-dropout

```
# Among all token bigrams, keep the ones that are in the merge list.
# Randomly drop each candidate merge with probability p.
possible_merges = [
    pair for pair in zip(pretoken, pretoken[1:]) if pair in ranks and random.random() > p
]
```

Code 3.7 – BPE-dropout injects randomness into the tokenization process by randomly dropping candidate merges with probability p at each iteration. This requires only a small change to the implementation in Algorithm 3.3, highlighted in pink. As a result, at some iteration, the highest priority merge may be dropped and a lower priority merge may be applied instead, which can result in a different final tokenization.

3.8 Joint Tokenizer-Task Training

So far, the tokenization algorithms have been disconnected from their downstream use—they are created before the language model training begins using an objective that is not related to what it will be used for later, and are never updated. This motivates the research direction of jointly training a tokenizer with the model and task that it will be used for. However, it is not straightforward to connect tokenizers and neural models, as the former are discrete while the latter are continuous. Hiraoka et al. (2020) propose a jointly learned tokenization inference algorithm by using a neural unigram language model as a tokenizer for classification tasks. The tokenizer is trained by sampling several sequences from the unigram distribution during model training and increasing the weight of the path corresponding to the sequence that is most favored by the downstream model and task metric. Hiraoka et al. (2021) later extended this work to various downstream tasks, including sequence-to-sequence tasks. Hiraoka & Iwakura (2023) further explored task-specific tokenization selection for pretrained models with frozen parameters.

Gradient-based tokenization methods learn to predict token boundaries from an input byte stream (see Section 9.2). They inherently learn the tokenization algorithm based on the task at hand, since the token boundary predictor is trained jointly with the model (Nawrot et al., 2023; Ahia et al., 2024; Hwang et al., 2025) and the tokenization can change during pretraining and fine-tuning (Owodunni et al., 2026a).

3.9 Questions

The learned tokenizers discussed here all start with a predetermined vocabulary and then jointly learn the *tokenization algorithm* with a downstream language modeling task. What options are there to construct the *vocabulary* jointly with the language model and tokenization function?

MinGram and ConvTok offer a flexible way to directly optimize for particular features (though they both use compression in their original formulations). What other metrics (possible of those covered later in Section 7) fit into this paradigm, and which can be used either in conjunction with a compression metric or at least without reducing compression too much?

Feeding different tokenizations into a model shows up commonly in tokenization research. Stochastic tokenization just feeds one tokenization in at a time (limiting exposure to different tokenizations at a given time), while lattice tokenization encodes every possible tokenization (which requires a special positional encoding and more computation during training and inference). Is there a good middle ground, where a small set of tokenizations are encoded using a specialized lattice encoding? What tokenizations should be chosen? How would decoding work, and when should new tokens get added to the input lattice?

4 Pretokenization

As introduced in Section 3, pretokenization is a preprocessing step that segments text into coarse **pre-tokens** before tokenization. Pretokenization exists to enforce constraints on the final vocabulary, limiting token complexity regardless of how common certain sequences may be in the training data (Kudo & Richardson, 2018). For example, the string `youtube.com/watch?v=dQw4w9WgXcQ` is unlikely to be a suitable single token for language models, despite being particularly common on the internet, so it is broken into parts like `[youtube][.][com][/] [watch][?][v][=][dQw4w9WgXcQ]` which are individually more amenable to tokenization. In addition, tokens that cross whitespace boundaries such as the middle token in `[aft][er][h][ours]` are typically viewed as inferior and reduce model performance (Mielke et al., 2021; Schmidt et al., 2024),¹⁰ and pretokenization that splits text on whitespace serves to prevent such tokens (as tokens cannot cross pretoken boundaries). Finally, relatively strict pretokenization allows token counts to be aggregated for a much higher efficiency in noncontextual vocabulary training schemes (see Section 3.1).

An example of a pretokenization pass is given below, where the raw text is pretokenized based on whitespace and punctuation, and spaces are converted by convention to the `__` symbol (U+2581). An alternative convention, popularized by the GPT tokenizers, is to use the `Ġ` (U+0120) to denote spaces. Note that the result is a sequence of *pretokens* and not *tokens*. Tokens will form within each pretoken.

Example

Plain Text:	<i>What a week, huh? Captain, it's Wednesday.</i>
Pretokenized:	<code>[What][__a][__week][,][__huh][?][__Captain][,][__it]['s][__Wednesday][.]</code>

4.1 Normalization

One common preprocessing step is to normalize the input text into a standardized form before tokenization. This may involve removing unwanted artifacts, mapping equivalent surface forms to a common representation, or applying lossy simplifications intended to reduce superficial variation in the data. Common surface-level transformations include:

- Punctuation Normalization: e.g., *«hello»* → *“hello”*
- Whitespace Normalization: e.g., *hello\tworld* → *hello world*
- Invisible Character Removal: e.g., *aU+200Db* → *ab*¹¹

These normalizations are often used to unify minor typographical differences across scripts for easier processing or to clean up erroneous text. Invisible character removal is also important for tokenizer safety, as control characters can be used to inject token breakpoints to alter a model’s output while leaving the surface form of the text unchanged (see also Section 14.3; Boucher et al., 2022).

Normalization may also include transformations that reduce orthographic or lexical variation, such as:

- Case Folding: e.g., *Hello World* → *hello world*
- Diacritic Removal: e.g., *naïve* → *naive*
- Width Folding: e.g., コーヒー → コーヒー
- Lexical Normalization: e.g., *idk* → *I don’t know*

¹⁰Note that SuperBPE and BoundlessBPE (Section 3.1.3) avoid this pitfall by limiting multiword tokens to ones delimited at pretoken boundaries.

¹¹U+200D is the Unicode *zero-width joiner*, an invisible formatting character that joins adjacent characters into a composite form (e.g., emoji sequences).

Another commonly applied normalization step is Unicode canonicalization, which converts sequences of Unicode code points into a single, canonical form representing the same character. For example, the character é could be represented as e + ´ (U+0065 followed by U+0301), which is the canonical form under NF(K)D normalization, or as the single code point U+00E9, which is canonical under NF(K)C normalization (Davis et al., 2023).

Notably, these transformations can be lossy¹² and can alter the meaning of the text. Since they are performed prior to tokenization, the full process may not produce the same text after detokenization. Nevertheless, they have historically been used in NLP pipelines to minimize the surface-level variance of language and to allow for direct comparison of NLP systems by ensuring that they are processing the same canonical form of text (Koehn et al., 2007; Post, 2018).

Case folding can be combined with special markers to preserve casing information while reducing the orthographic variation of text in a pretoken. The surface form *The MVP* would be normalized and pretokenized as [↑the][↑mvp], where ↑ (U+21E7) and ⇧ (U+21EA) are “title” and “uppercase” casing symbols encoding the case of the surface form held in the pretoken. The Claude tokenizer utilizes this style of casing normalization (Lindsey et al., 2025a).

4.2 Whitespace Delimitation

A common constraint enforced by a pretokenizer is preventing tokens from containing internal whitespace. Even SentencePiece, which got its name for intentionally encouraging minimal pretokenization (Kudo & Richardson, 2018), has the `--split_by_whitespace` flag enabled by default.¹³ As a consequence, subword tokens are generally limited to only spanning a single “word” unit and do not cross whitespace-delimited word boundaries.

This behavior often comes with removing the space from the set of possible tokens, or replacing it with a special marker denoting a pretoken boundary, ensuring the original space can be recovered during detokenization. Preserving this reversibility also allows a midword occurrence of a string such as `embankment` to receive different treatment than the same string at the start of a word, `mentality`. Gow-Smith et al. (2022) find that forcing spaces to be in their own pretoken mitigates this issue and leads to better tokenization.

The decision of where and how to mark space boundaries is not obvious. Jacobs & Pinter (2022) examine the history of word-initial versus word-final space marking, and compare the intrinsic properties of token vocabularies trained over each pretokenizer family in English text. Space-marker placement also influences how to compute word surface form probabilities (see Section 2.2), with word-initial placement requiring an additional normalization step (Oh & Schuler, 2024; Pimentel & Meister, 2024). When using a word-initial prefix marker, some systems opt to also prepend the marker to the very first token of a sequence, even though no literal space precedes it, so that every word-initial token is treated uniformly regardless of its position in the input.

This whitespace-splitting behavior has been shown to greatly constrain the space from which vocabulary training algorithms can create long-tail tokens, resulting in very rare pretokens taking up vocabulary space that is not used by the downstream model, which results in lower vocabulary efficiency (Reddy et al., 2025) and unbalanced token distributions. SuperBPE (Liu et al., 2025) and BoundlessBPE (Schmidt et al., 2025) propose the notion of **supertokens** (see Section 3.1.3), where pretokens that have been selected by the vocabulary training algorithm to be single tokens can be merged into larger tokens containing whitespace, such as `New York` and `of the`.

However, whitespace-based delimitation is fundamentally ill-suited for scripts that do not use whitespace at all (e.g., Chinese or Japanese), use whitespace for purposes other than *word* delimitation (e.g., Vietnamese uses whitespace for *syllable* boundaries in addition to word boundaries), or use whitespace inconsistently (e.g., Korean uses spaces, but they are often applied inconsistently in colloquial writing). These orthographic fea-

¹²For Unicode, NFC/NFD are lossless from a surface-form perspective but not from a byte-level perspective, while NFKC/NFKD are lossy at both levels, since they can fold visually distinct characters together.

¹³<https://github.com/google/sentencepiece/blob/439fec3ade0a9f8a1e6bc3903597582587ec0f9b/doc/options.md>

tures render whitespace-based pretokenization ineffective without additional language-specific preprocessing (see Section 4.4).

4.3 Rule-Based Pretokenization

Pretokenizers are often constrained to use simple rules such as splitting on whitespace or on changes in Unicode script category. Following GPT-2 (Radford et al., 2019), many pretokenizers are based on a single regular expression, where each match defines a pretoken. Such patterns have grown more intricate over successive model generations. The GPT-2 pattern handles English contractions, groups letters and digits separately, and uses lookahead assertions to manage whitespace. GPT-4’s pretokenizer modifies this pattern to limit numerical tokens to at most three consecutive digits, and permits single punctuation marks to merge with the start of words, presumably to accommodate programming languages, allowing sequences like `get` to form single tokens. These increasingly complex regular expressions¹⁴ are not without drawbacks, and often contain unintended edge cases, such as contraction patterns inadvertently splitting true words, whitespace lookahead assertions producing unexpected token boundaries, and “unstable” split points where minor input changes cascade into entirely different tokenizations (Land, 2024a;b).

Several alternatives to regular expressions also exist. **SentencePiece** (Kudo & Richardson, 2018) restricts tokens during the training process using optional rules, such as excluding tokens that contain internal whitespace, multiple digits, or multiple Unicode scripts. **SCRIPT** tokenization (Land & Arnett, 2025) optionally uses a similar rule-based approach based on Unicode script as the basis for pretokens.

4.4 Morphological Pretokenization

For scripts without whitespace word boundaries (like Chinese, Japanese, or Thai) or languages with complex morphology (like Korean or Greenlandic), rule-based pretokenization may be insufficient. Without whitespace or other obvious delimiters, simple boundary rules have nothing to split on, and against complex morphology, language-agnostic heuristics fail to capture meaningful semantic units.

Morphological analyzers such as **MeCab** (Kudo, 2005), **Juman++** (Morita et al., 2015; Tolmachev et al., 2018), and **Sudachi** (Takaoka et al., 2018) for Japanese, **MADAMIRA** (Pasha et al., 2014) and **CAMEL Tools** (Obeid et al., 2020) for Arabic, and **Morfessor** (Creutz & Lagus, 2002) for a variety of languages, can segment text into linguistically meaningful units before applying subword tokenization.

The example below demonstrates the use of **MeCab** as a morphological segmenter for pretokenizing 私は日本語を学ぶ。 (“I study Japanese.”). The pretoken boundaries help prevent commonly co-occurring but semantically distinct text spans from being fused into a single token.

Example						
私は日本語を学ぶ。	[私]	[は]	[日本語]	[を]	[学ぶ]	[。]
I study Japanese.	I	(TOPIC)	Japanese	(OBJECT)	study	.

Without morphological pretokenization, since 語 (language/word) and を (the object particle) appear so often together—as in 英語を (English language), 中国語を (Chinese language), 物語を (story), etc.—the downstream tokenizer may form a 語を token. With morphological pretokenization, the word and particle are separated into different pretokens, which blocks the 語を token from forming so that the tokenizer can produce more morphologically aligned token sequences like 日本 語 を.

While foundational subword tokenization methods are purely data-driven, injecting morphological information into the pipeline can improve downstream task performance while preserving the sequence compression and controlled vocabulary benefits of subword tokenization. Park et al. (2020) explore several pretokenization methods, including morphological pretokenization with **MeCab-ko**¹⁵ combined with BPE for Korean language modeling. They find that morphologically aware subword tokenization outperforms both pure

¹⁴See, e.g., exact annotated formulations in Appendix A of Schmidt et al. (2025).

¹⁵<https://bitbucket.org/eunjeon/mecab-ko>, a Korean extension of the MeCab morphological segmenter.

morphological tokenization (without a subword tokenizer) and subword tokenization (without morphological pretokenization) across many translation and NLU tasks. Vemula et al. (2025a) report a related effect for Telugu, where morphological pre-segmentation significantly improves BPE, but find that the choice of tokenizer algorithm matters more: Unigram outperforms BPE across most settings and does not benefit from morphological pretokenization.

In addition to the aforementioned inconsistent downstream gains, morphological pretokenization comes with downsides related to complexity and data requirements. Morphological segmenters require knowledge of the language being processed and vary widely in quality across languages. Often, morphological segmenters are learned models themselves and come with the same kinds of challenges that tokenization is meant to solve. Many segmenters rely on large, handcrafted dictionaries that make them brittle in contemporary use. Such lexicons also age and lose coverage of recent named entities, prompting continually updated resources like NEologd (Toshinori Sato & Okumura, 2017) for Japanese.

More broadly, the benefits of morphological alignment in pretokenization and tokenization are not unambiguous: gains tend to be inconsistent across languages and tasks, and the choice of tokenization algorithm can matter more than morphological pretokenization itself. These questions are discussed in greater detail in Section 5.4.1.

4.5 Pretokenizing Numbers

Proper treatment of digit sequences in pretokenization is an open question, with several studies investigating how numerical tokenization affects arithmetic reasoning and numerical understanding (Nogueira et al., 2021; Thawani et al., 2021; Garreth Lee & Wolf, 2024; Singh & Strouse, 2024; Yang et al., 2025b; Reddy et al., 2026). Some tokenizers, like the GPT-2 tokenizer, do not treat numbers differently from regular text. As a result, they will often tokenize digit sequences into irregular token sequences. For example, the string `8675309` would form the pretoken `[8675309]` which could be tokenized into a token sequence like `[8][67][5][30][9]` (Radford et al., 2019).¹⁶ On the other hand, using a specialized digit pretokenizer can help guarantee consistent digit grouping with desirable properties (Yang et al., 2025b). The Llama 2 tokenizer treats each digit as a separate pretoken, so each digit is guaranteed to be its own token in the final tokenized text (Touvron et al., 2023). This can improve numerical performance since the model has direct, fine-grained access to each digit. GPT-3.5+ and Llama 3+’s pretokenizers greedily segment digit sequences into groups of three from left to right (e.g., `8675309` $\rightarrow$ `[867][530][9]`) (Brown et al., 2020; Dubey et al., 2024; OpenAI et al., 2024). However, not all three-digit sequences are present as tokens in the vocabulary, so not all numerical pretokens will be represented as a single token (Singh & Strouse, 2024). The left-to-right greedy pretokenization process does not align with traditional mathematical notation, which groups digits from right to left, and can leave awkward, trailing digit pretokens. In the previous example, the trailing `[9]` is stranded as its own pretoken. Instead, right-to-left digit pretokenization could be used (e.g., `8675309` $\rightarrow$ `[8][675][309]`) to align better with standard digit grouping.

Non-arithmetic digit sequences may benefit from alternative grouping strategies. For example, a year like `2026` may be better kept as a single `[2026]` pretoken so it can map to a single `[2026]` token, rather than being split into `[202][6]` as in the GPT-3 and Llama 3 tokenizers.

Other techniques for processing numbers such as specialized embeddings and normalization algorithms are discussed in more detail in Section 14.1.

¹⁶The GPT-2 tokenizer actually produces the token sequence `[86][75][309]`. This example was intended to illustrate that multi-digit tokens were formed without a consistent pattern when no pretokenization is done.

4.6 Questions

Pretokenization takes many forms, from very minimal (e.g., the default `SentencePiece` pretokenizer, which only replaces spaces with a special space token and splits on whitespace, numbers, and differing Unicode scripts) to very complex (e.g., morphological pretokenizers or the GPT pretokenization schemes which use complex regular expressions). On one hand, complicated pretokenizers are intended to add domain-specific inductive bias to help models learn better, but on the other hand, having minimal pretokenization brings language model training closer to fully end-to-end. All else held equal, which is better for language modeling?

Pretokenizers are often script or language-specific, but language models are increasingly multilingual. Do modern pretokenizers introduce inequities in performance across languages, specifically those that are not similar to the primary language that the pretokenizer was designed for? Dynamically applying a pretokenizer based on the script is lossy and cumbersome. Are simple, language-agnostic pretokenizers better from an implementation, auditing, and language model performance perspective compared to even high-quality, language-specific pretokenizers?

Individual digit pretokenizers improve model performance on simple mathematical tasks by giving the model direct visibility of the digits involved. This is analogous to improving language model performance on simple text queries (e.g., “how many vowels are in this word?”) by using a character-level model or by explicitly breaking up a word’s tokenization into individual characters. What other tasks could benefit from similar manipulations, what other task-specific pretokenizers could we use, and how should this be integrated into training and inference? If we don’t include the task-specific pretokenizer during training, we could have a train-test mismatch during inference, but, on the other hand, language models have been shown to be able to process alternative (pre)tokenizations without major losses in quality (Section 8; Zheng et al., 2025a), and this form of task-specific pretokenization could be seen as an inference-time use of non-canonical tokenizations.

	Tamil	Sinhala	Devanagari
Grapheme	ஂ	ස්	वा
Unicode characters	ஂ + ஃ	ස් + ඌ	व + ा
Unicode code points	U+0BA9, U+0BCD	U+0DC3, U+0DCA	U+0935, U+093E
UTF-8 bytes	e0 ae a9 e0 af 8d	e0 b7 83 e0 b7 8a	e0 a4 b5 e0 a4 be

Table 5.1 – Text representations in digital systems. Adapted from [Velayuthan & Sarveswaran \(2025\)](#).

5 Language- and Script-Specific Considerations

Tokenization and other systems for rendering and processing text have been designed primarily for English and other high-resource European languages. But for languages which are typologically distant from English, tokenization interacts with structural linguistic differences. In this section, we discuss some known ways in which these crosslinguistic differences impact tokenization, focusing on the effect of writing system and morphological typology, as well as techniques to improve tokenization by incorporating language and script information.

5.1 Writing System Typology

Writing systems differ in what information each character encodes (**orthographic granularity**), in direction of writing (left-to-right, right-to-left, or top-bottom), in use of whitespace (both whether whitespace is used and to what extent), in use of diacritics and their goal (such as phoneme identification, vowel marking, and semantic disambiguation), in phonetic transparency (or *orthographic depth*, i.e., how directly graphemes correspond to pronunciation), and in degree of standardization. In this section, we focus on orthographic granularity and use of whitespace.

Orthographic Granularity and Information Density Different writing systems encode linguistic information in quite different ways. One typological difference, orthographic granularity, has to do with what kind of information is encoded by the individual units of a writing system, i.e., *graphemes*, or *characters*. In some writing systems, each character represents an individual sound, while in others a character may represent a syllable or even some semantic content directly. As a result, different systems require different numbers of graphemes to encode the same amount of information ([Arnett et al., 2024a](#)). This variation in information density has direct consequences for tokenization. Languages using writing systems with higher information density per character, such as logographic systems, can convey more meaning in fewer characters and thus may be encoded more efficiently in tokens. Conversely, languages with lower information density require more characters to express the same content, which could lead to longer token sequences and reduced efficiency in fixed-length context windows ([Shani et al., 2026](#)).

Graphemes A **grapheme** is the smallest unit of a writing system that distinguishes words; roughly, the analogue of a phoneme in spoken language. Each grapheme may be realized by one or more **glyphs**, the concrete visual representation with which it is written. The relationship between graphemes and the underlying digital representation of text is not uniform across writing systems. Unicode, the global standard character set, assigns a numeric **code point** to each character (e.g., U+0055 is the capital letter “U”). Graphemes in scripts such as Tamil, Sinhala, and Devanagari are often composed of multiple Unicode code points ([Velayuthan & Sarveswaran, 2025](#)), because a single grapheme such as a consonant-vowel unit in Devanagari can be built from a base character and one or more combining marks, each assigned its own code point (see [Table 5.1](#)). Some Latin graphemes such as *ÿ*, *ő*, *ï*, and *ř* can be encoded in several ways: either as a composition of a base character and a combining diacritic, or as a single precomposed code point (e.g., *ÿ* as U+0177). In general, a single grapheme may spans more than one code point, for example with emoji or scripts that require combining marks in Unicode. For basic, undiacritized Latin letters, the correspondence is one-to-one.

5.1.1 Encoding Writing Systems

Representing text in digital form requires a convention for converting between units of binary code (e.g., bytes) and units of text (e.g., characters). **Character encodings** such as ASCII, UTF-8, UTF-16, UTF-32, or GB 18030 map between Unicode code points and binary representations. In practice, UTF-8 is the standard encoding, used by approximately 90% of websites in the Common Crawl (Common Crawl, 2026), and is the *de facto* encoding for LLMs. This dominance is reinforced by the underlying layers of the LLM tech stack: Python and Rust, two programming languages prominently used for tokenization, both use UTF-8 by default.

UTF-8 is a variable-length encoding: code points corresponding to basic **ASCII** characters (i.e., unaccented Latin letters, digits, and common punctuation) are encoded as a single byte, a deliberate choice to ensure backward compatibility, and code points outside this range require two, three, or four bytes. Because code points in non-Latin scripts fall outside the ASCII range, they require more bytes, and graphemes composed of multiple such code points require still more. This manifests as systematic unfairness between languages in tokenizers (Ahia et al., 2023; Petrov et al., 2023), especially in byte-level models. For certain scripts, such discrepancies are compounded by graphemes being split across multiple code points (Arnett et al., 2024a), though the relationship between these forms of **byte premiums** and tokenizer features is not fully understood (Arnett et al., 2025).

Several studies have investigated alternatives to UTF-8 for language modeling. Wan (2022) found that simply changing the encoding of Russian can significantly lower perplexity. Limisiewicz et al. (2024) repurpose a subset of UTF-8 bytes to encode morphemes, substantially reducing crosslinguistic performance differences, and Land & Arnett (2025) propose the **SCRIPT** encoding scheme, which yields equal encoding length for all characters regardless of script. On the other hand, Moon et al. (2025) propose an alternative, sub-byte-level representation of text that improves sequence compression, especially on CJK and rare Unicode characters. Moryossef et al. (2025) propose a minimalist byte-level tokenizer that directly uses the extended ASCII range (i.e., exactly 256 tokens), encoding text as its ASCII IDs and adding biases to the token embeddings to reflect ASCII regularities like casing and script.

5.1.2 Effects of Pretokenization on Various Scripts

Pretokenization, discussed at length in Section 4, is an early optional step in the tokenization process, usually implemented as simple regular expressions splitting mostly on whitespace, although more sophisticated rule-based systems exist, such as **Moses** (Koehn et al., 2007), which can be repurposed from its original machine-translation tokenization goal.

While the intention behind regex pretokenizers is to split along punctuation, newlines, whitespace, and sometimes English contractions (e.g., the GPT-2 regex), they may have unintended consequences for other languages and scripts. For example, most regex pretokenizers split South Asian scripts down to the sub-grapheme level, due to how diacritics are recognized and how these scripts are encoded in UTF-8 (Velayuthan & Sarveswaran, 2025). The whitespace aspect of these pretokenizers has uneven effects which may prove extreme for languages whose writing systems do not separate words with whitespace, such as Chinese, where long, multi-word strings have been observed to be learned as tokens (Zhang et al., 2025a). For such languages, morphological analyzers historically filled this role in pre-LM tokenization (Chinese: Tseng & Chen, 2002; Japanese: Kudo, 2005; Kurohashi & Kawahara, 2007; Neubig & Mori, 2010; Neubig et al., 2011; Morita et al., 2015; Yoshinaga, 2023; Akabe et al., 2024; Korean: Kakao Corp., 2018; Eunjeon Project, 2025), and are now sometimes used as pretokenizers (see Section 5.4.1).

At the other extreme, languages that use whitespace at a finer granularity than the word level, such as Samoan or Scottish Gaelic, are also strongly affected: whitespace pretokenization limits the compression a tokenizer can achieve (Reddy et al., 2025; Arnett et al., 2025). Superword tokenization (see Section 3.1.3) offers a way to combine subword space-delimited segments into multi-pretoken tokens, mitigating this problem. **IndicSuperTokenizer** (Rana et al., 2025) combines subword and multi-word tokenization for Indic languages based on SuperBPE under a sentence-level constraint: merges are allowed within sentences but not across sentence boundaries. The tokens produced are better aligned linguistically and achieve better compression compared to standard multilingual LLM tokenizers.

5.2 Script-Specific Tokenizers

Most modern tokenizers are not explicitly designed around individual scripts, especially in multilingual settings, and instead rely on alphabets of bytes or Unicode code points without representing script-specific structure. This discards two distinct kinds of orthographic information: a single encoded character may contain individually useful subcharacter components and a single grapheme or orthographic syllable may span several code points. Both mismatches can produce inefficient or orthographically incoherent segmentations, especially for non-Latin scripts (Velayuthan & Sarveswaran, 2025; Dwivedi & Gopalan, 2026).

Vocabulary Parameter Allocation As discussed in Section 5.1, writing systems differ in orthographic granularity: logographic and syllabic scripts encode more linguistic information per character than alphabetic ones, which in turn means their base character sets are dramatically larger. For languages with a large base character set like Chinese, Japanese, or Korean, the number of characters that must be included in the tokenizer’s vocabulary in order to mitigate OOV issues is prohibitively large. A trade-off must be made between the size of the vocabulary and the coverage of the tokenizer, despite this being exactly what subword tokenization was designed to solve.

CJK languages have enormous base character sets: Korean has 11,172 *syllables* (or *eum-jeol*/음절) in its syllabic alphabet; Japanese uses three scripts including kanji (the Chinese characters used in Japanese), of which 2,136 are classified as “regular use”,¹⁷ and many thousands more exist in modern and archaic use; and Chinese, which has tens of thousands of cataloged characters. Using Korean as a representative example, including all 11,172 Korean syllables in a tokenizer would account for a substantial percentage of even large, modern tokenizers’ vocabulary allocation (see Table 2.1). In Gemma 3, for example, 4% of the 256k vocabulary would be devoted just to the set of Korean syllables (Gemma Team et al., 2025). This is suboptimal not only from a vocabulary allocation perspective, but also from a modeling perspective when considering that only about 1,200 Korean syllables account for 99.9% of Korean text on Wikipedia,¹⁸ and the rest primarily exist as combinatorial artifacts of the writing system and do not appear frequently (if at all) in any training corpus. These rare tokens receive few gradient updates, contribute to training instability, and do not learn robust embeddings, so even if they are observed during inference, they will harm performance (Land & Bartolo, 2024). Finally, since so much of the vocabulary would be taken up by the base characters, very few longer tokens could be formed. Table 5.2 shows the vocabulary allocation for CJK over several multilingual tokenizers.

Having worse token allocation for a script also increases the cost of using LLMs for languages that use that script by increasing the number of tokens required to encode an input text, which acts as a *token tax* on users based on their language of choice. We cover this more in Section 11.3.

Ignored Orthographic Information Being script-agnostic, tokenizers fail to capture semantically meaningful aspects of the script that could otherwise help with modeling. Many writing systems, including the logographic Chinese characters and the phonetic Korean syllables, encode semantic or phonetic information directly into the character, often in regular forms shared across many characters. Chinese characters used in Chinese and Japanese contain *radicals* (*bùshǒu*/部首 [Mandarin Chinese]; *bushu*/部首 [Japanese]), and Korean Hangul is a compositional writing system where characters are made up of visually distinct, phonetic subcharacters (*jamo*/자모). This information is typically not exposed during the tokenization process, as the discrete character-to-token mapping does not preserve any visual or semantic information.

Diacritic marks are another case of incomplete use of orthographic information. Gorman & Pinter (2025) point out that many normalization and preprocessing practices either fail to canonicalize diacritic information or strip diacritics altogether (see Section 4.1), harming downstream performance on various tasks but also influencing tokenizer output due to the inconsistencies created between corpora used for training a vocabulary, pretraining a model, and performing downstream tasks. An extensive analysis of the case of Ara-

¹⁷The *Jōyō Kanji* (常用漢字), a set of characters designated by the Japanese government as a baseline for literacy.

¹⁸Retrieved from <https://huggingface.co/datasets/wikimedia/wikipedia/viewer/20231101.ko>. All 11,172 syllables appear at least once, due to the article about the Korean Unicode Syllable Range. Otherwise, 1,238 syllables account for 99.9% of the observed Korean text and 1,714 syllables make up 99.99%.

Tokenizer	$ \mathcal{V} $	Chinese		Japanese [†]		Korean		CJK	
		$\ell=1$	$\ell \geq 2$	$\ell=1$	$\ell \geq 2$	$\ell=1$	$\ell \geq 2$	Total	%Total
Llama 2	31,995	700	0	138	0	111	0	949	3.0%
Llama 3	127,991	2,725	1,408	281	808	1,078	1,176	7,476	5.8%
GPT-2	50,255	42	4	103	30	1	1	181	0.4%
GPT-OSS	199,988	3,076	4,013	230	556	1,129	1,220	10,224	5.1%
Gemma 3 & Gemma 4	255,810	10,496	8,760	356	4,594	2,400	2,278	28,884	11.3%
Multilingual BERT	119,441	13,550	0	338	985	2,376	802	18,051	15.1%
Qwen-3.5	248,034	5,426	48,003	283	5,197	1,703	5,104	65,716	26.5%
DeepSeek-v4	127,997	5,255	29,938	154	736	706	416	37,205	29.1%
Mistral-3.5	130,068	1,727	1,650	246	1,382	1,326	3,084	9,415	7.2%

Table 5.2 – CJK token statistics across tokenizer vocabularies with the breakdown by token length ℓ . Note that the  and  form of a character are both counted as $\ell = 1$. For byte-level tokenizers, only tokens with valid UTF-8 prefixes are counted (they may contain incomplete *trailing* UTF-8 sequences). [†]The Japanese column contain tokens with Kana or mixed Kana and Kanji. Tokens with only Chinese characters are allocated to the Chinese column since we cannot easily distinguish between Japanese and Chinese usage.

bic, a language using a heavily diacritized writing system, has shown significant degradation in tokenization quality owing to the inconsistent treatment of diacritics (Inoue et al., 2026).

Byte-Level Fallback The vocabulary allocation problem can be partially mitigated via **byte-level fallback**: rather than including every character in the base vocabulary, only common characters are kept and the rest are represented as their UTF-8 byte sequence. This eliminates OOVs and keeps the vocabulary small. However, this practice inherits the byte-premium issues discussed in Section 5.1.1: non-Latin scripts use multi-byte UTF-8 encodings, so falling back to bytes inflates sequence lengths for languages whose characters were dropped, which continues to tax them. These languages require more tokens, which increases inference time and cost and diminishes performance and usability (Ahia et al., 2023; Petrov et al., 2023). Since fallback is usually triggered by corpus-level frequency thresholds, the characters represented as bytes are rare by definition. Consequently, although the individual byte tokens may be frequent, the specific byte sequences corresponding to these rare characters occur infrequently, making it difficult for the model to learn reliable representations. Breaking code points into constituent bytes also introduces the possibility of tokens forming across code point barriers but not fully encompassing one or both of the complete code points. One mitigation technique for this problem in BPE-style vocabulary training involves pretokenizing at composite character boundaries and applying the superword scheme that disallows merging cross-boundary tokens unless they already cover entire pretokens (Schmidt et al., 2026).

Byte fallback also fails to address the ignored orthographic information described above: byte sequences are determined by Unicode code points, which are not tied to semantic or orthographic structure. As a result, similar characters may have very different UTF-8 byte sequences. Figure 5.1 shows examples of Chinese and Korean characters that share orthographic features such as structural components or jamo, but the overlap is not reflected in their byte encodings. This limits transfer across rare characters because the byte mapping is unrelated to the relevant orthographic structure, and so semantically meaningful information learned about one character does not easily generalize to another with similar structure.

5.2.1 Using Script Information

Script-aware tokenizers, unlike byte-level fallbacks, break down characters into constituent parts that preserve orthographic information. Doing so reduces the number of unique tokens required to encode the base character set, which both mitigates the parameter count issue and preserves information from the original glyphs since the encodings now operate at sub-glyph granularity.

Work on Chinese has primarily exploited structural decomposition, where each character is broken into sub-character units, ranging from dictionary radicals (部首) to structural components (部件) to strokes (笔画) (Shi

	<i>share 人 (亻)</i>				<i>share ㄹ and ㄷ</i>		
Hanzi/Kanji	人	少	少	Hangul	럿	럿	럿
Components	人	人, 小, 丿	小, 丿	Jamo	ㄹ, ㅏ, ㄷ	ㄹ, ㅏ, ㄷ	ㄹ, ㅏ, ㅍ
		<i>share 小 and 丿</i>			<i>share ㄹ and ㅏ</i>		
UTF-8	e4 ba ba	e4 bb af	e5 b0 91	UTF-8	eb a0 b7	eb 9f bf	eb a0 80

Figure 5.1 – Kanji/Hanzi and component information (left) and Hangul and jamo information (right). Note that even though the Kanji contain similar components and the Hangul syllables contain overlapping jamo, their UTF-8 byte sequences do not reflect this.

et al., 2015; Saunders et al., 2020; Wang et al., 2020b; 2022; Si et al., 2023; Xiang & Liu, 2025). A parallel line of work draws on Chinese input methods—the keyboard-input encodings used by speakers to type Chinese on non-native (e.g., Latin-script) keyboards—to derive subcharacter representation (Zhang et al., 2019). These input-method encodings often combine phonetic information (such as pinyin or Bopomofo) with shape-based disambiguation, providing a complementary signal to orthographic decomposition.

For Japanese, because kanji are shared with Chinese, radical-based approaches transfer naturally, but require the additional consideration of interaction between kanji and the two phonetic scripts (hiragana and katakana), which can appear within a token (Nguyen et al., 2017; Saunders et al., 2020).

Hangul, the Korean script, has a compositional structure where each syllable is built from two or three *jamo* (initial consonant, vowel, and optional final consonant) drawn from a small alphabet. This makes subcharacter decomposition natural and avoids byte-level fallbacks while requiring few parameters for the base alphabet. Several works have decomposed syllables into jamo as a preprocessing step and found that the finer-grained encoding improves model performance and efficiency while decreasing the required parameters (Stratos, 2017b; Moon & Okazaki, 2020a; Park et al., 2020; Kim et al., 2021; Cognetta et al., 2023; Lee et al., 2025a).

Injecting Orthographic Information Directly Into Embeddings The methods discussed so far use orthographic information to design better tokenization algorithms, with the goal of allocating parameters more efficiently and encoding subcharacter structure into the tokens themselves. However, once a longer token is formed, its subcharacter components are obscured and the model must recover them from data. Alternatively, one can inject orthographic information directly into the token *embeddings*, in conjunction with or independently of the tokenization-level approaches described above. We discuss using script information directly as input in Section 10, and injecting script information into embeddings in Section 14.2.2.

5.3 Transliterating to a Different Script

Rather than using a secondary script that is derived from the components of the primary script, like jamo subcharacters for Hangul syllables or strokes and radicals for Chinese characters, we can abandon the primary script altogether and use a different script for a **transliterated encoding**. Pinyin for Chinese and romaji for Japanese are well-known romanization schemes, where the native script is transliterated phonetically into the Latin script. For tokenization, this avoids the costly encoding of the native script’s base character set and eliminates byte-level fallbacks.

Such transliterated encodings can also take advantage of **crosslingual transfer**, where languages that share features can mutually benefit after being converted to the same script, since they now share representations at the token level rather than just in a model’s latent representation. This has been applied to Indic languages, where it was found that transliteration benefited lower-resource languages by enabling them to more easily

Language	Script	UTF-8	
		Characters	Bytes
English	Latin (alphabetic)	12,321	12,333
<i>All human beings are born free and equal in dignity and rights.</i>			
German	Latin (alphabetic)	13,623	13,799
<i>Alle Menschen sind frei und gleich an Würde und Rechten geboren.</i>			
Czech	Latin (alphabetic)	11,518	12,829
<i>Všichni lidé se rodí svobodní a sobě rovni ve své důstojnosti a právech.</i>			
Russian	Cyrillic (alphabetic)	13,493	23,416
<i>Все люди рождаются свободными и равными в своем достоинстве и правах.</i>			
Chinese	Han (logographic)	4,676	10,256
人人生而自由，在尊严和权利上一律平等。			
Korean	Hangul (syllabic)	6,399	13,088
모든 인간은 태어날 때부터 자유로우며 그 존엄과 권리에 있어 평등하다.			
Hindi	Devanagari (syllabic)	13,165	31,565
सभी मनुष्य स्वतंत्र और समान सम्मान और अधिकारों के साथ जन्मे हैं।			
Hebrew	Hebrew (abjad)	8,924	14,709
כל בני האדם נולדו חופשיים ושווים בכבודם ובזכויותיהם.			
Amharic	Ethiopic (syllabic)	6,393	17,223
ሁሉም ሰዎች ነፃ እና በክብርና መብቶቻቸው ላይ እኩል የተወለዱ ናቸው።			

Table 5.3 – The length of Universal Declaration of Human Rights in UTF-8 characters and bytes across languages and writing systems, with a sample of the first sentence in Article 1.

leverage the information from the higher-resource languages, improving compression (Moosa et al., 2023; Husain et al., 2024). The **uroman** tool (Hermjakob et al., 2018) allows for transliteration of any script supported by Unicode into the Latin script, and has been shown to be helpful in improving language model quality across a variety of scripts, language families, and tasks (Purkayastha et al., 2023; Xhelili et al., 2024; Chari et al., 2025; Ebing et al., 2026; Jung et al., 2026; Zhang et al., 2026a). The International Phonetic Alphabet (IPA) has also been used to represent different languages and scripts in a unified format (Astrach & Pinter, 2025; Miletić et al., 2026).

The primary downside of transliteration is that it may be lossy, and the original script cannot always be recovered perfectly. This makes transliteration better suited for encoder-only architectures, where the downstream tasks often do not require generating native-script text. However, transliteration has found use in generative tasks as well (Husain et al., 2024). One could use a transliteration model to recover the native script, but this is also lossy and would suffer from the same native-script issues as before. Alternatively, for some scripts, lossless transliteration schemes do exist, like **IndicNLP** (Kunchukuttan, 2020). They may still require disambiguation mechanisms such as large character sets or long codewords, reducing their benefit compared to native-script encodings.

5.4 Morphological Typology

Morphology is the study of word structure and the combination of **morphemes**, the smallest units that convey lexical or grammatical information, into words. Morphology is a key typological dimension with respect to tokenization, since different languages exhibit distinct morphological strategies (Haspelmath, 2009; Wilson & Mihalicek, 2011; Haspelmath & Sims, 2013). Crucially, they differ in how many morphemes they encode in each word and in how many semantic features each morpheme encodes. Languages can be fuzzily classified into several morphological types based on how they form words: *isolating* languages like

Chinese, Vietnamese, Thai, and Khmer use minimal morphology; *agglutinative* languages such as Turkish, Finnish, Japanese, Korean, Hungarian, Swahili, Tamil, and Basque combine morphemes with clear boundaries; *fusional* languages (also called *flective* or *inflecting* languages), including Spanish, Russian, German, French, Latin, Greek, Hindi, and Sanskrit, fuse multiple grammatical features into single affixes; *polysynthetic* languages like Mohawk, Inuktitut, and many indigenous languages of the Americas and Australia form highly complex words that can fully contain event structures of entire sentences. While these fuzzy classes are helpful in a heuristic sense, languages actually fall along continuous spectra of morphological complexity, and the boundaries between categories are not clearly defined. Individual languages often exhibit characteristics of multiple types, and classifications can be contentious. For example, English, while historically a fusional language, has lost much of its inflectional morphology and now exhibits many isolating characteristics (Haspelmath, 2001).

Morphological variance has direct implications for vocabulary size. Languages with richer morphology produce significantly more word forms, resulting in larger vocabularies with longer tails in frequency distributions. Arnett et al. (2025) find that given comparable tokenizer training, some languages need larger vocabulary sizes to achieve the same compression rate across tokenization algorithms and vocabulary sizes. Still, they do not find a direct connection between vocabulary size and morphological typology.

Figure 5.2 illustrates these differences in word frequency distributions and type-token ratios across languages with varying morphological typologies. While fusional and agglutinative languages show longer tails in their word frequency distributions, these distributions remain Zipfian. Some evidence suggests that this affects subword tokenization, leading to shorter subwords and higher token premiums (Gerz et al., 2018; Cotterell et al., 2019). Conversely, some have argued that apparent effects of morphological typology can be accounted for by differences in dataset size measurement (Arnett & Bergen, 2025).

Morphologically rich languages exhibit additional challenges. For agglutinative languages like Telugu, naive Unigram tokenizers outperform alternative methods across most settings, though hybrid approaches that incorporate morphological segmentation can substantially improve performance within the BPE framework (Fujii et al., 2023; Vemula et al., 2025b). These findings highlight that tokenization does not provide a uniform representational basis across languages; instead, it embeds structural differences that may advantage some languages while disadvantaging others. Finally, while Park et al. (2021) find that aligning tokenizers morphologically leads to lower surprisal in morphologically rich languages, follow-up experiments (Arnett & Bergen, 2025) across a broader set of language families find low correlation. In terms of orthography, languages that share a script with a high-resource language tend to benefit from multilingual training as the shared script tends to generate more frequent tokens (Shani et al., 2026).

5.4.1 Morphologically Aware Tokenizers

In order to account for these and other shortcomings arising from relationships between morphological typology and tokenization, several **morphologically aware tokenization** methods have been developed. Semitic languages like Arabic, Hebrew, Amharic, Tigrinya, and Aramaic present a unique challenge through their use of non-concatenative morphology. In these languages, words are formed mostly around consonantal roots with vowel patterns interleaved between them. This structure is impossible to accommodate in any conventional substring-based tokenizer, as they all represent tokens as contiguous spans. As a result, the majority of inflected forms of a given root can never share a token, nor can markers of a given morphological inflection on different roots. **SPLINTER** (Gazit et al., 2025) is a statistically trained text preprocessor which re-linearizes text in a source-preserving manner, meaning that characters are shifted to the ends of pretokens while maintaining knowledge of their original locations. This allows non-concatenative input to be represented in a concatenative form where the root is separated from template material, which is then used as input for any tokenizer throughout its lifetime, from training the tokenizer to later inference scenarios. Splintering requires an additional preprocessing module but does not affect the algorithms and procedures of tokenization and detokenization themselves, and improves performance on both intrinsic metrics and downstream applications.

Another manner of incorporating morphological information into tokenizers is through direct morphological supervision. Morphological segmentation is an established task whose annotations can provide linguistically

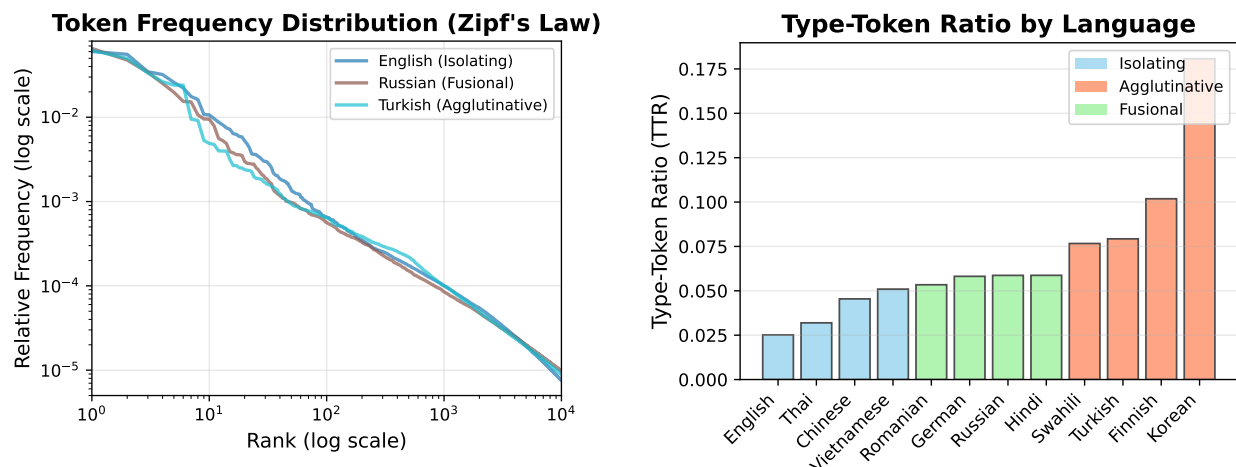


Figure 5.2 – Left: Comparison of word frequency distributions between English (here considered an isolating language, however, with fusional elements), Hindi (a fusional language), and Turkish (an agglutinative language). Right: Type-token ratio (the number of unique words over the total number of words) comparison between four languages with different morphology.

motivated boundaries or supervision for tokenization. Supervised segmenters require annotated data that is not available in many languages, but they achieve very high scores on human-annotated benchmarks (see Section 7.2.2): systems have achieved high F1 scores ($\sim 97\%$) on some supervised SIGMORPHON morphological-segmentation settings (Batsuren et al., 2022). Other datasets include the *Kyoto Corpus* (Kawahara et al., 2002) for Japanese and the *Sejong Corpus* and *OpenKorPOS* (Moon et al., 2022) for Korean.

Morfessor (Smit et al., 2014) is a widely used unsupervised tool that learns to segment words into subword units it calls **morphs** from raw text. It operates with two conflicting objectives: minimizing lexicon size, which favors fewer, longer morphs, while also minimizing corpus description length, which favors shorter, reusable morphs. A common misconception is that Morfessor’s outputs are definitionally morphemes; they are in fact subwords like those of any other tokenizer. While using Morfessor directly for tokenization in neural machine translation has shown mixed results, often underperforming compared to BPE (Macháček et al., 2018), it has been incorporated as a component in tokenization pipelines. MYTE uses it to derive morphologically motivated units (Limisiewicz et al., 2024), and recent work explores using it for pretokenization (Libovický & Helcl, 2024).

Building on resources such as the SIGMORPHON shared tasks, a growing body of work has explored morphology-aware tokenization (Hofmann et al., 2021a; Jabbar, 2024; García et al., 2025; Hu et al., 2025; Asgari et al., 2026, *inter alia*), typically relying on existing morphological segmentation datasets or applying external analyzers (Smit et al., 2014; Darwish & Mubarak, 2016) to obtain morpheme boundaries. **Morph-Piece** (Jabbar, 2024) first segments text using a morpheme lookup table before applying BPE. Similarly, **MorphBPE** (Asgari et al., 2026) constrains BPE merges to respect morpheme boundaries. These approaches are usually applied to typologically diverse languages such as English, Russian, Hungarian, and Arabic. Instead of using existing morphology-related techniques, the **Few Longest Token Approximation (FLOTA)** algorithm (see Section 3.6) takes advantage of pretrained tokenizers and recursively matches the longest token possible in a word, which often coincides with gradual morphological decomposition (Hofmann et al., 2022). Empirically, morphology-aware or morphologically better-aligned tokenization often improves downstream performance across languages (e.g., Bostrom & Durrett, 2020; Hofmann et al., 2020; García et al., 2025), although some studies report limited or inconsistent gains (Arnett et al., 2024b; Arnett & Bergen, 2025).

5.5 Syntactic Considerations

Beyond morphological awareness, other linguistic features like phonological syntax are also present as considerations for tokenizer performance. One example is *clitics*, linguistic units that are syntactically independent

like words, but phonologically (and thus often orthographically) dependent on a host word to which they attach, functioning as an intermediate category between free words and affixes. From a linguistic perspective, clitics belong to syntax rather than morphology, but the tokenization issues they create are similar to those posed by affixes. Examples include French *l'ami* (the friend), Italian *d'accordo* (agreed), and Spanish *dámelo* (give it to me), where the pronominal clitics *me* and *lo* attach to the imperative verb *da*. Mielke et al. (2021) provide a discussion of this case as **multi-word tokens** (not to be confused with superword tokens, see Section 3.1.3). In the age of rule-based language-specific (pre-)tokenizers, clitics were typically handled through specific rules designed for each language such as Arabic (Attia, 2007; Diab et al., 2007) or certain European languages (Moses, Koehn et al., 2007). Clitics are still treated separately in some low-resource scenarios such as Kurdish (Ahmadi, 2020) and Tunisian Arabic (Mekki et al., 2023). Additionally, Land & Arnett (2025) point out that pretokenizing along English contractions may split words in other languages at unnatural points, unnecessarily reducing compression.

5.6 Questions

Transliterated encodings (e.g., IPA, Pinyin) work well for *input*, but we may eventually want to produce the original script as output. Generating transliterated text and converting it back to the native script is lossy, and ingesting transliterated text but producing native-script text has similar parameter count and data sparsity issues as encoding native-script text. What is the right way to handle it? Is there a general framework for disambiguating transliterated output that isn't (parameter-)expensive?

Transliterated encodings have an issue (similar to byte-level encodings) where invalid sequences can be generated (transliteration sequences that do not map to any native-script character sequence). How can we design decoding modules to prevent this? Constrained generation is one choice. Korean solves this by having a three-step decode (generate subcharacters as initial consonant, vowel, final consonant triplets, with each component from distinct alphabets/output heads), but it is special in that the mapping is “dense” (every possible triplet maps to a syllable, and vice versa) whereas most transliteration systems are “sparse” (most transliteration script sequences don't map to native-script sequences). What other transliterated scripts have similar dense mappings or other ways to sidestep the invalid sequence problem?

6 Multilingual Tokenization

In the previous section, we discussed the impact of linguistic and orthographic features of an individual language on tokenization. In *multilingual* settings, different languages and scripts must be processed by the same tokenizer, and there is no guarantee that a design decision made to benefit one language will benefit the others. The tokenization algorithm itself is an example of this tradeoff. The relative performance of BPE, Unigram, and other algorithms varies by language, corpus characteristics, vocabulary size, and evaluation setting. For example, empirical observations suggest that Germanic languages such as German and English tend to perform better with BPE, whereas Romance languages such as French and Spanish benefit more from Unigram (Ali et al., 2024), as do morphologically complex languages (Bostrom & Durrett, 2020; Toraman et al., 2023; Vemula et al., 2025b; Držík & Kapusta, 2026). Data budget can also be language-dependent: in one experiment, Reddy et al. (2025) found that training a Russian tokenizer to comparable quality required roughly 33% more data than training an English tokenizer. These disagreements must be carefully balanced when designing a tokenizer intended to process multiple languages. We first discuss the challenges associated with multilingual tokenization, and then review existing approaches to addressing them.

6.1 Allocation Bias

Most multilingual tokenizers are trained on imbalanced language mixtures and consequently do not allocate vocabulary capacity evenly across the languages they support. Languages with more available data (Joshi et al., 2020b) and more financial influence in the global economy may represent a higher proportion of the (tokenizer) training data, while other languages are barely represented. This may lead to *allocational harms*, which Gallegos et al. (2024) define as “disparate distribution of resources or opportunities between social groups.” For vocabulary construction, this concerns how much of a multilingual tokenizer’s vocabulary is used for encoding data in a given language (Limisiewicz et al., 2023). Standard tokenization algorithms such as BPE and Unigram form tokens largely based on frequencies. As a result, high-resource languages tend to dominate the learned vocabulary, while low-resource languages or languages with distinct scripts are underrepresented. Even when corpora are balanced, languages that share scripts with high-resource languages or exhibit simpler morphological structures often occupy a disproportionate portion of the vocabulary. Consequently, higher-resource languages are generally tokenized more efficiently, requiring fewer tokens to represent the same unit of meaning, while lower-resource languages tend to be split into a greater number of less meaningful subword units, or **oversegmentation** (e.g., Ahia et al., 2024). In addition, lower-resource languages end up with fewer characters per token, reducing the amount of textual information represented by a fixed number of tokens relative to more compressed languages. As shown in Figure 6.1, pretraining data size is strongly negatively correlated with **corpus token count** (CTC), the number of tokens produced by a tokenizer on a given corpus, measured on FLORES-200 (NLLB Team et al., 2024), a multilingual parallel dataset. The downstream effects of oversegmentation have been shown empirically when comparing models trained on multilingual tokenizers versus counterparts with dedicated monolingual tokenizers (Rust et al., 2021; Goldman et al., 2024; Tsvetkov & Kipnis, 2024; Brahma et al., 2025; Lundin et al., 2025; Shani et al., 2026).

Beyond the loss of semantic granularity, token count disparity also imposes an imbalanced computational cost across languages (Ahia et al., 2023; Petrov et al., 2023; Kanjirangat et al., 2025; Lundin et al., 2025). Longer sequences increase both latency and resource utilization during training and inference. As a result, a sample of N tokens for a high-resource language, such as English, conveys substantially more information than a sample of N tokens from an oversegmented language, and the computational and financial costs of processing semantically equivalent sentences can differ substantially across languages (Ahia et al., 2023; Petrov et al., 2023; Arnett et al., 2025; Lundin et al., 2025). Further downstream, **token parity**, or the degree to which sentences diverge in token count for different languages, affects performance on syntactic and morphological tasks (Kanjirangat et al., 2025). Importantly, performance disparity among languages persists even under character- or byte-level tokenization, allegedly “tokenizer-free” approaches (see Section 9). Lundin et al. (2025) describe this phenomenon as a **token tax**: because self-attention scales quadratically with sequence length, doubling the number of tokens required to represent the same semantic content results in roughly a fourfold increase in computational cost.

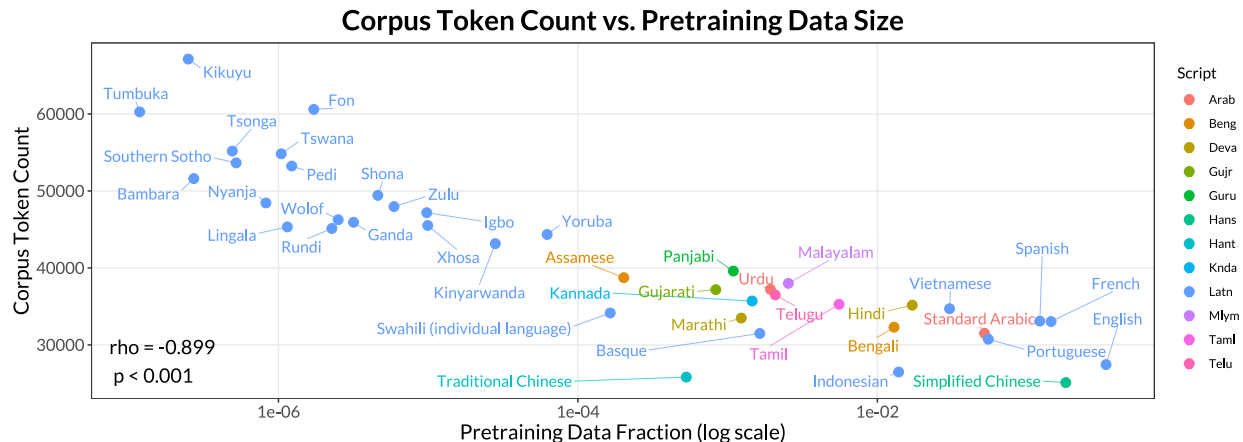


Figure 6.1 – Relationship between pretraining data size and tokenized corpus token counts across languages for the BLOOM-7b model (Workshop et al., 2023) on the FLORES-200 (NLLB Team et al., 2024) dev-test split. Each point represents a language, colored by writing system. The reported Spearman’s rank correlation coefficient is shown in the lower left corner.

6.2 Vocabulary Overlap and Transfer Learning

One way to obtain multilingual tokenizers is to explicitly allocate different portions of the vocabulary to individual languages (Neubig & Hu, 2018; Chung et al., 2020; Zheng et al., 2021a), but in many cases some of the vocabulary is *shared* across multiple languages. This **vocabulary overlap** has been found to be associated with the quality of crosslingual transfer in multilingual (Pires et al., 2019; Wu & Dredze, 2019) and bilingual settings (Kallini et al., 2025a), leading to the development of systems where overlap is explicitly enforced in order to improve zero-shot performance for low-resource languages (Patil et al., 2022). Vocabulary overlap is the strongest predictor of representational similarity between languages (Stap et al., 2023), which is thought to drive transfer (Patil et al., 2022; Kallini et al., 2025a; Itkonen & Tiedemann, 2026) despite the existence of *false friends*, words that are spelled the same in different languages but diverge in meaning, such as English vs. German *Gift* (“poison”) (Brilliant & Pinter, 2026).

Vocabulary overlap is largely impossible between languages with different writing systems, as those languages will not share any string overlap. For these cases, romanization and transliteration may enable vocabulary sharing, improving transfer and performance (Purkayastha et al., 2023; Husain et al., 2024). These approaches remain limited in practice, as users typically expect language models to generate text using conventional orthography. Hämmerl et al. (2025) propose aligning tokens with similar meanings but written in different scripts as part of a built-in system controlling overlaps, and find that such script-agnostic overlap is a better predictor of crosslingual knowledge transfer.

However, vocabulary overlap may not be necessary for crosslingual transfer. K et al. (2020) show that joint multilingual pretraining can lead to positive transfer, even without subword token overlap, and that the amount of overlap is not correlated with better performance. Limisiewicz et al. (2023) find that in some cases vocabulary overlap may be detrimental for word-level tasks but not sentence-level tasks. Similarly, Meyer & Buys (2024) note that representing multiple languages with a smaller vocabulary may *harm* crosslingual transfer and induce crosslingual negative interference, a phenomenon known as the “curse of multilinguality” (Conneau et al., 2020).

6.3 Code Switching and Language Variation

Language in the wild is rarely monolithic: speakers often draw on multiple languages within a single utterance, and even within one language, the same expression can vary systematically across dialects and communities. In multilingual settings, speakers alternate between two or more languages within a single utterance or text, a phenomenon known as **code switching**, which is constrained by various linguistic factors (Chi

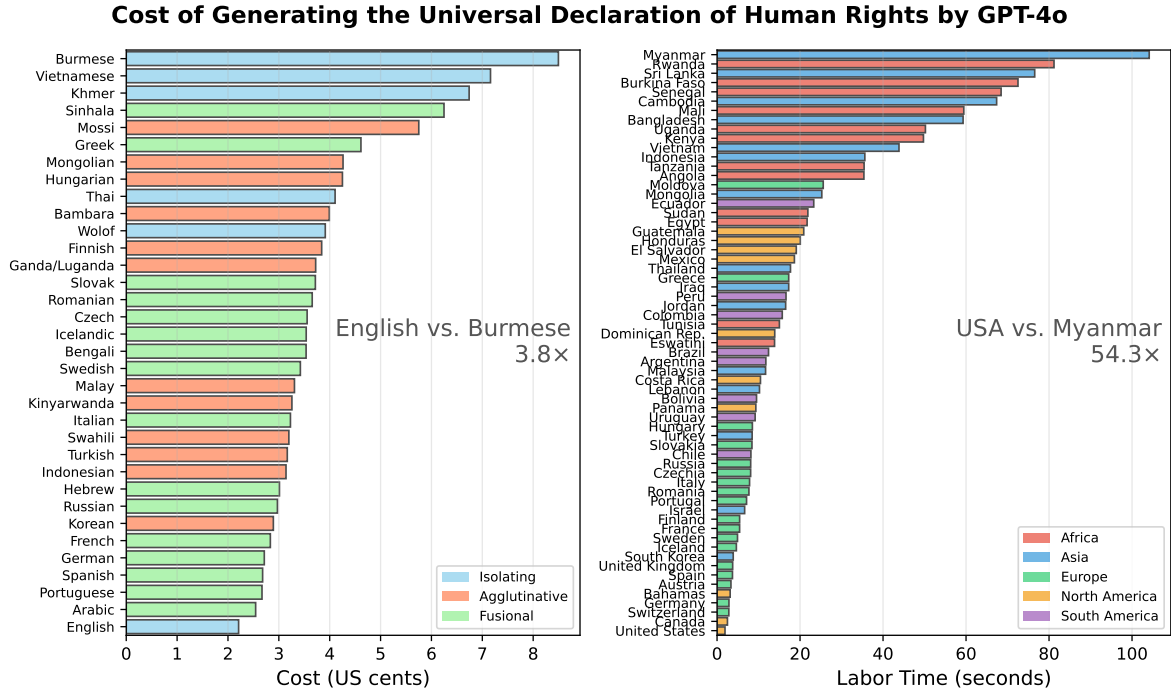


Figure 6.2 – Cost analysis of generating the Universal Declaration of Human Rights via GPT-4o (\$10/1M tokens). Left: absolute cost (USD cents). Right: labor-time equivalent at national average hourly wage (International Labour Organization, 2025) for majority-language generation.

& Bell, 2024; Sterner & Teufel, 2025). LLMs exhibit a related but distinct failure mode, termed **language confusion** or **language mixing**, in which a model cannot consistently generate the desired language or produces a language different from that requested in the prompt (Marchisio et al., 2024; Wang et al., 2025b). A related challenge arises even without switching languages: within a single language, the same meaning can surface in different forms across dialects (e.g., Standard American English vs. African American English), a phenomenon termed **language variation**. These phenomena all pose challenges for tokenizers, which are trained primarily on formal or monolingual data and tend to produce suboptimal segmentations when languages are mixed within the same document or sentence (Winata et al., 2023). Tasks involving language variation have been found to be more difficult for models with larger tokenizer vocabularies (Wegmann et al., 2025).

6.4 Mitigation Strategies

A range of approaches have been proposed to mitigate the aforementioned performance disparities across languages in multilingual models.

6.4.1 Multilingual Pretokenization

Pretokenization (Section 4) is a crucial step preceding subword tokenization, as it determines the basic units over which tokenization operates. Dominant pretokenization approaches are typically implemented using manually designed regular expressions, whose rules can often lead to brittle behavior, especially in multilingual settings. In particular, they may incorrectly segment characters or fail to preserve character integrity in languages with diverse scripts, such as Mandarin Chinese (Land & Arnett, 2025). While regex-based rules have well-documented limitations, eliminating pretokenization altogether also has downsides (Schmidt et al., 2024; Wegmann et al., 2025). Hence, the challenge is not simply whether to pretokenize, but how to do so in a more principled and multilingual manner.

Land & Arnett (2025) propose Script Category Representation in PreTokenization (SCRIPT) to address the limitations of regex-based pretokenization. Specifically, they represent each Unicode character as a pair of tokens: a block token, which encodes the character’s script and category, and an index token, which identifies the specific character within that block. This structured representation enables BPE to operate under constraints that preserve complete character representations, preventing merges that cross character boundaries. In a similar vein, Manodnya & Giri (2023) propose **Orthographic Syllable Pair Encoding (OSPE)** for Indic languages, which pretokenizes the text based on script-specific orthographic syllables before applying pair encoding. On Kannada language-modeling data, OSPE produces lower corpus token counts than standard BPE. K H & De Nardi (2026) extend this line of work to other languages, including Hindi, Nepali, Urdu, Arabic, and English. They evaluate models trained with OSPE and standard BPE on a polarity-detection task, and show that OSPE substantially improves performance for languages written in abugida and abjad scripts.¹⁹ Furthermore, OSPE leads to more stable optimization and preserves orthographic structure better than BPE. Addressing the phenomenon of false friends, Brilliant & Pinter (2026) propose replacing the first character of words shared by two languages in a bilingual tokenizer with language-specific characters, allowing disjoint representations for these words in the model.

6.4.2 Vocabulary Allocation

Various attempts have been made to adjust vocabulary size and allocation across languages either by changing their linguistic encoding or by designing new algorithms for fairer vocabulary allocation. Zhang et al. (2022) show that increasing vocabulary size can improve machine translation robustness across scripts. Rust et al. (2021) find that monolingual tokenizers often outperform multilingual tokenizers across a range of multilingual tasks.

To alleviate the *vocabulary bottleneck* across languages, XLM-V (Liang et al., 2023) clusters languages together and learns cluster-specific vocabularies, which are later combined by interpolating Unigram tokenization models. A related idea is explored by Chung et al. (2020), who propose constructing multilingual vocabularies by first clustering similar languages, learning vocabularies within each cluster, and then merging them into a single vocabulary. An alternative approach is to normalize representations across scripts. Patil et al. (2022) introduce **Overlap BPE (OBPE)** to maximize the vocabulary overlap between a low-resourced language and a closely related high-resourced one. This approach is shown to improve translation performance on southern African languages (Elmadani et al., 2022). Miletić et al. (2026) propose using the International Phonetic Alphabet (IPA) to represent languages written in different scripts. Their approach yields more robust and efficient tokenizers while achieving downstream performance comparable to that of standard text-based tokenizers. However, as the authors note, converting IPA representations back into the original writing systems remains a challenge.

Kautsar & Koto (2025) explores a parallel tokenizer framework that first constructs monolingual tokenizers and then aligns their vocabularies at the word level using machine translation. Semantically equivalent words across languages are then mapped to the same vocabulary index, enforcing shared embeddings across languages. Models trained with such parallel tokenizers show improved crosslingual transfer, more balanced compression rates, and stronger semantic alignment in representation space. Vico & Libovický (2025) extends the Unigram tokenization framework by conditioning target token probabilities on source language tokens. They evaluate these algorithms on intrinsic tokenization metrics, machine translation, and monolingual language modeling, but find improvement only in the latter.

Parity-aware BPE (PA-BPE) (Foroutan et al., 2026) is a variant of BPE (see Section 3.1.3) that prioritizes merge operations for the language with the worst current compression rate rather than maximizing global compression. Here, compression rate is defined as the ratio between the length of the original text and the number of resulting tokens, such that higher values indicate more efficient compression. They show that this approach leads to more equitable vocabulary allocation and tokenization costs across languages while maintaining comparable downstream performance.

¹⁹ Abugida scripts encode each syllable as a single grapheme in which the consonant serves as the base character and the vowel is marked by a diacritic attached to it, whereas abjad scripts represent only consonants, with vowels typically omitted or optional (Ramoo, 2021).

6.4.3 Multilingual Parity without Subword Tokenizers

Latent tokenizers, or byte-level representation methods (see Section 9), also face the problem of compression rate disparity across languages and scripts. Multilingual Adaptive Gradient-based Tokenization (MAG-NET) (Ahia et al., 2024) uses dynamic token pooling to introduce a script-aware tokenization module that predicts segmentation boundaries using gradient signals from the language modeling objective. This design encourages more balanced compression rates across languages with different scripts, but requires manually choosing a compression hyperparameter for each language or script. FlexiTokens (Owodunni et al., 2026a) replaces the script-specific, hand-tuned metric with automatically tuned compression hyperparameters. MYTE (Limisiewicz et al., 2024) retains the byte-level representation used by ByT5 but modifies the byte encoding scheme to incorporate morphological information. This encoding scheme reduces crosslinguistic differences in sequence length and improves multilingual performance, particularly for low-resource languages.

These architectures are covered further in Section 9.2.2.

6.4.4 Post-Hoc Multilingual Parity

Training a new language-specific LLM from scratch can be expensive, and one popular method for mitigating cost is based on tokenizer adaptation (see Section 12, as well as Wang et al., 2019b; Moon & Okazaki, 2020b; Wang et al., 2020c; Fujii et al., 2024). These techniques involve either extending pretrained tokenizers to new languages or domains, or constraining tokenizer output toward desired behavior. Tejaswi et al. (2024) experiment with training new tokenizers to apply LLMs to low-resource settings and appending their vocabularies to an existing pretrained tokenizer in order to expand its vocabulary. They observe a negative correlation between the expanded vocabulary size and tokens-per-word, although the gains diminish as vocabulary size increases. Nguyen et al. (2024b) extend the vocabulary of an English-centric pretrained tokenizer and then continue pretraining on Southeast Asian languages. This approach produces a multilingual LM outperforming GPT-3.5 on low-resource, non-Latin-script languages. Purason et al. (2026) propose continued BPE training, an approach that allows a pretrained tokenizer to be further adapted by continuing the BPE training process on new domains or languages. Their method reapplies the BPE algorithm to in-domain data by leveraging token pair frequencies computed from text tokenized with the original tokenizer to derive new merges, improving compression rates. **Language-Aware Token Boosting (LATB)** (Ukarapol et al., 2026) addresses language confusion by first identifying tokens belonging to the target language via Unicode-based filtering, and then boosting their logits at decoding time. This approach outperforms both fine-tuned models and explicit prompting approaches.

6.5 Questions

The field of continual learning, where new skills are added to a model without lessening performance on previously learned tasks, is an active area of research (Wang et al., 2024c). This is a similar problem to post-hoc multilingual parity. Can results from continual learning be applied improving the multilingual parity of pretrained models?

Language variation and code switching have received relatively limited attention in tokenization research, despite being common phenomena in interactions with multilingual speakers and increasingly important for conversational AI systems. To what extent does tokenization shape a model’s ability to perform downstream tasks, and how can we better understand its impact in these settings?

Various morphology-aware tokenization algorithms have been proposed, most of them in monolingual settings (e.g., Jabbar, 2024; Asgari et al., 2026), where morphological resources such as annotated

data or analyzers are typically available. Can morphology-aware tokenization scale to a viable multilingual paradigm, or will a purely statistics-driven approach remain more robust?

7 Evaluation

Evaluating and comparing different tokenization algorithms and settings in a meaningful way is not a straightforward endeavor. Arguably, the most intuitive approach is through **extrinsic evaluation**, where one measures the performance of a model trained with a given tokenizer. This provides a direct assessment of a tokenizer’s impact on the downstream use cases that we ultimately care about, such as machine translation, text classification, or question answering. However, because tokenization is the first step in an NLP pipeline, several problems arise with this evaluation strategy. First, it requires training models entirely from scratch: swapping a tokenizer necessitates replacing the embedding matrix, the softmax projection, and any other components that are dependent on the exact vocabulary and segmentation patterns induced by a tokenizer.²⁰ Second, extrinsic evaluation of a tokenizer is confounded by choices made for other components in the system, as well as by stochastic factors such as parameter initialization and training data ordering. These properties collectively make extrinsic tokenizer evaluation computationally expensive and time-intensive, more so than performing ablations over other parts of the LM training pipeline, such as post-training strategies. While some works have proposed the use of smaller models, hoping their performance would be predictive enough to that of larger, real-life models (Lotz et al., 2025a), these approaches rest on scaling assumptions that have not been widely verified and only partially mitigate the cost. This has motivated the development of **intrinsic evaluation** metrics that compare tokenizers independently of any downstream model or task, although they may rely on corpus data or on annotated resources.

7.1 Extrinsic Evaluation

Since the role of tokenizers is to work as components in NLP systems, and most saliently in language models, the most straightforward way to evaluate them is by measuring their impact on the tasks these systems are meant to solve. The use of general-purpose extrinsic tasks (cf. Maimon et al., 2025) to evaluate tokenizers has thus aimed to determine what tokenizer choice results in a better overall model.

The most direct candidate for extrinsic evaluation is to measure the impact of tokenization on next-token prediction. One way to evaluate models is through *raw cross-entropy loss* (Huang et al., 2025), but loss values from different tokenizers cannot be directly compared (Mielke, 2019). This value, and its surrogate *perplexity*, are computed at the token level, so its value depends on how many tokens a tokenizer assigns to a fixed span of text: the same string split into more tokens will typically have a lower per-token perplexity, because its negative log-likelihood is divided over more tokens. Consistent with this observation, lower perplexity produced by smaller vocabularies with more granular tokenizations is not predictive of performance further downstream (Luitel et al., 2025). Normalizing loss per word from a fixed dictionary (Chung & Kim, 2025) or by the vocabulary’s unigram distribution (Roh et al., 2020; Tao et al., 2024) corrects for vocabulary size, but vocabulary-based corrections are not sufficient: two tokenizers with the same vocabulary can still assign different tokenizations to the same string if their inference algorithms differ. The standard remedy is therefore to normalize the negative log-likelihood of a string by a count that does not depend on the tokenization at all. **Bits-per-byte** divides the total cross-entropy loss of a document by its size in bytes (Gao et al., 2020a); **bits-per-character** (Brown et al., 1992; Poelman & de Lhoneux, 2026) also accounts for differences among scripts in the number of bytes Unicode assigns per character. These metrics are all sensitive to aspects of either tokenization or text encoding, and so are not suitable for making comparisons across both tokenizers and languages. Instead, aggregating negative log-likelihood at the *sentence level* allows comparing performance across languages that share parallel data (Yang et al., 2026). We list these metrics and others in Table 7.1.

These data-invariant normalization methods address the granularity mismatch, but they do not remove differences in the trained model that arise from inductive biases imparted by the use of a subword tokenizer, such as the potentially significant difference in probability assigned to a given orthographic word by a language model that has that word as a token in its vocabulary compared to one that does not (Lesci et al., 2025). In addition, these metrics are typically computed only over the canonical tokenization of the document, ignoring the marginalization necessary in theory across all possible segmentations (see Section 8) (Mielke, 2019; Cao & Rimell, 2021). Computing this marginal probability is prohibitively expensive, but luckily the

²⁰We discuss ways to avoid entirely retraining a model when modifying a tokenizer in Section 12.

Metric	Formula	Description
Bits-per-byte (BPB)	$-\frac{1}{N_b} \sum_i \log p(t_i \mathbf{t}_{<i})$	Log-likelihood of the corpus normalized by the number of UTF-8 bytes N_b . Comparable across tokenizers since the denominator is tokenization-independent.
Bits-per-character	$-\frac{1}{N_c} \sum_i \log p(t_i \mathbf{t}_{<i})$	Log-likelihood normalized by the number of Unicode characters N_c to mitigate the effect of UTF-8 byte encoding lengths.
Mean token rank	$\frac{1}{N_t} \sum_i \text{rank}(p(t_i \mathbf{t}_{<i}))$	Average rank of the true token in the model’s next-token distribution. Depends only on the <i>ordering</i> of token probabilities, not their magnitudes, so it does not overly penalize low-resource languages.
Bits-per-word	$-\frac{1}{N_w} \sum_i \log p(t_i \mathbf{t}_{<i})$	Log-likelihood normalized by the number of orthographic words N_w in the document.
Unigram-normalized Loss	$-\frac{1}{N_t} \sum_i \log \frac{p(t_i \mathbf{t}_{<i})}{p_{\text{uni}}(t_i)}$	BPB rescaled by the corpus log-likelihood under a unigram baseline p_{uni} to mitigate the influence of token frequency across different tokenizers.
Sentence-level NLL	$-\sum_i \log p(t_i \mathbf{t}_{<i})$	Total negative log-likelihood. Useful for mitigating the differences in tokenizers when comparing parallel strings from different languages or the same string with different tokenizers.

Table 7.1 – Metrics that can be used to compare across different tokenizers.

difference between the marginal and canonical log-likelihood is empirically small in most settings: [Chirkova et al. \(2023\)](#) report that it stays under 0.5% for most text, though it grows for text with long, complex words and under distribution shift, where accounting for the marginal can change the outcome. [Geh et al. \(2024\)](#) likewise find the estimated marginal to be nearly indistinguishable from the canonical probability. While there appear to be cases where considering non-canonical tokenizations during evaluation helps ([Geh et al., 2024](#)), empirically, evaluating only the canonical tokenization seems to be sufficient for most purposes. For comparing a single model across different tokenizations, [Jain et al. \(2023\)](#) evaluate *tokenization sensitivity*, which measures the divergence of a model’s next-token probability distributions between the canonical tokenization of a context and a perturbed tokenization of the same context.

In principle, any NLP task can serve as a downstream evaluation for tokenization, and in practice many do. For some time, machine translation was a primary driving force behind NLP innovation, and BPE ([Sennrich et al., 2016](#)) and Unigram ([Kudo, 2018](#)) tokenizers were both proposed in this context. As a result, the effects of tokenization on neural machine translation have been studied extensively (e.g., [Sennrich, 2017](#); [Ding et al., 2019](#); [Gowda & May, 2020](#); [Zhang et al., 2022](#); [Saleva & Lignos, 2023](#); [Zouhar et al., 2023a](#); [Cognetta et al., 2024a;b](#)). Tokenization-focused studies have also used other tasks, such as language understanding ([Yehezkel & Pinter, 2023](#)), dependency parsing ([Bauwens et al., 2025](#)), multiple-choice QA ([Schmidt et al., 2024](#); [Sanz-Guerrero et al., 2025](#)), and summarization ([Goldman et al., 2024](#); [Lotz et al., 2025a](#)).

7.1.1 Model Benchmarks Targeting Tokenization

Rather than evaluating a tokenizer through a model on a generic downstream task, one can use targeted tasks that isolate what information a model can extract from its tokens. Such tasks are typically cheap to create and easy for humans to solve, and they often require character-level knowledge, which makes them challenging for token-based models that do not have access to the individual characters that make up a

token. These tasks are not used during the development of the tokenizer, but instead examine how well a model overcomes the limitations imposed by its tokenization.

One common set of tasks is where a model must make character-level decisions about an input string. For models with subword tokenizers that do not have direct character access, these seemingly simple tasks can be challenging. CUTE (Edman et al., 2024b), CharBench (Uzan & Pinter, 2026), and LangGame (Sims et al., 2025) provide a set of character-level tasks such as the following, presented to the model as natural-language prompts that the model must answer in generated text.

Example	
Character Counting	<i>How many r's are there in the word strawberry? → 3.</i>
Spelling	<i>Spell out strawberry. → s t r a w b e r r y.</i>
Inverse Spelling	<i>Write the word s t r a w b e r r y. → strawberry.</i>
Character Inclusion	<i>Does the word strawberry contain an r? → Yes.</i>
Character Manipulation	<i>Change every r in the word strawberry to a z. → stzawbezzy.</i>

This style of task is difficult for subword language models, as the granularity of tokens does not always match with the granularity of the task. For example, in the Llama 3 tokenizer, *strawberry* is tokenized as `strawberry`, but *stzawbezzy* has the fragmented tokenization `st z aw be z zy`, so a language model must have a good understanding of its tokens’ internals in order to successfully complete the manipulation task.

LMentry (Efrat et al., 2023) provides a list of tasks that are trivial for humans, including word and character manipulation tasks similar to those in CUTE and CharBench, as well as others such as sorting words alphabetically and by length. StringLLM (Wang et al., 2025c) provides a set of simple, templated tasks such as “Count the instances of {x} in {y}.”, but applies them to difficult inputs such as multilingual sentences, hashes, and random alphanumeric strings. These tasks are still trivial for humans, but are difficult for language models due to irregular token sequences and because the inputs are unlikely to have appeared in any training data. Indeed, while humans solve each task nearly perfectly, models as advanced as GPT-4o perform relatively poorly.

EXECUTE (Edman et al., 2025) extends the CUTE test suite to non-English languages with diverse script families. EXECUTE additionally includes subcharacter tasks that probe the model’s knowledge of writing systems below the level of the base character, such as extracting radical information from Chinese characters. Outside of natural language, NUPA (Yang et al., 2025b) measures performance on simple numerical tasks (e.g., arithmetic, comparison, and conversion) which are highly sensitive to individual digit knowledge, Reddy et al. (2026) test these capabilities under different numerical scripts and formats, and SubTokenTest (Hou et al., 2026) provides tasks such as spatial comprehension of text-rendered diagrams and processing of tabular data, which are both very sensitive to tokenization.

A challenge for these evaluations is the growing tool-use capabilities of LLMs. Modern agentic language models can often “cheat” on these tasks by recognizing the task, writing a program to solve it, and executing it on the given input, which bypasses the capabilities we are trying to measure. These evaluations must therefore take care to disable tool use to ensure an accurate assessment.

A more direct way to assess a model’s knowledge of its tokens’ composition is through learned probes. In this setting, simple classifiers are trained to answer certain queries about tokens (e.g., “What is the third letter in the token {X}?”) from their embeddings. These probes can help pinpoint the character-level information encoded in embeddings without the confounding variables that come with a generative process. Experiments have shown that language models do encode information about their tokens’ character composition, albeit imperfectly (Itzhak & Levy, 2022; Kaushal & Mahowald, 2022; Hiraoka & Okazaki, 2024; Hiraoka & Inui, 2025).

7.2 Intrinsic Evaluation

In intrinsic evaluations, one evaluates a tokenizer in isolation, without training a downstream model. There are several reasons for choosing this mode of evaluation, the most common of which are cost and speed:

intrinsic metrics can often be computed in seconds, whereas measuring the downstream effect of a tokenizer choice requires training a language model and then evaluating it, each of which takes orders of magnitude more computational resources and time. Having good intrinsic evaluations also allows for tuning tokenizer hyperparameters, such as vocabulary size or the choice of pretokenization regex, together before training rather than as separate ablation experiments.

Intrinsic evaluations are often treated as hopeful proxies for downstream performance, and the correlation between the two kinds (or lack thereof) has been the subject of many studies (Cognetta et al., 2024b; Meister, 2026, *inter alia*). Some intrinsic evaluations, however, test for properties that one may want a tokenizer to have even when the effect of those properties on downstream performance is unknown. For instance, a morphological evaluation dataset can be developed to test whether a tokenizer handles morphologically rich languages without excessive fragmentation; a compression benchmark can serve to assess the expected monetary cost of deploying a tokenizer to downstream data; a vocabulary utilization test can inform what fraction of a vocabulary is used in a given corpus, providing an estimate of parameter waste in the embedding layer of a model trained with that tokenizer.

Intrinsic metrics can be divided along several axes. Here, we organize them by the inputs they require, which also roughly tracks their cost to compute and their availability as a function of language resource status. The simplest metrics examine the vocabulary alone, and possibly some of the auxiliary tokenizer components required for inference. The second group requires encoding a raw corpus and analyzing the resulting token sequences. The third requires externally annotated data, such as gold-standard morphological segmentations or parsed syntax trees. The metrics are summarized in Table 7.2. Many of these metrics have been centrally implemented by tokenizer evaluation tools like **tokenizer-scorer** (Zouhar et al., 2023a), **TkTkT** (Bauwens, 2024), **QTok** (Chelombitko et al., 2024), **Tokenizer Intrinsic Benchmark** (Uzan et al., 2024), **TokSuite** (Altıntaş et al., 2025), **TokEval** (Meister, 2026), and **TokCollate** (Variš et al., 2026).

7.2.1 Vocabulary-Based Metrics

One vocabulary-based metric is measuring bytes per token at the vocabulary level (rather than relative to a corpus), with the implication that a vocabulary with longer tokens encodes more information into each token and provides models with a richer representation and shorter tokenized sequence lengths.

In multilingual settings, crosslingual vocabulary overlap (the share of identical tokens that appear in multiple languages) has been found to promote better crosslingual transfer and multilingual performance (Kallini et al., 2025a). When swapping tokenizers between model application phases (see Section 12), it is useful to compute the Jaccard or weighted overlap between two tokenizers’ vocabularies to measure their compatibility (Nair & Resnik, 2023).

7.2.2 Corpus-Based Metrics

These metrics require encoding a raw, unannotated text corpus and analyzing the resulting token sequences.

Compression All else held equal, compression, i.e., how many tokens are required to encode a given input, is an attractive intrinsic metric for tokenization due to its direct impact on any downstream model’s runtime and memory requirements. Compression can be measured in different ways. The simplest is to count the number of tokens needed to encode a given text, or *corpus token count* (CTC; Schmidt et al., 2024). One benefit of this method is that by calculating CTC over parallel texts, one can compute the tokenizer parity and compare compression rates across different languages (Petrov et al., 2023, *inter alia*).

Token counts can be normalized by the number of bytes in the surface form of the tokenized text to compute a tokenization-agnostic compression metric (Schmidt et al., 2025; Limisiewicz et al., 2026). However, because different languages encode different amounts of information per byte (Arnett et al., 2024a), this measure is not always suitable for comparing compression across languages. Alternatively, Land & Arnett (2025) use tokens-per-character for normalization. Other popular compression metrics include **fertility**, which counts the average number of tokens per word (Rust et al., 2021), and **single token retention rate** (Nayeem et al., 2025), which measures the fraction of surface words preserved as single tokens. However, these metrics rely on an operationalization of wordhood. Most often, for English, this is determined by whitespace and

Type	Metric	Domain(s)	What it measures
Corpus-based	Token-length distribution	Tokenizer swap	Subwords that are decoded to valid UTF-8 sequences
	UTF-8 consistency		
	Vocabulary overlap	Tokenizer swap	Jaccard or weighted overlap between vocabularies
	Compression	Crosslingual fairness*	Number of tokens
	↳ Corpus token count		
	↳ Bytes-per-token		Tokens per word
	↳ Tokens-per-character		
	↳ Fertility		It compares the compression of parallel text between two languages.
	↳ Tokenizer Parity		
	Information-theoretic		Fraction of words that are a single token
	↳ Upper bound on Kolmogorov complexity		
	↳ Rényi efficiency		Frequency of the tokens in the 95th percentile
	↳ Single token retention rate		
Annotation	Frequency	Crosslingual fairness*	Frequency of the tokens in the 95th percentile
	↳ Zipfianness		
	↳ Percentile frequency ($F_{95\%}$)		Whether the subword boundaries correspond to morpheme boundaries
	Contextual coherence		
	Tokenizer Gini coefficient		Proportion of morpheme boundaries recognized correctly
	Self-supervised divergence		
	Vocabulary overlap (distribution)		Proxy for cognitive plausibility, computed as $1 - \frac{\# \text{ tokens}}{\# \text{ chars}}$
	Semantic overlap		
	Precision, recall, F_1	Morphological alignment	Whether the subword boundaries correspond to morpheme boundaries
	MorphScore	Morphological alignment	

Table 7.2 – List of intrinsic tokenization metrics. * denotes multilingual settings.

punctuation, meaning that sequences like *ice cream*, which may meet some criteria of wordhood, will be treated as multiple words (Arnett et al., 2025). In scripts where word boundaries are not easily definable, these metrics essentially cannot be computed.

Since more compression leads to greater information density for a sequence of a fixed token length, increased compression has been argued to lead to improved model performance (Deletang et al., 2024). While several empirical findings support the correlation between compression and better downstream task performance (Gallé, 2019; Goldman et al., 2024), others have found no clear relationship between the two (Zouhar et al., 2023a; Cognetta et al., 2024b; Schmidt et al., 2024).

Information-Theoretic Measures **Rényi Efficiency** measures the optimality of the frequency distribution of tokens in a given tokenized text (Zouhar et al., 2023a), seeking to balance the frequency of both the

most-common and least-common tokens. Rényi Efficiency is parameterized by a hyperparameter α , which changes the impact that different attributes of the frequency distribution (e.g., concentration in the tails) have on final metric, and is normalized by the vocabulary size. At higher values of α , this metric has been shown to correlate well with downstream machine translation performance, which suggests that balanced-ness of token distributions is advantageous. Under some adversarial conditions, however, Rényi Efficiency is “gameable”: counterexamples can be designed in the form of tokenizers that have high efficiency but degrade model performance (Cognetta et al., 2024b).

Chung & Kim (2025) compute an upper bound of the Kolmogorov complexity of the tokenized text of a (BPE) tokenizer and find that increasing the vocabulary size lowers this complexity of the tokenized text and improves model performance.

Frequency Rank and Distributional Shape **Percentile frequency** tracks how deep into the vocabulary a corpus reaches (Gowda & May, 2020). **Zipfianness** measures how closely the token-frequency distribution follows a power law, which is a commonly found distributional shape in language (Lotz et al., 2025a; He et al., 2025). Gowda & May (2020) propose a heuristic rule for choosing the token vocabulary size that works well empirically: Use the highest possible vocabulary size such that 95% of tokens occur at least 100 times in the data. We note that this heuristic was determined on a relatively small-scale dataset, and can be replaced with a more robust generalized metric that measures the probability mass contained in a percentile range (Zouhar et al., 2023a).

Contextual Coherence **Contextual coherence** (Yehezkel & Pinter, 2023) is motivated by the role of token vocabularies in the downstream language model. The argument is that as token types appear in fewer disparate contexts in the pretraining corpus, they provide a crisper signal to the model and are easier to learn better representations for. For each token, the number of unique tokens appearing in a window of $\pm w$ tokens from its occurrences in the corpus is counted, and then these counts can be aggregated into various vocabulary-level distribution statistics such as mean count, quartiles, and variance.

7.2.3 Multilingual Corpus-Based Metrics

Crosslingual Fairness Gini coefficients and Lorenz curves over per-language tokenization costs quantify how equitably a tokenizer serves different languages (Meister, 2026).

Tokenizers that do not exhibit crosslingual fairness exhibit a difference in tokenization counts and fertility between different languages, where lower-resourced languages can have worse segmentation, leading to worse model quality and more expensive inference (Ahia et al., 2023; Petrov et al., 2023).

Vocabulary and Semantic Overlap Tokenizers trained on multilingual corpora can contain tokens that are meaningful in multiple languages. Vocabulary overlap is therefore a predictor for multilinguality and crosslingual transfer, and can be quantified by the Jensen-Shannon divergence (JSD) between the probability distributions of the vocabulary in corpora in different languages (Limisiewicz et al., 2023). Tasks like named-entity recognition and sentence-level tasks benefit from high overlap, while part-of-speech tagging and dependency parsing suffer from it.

Semantic overlap captures the alignment between tokens which may have different forms or be in different scripts across languages. Hämmerl et al. (2025) use token alignability instead of literal overlap, as it does not depend on the surface form or script. The first step is to run a statistical word aligner such as eflomal (Östling & Tiedemann, 2016), and then to measure assignability. This can be done in two ways: by counting the proportion of tokens in the target language with one-to-one alignments; or by using the eflomal score as the maximum unnormalized log-probability of links in the last sampling iteration (Vázquez et al., 2019).

7.2.4 Metrics Requiring Auxiliary Data

Morphological Alignment While data-driven tokenization training algorithms like BPE do not directly optimize for linguistic structure, they nevertheless often learn morphologically meaningful tokens. **Morphological alignment** is a measure of the extent to which a tokenization aligns with the morphological

boundaries within a word. Alignment is measured using precision, recall, and F_1 of token boundaries against gold-standard morpheme boundaries (Kurimo et al., 2010), and is available for any language where morphological resources exist (Bostrom & Durrett, 2020; Batsuren et al., 2022; 2024; Uzan et al., 2024). Batsuren et al. (2024) developed an evaluation protocol in which the tokenization of a given word is classified according to whether it is split into morphemic tokens or non-morphemic tokens, or stored whole as a single token. The authors found that morphemic tokenization was correlated with better performance. MorphScore (Arnett & Bergen, 2025) expands on this idea and measures tokenizer recall of the last morpheme boundary of words in 22 languages. It assigns a score of 1 if the tokenizer assigns a token boundary at the last morpheme boundary and 0 otherwise, without penalizing the tokenizer for unnecessary extra token boundaries or for disregarding other morpheme boundaries, which introduces limitations (Poelman et al., 2025). Stephen & Libovický (2026) propose a metric that uses morpho-syntactic features instead of gold segmentations to evaluate morphological alignment that extends to languages where reliable segmentation data is unavailable.

While multiple studies have argued that increased morphological alignment is associated with improved performance on a variety of NLP tasks (Bostrom & Durrett, 2020; Park et al., 2020; Vasii & Potolea, 2020; Hofmann et al., 2021a; Nzeyimana & Niyongabo Rubungo, 2022; Erkaya, 2022; Toraman et al., 2023; Bauwens & Delobelle, 2024; Držík & Forgáč, 2024; Libovický & Helcl, 2024; Jabbar, 2024; Uzan et al., 2024; Asgari et al., 2026; Kildeberg et al., 2025), there is also evidence from a variety of different languages that morphologically aligned tokenization did not benefit model performance (Macháček et al., 2018; Saleva & Lignos, 2021; Choo & Kim, 2023; Arnett et al., 2024b; Arnett & Bergen, 2025).

Cognitive Plausibility Tokenizers can also be measured by how they segment text compared to how humans process it. **Cognitive plausibility** (Beinborn & Pinter, 2023) measures the correlation between tokenizer segmentation and human processing signals such as reading times or eye-tracking fixations. The idea behind cognitive benchmarking is that tasks that are difficult for humans should be difficult for automatic systems, in this case tokenizers, and vice versa. Human performance is proxied through a *word recognition task* (Balota et al., 2007), where response time and accuracy are recorded for deciding whether a string of characters is a word in their language. Both human metrics are then correlated with a measure derived from a tokenizer’s behavior over the same string, typically characters-per-token, and a tokenizer performs better if its correlation in any or all four setups ($\{\text{accuracy, response time}\} \times \{\text{words, nonwords}\}$) is higher in absolute value. The metric is thus computable for any language in which lexical recognition data exists, and is particularly beneficial in cases where morphological resources cannot be used for evaluation, such as in nonconcatenative languages (Gazit et al., 2025). Beinborn & Pinter (2023) use **chunkability** as their tokenizer proxy metric: $1 - \frac{\# \text{ tokens}}{\# \text{ chars}}$.

7.3 Questions

Many intrinsic metrics have been proposed to better design tokenizers without trial-and-error training runs. However, they often require some calibration on a downstream task. For example, percentile frequency and Rényi efficiency have tunable hyperparameters that can be tuned via downstream tasks (defeating their purpose as purely intrinsic metrics). What intrinsic metrics can we design that require no tuning?

Intrinsic metrics often implicitly assume that the rest of the model is held equal and that we are just comparing the effect of a change in tokenizer. However, the modern LLM development pipeline often runs in stages where ideas are validated on small models before being scaled to a larger model. What intrinsic metrics work well in this domain (predicting tokenizer performance across scale rather than across tokenizers)?

8 Non-Canonical Tokenizations

We have so far only considered the *canonical* tokenization of text: the token sequence produced by a given deterministic tokenizer for a specific input. We now come back to a notion mentioned in Section 2.2, that of **non-canonical tokenization**: sequences other than the canonical one that still match the surface form of the text. These arise from the many-to-one nature of subword token sequences. For a given subword vocabulary, there often exist several different token sequences that map back to the same surface form. For example, consider the word *tokenization*. Under a specific subword tokenizer, this input may be converted to the sequence `tokenization`, which would be the canonical tokenization under that tokenizer. However, if the tokens `to`, `tok`, `en`, `ken`, `niz`, etc. also exist in the vocabulary, then alternative tokenizations `tokenization`, `to ken ization`, etc. are also possible. In general, because a string s can be split between each of its $|s|$ characters, it has up to $2^{|s|-1}$ possible segmentations (Bauwens et al., 2025; Lee et al., 2025a).

Non-canonical tokenizations have found use in both training and inference of neural language models. During training, sampling non-canonical sequences exposes the model to alternative segmentations of the same text and can improve model robustness and act as data augmentation. During inference, using non-canonical tokenization provides alternative representations of an input text, and can be used to account for the probability mass of a given surface form that is not associated with the canonical tokenization.

The presence of non-canonical tokenizations can also harm model quality, especially during inference. We cover some instances of this in Sections 14.3 and 14.5.

8.1 In Language Model Training

Originally, non-canonical tokenization was motivated by regularization and data augmentation during training (Kudo, 2018; Provilkov et al., 2020). From the regularization perspective, a model trained only on the canonical tokenization of each string rarely or never sees the fragmented segmentations that arise from typos, variant capitalization, unconventional spelling, or changed whitespace (see Figure 8.1 for examples); these segmentations are out-of-distribution for the model, and models trained solely on canonical tokenizations have been shown to perform worse on such inputs (Belinkov & Bisk, 2018; Moradi & Samwald, 2021). From the data augmentation perspective, non-canonical tokenizations provide “new” training examples to learn from: even though the concepts represented by the underlying text remain unchanged, the model sees a distinct input sequence. This is particularly useful when training data is limited, e.g., for low-resource languages. **Subword regularization** can be considered analogous to many of the techniques used in data augmentation for image processing, where models are made more robust by being trained on more examples created by transforming the training images using rotations, flips, random crops, occlusions, and other operations that change the input’s form while preserving its content (Shorten & Khoshgoftaar, 2019). Incorporating non-canonical token sequences during training serves the same purpose: it enlarges the training set and makes the model less dependent on any single tokenization of a string by modifying the tokenization but preserving the surface form text (Provilkov et al., 2020).

8.1.1 Subword Regularization

Subword regularization was first supported by the Unigram tokenizer (Kudo, 2018) (see Section 3.2), which defines a probability distribution over the segmentations of a string using the product of the unigram probabilities of each token. Although by default Unigram inference produces the probability-maximizing segmentation, it is also possible to sample from the distribution of all segmentations and thus expose the model to alternative, perhaps comparably likely segmentations during training. The idea was extended to BPE by BPE-dropout (Provilkov et al., 2020) (see Section 3.7): whereas BPE normally applies the highest-priority merge at each time step during inference, BPE-dropout drops each candidate merge with probability p at each step, so lower-priority merges are sometimes applied before higher-priority ones, yielding a different segmentation of the same string (a reference implementation is given in Code 3.7). A corresponding variant for MaxMatch (WordPiece) tokenization, MaxMatch-Dropout, drops candidate tokens during the longest-match search with probability p , so that a shorter matching token is sometimes selected in place of the longest one (Hiraoka, 2022).

Example		
Surface Form	Tokenization	Token IDs
intelligence		[11478]
“intelligence”		[330, 93375, 1]
intellegence		[14490, 1978, 768]
intelligent		[25530]
“intelligent”		[330, 396, 21149, 1]

Example 8.1 – The tokenizations of *intelligence*, “*intelligence*”, *intellegence* [sic], *intelligent*, and “*intelligent*” and their token IDs using the GPT-4 tokenizer. Despite the fact that these all share very similar surface forms and meanings (with *intellegence* being a common typo), they produce vastly different tokenizations.

Cognetta et al. (2024c) observed that in practice, BPE-dropout and MaxMatch-Dropout sample the set of possible segmentations *quasi-deterministically*, i.e., in such a way that only a handful of segmentations receive non-negligible probability. *Tokenizer-agnostic* algorithms address this by s‘ampling more broadly: for example, sampling uniformly over the tokenizations of each word showed improved modeling performance over BPE-dropout and Unigram baselines (Cognetta et al., 2024c). Bauwens et al. (2025) find that quasi-deterministic tokenizers work better for memorization-based tasks like part-of-speech tagging and named-entity recognition due to the stable connection between segmentations and labels. They argue that the trade-off between determinism and subword regularization should be a hyperparameter p_{mux} that defines the amount of multiplexing, and that model performance deteriorates at lower quantities p_{mux} in some subword regularizers (e.g. BPE-dropout) than others due to the quality of their distributions. Moreover, in uniform sampling, tokenized sequence lengths follow a binomial distribution (for sufficiently large $\mathcal{V}$) and token lengths themselves follow a truncated geometric distribution, so that segmentations tend to be made up of shorter tokens with less semantic information. To address this, they introduce **GRaMPa**, where samples are biased towards longer individual tokens and shorter overall sequences. They find that for certain values of minimal token length ℓ and sampling temperature τ , longer tokens can be generated consistently, outperforming uniform sampling on various downstream tasks.

Non-canonical tokenization can also be mixed with canonical tokenization during training. Wang et al. (2021) observe an improvement in downstream multilingual tasks by tokenizing a model’s input twice during training: once with the canonical tokenization and once with a sampled segmentation. Cross-entropy loss is applied to both, but an additional *self-consistency* loss is used to ensure that the model’s predictions are aligned regardless of the input tokenization.

Empirically, subword regularization has been shown in many cases to improve model performance across a range of downstream tasks including translation (Kudo, 2018; Provilkov et al., 2020; Haddow et al., 2022; Takase et al., 2022) and text classification (Hiraoka et al., 2019; Zheng et al., 2021b; Hiraoka, 2022). These gains are typically largest in low-resource settings and shrink as the training corpus grows (Provilkov et al., 2020; Cognetta et al., 2024c); at larger pretraining scale, some studies report negligible gains (Mroczkowski et al., 2021). Recent work has nonetheless applied stochastic tokenization at larger model and dataset scales. Sims et al. (2025) report no loss on standard benchmarks when applying a form of stochastic tokenization to a 50M-parameter model trained on OpenWebText (Gokaslan & Cohen, 2019), and later to a 275M-parameter model trained on FineWeb (Penedo et al., 2024). Their method improves performance on character-level tasks such as CUTE (Edman et al., 2024b) (see Section 7.1.1) and on multi-digit arithmetic, which they attribute to the model acquiring a more detailed representation of the characters that make up its tokens. They further find that a short continued-pretraining step with stochastic tokenization improves an already pretrained, deterministically tokenized model on the same tasks.

One frontier model, ERNIE5, is known to use BPE-dropout during pretraining (Wang et al., 2026).

8.1.2 Intentional Shift

Another use of non-canonical tokenizations is to deliberately modify the tokenization algorithm to expose the model to better tokenizations than what the tokenizer would have produced by default. Hofmann et al. (2021a) fine-tune a BERT model using the same vocabulary but replacing some words’ canonical tokenizations with an alternative tokenization guided by derivational morphology and find that it improves the understanding of new, complex words in a linguistic probing task compared to a model fine-tuned with the canonical tokenization. Bauwens & Delobelle (2024) find a similar result when using BPE-knockout, a BPE variant that helps to better align tokens to morphological boundaries while preserving the subword vocabulary, when fine-tuning RoBERTa (trained with BPE) on a series of NLU tasks.

Modifying the tokenization algorithm can also be used to mitigate some security risks, as BPE merge lists may leak information about the tokenizer training data (Hayase et al., 2024). Sawada & Goyal (2025) replace the BPE tokenizer of a pretrained model with a vocabulary-only tokenizer that does not leak such information.

These can be considered examples of tokenizer transfer, which is covered more in Section 12.

8.2 In Language Model Inference

When models are trained on the canonical tokenization of input text, they are better fit to handle such sequences, making it natural to use canonical segmentation during model inference. There are nonetheless reasons to consider non-canonical tokenizations at inference time, both theoretical and empirical. For example, language models should theoretically sum the probabilities of all tokenizations that decode to a given surface form in order to properly model the probability of text, rather than taking the canonical tokenization alone; that is, one should *marginalize* over the set of all possible tokenizations of the input s .

Cao & Rimell (2021) show that marginal perplexity can be substantially better than one-best (canonical) perplexity, especially in out-of-domain settings, and relate the size of this gap to the tokenizer’s uncertainty; Chirkova et al. (2023) find the gap is usually small. This approach has been shown to be valuable in language-modeling applications (Phan et al., 2024; Chatzi et al., 2025; Vieira et al., 2025b). Pimentel & Meister (2024) and Oh & Schuler (2024) derive formulae for the probability of a word under subword tokenizers and show the effect concretely in linguistic and psycholinguistic analyses. The correction is largest for tokenizers that mark word boundaries at the start of a token, such as the GPT family: there, the probability of a word must include the word-boundary marker that begins the following token, and omitting it changes measured outcomes in sentence comprehension and lexical-optimization analyses. Giulianelli et al. (2024) use marginalization to convert a token-level model into a character-level model that assigns a probability to any character substring rather than only to whole tokens; surprisal of substrings other than the full region of interest predicts reading times more accurately. Vieira et al. (2025a) give exact and approximate algorithms for this token-to-character conversion.

Models can often handle non-canonical tokenizations even without any regularization, marginalization, or correction without a huge loss in performance (Zheng et al., 2025a). Geh et al. (2024) find that using a weighted mixture of the canonical and non-canonical tokenizations as a proxy for the marginal probability can lead to large gains in accuracy on downstream tasks like multiple-choice question answering.

Non-canonical tokenizations are advantageous beyond language modeling. In machine translation, Kudo (2018) sample several tokenizations and select the one that scores highest on a proxy metric, and Takase et al. (2022) feed a subword-regularized model multiple tokenizations of the same input and combine their predictions as a single-model ensemble, improving low-resource translation. A single-model ensembling approach has also been adapted to reweight training examples based on annotation reliability (Tsuji et al., 2025). Song et al. (2024) merge, during inference, the subword sequences that share a surface form, aggregating probability mass spread across equivalent segmentations before the best candidate is chosen. This recovers mass that single-tokenization decoding discards in subword-regularized models. In reasoning, Sathe et al. (2025) use multiple sampled tokenizations of a problem to generate diverse reasoning paths and aggregate them, improving self-consistency and accuracy.

A practical obstacle to marginalization is that the number of tokenizations generally grows exponentially in the length of the text. Geh et al. (2024) show that, under a neural language model, finding the most probable tokenization is NP-complete and computing the exact marginal is #P-hard. A common alternative is to estimate the marginal with a randomized approximation. The naive approach is *rejection sampling*: draw samples from the language model and keep only those whose surface form matches the target $\mathbf{s}$. When $p(\mathbf{s})$ is small, this rarely produces valid samples and converges slowly. *Importance sampling* instead draws from a proposal distribution q whose support is $\mathcal{T}(\mathbf{s})$, the set of valid tokenizations of $\mathbf{s}$. The contribution from each of the samples towards the estimand is then reweighted to correct for the difference in sampling probability under the original and proposal distributions:

$$p(\mathbf{s}) = \sum_{\mathbf{t} \in \mathcal{T}(\mathbf{s})} p(\mathbf{t}) = \mathbb{E}_{\mathbf{t} \sim q} \left[\frac{p(\mathbf{t})}{q(\mathbf{t})} \right] = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N \frac{p(\mathbf{t}^{(i)})}{q(\mathbf{t}^{(i)})},$$

where $\mathbf{t}^{(i)}$ is the i -th tokenization sampled from q . Cao & Rimell (2021) estimate $p(\mathbf{s})$ using the (stochastic) Unigram tokenizer to define the proposal distribution q , as the estimator requires access to the sampled tokenizations' probabilities, which Unigram provides directly but, e.g., BPE-dropout does not.

Chirkova et al. (2023) use the same importance-sampling framework but introduce a proposal distribution based on constrained generation (see Section 14.4) that applies to any tokenizer, and use it to measure the gap between the marginal and canonical log-likelihood directly. Geh et al. (2024) also estimate the marginal by importance sampling, using the same constrained proposal distribution, and find the marginal to be nearly indistinguishable from the canonical probability.

8.3 Questions

The results of Sims et al. (2025) suggest subword regularization can help improve model performance on character-level tasks and arithmetic even for GPT-2 sized models. Does it help in other domains? Does the effect have diminishing returns as model or data size increases? Sims et al. (2025) use subword regularization as a short continual pretraining step. What other training schedules could be used? Subword regularization also can worsen perplexity. Can we perform subword regularization earlier during training and then stop, to get the best of both worlds?

Marginalization is very expensive. What modifications to tokenization algorithms or heuristics for sampling can we use to produce fast, but accurate, approximate marginals?

In what ways can model providers add an interface by which users can request non-canonical tokenizations in specific cases? What are the security concerns (DDoS, jailbreaking, tokenizer leakage, etc.) that arise?

9 Byte and Character Representations

Although subword token vocabularies have become the standard in modern NLP systems, the approach itself is not without fault. Relying primarily on precomputed subword vocabularies can introduce difficulty processing noisy or out-of-vocabulary text (Xue et al., 2022), inconsistent behavior across languages (Rust et al., 2021; Ahia et al., 2023; Petrov et al., 2023), and obfuscation of character-level information from the model (Kaushal & Mahowald, 2022; Hiraoka & Okazaki, 2024, *inter alia*). One main alternative to subword vocabularies is to process text at the byte or character level directly, without *explicit* subword (or word) tokens. For brevity, we use *byte* to refer to both bytes and characters when clear.²¹

In addition to alleviating the shortcomings listed above, byte-level models avoid the computational cost of a softmax over a large vocabulary as they use a small vocabulary of only 256 tokens (Grave et al., 2017; Liang et al., 2023). However, the primary price for byte-based approaches is efficiency. When the basic representation level is not made up of multi-character tokens, input sequences become much longer, which increases the compute and memory requirements of the model. As a result, byte-level models typically incorporate *hierarchical* mechanisms to aggregate fine-grained inputs into intermediate-level representations, such as downsampling modules (e.g., convolutions, Tay et al. 2022; pooling, Nawrot et al. 2023), heuristic rules (e.g., whitespace-based, Slagle 2024; entropy-based segmentation, Pagnoni et al. 2025), or end-to-end learned strategies (Godey et al., 2022; Nawrot et al., 2022; Hwang et al., 2025; Kallini et al., 2025b; Dauncey & Wattenhofer, 2026).

This section surveys the current landscape of byte and character representations, focusing on two major directions: 1) *pure byte-level and character-level modeling*, where models operate directly on raw bytes or characters without introducing coarser-grained units; and 2) *latent tokenization*, where models internally construct hierarchical representations to improve efficiency.²² The primary granularity and architectural patterns along with the models that use them are listed in Table 9.1.

9.1 Pure Byte- and Character-Level Representations

Pure byte-level or character-level models use architectures that process the original byte- or character-level sequence directly.

Early examples of this paradigm include character-level recurrent language models such as the multiplicative RNN (Sutskever et al., 2011a) and the multiplicative LSTM (Krause et al., 2017). For part-of-speech tagging and named-entity recognition, Gillick et al. (2016) introduced a byte-to-span LSTM that directly processed UTF-8 bytes and demonstrated improved multilingual performance. Byte-level recurrent language models were also used for sentiment classification (Radford et al., 2017), while deep character-level sequence-to-sequence models were explored for neural machine translation (Cherry et al., 2018). Collectively, these models illustrate an early line of pure byte- and character-level modeling in which the main sequence architecture continued to operate at the original input resolution.

With the proliferation of transformers, pure byte- and character-level transformer architectures were occasionally explored as competitive alternatives to subword-based models. Al-Rfou et al. (2019) demonstrated that a very deep (64-layer) transformer decoder trained purely on characters can achieve state-of-the-art performance on the `text8` and `enwiki8` language modeling benchmarks, illustrating that causal attention mechanisms can work effectively without subwords. Choe et al. (2019) extended this result to raw byte-level language modeling at a larger scale, showing that a vanilla transformer operating directly on bytes can achieve performance competitive with word-level language models on the One Billion Word benchmark. Pure byte- and character-level transformers were also explored for neural machine translation (Gao et al., 2020b; Shaham & Levy, 2021).

²¹In Latin script languages, basic (ASCII) characters are represented by single bytes and so the terms *byte* and *character* were often used interchangeably. In other scripts, the UTF-8 encoding of a character may be more than one byte.

²²A note on terminology: The methods discussed in this section are often referred to as “tokenizer-free” (e.g., Hwang et al., 2025). In this survey, we distinguish between methods that do not rely on a pretrained subword tokenizer but still represent text as bytes or characters, discussed in this section, and methods that represent text visually as pixels, discussed in the next section. We therefore avoid using “tokenizer-free” as an umbrella term.

Model class	Architecture type	Models
Pure byte-level	Decoder-only	Character Transformer (Al-Rfou et al., 2019), Sparse Transformer (Child et al., 2019), MambaByte (Wang et al., 2024b), EvaByte (Zheng et al., 2025b)
	Encoder-decoder	ByT5 (Xue et al., 2022)
Latent tokenization ▷ fixed-width	Encoder-only	CANINE (Clark et al., 2022b)
	Decoder-only	Hourglass Transformer (Nawrot et al., 2022), MegaByte (Yu et al., 2023b), MBLM (Egli et al., 2025)
	Encoder-decoder	Charformer (Tay et al., 2022)
Latent tokenization ▷ dynamic-width ▷ predetermined	Encoder-only	CharacterBERT (El Boukkouri et al., 2020), XRayEmb (Pinter et al., 2021)
	Decoder-only	eByte (Thawani et al., 2023), BLT (Pagnoni et al., 2024), SpaceByte (Slagle, 2024), HAT (Neite-meier et al., 2025), AU-Net (Videau et al., 2025), Fast BLT (Kallini et al., 2026)
Latent tokenization ▷ dynamic-width ▷ learned	Decoder-only	DTP (Nawrot et al., 2023), MAGNET (Ahia et al., 2024), H-Net (Hwang et al., 2025), MrT5 (Kallini et al., 2025b), Bolmo (Minixhofer et al., 2025a), FlexiTokens (Owodunni et al., 2026a), MBP (Owodunni et al., 2026b)
	Encoder-decoder	MANTa (Godey et al., 2022)

Table 9.1 – Taxonomy of byte- and character-level LLM architectures based on their design choices.

Transformer-based byte-level modeling was further popularized by **ByT5** (Xue et al., 2022), a byte-level encoder-decoder model in the T5 family (Raffel et al., 2020; Xue et al., 2021). ByT5 was pre-trained on over 100 languages using a span-corruption objective in which the encoder receives corrupted byte sequences and the decoder predicts the masked spans of bytes. ByT5 outperformed parameter-matched, token-level mT5 models across a range of multilingual, generative, and noise-robustness benchmarks. However, processing raw bytes without any explicit downsampling or latent structure led to significantly longer sequence lengths and worse training and inference efficiency. Edman et al. (2024a) compared ByT5 and mT5 on machine translation and observed that byte-level modeling helped with the translation of rare or orthographically similar words and where fine-tuning data was scarce, but also noted the large gap in efficiency.

The **Sparse Transformer** (Child et al., 2019) used sparse attention to reduce the cost of self-attention while modeling raw byte sequences. More recently, **MambaByte** (Wang et al., 2024b) adapted the Mamba selective state space model (SSM; Gu & Dao, 2024) to byte-level inputs, offering an architecture that avoids the quadratic cost of self-attention while retaining strong language modeling performance. Other approaches mitigate the inefficiency of byte-level generation, either by adopting a multi-byte prediction strategy to improve inference speed of transformer-based language models (Gloeckle et al., 2024) or by introducing a linearized attention mechanism in the transformer architecture, enabling scalable modeling while still processing directly at the byte level (**EvaByte**; Zheng et al., 2025b).

9.2 Hierarchical Models and Latent Tokenization

Language modeling at the byte level is challenging due to the computational and memory costs of processing long sequences. One common approach to address these limitations is to aggregate the representation over byte spans to obtain a sparser sequence. While different names for these byte subsequences are used in the literature, such as *patches*, *chunks*, *spans*, or *segments*, here we opt for the term **latent tokens**. We retain the word *tokens* as we view grouping bytes as analogous to subword tokenization despite some salient

differences. The adjective *latent* emphasizes that, unlike subword tokens, latent tokens exist only in the model’s internal representation, while the input and output remain byte sequences.

In the following, we distinguish between early approaches developed mostly before 2022, which feature a variety of architectural designs, and later approaches which have increasingly converged on autoregressive, hierarchical architectures.

9.2.1 Early Approaches

Some approaches used character-level encoders to construct representations of explicitly segmented words, including transition-based dependency parsers (Ballesteros et al., 2015) and word-level language models (Kim et al., 2016; Labeau & Allauzen, 2017). Although these models reduced reliance on word or subword lookup tables at the input, their main sequence models still operated over word-level units, and we therefore distinguish them from the pure byte- and character-level models discussed above and from learned latent models.

CANINE (Clark et al., 2022a), a character-level masked language model, uses local-attention transformer layers (Beltagy et al., 2020) and sequence downsampling to improve the efficiency of processing long sequences. Latent vectors corresponding to multiple neighboring characters are aggregated through hash embeddings. **CharacterBERT** (El Boukkouri et al., 2020), **XRayEmb** (Pinter et al., 2021), and **Charformer** (Tay et al., 2022) also aggregate byte-level input into latent representations, but use a convolutional neural network instead of a shallow local transformer. One significant difference between these approaches is the method of segmenting text into latent tokens: whereas CharacterBERT and XRayEmb use whitespace as the token delimiter to aggregate the bytes within words, Charformer instead applies a convolution followed by a neural function that scores candidate byte spans of different lengths and averages their representations.

Libovický et al. (2022) adapted CANINE- and Charformer-style downsampling to character-level machine translation. They found that while character-level models generally did not outperform subword systems, their main advantage was robustness for noisy inputs. Edman et al. (2022) modified this architecture to use variable-sized character spans determined by subword boundaries drawn from a pre-existing static subword tokenizer. The subword-delimited latent tokenization approach outperformed traditional subword models on the majority of the tested translation directions, but at a cost of slower inference.

9.2.2 Hierarchical Architectures

Recently, autoregressive, hierarchical architectures have become the most popular solution for byte-level modeling. They operate on representations of varied granularity. Following the **Hourglass Transformer** (Nawrot et al., 2022), hierarchical models are typically comprised of three modules (see Figure 9.1):

1. An *encoder* that converts byte input into latent tokens;
2. A *global module* that performs computation on latent token embeddings; and
3. A *decoder* that takes a latent representation and generates a byte output.

Although the original Hourglass Transformer used subword tokens in the input layer, similar architectures have been widely adopted for byte-level modeling. **MegaByte** (Yu et al., 2023b) is a hierarchical model with a byte-level encoder and decoder that is distinct from prior hierarchical architectures (CharacterBERT, CANINE, etc.) in that each level in the hierarchical structure is autoregressive. The output embeddings for fixed spans of text (8 bytes long) are concatenated into the latent representation processed by a large global module. The output of the global module is then fed to a small decoder module that predicts one byte at a time. Both the global and local modules are decoder-only transformers. MegaByte can be applied to modalities other than text, such as audio and images. **MBLM** (Egli et al., 2025) further generalizes MegaByte to multiscale decoder hierarchies.

SpaceByte (Slagle, 2024) is a byte-level decoder architecture which applies a global module over varied-length latent tokens defined by whitespace boundaries. This approach, also used by **eByte** (Thawani et al., 2023), is limited to scripts where words are space-delimited, leaving out scripts such as Chinese and Japanese.

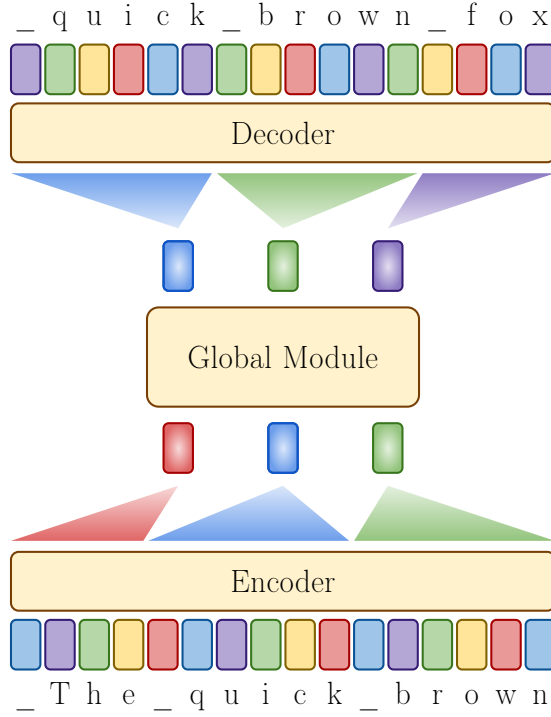


Figure 9.1 – An illustration of a generic hierarchical byte-level language model with encoder, global, and decoder modules. The input is at the byte (or character) level, but downsampling is applied so that the global model operates over latent subwords with a shorter sequence length. The latent representations are projected back to the byte granularity for decoding.

Furthermore, even when whitespaces are used as markers, they are not exclusive. For example, when processing text encoding polysynthetic languages such as Inuktitut or Greenlandic, using whitespace boundaries may produce over-compressed tokens containing an entire sentence’s worth of content.

Dynamic Token Pooling (DTP) (Nawrot et al., 2023) follows the same hierarchical architecture consisting of an encoder, a global module, and a decoder, but introduces *self-learned* latent tokenization: for each byte, a binary classifier predicts whether to end the current latent token and create its representation. While this technique provides flexibility to the overall system, it is possible that the latent tokenizer will learn to predict a boundary at each position, reducing to an undesired byte-level model. To avoid this, average token length is controlled by a regularizing loss function:

$$\mathcal{L}_{\text{boundary}}(N, k, \alpha) = -\log \left(\binom{N}{k} \alpha^k (1 - \alpha)^{N-k} \right), \quad (4)$$

where N is the length of the input sequence, k is the number of predicted boundaries, and α is a hyperparameter that specifies the desired expected boundary rate (or, equivalently, the inverse of the desired expected token length). This loss is minimized when $k = \alpha N$.

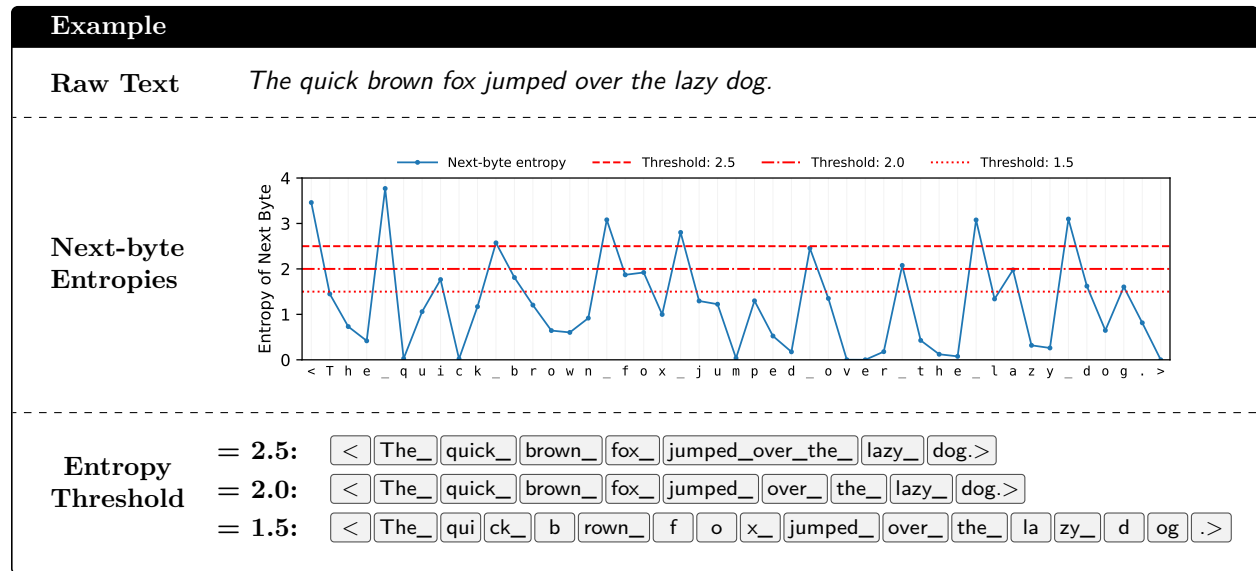
Similarly to DTP, **MANTa** (Godey et al., 2022) introduces learned tokenization in a T5-style model. It softly assigns input bytes to latent tokens that are processed by the encoder, while the decoder continues to generate at the byte level. **MrT5** (Kallini et al., 2025b) introduces an alternative form of learned compression within ByT5, using a deletion gate after the initial encoder layers to select which contextualized byte representations to retain. Information from deleted positions is kept within the retained representations, shortening the sequence processed by subsequent encoder layers at inference while preserving byte-level decoding.

MAGNET (Ahia et al., 2024) adapts DTP to a multilingual setting based on the observation that different languages and scripts encode text with a different number of bytes. Its modified version of (4) includes script-specific latent token length priors (α) that account for the information granularity of byte sequences

in specific languages. Accounting for each script mitigates tokenization unfairness and improves speed and performance in multilingual tasks. **FlexiTokens** (Owodunni et al., 2026a) further improved the language adaptability of latent tokens by adding a module that automatically controls the size of tokens for specific languages or domains and by replacing (4) with script-specific upper and lower bounds on the number of tokens to be predicted. These changes led to better performance and more adaptability in multilingual tokenization.

Dauncey & Wattenhofer (2026) formulate latent tokenization as a reinforcement learning problem. Token boundaries are sampled from a learned Bernoulli policy with the reduction in byte-level language modeling loss serving as the reward. This approach directly optimizes the expected loss under discrete token boundary decisions without having to rely on approximate solutions used by straight-through estimators in DTP.

The **Byte Latent Transformer (BLT)** (Pagnoni et al., 2025) uses *entropy* as a signal for latent tokenization. This heuristic, first considered in DTP, is based on the principle that the global model should only be applied for difficult predictions—specifically, the cases where the language model is uncertain about the next byte. This uncertainty is quantified by the next-byte entropy, and “entropy spikes” indicate that the global model should be used. Altering the entropy threshold can create coarser- or finer-grained representations.



Example 9.2 – Examples of text segmentations produced by spikes in next-byte entropy from a small byte-level language model. These entropies were extracted from BLT’s entropy model. Using different entropy thresholds yields different segmentations, and therefore different representation granularities, of the same input text.

A key architectural novelty of BLT is the use of cross-attention between byte and latent token representations, in both the local encoder and decoder modules. Similar to CANINE, multi-byte representations are encoded using hash embeddings, thereby reducing the number of encoder layers without harming performance.

H-Net (Hwang et al., 2025) is a hierarchical model that uses Mamba (Gu & Dao, 2024) layers in both the local encoder and decoder. The authors conduct multiple steps of latent tokenization by stacking multiple encoder and decoder modules. This approach enables processing representations of various granularity (i.e., tokens corresponding to morphemes, words, and multi-word expressions). To find boundaries between tokens, H-Net uses a binary classifier similar to the one introduced in DTP. However, unlike DTP, H-Net’s classifier takes the embeddings of two consecutive bytes as input, and the Bernoulli loss is replaced with a balancing loss (inspired by the mixture-of-experts literature; see Fedus et al. (2022)) for controlling average token length. Furthermore, the architecture accounts for uncertainty in predicting token boundaries by mixing the representation of neighboring tokens with weights corresponding to the probability of the boundary.

Autoregressive U-Nets (AU-Nets) (Videau et al., 2025) are a hierarchical language model inspired by the U-Nets used in computer vision (Ronneberger et al., 2015). Like H-Net, AU-Nets, as well as the **Hierarchical Autoregressive Transformer (HAT)** (Neitemeier et al., 2025), implement multiple steps of latent tokenization,

but token boundaries are determined by the appearance of a space, thus first-level latent tokens always correspond to words, second-level to pairs of words, and so on. In these -Net systems, while FLOP-matched results on downstream tasks are promising, inference performance remains challenging.

An alternative to a heuristic or self-learned binary classifier for predicting token boundaries is to use supervision from an existing subword tokenizer. This approach, introduced as a variant of DTP (Nawrot et al., 2023), is adopted by **Bolmo** (Minixhofer et al., 2025a), where the boundary predictor is trained to reproduce token boundaries from OLMo 3’s (Olmo et al., 2026) BPE tokenizer. **Bolmo** training is initialized from a checkpoint of a pre-trained, subword-tokenized OLMo 3 model, making it a special case of tokenizer transfer from a subword tokenizer to a latent tokenizer (see Section 12). This *byteification* of the BPE tokenizer enables direct control over token granularity (i.e., compression) and can thus change the trade-off between model efficiency and performance. Moreover, **Bolmo** gains the ability to better understand the character-level composition of text, substantially improving over the base OLMo 3 model on CUTE, a benchmark dedicated to measuring this capability (Edman et al., 2024b). A concurrent distillation approach converted multiple model families, such as Llama, Qwen, and OLMo, to byte-level models through a hierarchical byte-level model (Bao et al., 2026). Similarly, Alpha et al. (2026) train a latent-tokenizer model from a BPE-tokenized Llama 3 checkpoint to form HAT 70B, the largest hierarchical language model to date.

All the works described in this section so far focus on improving the efficiency of processing byte sequences via latent tokens. However, these models still generate text one byte at a time, making inference considerably slower than subword-based models. Recent work has attempted to improve inference speed by predicting all bytes within a latent token in parallel. **Fast BLT** (Kallini et al., 2026) introduced *BLT Diffusion*, a new BLT model variant that uses text diffusion for faster parallel byte decoding. Owodunni et al. (2026b) introduced a new FlexiTokens variant where all bytes in a latent token are predicted in parallel using an additional language modeling head. Both methods use additional inference techniques inspired by speculative decoding to improve generation efficiency.

9.3 Questions

Latent tokenizers segment texts *on the fly*; i.e., at each byte, they must decide whether to construct a latent representation and use a global transformer module. In this chapter, we presented multiple methods for delimiting latent tokens (see Table 9.1). Which method for setting boundaries between latent tokens offers the best trade-off between model performance and computational efficiency?

Byte models have a speed disadvantage compared to subword-based models because they process longer byte-level sequences. In hierarchical models, this problem can still affect the local encoder and decoder. Autoregressive generation is time consuming due to communication overhead incurred by processing sequence byte by byte. Thus, slow decoding remains an important problem for byte models. Will fast decoding approaches (e.g., diffusion, speculative decoding, more efficient batching) accelerate byte models sufficiently to enable their wider adoption?

10 Visual Text Representations

The tokenization strategies discussed elsewhere in this survey split text into symbolic units such as subwords, characters, or bytes. These symbolic units can create structural biases related to language support or inequities in input sequence lengths (Ahia et al., 2023); drop semantically meaningful orthographic information, especially for CJK languages (see Section 5.1.1); and introduce a trade-off between vocabulary-dependent parameter count and sequence compression. One approach to mitigating these problems is to process text using a **visual text representation** that renders text as a rasterized image.

The earliest motivation for visual text representations was for processing Chinese, Japanese, and Korean, since these scripts have large base vocabularies and semantically meaningful visual components (see Section 5). Here, simple glyph encodings were used instead of one-hot character encodings or other script-specific encodings in order to expose character shape information to the model and avoid the computational challenges of large vocabulary sizes (Dai & Cai, 2017; Zhang & LeCun, 2017; Meng et al., 2019a).

Compared to traditional subword tokenizers, visual representations have proven useful for handling visually confusable characters (Salesky et al., 2021), processing code-switched text (Rust et al., 2023), transferring to unseen languages and scripts (Kesen et al., 2025), and effectively compressing input sequence lengths (Wei et al., 2025). Additionally, purely visual representations overcome many of the limitations of text tokenizers and enable truly open-vocabulary language modeling as they do not require a fixed-vocabulary tokenizer. Despite these advantages, using purely visual representations has one critical limitation: *generating* text, which requires either the use of a post-processing optical character recognition (OCR) step to recover the visually generated text or the use of hybrid visual-text architectures (Tai et al., 2024).

This section covers approaches for rendering visual text (Section 10.1), models for purely visual text processing (Section 10.2), hybrid models that jointly process visual text and tokenized inputs (Section 10.3), and concludes by discussing some open challenges in visual text representations.

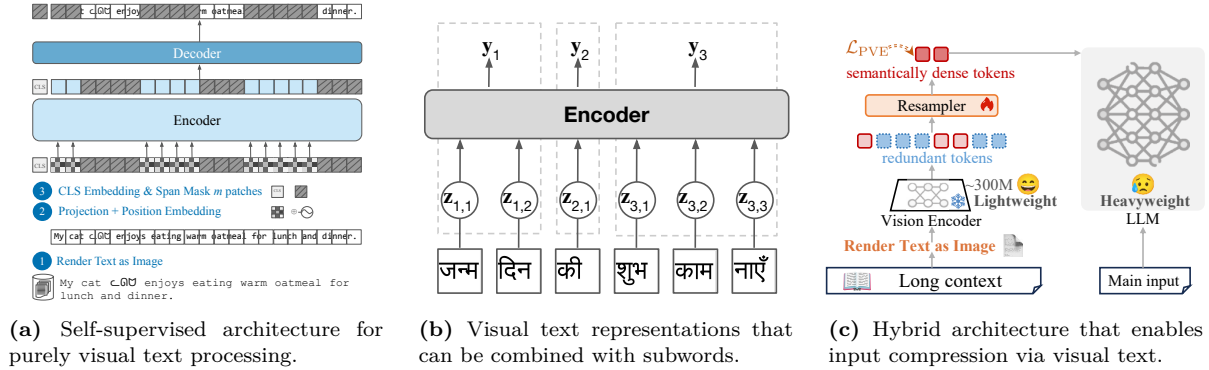


Figure 10.1 – Visual text representations has been used in (a) models that operate on purely visual text without any direct influence from tokenization in the preprocessing or model, such as Rust et al. (2023)’s model which learns language representations end-to-end from visual text inputs that reconstruct visual text outputs; (b) approaches that combine visual text representations with tokenization-based representations in hybrid models (Lotz et al., 2025b); (c) models which compress long contexts for more efficient processing (Xing et al., 2025). Images used under license from the respective articles.

10.1 Rendering Text for Visual Language Modeling

Representing text as images starts with a *text renderer*, which converts Unicode sequences into *glyphs*, the visual shapes that represent text. While implementations vary, the pipeline shown in Figure 10.2 is representative of the high-level flow (Rougier & Esfahbod, 2018; Desires, 2019; Levien, 2020; Esfahbod, 2024). A sequence of segmentation stages first partitions the input into *runs* according to paragraph boundaries, formatting rules, text direction, font coverage, and Unicode script (UAX #9; UAX #24; UAX #29). Each

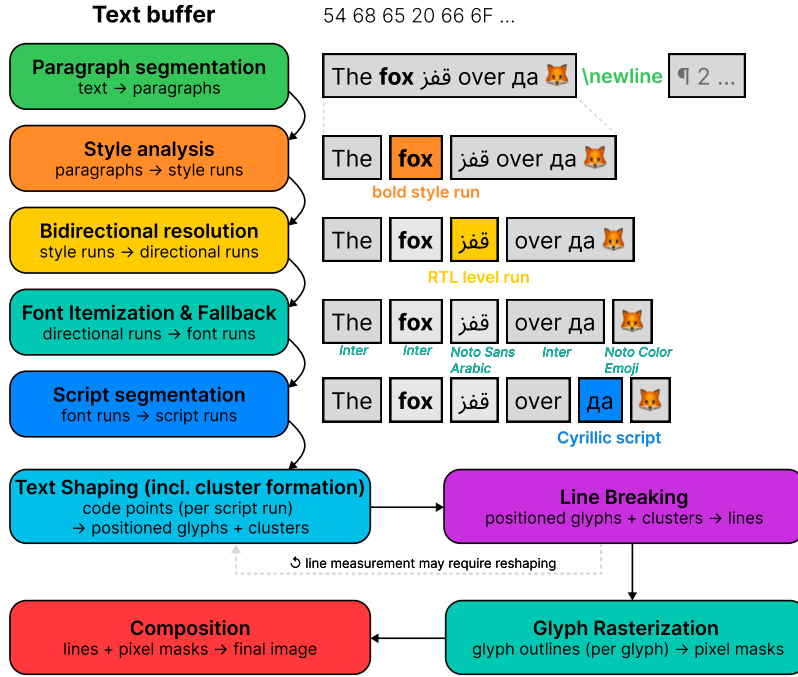


Figure 10.2 – Illustration of the text rendering pipeline with one example per segmentation stage: a newline (paragraph segmentation), a bold span (style analysis), an Arabic word (bidirectional resolution), an emoji (font fallback), and a Cyrillic word (script segmentation); assigned fonts are shown beneath the runs.

resulting *script run* is uniform in style, direction, font, and script.²³ Text *shaping* then converts the code points of each script run into positioned glyphs, applying script-specific substitution and positioning rules, and groups code points and glyphs into *clusters*, as shown in Figure 10.3. Finally, line breaking selects break points to satisfy line-width constraints, glyph outlines are rasterized into per-glyph *pixel masks*, and composition places the rasterized glyphs at their computed positions to produce the final image.

There is no canonical approach to rendering text for language models, and choosing one involves several design decisions. One decision is the layout and style of rendered text. Text can be rendered as a document, where the content is laid out on a page, whether as a simple single stream of text, where layout reduces to line breaking; or as a complex arrangement of blocks such as a receipt, a form, or a webpage, where the text renderer is one component of a larger layout engine. Alternatively, text can be rendered as text *chunks*: short spans rendered separately into narrow strips, with a rendering strategy fixing whether and how text aligns with image patches (e.g., continuous rendering, a fixed number of characters per patch, or words aligned to patch boundaries; Lotz et al., 2023; Salesky et al., 2023). Another issue is the number of pixels per character: for a given text, the glyph scale sets the pixel count, and hence the patch count and sequence length. Larger glyphs require more patches per character, whereas smaller ones pack more text per patch but degrade legibility.

Rendering can happen at different stages relative to training. *Pre-rendering* does it once, offline, and stores the images (Mansimov et al., 2020; Salesky et al., 2021; Rust et al., 2023), whereas *on-the-fly rendering* generates each image at training time, per sample (Tschannen et al., 2023; Kesen et al., 2025). Pre-rendering text pays an upfront cost but ensures reproducibility across environments, easier validation, and training-time speed. On-the-fly rendering, on the other hand, pays a cost at each training sample to re-render the text and potentially breaks reproducibility if the environment changes, but also allows for data augmentation and regularization effects (varying font, size, color, etc.).²⁴

²³This stage may be considered analogous to pretokenization, which pre-segments the text into canonical, normalized pieces.

²⁴These may be seen as analogous to subword regularization (Sections 3.7 and 8.1).

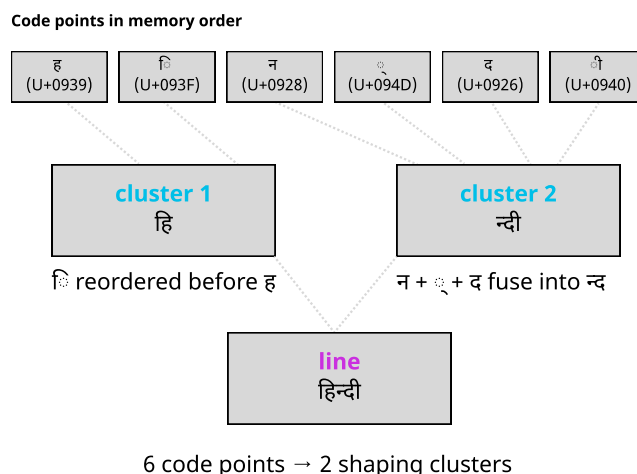


Figure 10.3 – Text shaping for the word हिन्दी, made up of the Devanagari ह (U+0939), ि (U+093F), न (U+0928), ् (U+094D), द, (U+0926), and ी (U+0940) and producing 2 shaping clusters, हि and न्दी. The clustering is a not one-to-one mapping from code points to glyphs and is context dependent, so rendering this word correctly requires a shaping engine rather than a direct lookup.

10.2 Purely Visual Text Processing

Purely visual approaches operate only on visually rendered text, thereby eliminating the need for any type of discrete vocabulary. The earliest attempt at this approach is the in-image translation model of Mansimov et al. (2020), which maps an image of a source-language sentence directly to an image of its translation, trained end-to-end with pixel-level supervision only.

Current systems commonly use the masked autoencoder vision transformer architecture (**ViT-MAE**) (He et al., 2022) as a visual backbone. This is a self-supervised visual representation learning method, pretrained through the *masked patch reconstruction objective*, where the model attempts to reconstruct randomly masked image patches using the mean squared error (MSE) loss function. This architecture consists of a bidirectional encoder, which processes non-masked patches, and a lightweight bidirectional decoder, which processes contextualized hidden representations and mask embeddings to reconstruct masked patches. ViT-MAE-based language models differ from the original model in two aspects. First, visual language models replace natural images with images of rendered text. Second, since the input is always images of rendered text, they are trained using a modified objective, masked *span* patch reconstruction, which masks contiguous spans of rendered text, rather than individual patches at random, and thereby mimics textual span-based masked language modeling (Joshi et al., 2020a).

Rust et al. (2023) introduced **PIXEL**, the first end-to-end language model pretrained using purely visual representations, built upon the ViT-MAE architecture, as illustrated in Figure 10.1a. PIXEL demonstrated strong performance against its subword tokenizer-based counterpart BERT, outperforming it by a large margin on crosslingual transfer to unseen writing systems with increased robustness against orthographic attacks. Lotz et al. (2023) experimented with different structured text rendering strategies to reduce patch diversity: *continuous*, where text is rendered in fixed-sized patches throughout the input; *bigram*, where each patch contains exactly two characters; *monospace*, which uses a monospaced font and then applies bigram patching; and *words*, where patches are aligned to word beginnings, but each word uses a continuous patching strategy. An example of each rendering approach is shown in Example 10.4. Each proposed strategy outperformed the default continuous text rendering. In particular, grouping bigrams of characters in a patch allowed a smaller, 22M-parameter PIXEL model to perform on par with the original 86M-parameter PIXEL model on the GLUE benchmark (Wang et al., 2019a). The PIXEL architecture has also been pretrained on historical documents (Borenstein et al., 2023), operating directly on scanned pages rather than on rendered OCR output. Kesen et al. (2025) introduced the first multilingual pretrained model PIXEL variant,

PIXEL-M4, trained on English (Latin script), Ukrainian (Cyrillic script), Hindi (Devanagari script), and Chinese, for visual and linguistic diversity. PIXEL-M4 demonstrated strong crosslingual transfer learning performance to unseen writing systems through visual similarities (Muñoz-Ortiz et al., 2025). However, recent results on transfer to four low-resource languages suggest that multilingual continued pretraining of a monolingual PIXEL model can produce a better text classifier than starting from the multilingual PIXEL-M4 model (Zhang et al., 2026b).

Example	
Continuous	I must be growing small again. ■
Bigram	I must be growin g smal l ag ai n. ■
Monospace	I mu st b e gr ow in g sm al l ag ai n. ■
Words	I must be growin g small again. ■

Example 10.4 – Rendering strategies for the sentence *I must be growing small again*. Black patches mark the end of a sequence, following Rust et al. (2023). Images and caption adapted from Lotz et al. (2023).

Tatariya et al. (2024) examined the linguistic knowledge learned from pretrained visual representations. They performed vision- and language-centered probing analyses and found a layer-wise division between PIXEL models’ visual and language capabilities: lower layers capture visual features of the rendered language, while higher layers perform language processing. Similarly, Kesen et al. (2025) used LINSPECTOR (Şahin et al., 2020) to show which types of word-level linguistic knowledge were learned from multilingual pretraining.

Purely visual language modeling methods are not confined to the ViT-MAE architectural backbone. Tschanen et al. (2023) introduced **CLIPPO**, which processes both natural images and images of rendered text using a shared vision encoder. CLIPPO is pretrained through a multimodal contrastive learning objective, which forces the model to associate images and short description pairs, similar to the CLIP model (Radford et al., 2021). **PIXAR** (Tai et al., 2024) implements an end-to-end patch-level autoregressive generative model consisting of two stages. In the first stage, the model is pretrained using per-pixel maximum log likelihood estimation objective. The second stage implements a patch-level adversarial objective to improve readability of visual predictions. For inference, a lightweight Tesseract OCR module (Smith, 2007) maps the generated visual representation back to discrete, machine-processable text. This approach was extended to a multilingual setting with **MIXAR** (Hu et al., 2026), which is pretrained on eight languages, improving performance on multilingual downstream tasks.

10.3 Combining Visual Representations with Tokenizers

The reliance of purely visual language models on OCR modules to convert output images back to text can be seen as an unnecessary lossy step, considering the need for textual output from the system which is not subject to the constraints of unknown inputs. *Hybrid* models combine visual and subword tokens, allowing the direct generation of text. Early work in this direction explored hybrid sequence-to-sequence encoder-decoder architectures for machine translation tasks, where the encoder processes image-rendered source text and the decoder predicts the corresponding English translation using a traditional subword text decoder. Salesky et al. (2021) proposed to use visual text representations only for the source language, with a subword-based decoder generating the target language. Their model outperformed BPE-based baselines on perturbed German-to-English translation, showing increased robustness against source-text noise. Salesky et al. (2023) expanded this work to multilingual settings and demonstrated that visual representations provide a data-efficient solution with increased crosslingual transfer learning performance on unseen scripts.

Aside from language modeling and other generative applications, DiVRit (Elboher & Pinter, 2026) showed that a hybrid visual encoder can be used to provide representations in a scoring framework, namely for ranking candidates in a Hebrew diacritization model.

Gao et al. (2024) extended PIXEL with a subword tokenizer-based decoder and pretrained the model by teaching both encoder and decoder to predict masked text, which resulted in substantial performance gains. Chai et al. (2024) pretrained **PixelGPT**, a decoder-only hybrid model using next-patch prediction and next-token prediction objectives simultaneously.

Lotz et al. (2025b) introduced a pixel-level fallback module, which allows LLMs to handle scripts unknown to their tokenizers, avoiding both `<UNK>` tokens and byte-level fallbacks. Pixel-level fallback is architecture-agnostic, meaning it can be plugged into encoder-only, decoder-only, and encoder-decoder models. See Figure 10.1b for an illustration. The proposed pixel fallback mechanism outperformed alternative approaches such as byte-level processing and vocabulary expansion (see Section 12.1).

10.3.1 Long-Context Processing with Visually Rendered Text

Visual text representations have also been applied to the problem of long-context processing. In this setting, a long-context document is split into smaller chunks, each of which is rendered into visual text, which is then processed by a lightweight vision encoder. Since the vision encoder is not bound by a subword vocabulary and can tune the patch size and other text rendering parameters, it can produce fewer tokens than standard text-only LLMs, allowing the model to operate on much longer contexts at lower computational cost.

Xing et al. (2025) introduced a parameter-efficient, vision-centric token compression framework (**VIST**) which enables long-context processing for LLMs by combining visual and text tokenization, shown in Figure 10.1c. The input text is split into a local section and a long-context section. The local section consists of the most recent k text tokens as produced by a traditional subword tokenizer, while the long-context section contains all text prior to it. The local section’s subword tokens are passed directly to a language model, whereas the long-context text is visually rendered. Then, another visual component, the **Perceiver Resampler** (Alayrac et al., 2022), takes the variable-length image encoding as input and produces a fixed-length latent representation as output, which is fed into the downstream language model via a cross-attention mechanism with the local text’s embeddings. To ensure that the visual representation of the long context is aligned with what a language model would have produced, during training, a pooled embedding of the visual representation is compared with a pooled embedding of the long-context when encoded with a subword tokenizer and embedded by the language model. To further reduce the context length and improve performance, a heuristic token filtering strategy removes unnecessary and low-information tokens from the context.

Li et al. (2025a) applied text-as-image input to vision-language models (VLM) in a zero-shot manner, where visually rendered long context is fed into the vision encoder as a single image. This approach performs on par with text-only LLMs on retrieval and summarization, while significantly reducing the number of tokens. Building upon this idea, **Glyph** (Cheng et al., 2025) adapts a pretrained VLM to long-context text processing tasks by applying continual pretraining with three objectives: optical character recognition, interleaved image-text processing with rendered text, and text generation with partially rendered linguistic context. **DeepSeek-OCR** (Wei et al., 2025) further investigated the optical compression ratios for visually rendered text through an end-to-end pretrained OCR model, recovering the original text at 97% accuracy with a $10\times$ compression ratio.

10.4 Open Challenges with Visual Text Representations

While visual text representations mitigate some issues related to text tokenizers, they nevertheless introduce new problems beyond those of subword tokenization. For one, visual language models need input to be segmented, introducing the question of whether to minimize or maximize the splitting of words across image patches. It is unknown whether keeping inputs intact or decomposing them into base glyphs improves model performance (Lotz et al., 2023; Salesky et al., 2023; Tatariya et al., 2024). Another issue is that there is no fixed, recoverable output after Unicode has been rendered into visual text. This is because the final visual text representation has no record of which code points produced it: recovering that mapping requires either capturing it at render time or reconstructing it with OCR (Zhang et al., 2021; Blecher et al., 2023; Siebensschuh et al., 2025). While traditional text tokenizers have a canonical surface form that is recoverable by the DETOKENIZE function, there is no unique pixel-level representation of a glyph even for a fixed string and font, and identical glyph outlines rasterize differently depending on the engine, target resolution, sub-



Figure 10.5 – Effect of rasterization on character legibility across pixel size and rendering configuration. The sample string combines the Latin-script W and V, which can be ambiguous when placed next to each other; the Devanagari व (U+0935) and ब (U+092C), an easily confusable pair; and the Chinese characters 我 (U+6211) and 找 (U+627E), which have high stroke counts and are difficult to fit in a small cell. Rows show the combinations of anti-aliasing and hinting toggled on and off. At low pixel sizes, fine glyph features approach or exceed the spatial-frequency limit that can be reliably represented by the pixel grid, leading to feature loss (Stamm, 2011).

pixel positioning, and locale-specific rules that are not encoded directly into text (Stamm, 2011). The choice of the exact rasterization process can also cause some glyphs to become illegible or visually ambiguous due to rendering artifacts, as shown in Figure 10.5. Furthermore, font availability can also cause silent failures: when no available font contains a required glyph, the renderer may replace it with a generic missing-glyph symbol rather than report an error.²⁵

There is also the fundamental problem of ambiguity in the many-to-many mapping between code points and glyphs (Davis, 1999; Hadley, 2023). For example, ligatures and emojis fold multiple code points into a single glyph, a single code point can expand into several glyphs, and invisible characters (e.g., the zero-width joiner) modify their neighbors but are not directly visible to visual language models. Traditional tokenizers covered elsewhere in this survey do not suffer as badly from these problems because, for example, the visually identical Latin o (U+006F) and Cyrillic o (U+043E) can be disambiguated by inspecting the byte encoding of each, which is available to text encoders but not to visual language models. Likewise, while the zero-width joiner is not directly visible after rendering, in a traditional text-processing pipeline, its code point would still be included in the Unicode code point sequence. Thus, while traditional token-level representations may obscure the internal characters and orthographic structure of tokens (Section 5.2), visual representations can instead collapse symbolically distinct strings that happen to render identically but which would have been disambiguated by the Unicode code point sequence information.

Finally, it is not clear how to losslessly convert visual text representations back to discrete, symbolic text units. Thus, it is easier to design encoder-only visual language models, as decoder models require the use of a subword token output layer or a lossy OCR system to recover the text from a generated image.

10.5 Questions

Visual text representations are processed as a sequence of fixed-sized image patches, which causes problems with input segmentation. How can we design an input representation and architecture that allows for dynamically sized patches that depend on the inputs?

²⁵This can be considered analogous to a traditional text tokenizer’s use of the `<UNK>` token.

Visual language models suffer from a similar issue to subword models that use transliterated encodings (Section 5.3): encoding text is easy, but generating text is difficult. One strategy is to mix a visual input with a (subword tokenized) text output. What features of a visual encoding and tokenizer design would benefit such a setup?

11 Tokenizers in Modern LLMs

Tokenization is a foundational component of modern language models, yet it receives comparatively little attention in state-of-the-art model development. In this section, we survey tokenization practice through two complementary lenses: the state-of-the-art models currently achieving the highest performance on public leaderboards, and the broader ecosystem of models available through Hugging Face. We additionally cover nonstandard design decisions taken by some modern LLM systems for incorporating tokenizers in practice and the implications of these and other tokenizer-related decisions for the end user.

11.1 State-of-the-Art LLMs

For our study of tokenizers used in practice by state-of-the-art models, we consider the top-rated models on *Arena*²⁶ and identify the model families and their tokenizers. The results in Table 11.1 show that BPE remains the near-universal choice, with vocabulary sizes typically in the 100–275k range, reflecting the increasing multilinguality of modern models.

Model Family	Type	Vocab Size	Longest Alphabetic Token
Gemini 2/3, Gemma 3/4	BPE	262k	ylmethylsulfanyl
Qwen 3.5–3.7	BPE	248k [†]	latesAutoresizingMaskIntoConstraints
Muse/Llama 4 (Meta)	BPE	202k	abcdefghijklmnopqrstuvwxyz
GPT-4o/GPT-5.x/o3/Inkling	BPE	200k	abcdefghijklmnopqrstuvwxyz
MiniMax M3	BPE	200k	scriptscriptstyle
Kimi K2–K3	BPE	163k	fontawesomeextension
LongCat 2.0/Flash Chat	BPE	163k	InvalidArgumentException
GLM-5.x	BPE	154k [†]	latesAutoresizingMaskIntoConstraints
Qwen 3/MiMo V2–V2.5	BPE	151k [†]	latesAutoresizingMaskIntoConstraints
GLM-4.6/4.7	BPE	151k [†]	latesAutoresizingMaskIntoConstraints
Mistral Medium 3.5/Nemotron 3	BPE	131k	Chartplatzierungen
DeepSeek V3.2/V4	BPE	128k/129k	superscriptsubscript
Hunyuan Hy3	BPE	120k	IllegalArgumentException
Grok 4.5	BPE [?]	250k [?]	(closed)
Grok 4.3	BPE [?]	200k [?]	(closed)
Dola Seed 2.0 (ByteDance)	BPE [?]	155k [?]	(closed)
Amazon Nova	BPE [?]	128k [?]	(closed)
ERNIE 5.0 (Baidu)	BPE	128k [?]	(closed), abcdefghijklmnopqrstuvwxyz [?]
ERNIE 5.1 (Baidu)	BPE	90k [?]	(closed), enthusiastically [?]
Claude 3–4.6	Shortest Path [?]	49k [?]	(closed), telecommunications [?]
Claude 4.7+	Shortest Path [?]	16k [?]	(closed), characteristics [?]

Table 11.1 – Overview of tokenizers used by major language model families, showing vocabulary sizes, tokenization methods, and characteristic longest alphabetic tokens. Entries marked with [?] indicate uncertain or estimated values based on maximum token IDs returned by tokenization endpoints (Grok, Dola Seed), similarity to existing tokenizers determined using a token count endpoint (Muse Spark), reverse engineering (Claude), or compression estimates based on token counts (Amazon Nova, ERNIE). [†] indicates tokenizers that retain $\geq 96\%$ of tiktoken’s cl100k vocabulary by content.

Limited public information is available about the tokenizers of several frontier models, including xAI’s Grok 4.x, Anthropic’s Claude 3–4.6 and 4.7+, Amazon’s Nova, and ByteDance’s Dola Seed 2.0.²⁷ Among these, most appear to use a standard tokenization approach but either lack a technical report or omit tokenization details from it. The ERNIE 5.x models use a closed tokenizer, but their technical report describes the use of BPE with BPE-dropout (Wang et al., 2026). Google’s Gemini and Meta’s Muse Spark models do not divulge

²⁶<https://arena.ai/> in the text/overall category, with rank ≤ 100 as of July 27, 2026.

²⁷On request for information: xAI and Baidu did not respond, Amazon Support forwarded the inquiry internally but received no reply, Anthropic responded that they “cannot comment at this time”, and the email address that ByteDance lists for Dola Seed inquiries does not exist.

explicit public information about their tokenizers, but have sibling open-weight models that share the closed models’ tokenizers.

Claude 3+ remains the least documented, with no official information disclosed about its tokenization approach. There have been attempts to reverse-engineer the Claude tokenizer based on information revealed in Anthropic blog posts as well as analysis of token counts provided by their API. Its behavior appears inconsistent with standard BPE but consistent with shortest-path tokenization. In addition, it appears to use “shift” and “caps” tokens (Lindsey et al., 2025a), as well as word-boundary markers. Its vocabulary size is estimated at 49k in Claude 3–4.6, and 16k in Claude 4.7+ (Land, 2026a).

All models are heavily multilingual, and all except ERNIE use UTF-8 encoding. ERNIE uses UTF-16BE, which has better byte efficiency for some multilingual data—in particular for CJK characters, assigning most of them two bytes compared to three in UTF-8 (Wang et al., 2026). Among the surveyed frontier models, only Google’s Gemini/Gemma models use a byte fallback as implemented in SentencePiece. Most use standard byte-based BPE encoding (Wang et al., 2020a), which allows tokens that include partial character encodings. Some tokenizers avoid almost all such partial UTF-8 tokens. Qwen 3.5+ and Claude contain no tokens that join a complete, non-ASCII character with a partial UTF-8 encoding of another character. While they contain some partial-character tokens, they consist only of isolated bytes or prefixes of a single UTF-8 character, though Qwen 3.5+ has a small exception set of space-prefixed UTF-8 prefixes.

	Family	Models	Share	Downloads
1	Llama 3.1	29,332	6.9%	356M
2	Qwen 2.5	19,704	4.7%	911M
3	Mistral 7B v0.1	13,970	3.3%	105M
4	BERT-uncased	13,091	3.1%	6480M
5	Llama 1/2	12,760	3.0%	96M
6	Qwen 3	11,770	2.8%	464M
7	BERT-uncased (truncation variant)	11,226	2.6%	117M
8	Llama 3	10,656	2.5%	39M
9	GPT-2	6,608	1.6%	1196M
10	Gemma 3	6,580	1.6%	96M

Table 11.2 – The 10 most replicated tokenizers on the Hugging Face Hub and their number of downloads.

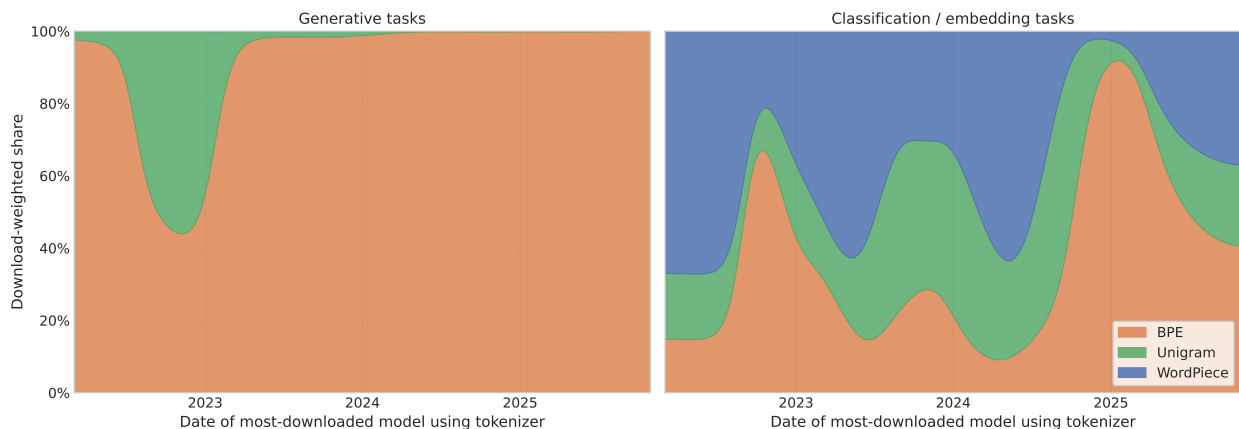
In order to measure how tokenizer design decisions propagate beyond the frontier, we conducted a census of all tokenizers on the Hugging Face Hub. To that end, we analyzed tokenizers using the “fast” tokenization backend of the `transformers` library, whose `tokenizer.json` serialization fully specifies the tokenization pipeline. Querying the Hub API for all public models tagged with either `library:transformers` or `library:tokenizers` yielded a total of 739,147 candidate models as of 2025-11-03, of which 423,650 (57.3%) could be loaded. Hashing each serialized tokenizer identified 24,798 distinct tokenizers among them and 94% of surveyed models use a tokenizer that is byte-identical to another model’s. We release the resulting dataset under the MIT license (Akiki et al., 2026).²⁸

Frontier tokenizers propagate widely. Sixteen tokenizers that are byte-identical to those in Table 11.1 at the time of the census scrape account for 19.8% of all census models and 7.1% of all-time model downloads.²⁹

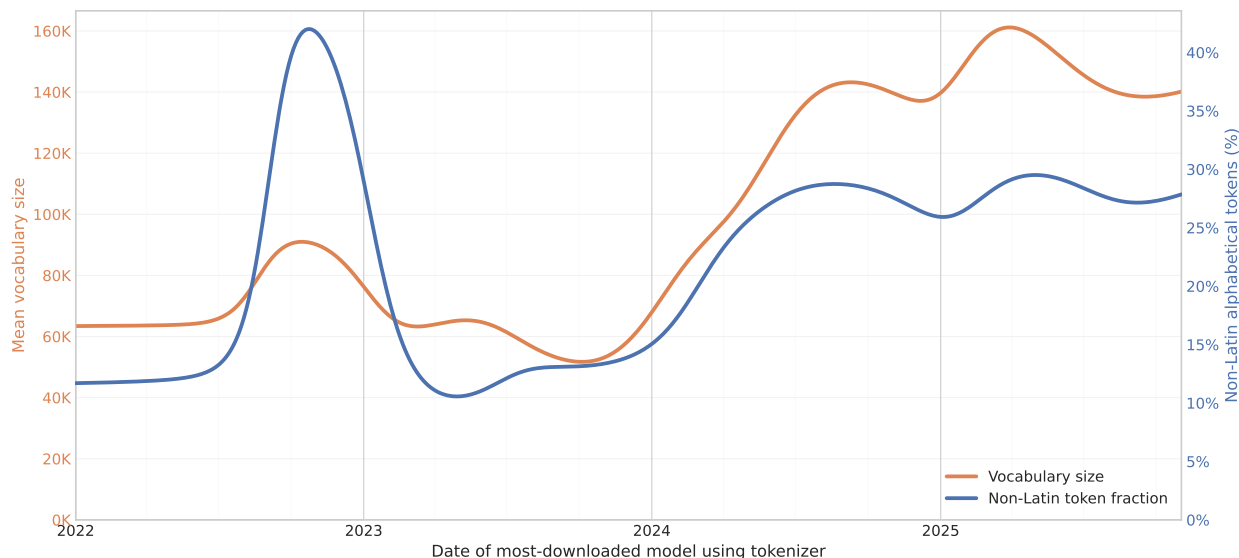
Tokenizer reuse is heavily concentrated, with the ten most replicated tokenizers shipping with 32.0% of all models, as shown in Table 11.2. The single most replicated tokenizer is Llama 3.1, which accounts for 6.9% of all census models. When ranked by downloads, the top 10 most downloaded tokenizers account for 65.9% of the 27.9 billion all-time downloads in the census, with the BERT_{base-uncased} tokenizer accounting for 23.2%. The most-replicated tokenizer is therefore a 2024 frontier design, while the most-downloaded is one from 2018, suggesting that newer models frequently adopt current designs.

²⁸<https://hf.co/datasets/tokenizer-eval/tokenizer-census>

²⁹This is a lower bound. Several frontier tokenizers cannot be analyzed directly. Kimi K2 and Hunyuan-Large ship custom tiktoken-format tokenizers (e.g., `hy.tiktoken`) with no `tokenizer.json`, and closed models are excluded outright.



(a) Download-weighted share of tokenizer algorithms over time, split by task category. *Left*: generative tasks (text generation, summarization, translation, and text-to-text generation). *Right*: classification and embedding tasks. Curves show Gaussian-smoothed ($\sigma = 60$ days) shares of all-time Hugging Face downloads. Models sharing an exact tokenizer configuration are aggregated and located at the creation date of their most-downloaded associated model. BPE dominates most of the 2023–25 generative trajectory. The late-2022 Unigram peak is driven chiefly by the *Flan-T5* family (Chung et al., 2022). In classification and embedding, WordPiece remains popular through the continued use of BERT-family tokenizers (Devlin et al., 2019).



(b) Download-weighted mean vocabulary size (orange, left axis) and non-Latin alphabetical-token fraction (blue, right axis), smoothed with a Gaussian kernel ($\sigma = 60$ days). As in Figure 11.1a, tokenizer configurations are weighted by all-time downloads and located at the creation date of their most-downloaded associated model. Vocabulary size is computed over all included text-task configurations. Vocabulary sizes increase substantially after 2023. The non-Latin fraction reaches a local maximum in late 2022, driven primarily by the *BLOOM* tokenizer (Workshop et al., 2023). It declines during 2023 and rises again from 2024 as *Qwen*-based models, and later *DeepSeek* models, make highly multilingual vocabularies more prominent.

Figure 11.1 – Trends in tokenizer configurations from 2022 to 2025 across 17,178 unique tokenizer hashes from text NLP models on the Hugging Face Hub, weighted by all-time model downloads.

Grouping only byte-identical tokenizers obscures the fact that many tokenizers are only slight modifications of others. This is especially common in frontier models, where models often share the same base tokenizer and vocabulary but have a different set of special control tokens (see Section 11.2). Treating such variants in Table 11.2 as members of the same tokenizer family would merge the *Llama 3* and *Llama 3.1* tokenizers (which differ by 256 control tokens), the *Qwen 2.5* and *Qwen 3* tokenizers (which differ by 4 control tokens), and the two BERT tokenizers (one of which has a vocabulary that is a subset of the other’s).

Figure 11.1 shows the divide between tokenizer choices for generation and for classification: BPE dominates generative decoder models after 2023, while WordPiece remains important in BERT-style encoders used in classification. The lower panel shows a parallel change in vocabulary composition: mean vocabulary size rises substantially after 2023, alongside a growing share of non-Latin tokens in widely downloaded vocabularies. This reflects the broader rise of multilingual model families, with Chinese labs contributing notably to the later increase.

11.2 Special Uses of Tokenizers in LLMs

Modern LLMs rely on their tokenizers to carry information beyond raw text. This is accomplished primarily through **control tokens**: reserved tokens that mark structural boundaries (start/end of a sequence, user/system roles, etc.) or trigger specific model behaviors (tool calls, reasoning blocks, role transitions, etc.). Control tokens also form the basis for **chat (prompt) templates**, which assemble structured multi-turn conversations including the system instructions, user inputs, and model responses, into a single tokenizable string in the form that the model is trained to process (Burtenshaw et al., 2025).

Control Tokens Language models often reserve special token IDs for structural markers that do not behave like ordinary text. These *control tokens* may mark the beginning or end of a sequence, separate segments of an input, identify speaker roles, or trigger model-specific behavior. Although they are often displayed using textual surface forms such as `<s>` and `</s>` (the beginning- and end-of-sequence tokens), they should correspond to special token IDs rather than to the ordinary tokenization of their surface forms. For example, the control token `<s>` should be distinct from the tokenization of the literal string `<s>` (which may produce `<` `s` `>`).

BERT popularized the use of a `[CLS]` token, which was commonly used as an aggregate representation of the input for downstream classification tasks, and the `[SEP]` token for separating parts of the input for the next-sentence-prediction pretraining objective (Devlin et al., 2019). More recent LLMs also use control tokens to delimit chat roles (covered below), reasoning blocks (which a provider may want to hide from the user, so it is helpful to fence them), and specialized tool calls and responses.

Control tokens can be added as raw text to the input during pretokenization by designing the tokenizer to parse these tokens in isolation, bypassing any other pretokenization considerations or side effects, so that the desired control token ID is produced. Alternatively, they can be added to the tokenized sequence as a “post-tokenization” step, by tokenizing the non-control text spans separately and then inserting the desired control tokens into the tokenized sequence at the correct locations. This method avoids any surface form ambiguity between regular text and control tokens, but finding the correct insertion points can be challenging as normalization and pretokenization steps can alter the original raw text.

Prompt templating is a common technique in modern LLM pipelines for separating text that defines how the model should act (the *system prompt*), the user’s input(s) (the *user text*), and the model’s response(s) (the *assistant text*), as well as for representing other structures such as tool use and reasoning blocks. These inputs are often serialized in a JSON format for structure, but are converted to text by a chat template into the exact prompt format expected by the model. The template inserts the necessary control-token surface forms, after which the rendered prompt is passed to the tokenizer.

One issue with using chat templates to convert structured inputs into raw text is that user-provided text may contain strings that resemble prompt-template control tokens, potentially altering the behavior of the model if they are interpreted as template structure rather than ordinary text. Malicious actors can exploit this by manipulating or forging template tokens to bypass safety mechanisms or hijack agent behavior via indirect *prompt injection* (Jiang et al., 2025; Chang et al., 2026).³⁰ Thus, special care must be taken to ensure that user-supplied text that shares a surface form with control tokens is interpreted correctly.

In Example 11.2, not only must the special delimiters `<bos>`, `<|turn>`, and `<turn|>` be interpreted correctly as their own tokens, but also potentially the identifiers `system`, `user`, and `model`, as these are common words that are often used in other contexts.

³⁰We cover security aspects of tokenization more in Section 14.3.

Example	
System Prompt	You are an expert chef. Answer questions about cooking.
User Input	I'd like to make spaghetti aglio e olio. What is a good recipe?
Assistant	Here is a simple recipe for spaghetti aglio e olio...
Formatted Prompt	<pre> <bos>< turn>system You are an expert chef. Answer questions about cooking.<turn > < turn>user I'd like to make spaghetti aglio e olio. What is a good recipe?<turn > < turn>model Here is a simple recipe for spaghetti aglio e olio... </pre>

Example 11.2 – An example using the Gemma 4 chat template. While `<bos>`, `<|turn>`, and `<turn|>` are tokenized as special control tokens, the tokenization of `system`, `user`, and `model` all match that of when these words are tokenized as part of regular text (i.e., they are not treated as special identification control tokens), despite carrying a special meaning.

11.3 Tokenization and Language Model Inference Cost

With the rise of *pay-per-token* LLM-as-a-Service APIs, tokenization now has a direct and tangible impact on users. The cost per token depends on factors such as the model used, with larger and more capable models charging more, and the length of the input and output, with output text typically costing more per token. Some providers also charge a higher per-token rate once a prompt crosses a length threshold.³¹ Tokenization thus directly influences cost: a tokenizer that compresses text better will, all else held equal, produce fewer tokens and cost less. However, this disregards other downstream aspects such as verbosity, effort/thinking level, and per-token quality, since more compressed tokenization may not necessarily yield gains in these areas.

Ahia et al. (2023) analyzed 2023 versions of ChatGPT and found that speakers of lower-resourced languages are charged more for the same information content, due to a combination of worse tokenization inflating the number of tokens and lower-quality language modeling from a lack of data in those languages. There was also a disparity across scripts, where lower-resourced Latin-script languages had better performance and lower cost compared to higher-resourced, but non-Latin scripts, such as the Indic-script families (though this was not universally true, as Chinese, Japanese, and Korean had a lower disparity from English) due to the lower fragmentation of Latin-script languages.

Lundin et al. (2025) explored token fertility (see Section 7.2) as a predictor of LLM quality and found that low-resource languages with poor tokenization support (in the form of higher fertility) lagged behind higher-resourced languages in modern LLMs (circa 2024). The authors named this disparity the *token tax*. At training time, lower-resource languages are taxed both by worse performance resulting from the smaller quantities of training data available for them and by the increased computational resources required to train models on languages with higher fertility. At inference time, it costs more to generate the same semantic information in a high-fertility language. However, the authors also note that the gap between high- and low-resource languages has narrowed with the advent of modern reasoning models.

Velasco et al. (2025) noted that, because LLM-as-a-Service APIs return results as raw, detokenized text but there are many ways to tokenize the same surface form (see Section 8), language model providers could artificially inflate the output token cost by using non-canonical tokenizations. They recommend decoupling the cost of the API from the tokenization by using pay-per-character rather than pay-per-token metering. In a similar vein, Chatzi et al. (2026) observed that artificially higher token costs resulting from non-canonical tokenization can often happen simply as a consequence of the probabilistic nature of language modeling. They

³¹For example, as of mid-2026, Google’s Gemini 3 Pro API charged a roughly double per-token rate for prompts exceeding a 200k-token threshold, with the higher rate applied to the entire request rather than only to the tokens past the threshold.

measured the frequency of a model generating text non-canonically for a given input, as well as the relative increase in tokenization length compared to the optimal tokenization. Across models and languages, they observed a non-negligible likelihood of non-canonical tokenizations being returned by the model, increasing for lower-resourced languages, which again acts as an additional tax on low-resource language speakers.

As a partial solution to token-cost auditability, several proprietary model providers also offer a token API,³² which allow users to measure the canonical token cost of inputs.

11.4 Tokenization and Scaling Laws

Although tokenizers are typically separated from the language model architecture, they have a crucial impact on LLM parameter counts and computational budgets, as the vocabulary size influences the model’s parameter count through the embedding and unembedding layers. For example, in an early multilingual model, XLM-RoBERTa_{base} (Conneau et al., 2020), vocabulary embeddings accounted for more than 70% of the total parameters. Additionally, the compression rate of tokenizers controls the length of the text input sequence, thereby affecting its computational cost during training and inference, as described in Section 11.3. Furthermore, many aspects of the training process, such as the batch size and therefore the number of gradient updates for a corpus, depend on tokenization, and *scaling laws* are often expressed in units of tokens.

These aspects show the need to account for tokenization and compression when designing reliable scaling laws. For example, Pearce & Song (2024); Porian et al. (2024) pointed out that the decision of whether to include vocabulary embeddings in the scaling recipe was one contributor to the discrepancy between the seminal scaling works of Kaplan et al. (2020) and Hoffmann et al. (2022).

Notable recent works have focused on the role of tokenization in model scaling. Tao et al. (2024) focused on the trade-off between embedding layer size (which increases the compute required for the final output softmax layer) and data compression rate (which decreases the compute required by reducing the sequence length). They derived a scaling law for the optimal vocabulary size given a compute budget, and showed that this optimal vocabulary size should increase with model scale. In line with these results, Takase et al. (2025) observed the benefits of large vocabularies on downstream tasks in English and Japanese, although they do not fit a scaling law.

Limisiewicz et al. (2026) generalized scaling laws across tokenizers, including latent tokenizers (see Section 9.2), by producing tokenizations across a broad range of compression rates. They showed that the optimal data-to-model-size ratio holds across tokenizers when expressed as *bytes* per parameter instead of *tokens* per parameter (Hoffmann et al., 2022). Moreover, their scaling analysis revealed the existence of an optimal compression rate, whose value depends on the training compute budget and domain. Optimal compression decreases at larger scales, indicating that large models benefit from smaller vocabularies, which contrasts with the findings of Tao et al. (2024). The authors attribute this discrepancy to the diminishing contribution of embeddings at larger scales and to the hypothesis that less-compressive tokenizers allow the model to use more compute at inference time by dividing each evaluation sample into more tokens.

11.5 Questions

Moving from a pay-per-token to a pay-per-character pricing mitigates poor tokenization (up to the downstream model quality) as a tax on users. However, it then penalizes information-sparse languages (e.g., English and other Latin-script languages) relative to information-dense languages (e.g., Chinese), and using pay-per-byte would unfairly penalize languages based on their UTF-8 encoding. Can we work on crosslingual tokenizer parity to design fair, token-based payment schemes?

³²Notably Claude, whose tokenizer is not publicly released, so the token-counting API is the only way to directly query tokenization metadata.

A tokenizer’s vocabulary is a finite resource, so improving compression for one language or domain can come at the expense of another. What should a tokenizer optimize for across languages? Equal token counts may be the wrong target: languages differ in numbers of users and speakers, prevalence as working languages, and actual workload composition; code forms a substantial share of many workloads; and model-generated reasoning is often disproportionately in English. How should these factors be weighted in allocating tokenizer capacity? (Consult Section 6.)

For pay-per-token systems, users may want to audit spending. However, providing finer-grained token details (such as the tokenization sequence) can leak information about the model and training data (Section 14.3). In what other ways can LLM-as-a-Service providers expose verifiable token information so that users can be confident they are not being overcharged?

12 Tokenizer Modification and Cross-Tokenizer Knowledge Transfer

Large language models are usually trained with a fixed tokenizer determined at pretraining time. Users of such LLMs are therefore “stuck” with this choice of tokenizer for inference, even though it might not be well suited for downstream use cases. This can happen due to the fundamentally different pressures that act on pretraining and inference. At pretraining time, downstream tasks may be unknown, so it is beneficial to optimize for amortized encoding efficiency over generic text. At inference time, however, a specific domain is known, and the tokenizer might not handle it well. Efficiently encoding this domain is important: excessive over-tokenization increases computational costs through longer sequence lengths (see Section 6.1, Ahia et al., 2023; Yamaguchi et al., 2024) and can potentially harm downstream task performance (Rust et al., 2021; Ali et al., 2024). Additionally, tokenizer incompatibility forms a barrier between LLMs. Methods such as representation-level knowledge distillation (Hinton et al., 2015), speculative decoding (Leviathan et al., 2023), model merging (Ilharco et al., 2023), and token-level ensembling (Mavromatis et al., 2024; Wang et al., 2024a) typically assume equivalent tokenizers. This fractures the ecosystem of LLMs by limiting these forms of interaction across tokenizer boundaries.

To solve these problems, various approaches to modify the tokenizer of already-trained LLMs have been developed. These may be limited to adding new tokens to an existing tokenizer’s vocabulary to better represent specific languages or domains, or go as far as completely replacing the tokenizer, i.e., **tokenizer transfer**. This may include both vocabulary changes and modifications of the tokenizer’s *inference algorithm*.

Another method for bridging the gap between models that have different tokenizers involves directly sharing knowledge across the models. Here, the goal is to bypass explicit tokenizer transfer and develop **knowledge transfer** training objectives which take a potential tokenizer mismatch into account.

12.1 Vocabulary Modification

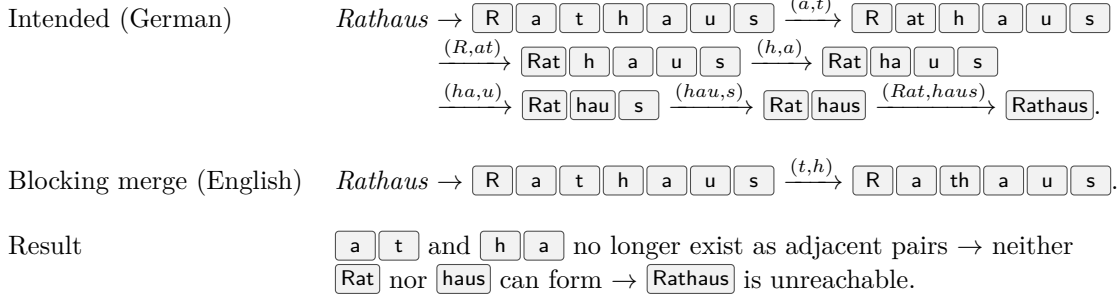
Adapting a model to a new domain or language can require modifying the vocabulary to better match the target, as the original vocabulary might be suboptimal for the new domain for reasons such as specific terminology or a previously unseen or underrepresented language. A common approach is to extend the original vocabulary with domain- or language-specific tokens. One can also trim the original tokenizer or create a custom mix of old and new vocabularies to avoid increasing the proportion of parameters devoted to vocabulary embeddings. However, vocabulary modification is not always trivial, as it should preserve the original tokenizer structure. For instance, in BPE-like tokenizers, it is important that the modification is compatible with existing merges so that it preserves original tokenizations where possible and does not create unreachable tokens, especially for the domains other than what is being targeted for extension.³³

The most common approach focuses on extending a base vocabulary with language-specific tokens (Sachidananda et al., 2021; Yao et al., 2021; Delobelle et al., 2022; Csaki et al., 2024; Cui et al., 2024; Fujii et al., 2024; Lin et al., 2024; Tejaswi et al., 2024; Yamaguchi et al., 2026). For BPE, this implies keeping the original vocabulary in full and training a new BPE tokenizer on the target language or domain. The two tokenizers are then merged by adding the new tokens from the target tokenizer to the original one. Sachidananda et al. (2021) also suggested a procedure for selecting domain-specific sequences based on token-frequency distributions. Other similar methods include trimming or filtering the original tokenizer first and adding the new tokens to fill the remaining space or achieve the desired final vocabulary size (Csaki et al., 2023; Kajiura et al., 2023; Kiulian et al., 2025).

This extension approach has been shown to improve model performance on the target language or domain while retaining the original model’s capabilities. However, care must be taken: for BPE, expanding the vocabulary by simply concatenating merges for new subwords to the old tokenizer’s merges may not have any effect at all, with the new tokens being blocked by the original merges (Bauwens, 2023, §4.9). In the example below, the original English token `th`, which typically appears very early in the merge list, blocks the newly added German token `Rathaus`.

³³See, for example, some of the vocabulary modification techniques in Section 3.1.3.

Example



Several modifications have been suggested to avoid this problem. [Nakash et al. \(2025\)](#) replaced low-frequency tokens with high-frequency, domain-specific n -grams of existing tokens and added a tokenization patching step that scans for occurrences of new tokens. [Balde et al. \(2024\)](#) used FLOTA inference (see Section 3.6) for the newly added tokens to find the longest subword that is contained in the original vocabulary and then add remaining merging rules. [Balde et al. \(2025\)](#) first tokenized the new vocabulary token with the old tokenizer and add all necessary scaffolding merges to ensure these tokens are reachable in the resulting tokenizer.

[Purason et al. \(2026\)](#) suggested *continued* tokenizer training for domain or language adaptation. This entails extending a BPE vocabulary by simulating the continued training of the original BPE tokenizer, but on domain-specific data. Optionally, the original vocabulary is first pruned to free space for new tokens. To preserve the BPE structure, only **leaf tokens**, which do not appear in a larger merge, are chosen as candidates for pruning. This procedure ensures the merge list structure remains consistent and does not produce unreachable tokens.

Vocabulary modification can be applied selectively across a language model’s components. [Hong et al. \(2024\)](#) adapted only the *output* vocabulary for more efficient generation and retokenized generated tokens before feeding them back to the model. [Huang et al. \(2025\)](#) kept the original BPE tokenizer fixed, but expanded the effective *input* vocabulary via large, hashed n -gram embedding tables. This consistently improved model performance at fixed compute.

12.2 Embedding (Re-)Initialization

Adapting the vocabulary typically also involves modifying the embedding and unembedding layers of the model to account for the new tokens. One natural solution is to randomly initialize the new embeddings and conduct a “warmup” training phase in which all weights except for the embedding matrices are frozen ([Lampinen & McClelland, 2018](#); [de Vries & Nissim, 2021](#)). [Marchisio et al. \(2023\)](#) showed that a shallower “mini-model” aligned with the target model can be used to make this phase more efficient. [Pfeiffer et al. \(2021\)](#) factorized pretrained embedding matrices into low-dimensional token representations and shared projections intended to capture crosslingual structure, allowing new languages to reuse the projections while learning new compact language-specific embeddings.

Rather than randomly initializing the new embeddings, one could include information from the pretrained weights by initializing the new embeddings based on the per-channel mean and standard deviation of the original embeddings ([Hewitt, 2021](#)). [Downey et al. \(2023\)](#) limit computation of these statistics to tokens sharing the same script. However, this strategy does not yet yield a *semantic initialization*: the resulting embeddings do not carry any particular meaning of their respective tokens. Instead of averaging over all embeddings, other works average only over the *subtoken* embeddings of each new token—that is, over all tokens that the new token would have been split into under the original tokenization ([Koto et al., 2021](#); [Sachidananda et al., 2021](#); [Gee et al., 2022](#)). If we were to add a new token `tokenization`, which would have been split into subtokens `token``ization` in the prior tokenization scheme, we can reasonably expect the individual subtoken embeddings to encode information that compose together to form the meaning of the full token.

One can consider the multiple possible partitions of a new token under the original tokenizer (Mosin et al., 2023; Yamaguchi et al., 2026), or assign different weights to subtokens based on their position (Nakash et al., 2025). However, these approaches can fail to fulfill their semantic purpose in cases such as languages which are heavily over-tokenized or even split into byte-level tokens. Additionally, while compositional semantics can be expected of some subtoken combinations, this will not always be the case for all new tokens: for example, an embedding for `palatable` derived from the pooled embeddings of `pal` `at` `able`, has no guarantee of an encoding a faithful semantic representation.

To avoid the limitations of unsupervised compositional semantics in these cases, several approaches leverage auxiliary resources to obtain better embedding initializations, including bilingual dictionaries, parallel corpora, and external embedding spaces such as `fastText` or another pretrained model’s embedding matrices. Tran (2020) estimated token alignments between old and new tokens on a parallel corpus using `fast-align` (Dyer et al., 2013). Remy et al. (2024) instead first estimated word-level alignments and then calculated (potentially many-to-many) mappings between individual tokens in the words. These mappings were then used to derive coefficients for initializing each new token embedding as a weighted combination of existing token embeddings. Zeng et al. (2023) instead simply “translated” whole-word tokens into new languages using a dictionary.

Alternatively, we can attempt to source embeddings for new tokens from an auxiliary embedding space in which we already have a semantically meaningful embedding, and map them to the new token’s representations in the embedding space of our target model. Initially, these auxiliary embedding spaces were often pretrained language-specific `fastText` embeddings (Bojanowski et al., 2017), but alignment of such embeddings and the embeddings of a pretrained language model was found to be difficult (Søgaard et al., 2018; Wang et al., 2019b). Thus, many works represented each new token as a weighted combination of the embeddings of existing tokens from the pretrained model. In early work, similarity coefficients were obtained by aligning two auxiliary embedding spaces, one in the target language and one in English, and selecting close neighbors of new target language tokens in the aligned English space (Wang et al., 2019b; Tran, 2020; Minixhofer et al., 2022). Ostendorff & Rehm (2023) and Dobler & de Melo (2023) directly mapped from the auxiliary embedding space into the target model’s embedding space using shared tokens as *anchors*, which circumvented instabilities during embedding-space alignment. While Dobler & de Melo (2023) trained cheap `fastText` embeddings for the new tokenizer as an auxiliary space, Ostendorff & Rehm (2023) trained a smaller language model from scratch with the new tokenizer in order to use its embedding matrices as higher-quality auxiliary embedding spaces. Recently, several approaches using such shared anchors proposed using better auxiliary embedding spaces and improved ways of mapping to the target model’s embedding spaces (Liu et al., 2024; Mundra et al., 2024; Yamaguchi et al., 2024; Lee et al., 2025b; Goddard & Neto, 2025; Singh et al., 2025). Rather than explicitly constructing each new embedding as a weighted combination of existing embedding rows, Moroni et al. (2025) learned a single affine mapping from an auxiliary embedding space using shared vocabulary items as supervision. Instead of mapping from an auxiliary embedding space, Han et al. (2025) trained lightweight vocabulary adapters while keeping all other weights frozen to learn the linear combination weights of existing embeddings.

While better initializations have been shown to outperform naïve random initialization approaches, they still require at least a short continued training phase to yield good performance. In contrast, **zero-shot tokenizer transfer** seeks to adapt the tokenizer of a pretrained model without requiring further training. One approach is to train a *hypernetwork* to predict new token embeddings in a pretraining phase involving the entire language model (Minixhofer et al., 2024). After the hypernetwork is trained, it can be cheaply used to generate embeddings for new tokens, yielding significantly better performance. Subsequent work takes parts of the language model parameters into account to arrive at better embedding initializations: Kaplan et al. (2025) proposed a *neural detokenization* process, in which representations of units of meaning are gradually composed from tokens layer by layer. They utilized this by extracting such representations for new whole-word tokens from intermediate layers using *PatchScopes* (Ghandeharioun et al., 2024) and map them back into the input and output embedding spaces. Dobler et al. (2026) similarly argued that the semantics of multi-token words are gradually added to hidden states along the layers and distilled hidden states obtained using the original tokenization into a single embedding for new tokens. They further showed that such (self-)distillation yields better results than simply learning new token embeddings via next-token

prediction. In the same vein, [Haltuik & Smywiński-Pohl \(2025\)](#) matched attention patterns between the original and the new tokenizer. [Minixhofer et al. \(2025b\)](#) more generally framed tokenizer transfer as cross-tokenizer self-distillation. They proposed a novel cross-tokenizer distillation objective and showed that it outperformed next-token prediction training for direct tokenizer transfer as well as training hypernetworks for generating embeddings for new tokens.

A notable blind spot of current embedding initialization approaches is that they typically only consider the vocabulary, although the tokenization also depends on the *inference algorithm*—even for two tokenizers sharing the exact vocabulary, different segmentations of the same text are possible. [Minixhofer et al. \(2024\)](#) *amortize* over different inference algorithms during the pretraining of their embedding-prediction hypernetworks. A different line of work on *non-canonical tokenization* (see Section 8) explicitly studies changing the inference algorithm after training (e.g., [Hofmann et al., 2021b](#); [Bauwens & Delobelle, 2024](#)).

12.3 Knowledge Distillation Across Tokenizers

The fact that different models rely on different tokenizers complicates *knowledge distillation*, which has emerged as an effective technique for transferring knowledge from a larger or more specialized “teacher” model to a smaller “student” model ([Buciluă et al., 2006](#); [Hinton et al., 2015](#)). By directly aligning intermediate feature representations or learning from the output distributions of the teacher, the student model acquires richer information with fewer training examples. The release of powerful families of LLMs developed through distillation ([Gemma Team et al., 2025](#); [Yang et al., 2025a](#), *inter alia*) highlights the crucial role of this technique in modern LLM development. Nevertheless, standard methods for LLM distillation require the teacher and student models to share a tokenizer ([Gu et al., 2023](#)), which is typically only the case for models of the same family.

The standard objective for distillation is to minimize the dissimilarity between the teacher’s and the student’s next-token distributions p_T and p_S , respectively. This dissimilarity is often quantified using measures like the *Kullback-Leibler (KL) divergence* ([Kullback & Leibler, 1951](#)):

$$D_{\text{KL}}(p_T \parallel p_S) = \sum_{t_i \in \mathcal{V}} p_T(t_i \mid \mathbf{t}_{<i}) \log \frac{p_T(t_i \mid \mathbf{t}_{<i})}{p_S(t_i \mid \mathbf{t}_{<i})}. \quad (5)$$

Crucially, KL requires the two distributions to share the same support, i.e., the same vocabulary $\mathcal{V}$, or this loss is infinite.

Cross-tokenizer distillation allows joint supervision of a student language model from many diverse teachers with potentially heterogeneous tokenizers ([Wan et al., 2024](#)). Many cross-tokenizer distillation methods have been proposed, such as exploiting *chunk alignments*, the token sequences produced by multiple tokenizers for the same chunk of text, and *shared vocabulary* tokens, which appear in multiple tokenizers’ vocabularies. Rather than trying to match the student and teacher distributions exactly, **ULD** ([Boizard et al., 2024](#)) applied Wasserstein distance ([Kantorovich, 1960](#)) to align the *shape* of the distributions. As this distance between the *sorted* distributions is being minimized, the student learns to match the teacher’s confidence spread. For example, the student learns not to be overly confident about the next token if the teacher assigned relatively high probabilities to multiple tokens. **GOLD** ([Patiño et al., 2025](#)) combined ULD with KL divergence at aligned sequence positions over tokens in the shared vocabulary. **X-Token** ([Greenivas et al., 2026](#)) addressed failure modes of **GOLD** by conditionally selecting the way to transfer probability mass across vocabulary distributions based on a measure of compatibility between the teacher and the student tokenizers.

To make alignment easier, **MinED** ([Wan et al., 2024](#)) loosened the exact-match alignment requirement by constructing a mapping between lexically similar tokens from the student and teacher tokenizers. Once the mapping is applied, a standard KL divergence loss is then used. **DSKD** ([Zhang et al., 2024](#)) circumvented vocabulary alignment by instead learning projections between the student and teacher hidden states and optimize for reconstructability of the teacher’s predictions from the student hidden states, and vice versa.

Minixhofer et al. (2025b) proposed **ALM**, a pure distillation loss that eliminates the need for a primary objective, such as next-token prediction. ALM applies both tokenizers to a given text. Then, token chunks representing the same string are identified and their probabilities are aligned using a *binarized divergence*.

Phan et al. (2026) used byte-level probabilities (Phan et al., 2025) extracted from the teacher until a natural word boundary is encountered. Then, the byte paths are tokenized using the student’s tokenizer to obtain the target next-token probability. The methods to extract byte- or character-level probabilities introduced by Hayase et al. (2025) and Vieira et al. (2025a) may be used for cross-tokenizer distillation in a similar manner.

Cross-tokenizer distillation is also relevant to tokenizer transfer: as discussed in Section 12.2, tokenizer transfer can be seen as a *cross-tokenizer self-distillation* problem. In this light, the teacher is the model with the original tokenizer, and the student is the same model with a changed tokenizer, and the objective is for the student to exhibit the same behavior as the teacher.

12.4 Questions

What does tokenizer transfer entail in the case of dynamic vocabularies, as appearing in tokenization of images, video, audio, and latent tokenization (Section 9.2)? Do techniques from existing methods transfer?

Are there unintended consequences of modifications to the vocabulary, such as performance loss on domains insufficiently covered by the training data, or degradation of alignment? To what extent does effective tokenizer transfer generalize beyond domains covered by the data used for training?

Language models typically assign probability to tokens agreeing with the canonical tokenization (see Section 2.2). How does the transition of probability mass from the original canonical tokenization (under the original embeddings) to the new canonical tokenization look like during tokenizer transfer training? Does a substantial amount of probability mass stay on the original canonical tokenization? Is it possible to detect the original tokenizer in this way?

13 Theoretical Aspects of Tokenization

Our discussion of tokenization so far has been largely empirical, with tokenization being presented as a series of ad hoc algorithms combined in a pipeline to produce token sequences from text. We turn now to theoretical aspects of tokenization, where we attempt to describe tokenization in terms of computational power and efficiency. Formalizing tokenization allows us to study it independently from the context in which it is being used. It further allows us to probe the limits of current tokenization techniques, which can suggest directions for more efficient or expressive tokenizer designs.

13.1 Tokenization and Automata Theory

The MaxMatch and BPE tokenization algorithms can be expressed in terms of finite-state automata and transducers.³⁴ These are powerful, yet simple and conceptually well-understood, models of computation, so characterizing tokenization in these terms unlocks a rich set of theoretical results.

When considering tokenization through the lens of finite automata, two structures are particularly important. One is the **subword lattice**: a labeled, directed graph encoding the possible tokenizations for a given input (Xiao et al., 2019).³⁵ Subword lattices can be defined for a single string (e.g., *token* as in Figure 13.1), or for arbitrary regular languages (such as in Figure 13.2). Second, the **subword transducer**: a finite-state transducer that maps sequences of characters from the tokenizer’s base alphabet Σ to the tokens they spell out (Koo et al., 2024; Cognetta & Okazaki, 2025). The subword transducer is useful for constructing subword lattices. Given any input represented as a finite automaton, we can *compose* that automaton and the subword transducer to produce the subword lattice. Effectively, the composition operation uses the subword transducer to replace character subsequences in the character-level automaton with tokens from the subword vocabulary that have the same surface form. We call this procedure **subword promotion**, as it *promotes* an automaton representing a string over Σ to an automaton over the subword vocabulary $\mathcal{V}$.

Importantly, both structures above can be constructed to be either tokenizer-agnostic, encoding both canonical and non-canonical tokenizations of the input, or tokenizer-aware, restricted to the canonical sequences a tokenizer would output.

13.1.1 Tokenizer-Agnostic Subword Promotion

While we typically think of tokenization as a deterministic, one-to-one string transformation where the same input sequence always produces the same output token sequence, Section 8 discusses the many ways in which alternative tokenizations of the same input sequence can be meaningful. An efficient way of representing all possible tokenizations is via a subword lattice, exemplified in Figure 13.1. When a subword lattice encodes all token sequences compatible with some input, it is independent of the inference algorithm, requiring only the vocabulary. We thus term it **tokenizer-agnostic subword lattice** here. To build this tokenizer-agnostic subword lattice, we use a subword transducer encoding all possible tokenizations compatible with a certain vocabulary.³⁶ We call this procedure **tokenizer-agnostic subword promotion**.

Given a tokenizer-agnostic subword lattice for a string, we can directly inspect all possible tokenizations. This structure is used by Unigram and PathPiece to compute, respectively, the most likely or shortest tokenization compatible with an input. It also enables techniques such as tokenization sampling (Section 8.1; Cognetta et al., 2024c) and marginalization (Section 8.2; Geh et al., 2024), which require access to the entire set of possible tokenizations in a succinct form. When applied to a multilingual context, this lattice structure can also be leveraged for language identification (Meister et al., 2026b) and language modeling (Meister et al., 2026a). For arbitrary regular languages, the subword lattice is useful for constrained generation (Section 14.4), as the transition function of the lattice exactly describes the set of valid next tokens given an input and pattern constraint (Willard & Louf, 2023; Koo et al., 2024).

³⁴Finite-state transducers are less commonly known than their automata counterparts (and, in fact, automata are a special case of transducers). Informally, while automata recognize strings, transducers recognize *pairs* of strings (x, y) and are said to *transduce* the input string x into the output string y , where x and y are possibly strings over different alphabets.

³⁵Also known as the Multi-valued Decision Diagram (MDD) (Geh et al., 2024).

³⁶This is not the only way to produce the lattice. Several other works have used bespoke algorithms to construct it, but the final results are equivalent (Willard & Louf, 2023; Geh et al., 2024).

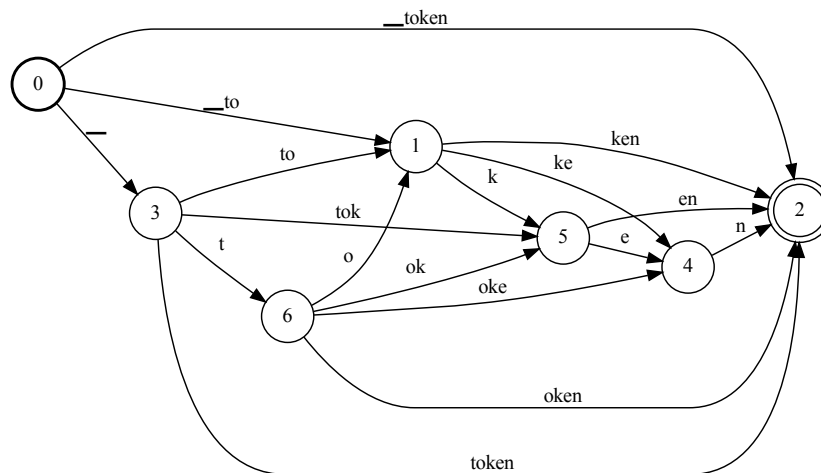


Figure 13.1 – A subword lattice for *token* using the Llama 2 tokenizer’s vocabulary. Each path from the start state to a final state encodes a valid tokenization, but only one (token) is the canonical tokenization according to the tokenizer.

13.1.2 Tokenizer-Aware Subword Promotion

The tokenizer-agnostic subword promotion procedure does not necessarily respect the underlying tokenization scheme. All possible tokenizations of the input are encoded, whereas a typical tokenizer has only a single canonical tokenization for a given input. We now turn to **tokenization-aware subword promotion**, where we can produce subword lattices that only encode canonical tokenizations of the strings in a regular language.

Figure 13.2 gives an example of tokenizer-agnostic and tokenizer-aware promotion under a MaxMatch tokenizer. For every sequence accepted by the character-level automaton, the MaxMatch-promoted subword lattice accepts that sequence’s canonical tokenization and no non-canonical tokenizations. Note that producing a tokenizer-aware lattice is not as simple as enumerating the character-level automaton, tokenizing each sequence, and then forming a new regular language, since the number of strings accepted by the character-level automaton may be infinite. Furthermore, the subword transducers required for tokenizer-aware promotion are themselves tokenizer-specific; we now discuss the MaxMatch and BPE cases.

MaxMatch Promotion The foundation for MaxMatch-aware promotion was first noted by Song et al. (2021), who found a connection between the MaxMatch tokenization algorithm and the Aho-Corasick Pattern Matching algorithm (Aho & Corasick, 1975). The Aho-Corasick pattern-matching trie, itself a type of automaton, can be used to provide an $O(n)$ -time algorithm, as opposed to the naïve $O(nm)$ -time algorithm, where n is the length of the string to be tokenized and m is the length of the longest token in the tokenizer’s vocabulary. Song et al. (2021) also briefly discuss the connection between their trie data structure and φ -failure automata,³⁷ later formalized and implemented by Cognetta & Allauzen (2025), enabling MaxMatch promotion to be applied to arbitrary regular languages rather than just single-string inputs.

BPE Promotion Berglund & van der Merwe (2023) and Berglund et al. (2024) provide an explicit algorithm for capturing BPE’s merge inference method by automata which is polynomial-time in the size of the automaton and of the tokenizer’s merge list and vocabulary. Cognetta & Okazaki (2025) provide a similar pipeline for subword promotion using only finite-state transducer composition, which shows that BPE can be

³⁷ φ -failure automata are automata with φ -failure arcs: arcs which can only be traversed when no matching arc is available at a state (Aho & Corasick, 1975).

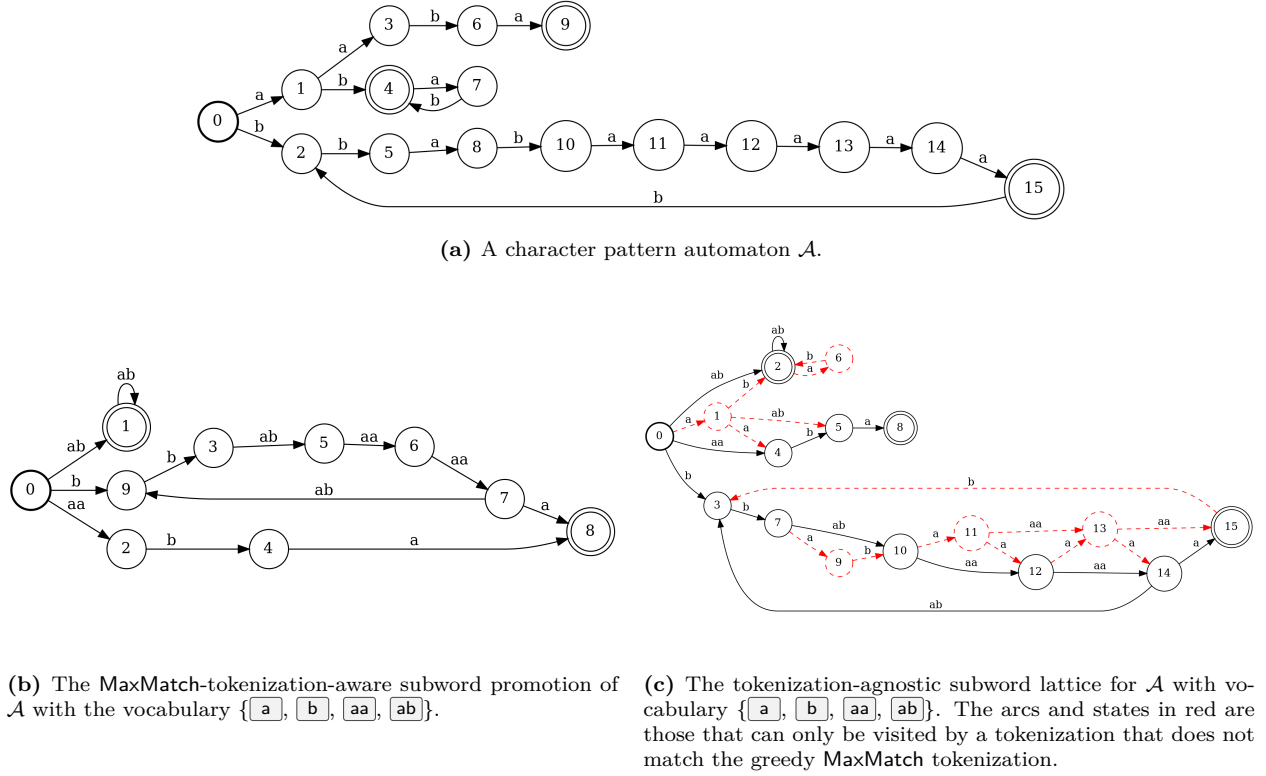


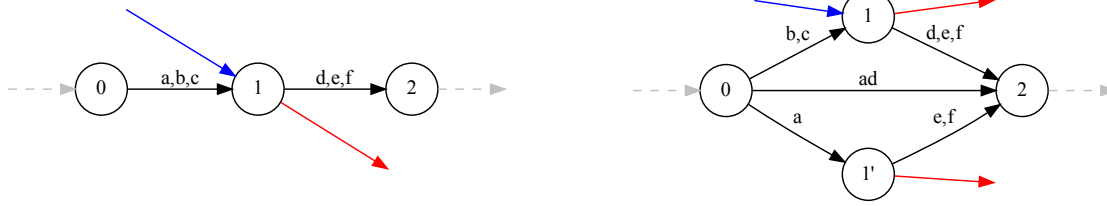
Figure 13.2 – An example of tokenizer-aware and tokenizer-agnostic promotion of a character-level automaton. The tokenization-aware automaton (Panel (b)) accepts only the canonical tokenization of every sequence accepted by the character-level automaton, while the tokenization-agnostic automaton (Panel (c)) accepts any tokenization that, when detokenized, is accepted by the character-level automaton.

implemented using only automata-theoretic primitives. Both methods work by iteratively applying merges in order of priority from the BPE merge list and modifying the underlying automaton to reflect the merge: [Berglund et al. \(2024\)](#) do this by directly modifying the underlying automaton transition function to apply a merge, while [Cognetta & Okazaki \(2025\)](#) do this by constructing a transducer that represents the merge and composing it with the automaton. The result of both methods is the same, and an example of how a merge modifies an automaton is shown in [Figure 13.3](#).

Despite the polynomial space complexity of BPE promotion, the resulting automata can be very large in practice, even when dealing with small input automata. To address this, [Berglund et al. \(2025\)](#) provide algorithms to incrementally build BPE-promoted automata in a space-efficient way in practice.

The Universal BPE Automaton A special automaton known as the **Universal BPE Automaton** ([Berglund et al., 2024](#)) encodes all possible valid BPE sequences for a BPE tokenizer. This automaton can be constructed by BPE promotion of the language Σ^* , which accepts all finite strings made of symbols in Σ and has a single state and $|\Sigma|$ arcs. Despite the simplicity of language Σ^* , however, the promoted automaton can have an enormous number of arcs and require significant disk space even for relatively small tokenizers ([Cognetta & Okazaki, 2025](#)). As this automaton is expensive to produce, [Vieira et al. \(2025b\)](#) and [Chatzi et al. \(2025\)](#) provide implicit representations for it, based on a bigram property of BPE which side-steps the construction entirely. Informally, the bigram property is that a token sequence $\mathbf{t} = t_1 t_2 \dots t_n \in \mathcal{V}^*$ is canonical for BPE if and only if for each bigram $t_i t_{i+1}$, $\text{TOKENIZE}(\text{DETOKENIZE}(t_i t_{i+1})) = t_i t_{i+1}$.³⁸

³⁸[Berglund & van der Merwe \(2023\)](#) prove a result similar to this property for canonical tokenization prefixes.



(a) A subset of an automaton with a subpath $\boxed{a}\boxed{d}$.

(b) After applying the merge (a, d) .

Figure 13.3 – An example of one iteration of a merge $(\boxed{a}\boxed{d} \rightarrow \boxed{ad})$ in BPE promotion being applied to an automaton. When the merge is applied, all subpaths labeled $\boxed{a}\boxed{d}$ are removed and replaced with a single arc labeled $\boxed{ad}$. An intermediate state (in this example, state $1'$) is constructed to preserve path information for all paths that either begin with the first component of the merge ($\boxed{a}$) but do not match the merge rule exactly, or which do not match any component of the merge. Figure adapted from [Cognetta & Okazaki \(2025\)](#).

The bigram property also implies a linear-time, left-to-right tokenization algorithm for BPE. This has been realized by *streaming* BPE algorithms, which have a linear runtime in the length of the input but require a buffer that depends on properties of the vocabulary ([Jiang & Gong, 2026](#); [Mamouras et al., 2026](#)).

The universal BPE automaton can be built once and used to promote other regular languages without going through the time-consuming iterative promotion steps mentioned above. Let $\mathcal{A}_{sub}$ be the tokenizer-agnostic subword lattice corresponding to a regular language $\mathcal{A}$ for some vocabulary $\mathcal{V}$. Then, intersecting $\mathcal{A}_{sub}$ with the universal BPE automaton produces an equivalent automaton to what would be produced by the BPE promotion method described above, but can be much faster in practice ([Cognetta & Okazaki, 2025](#)).

13.2 Tokenizer Complexity and Hardness Results

The computational complexity of tokenization has typically been explored through the lens of optimization problems ([Kozma & Voderholzer, 2024](#); [Kastreva et al., 2025](#); [Lim et al., 2025](#); [Whittington et al., 2025](#)). Let $\mathcal{D} = \{\mathbf{b}_i\}_{i=1}^N$ be a dataset composed of strings $\mathbf{b}$ over a certain alphabet Σ . Let $\mathfrak{G}$ be a function which, given a dataset $\mathcal{D}$ and a tokenizer tok , outputs a score reflecting how good that tokenizer is. We can formalize the tokenization optimization problem as:

$$\text{tok}^* = \arg \min_{\text{tok} \in \mathbb{T}} \mathfrak{G}(\text{tok}, \mathcal{D}), \quad (6)$$

where $\mathbb{T}$ defines a class of considered tokenizers. In words, [Equation 6](#) asks what is the optimal tokenizer tok^* from the class $\mathbb{T}$ according to the scoring function $\mathfrak{G}$. The tokenization algorithms described in previous sections can then be seen as heuristic procedures to try to approximate this optimization problem: e.g., BPE is a greedy bottom-up procedure to optimize the compression of dataset $\mathcal{D}$ once it is tokenized; Unigram is a heuristic pruning method to optimize the unigram log-likelihood of dataset $\mathcal{D}$ once tokenized. These algorithms only approximate the optimization above, and do not necessarily find optimal solutions.

Framing tokenization this way leaves open a few design choices. First, the choice of scoring function is strongly related to the discussion of evaluation in [Section 7](#). While an extrinsic downstream objective would be ideal, it is intractable to optimize directly, and so this choice is typically disregarded. While there are other reasonable choices of $\mathfrak{G}$, prior theoretical work mostly focuses on compression, formalized as either:

$$\underbrace{\mathfrak{G}(\text{tok}, \mathcal{D}) = \sum_{\mathbf{b} \in \mathcal{D}} |\text{Tokenize}(\mathbf{b})|}_{\text{Compressed length}} \quad \text{or} \quad \underbrace{\mathfrak{G}(\text{tok}, \mathcal{D}) = - \sum_{\mathbf{b} \in \mathcal{D}} \left(|\mathbf{b}| - |\text{Tokenize}(\mathbf{b})| \right)}_{\text{Compression reduction}}, \quad (7)$$

where $\text{TOKENIZE}(\mathbf{b})$ describes the application of tok’s tokenization function TOKENIZE to $\mathbf{b}$. While this distinction does not change the optimal value and solution(s) of the objective function above, it does change the approximation ratio of an algorithm approximating this objective and is thus relevant.

As an example, consider the dataset $\mathcal{D} = \{aab, aac, aad, ab, ac, ad\}$, which has an initial length of 15. We can look at how BPE approximates the compression of $\mathcal{D}$ with a vocabulary size of 3 (not counting the four atomic tokens corresponding to Σ). BPE greedily chooses the token $\boxed{aa}$ as it is the only bigram appearing (at least) three times. Afterwards, every bigram only appears once, which results (up to tie-breaking) in the following tokenization of the dataset:

$$\text{TOKENIZE}(\mathcal{D}) = \{\boxed{aa}\boxed{b}, \boxed{aa}\boxed{c}, \boxed{aa}\boxed{d}, \boxed{ab}, \boxed{ac}, \boxed{a}\boxed{d}\}$$

This BPE tokenizer thus achieves a compressed length of 10 and a compression reduction of 5. Optimally, however, one could instead propose a tokenizer which includes the tokens $\boxed{ab}$, $\boxed{ac}$, $\boxed{ad}$, yielding a tokenization:

$$\text{TOKENIZE}(\mathcal{D}) = \{\boxed{a}\boxed{ab}, \boxed{a}\boxed{ac}, \boxed{a}\boxed{ad}, \boxed{ab}, \boxed{ac}, \boxed{ad}\}.$$

This optimal tokenizer achieves a compressed length of 9 and a compression reduction of 6. We see here that BPE does not produce an optimal solution, but achieves a $10/9$ -approximation in terms of compressed length and a $5/6$ -approximation in terms of compression reduction.³⁹ While for this example the two approximation ratios are relatively similar, they may be arbitrarily different in other datasets. Kozma & Voderholzer (2024) show that the single-string dataset $\mathcal{D} = \{\bigcirc_{i=1}^n x_i a a y_i z_i x_i a t_i a y_i\}$, where $\bigcirc$ denotes concatenation, can be optimally reduced to a single token with $8n - 1$ merges; BPE, however, only reduces this dataset to $n + 2$ tokens with the same number of merges, resulting in a $n^{+2}/1$ -approximation for compressed length and a $9n-2/10n-1$ -approximation for compression reduction.

The second important design choice is the class of tokenizers being optimized. One possible classification is into *direct*, *bottom-up*, and *optimal-pair-encoding* tokenizers, defined as follows:

$$\mathbb{T}_{\text{dir}} = \{\text{tok}_{\text{dir}}[\mathcal{V}] \mid \mathcal{V} \subset \Sigma^+, |\mathcal{V}| = K\} \quad \text{optimal choice of vocabulary under PathPiece} \quad (8a)$$

$$\mathbb{T}_{\text{bup}} = \{\text{tok}_{\text{bup}}[\mathbf{m}] \mid \mathbf{m} \in \mathcal{M}^*, |\mathbf{m}| = K\} \quad \text{optimal choice of merges under BPE’s inference function} \quad (8b)$$

$$\mathbb{T}_{\text{ope}} = \{\text{tok}_{\text{dir}}[\mathcal{V}_{\mathbf{m}}] \mid \mathbf{m} \in \mathcal{M}^*, |\mathbf{m}| = K\}, \quad \text{optimal choice of merges under PathPiece} \quad (8c)$$

where $\mathcal{M} = (\Sigma^+ \times \Sigma^+)$, and $\mathbf{m} \in \mathcal{M}^*$ represents a sequence of pairs of tokens to be merged. In these equations: $\text{tok}_{\text{dir}}[\mathcal{V}](\mathbf{b})$ returns a string’s shortest tokenization under $\mathcal{V}$, as in the PathPiece inference method; $\text{tok}_{\text{bup}}[\mathbf{m}](\mathbf{b})$ iteratively applies merges $\mathbf{m}$ to $\mathbf{b}$, as is the default in BPE; and $\mathcal{V}_{\mathbf{m}}$ is a vocabulary extracted from the merges $\mathbf{m}$.

When using any combination of compression function and class of tokenizers, the tokenization optimization problem is NP-hard when applied to strings formed of characters from unboundedly large alphabets (Kozma & Voderholzer, 2024; Lim et al., 2025; Whittington et al., 2025). Considering only compression reduction as the scoring function, Kozma & Voderholzer (2024) further showed that tokenization is not simply NP-hard, but APX-complete, meaning that this problem is hard to approximate arbitrarily well in polynomial time. Kastreva et al. (2025) showed that a similar inapproximability result holds for both direct and bottom-up tokenization (respectively, $\mathbb{T}_{\text{dir}}$ and $\mathbb{T}_{\text{bup}}$) even when datasets are restricted to n -ary alphabets (for any $n \geq 2$). Finally, Kastreva et al. (2025) also showed that direct tokenization $\mathbb{T}_{\text{dir}}$ is NP-hard even when restricted to unary alphabets. These results are summarized in Table 13.1.

An important question specifically for BPE concerns whether there are bounds on the approximation it achieves on any dataset. Zouhar et al. (2023b) were able to bound BPE’s compression reduction as a parameter of the total backwards curvature, a measure of how much an optimal solution can be negatively affected by prepending a suboptimal prefix. The first *unconditional* bound was found by Kozma & Voderholzer (2024), who showed that BPE is always at least $1/3$ of optimal, even when compared to an optimal solution for $\mathbb{T}_{\text{ope}}$.⁴⁰ Considering compressed length, however, they also showed that BPE’s approximation

³⁹Note that here, minimization problems admit approximation ratios greater than or equal to 1, whereas maximization problems admit approximation ratios smaller than or equal to 1. This is generally not consistent throughout the approximation literature.

⁴⁰More precisely, that BPE’s approximation ratio lies between $1/3$ and $5/8$.

Tokenizer class	Alphabet	Scoring function $\mathfrak{S}$	
		Compressed length	Compression reduction
Direct ($\mathbb{T}_{\text{dir}}$)	unbounded	NP-complete (Kastreva et al., 2025)	NP-complete (Whittington et al., 2025)
	n -ary ($n \geq 2$)	APX-hard (Kastreva et al., 2025)	NP-complete (Kastreva et al., 2025)
	unary	strongly NP-complete (Kastreva et al., 2025)	
Bottom-up ($\mathbb{T}_{\text{bup}}$)	unbounded	APX-hard (Kozma & Voderholzer, 2024)	APX-complete (Kozma & Voderholzer, 2024)
	n -ary ($n \geq 2$)	APX-hard (Kastreva et al., 2025)	NP-complete (Kastreva et al., 2025)
	unary	open	
OPE ($\mathbb{T}_{\text{ope}}$)	unbounded	APX-hard (Kozma & Voderholzer, 2024)	APX-complete (Kozma & Voderholzer, 2024)
	n -ary ($2 \leq n < \infty$)	open	
	unary	weakly NP-complete (Kastreva et al., 2025)	

Table 13.1 – Hardness of the tokenization optimization problem, per tokenizer class, scoring function, and alphabet size. Strong vs. weak NP-completeness refers to string vs. string-length input representations of unary datasets.

ratio is not bounded by *any* constant, meaning that there exist datasets where BPE is arbitrarily worse than optimal; in fact, this is shown by the example above, where the approximation ratio of $n + 2$ grows to infinity as $n \rightarrow \infty$. Together, these results also show why both metrics must be studied separately, as different theoretical properties can be proven about tokenization depending on which metric is chosen.

Given the inapproximability results above, one might expect these optimization problems to be intractable. In practice, this seems to not be the case. *ConvexTok* (Tempus et al., 2026) recasts tokenization as an *integer program* problem; when relaxed to a linear program, the optimal compression value can be efficiently lower-bounded. Both this system and BPE have been shown to achieve a compressed length within 1% of optimal in practice. Extending this linear-programming-based approach, Nichol (2026) managed to find provably optimal tokenizers for the book *Pride and Prejudice*. While this is a relatively small dataset by modern standards, it shows that natural language may have properties which make tokenization *in practice* a much easier problem than in theory.

13.3 Questions

BPE and MaxMatch are both regular transductions. What implications does this have for these tokenizers? What are some possible non-regular tokenization algorithms, and what linguistic features would they give tokens access to? Does the insufficient representational capacity of regular languages show up as a failure of tokenizers to capture non-regular linguistic features in practice?

What is the best approximation guarantee for compression-based tokenization achievable by efficient algorithms?

The presented complexity results all consider tokenization with a compression objective. What is the complexity for other objectives, such as unigram log-likelihood?

Can properties of natural language, such as following Zipf’s law, be exploited to achieve better guarantees?

14 Additional Topics

We provide a short survey of research directions that are not directly related to tokenization, but are closely adjacent or are immediate artifacts of the use of subword tokenizers.

14.1 Tokenization in Non-Linguistic/Text Domains

Tokenization, as a task of converting raw (text) inputs into a representation that is suitable for model training, is typically associated with linguistic domains, but it can also refer to ways of partitioning a raw data stream into a more manageable set of segments from a well-defined vocabulary in non-natural language domains and modalities.

14.1.1 Audio, Image, and Video Tokenization

Audio, image, and video tokenization are major departures from text tokenization, as none of them have obvious sequences of meaningful discrete units (analogous to those in text processing) that would be amenable to the tokenization techniques discussed above. Nevertheless, these modalities are often processed by language models (in both unimodal and multimodal settings) or similar architectures and thus need to be represented in a useful format.

Audio Audio processing broadly comes in two forms: *continuous* modeling, where a high-dimensional, continuous representation of the input audio is fed directly into a model; and *discrete* modeling, where the continuous audio signal is discretized before being processed (Peng et al., 2026). Discrete modeling typically converts the continuous input into discrete units, each drawn from a fixed-size inventory, for example by quantization or by producing a *code*—a discrete vector that maps to a continuous *codebook embedding* (Chung et al., 2021; Hsu et al., 2021).

Text tokenization techniques have been applied to discrete audio tokens. For example, **Acoustic BPE** (Shen et al., 2024) takes representations produced by **HuBERT** (Hsu et al., 2021), a speech representation model, discretizes them, and maps the discrete units to Unicode code points. The resulting text-like Unicode representation is used to train and apply a traditional BPE tokenizer to drastically reduce the input sequence length. **MegaByte** (Yu et al., 2023b) was applied to audio in order to compress the relatively long (compared to typical text inputs) raw audio file byte sequences by forming latent tokens from fixed-sized byte patches.

For a more detailed survey on discrete audio processing, we point the reader to Mousavi et al. (2025).

Image Motivated by the inefficiency of direct modeling over pixels and the typically high correlation between nearby pixels, *patch-based tokenization* and learned tokenization have become a norm in image tokenization (Chen et al., 2020; Nguyen et al., 2024a, *inter alia*). Patch-based tokenization methods such as ViT (Dosovitskiy et al., 2020) divide images into non-overlapping square patches of fixed size (e.g., 16×16 pixels), which are associated with spatial locations, arranged in scan order, and treated as “tokens” (Esteves et al., 2025). Other approaches to image tokenization use feature reconstruction objectives and produce discrete codebook tokens, similarly to audio tokenizers (Wang et al., 2024d).

Some methods for image tokenization are inspired by the text tokenization literature. For example, one way to tokenize visual inputs is to group pixels by the “object” they belong to (e.g., a truck in an image). Chen et al. (2025a) introduced *subobject*-level token segmentation methods, which are inspired by subword tokenization techniques and which group pixels by components of the objects in the scene. Subobject tokens thus occupy a granularity between pixels (analogously, bytes or characters) and objects (analogously, words). Zhang et al. (2025b) applied BPE to visual data by first quantizing the input image and then applying two-dimensional BPE to merge commonly occurring pairs of horizontally or vertically adjacent quantized tokens.

Video Standard methods for video tokenization typically encode each frame or video segment with a fixed number of tokens produced by patching or some form of discretization (van den Oord et al., 2017; Esser et al., 2021; Yu et al., 2023a). However, this can lead to either lossy encodings when the number of tokens is too small to fully capture the scene, or prohibitively long sequences when unnecessarily many tokens are used for

each frame. Some video tokenization methods instead use adaptive, content-aware token allocation, encoding different frames within the same video using varying numbers of tokens, conditioned on the content of the current frame and on prior frames. **ElasticTok** (Yan et al., 2025) learned a *coarse-to-fine* latent representation for each frame using a masked-reconstruction loss where in the sequence of latent tokens encoding a frame, the earlier tokens learn to encode general details while later ones learn progressively finer details. During video processing, the latent token sequence for each frame can be truncated to reduce the length of the latent representation of the video by a desired amount while mitigating catastrophic information loss.

14.1.2 Code Tokenization

Code is closer to the linguistic domains than the other modalities discussed here: it is largely written in Latin script, draws its keywords and identifiers from English, and has an underlying formal grammar. However, most code tokenization algorithms rely on generic subword tokenization methods like BPE that do not take into account the grammatical properties of the programming language, so semantically identical code snippets can be tokenized differently based on whitespace, formatting, or identifier naming (Chirkova & Troshin, 2023). This misalignment matters in practice, since simple rewrites that preserve program semantics but shift token boundaries can substantially alter the outcome of code modeling tasks (Li et al., 2025b).

A code-tokenizer metric that is analogous to morphological alignment (see Section 7) is *abstract syntax tree* (AST) alignment (Meister, 2026). This measures how often token boundaries align with the boundaries of leaf nodes in the code’s AST to see how well the tokenizer captures the language’s syntax.

14.1.3 Numerical Tokenization

The tokenization of numbers has received particular attention, as digit sequences often have to be processed and manipulated at a fine granularity and must encode magnitude and other numerical properties, all of which are complicated by the use of subword tokenization (Nogueira et al., 2021; Thawani et al., 2021; Liu & Low, 2023; Garreth Lee & Wolf, 2024; Singh & Strouse, 2024; Kreitner et al., 2025; Yang et al., 2025b; Reddy et al., 2026) and specialized pretokenization rules (see Section 4.5). Some tokenization methods reverse digit order to generate the least-significant digits first, matching the order of standard hand-arithmetic algorithms (Lee et al., 2024; Zhang-Li et al., 2024). Other approaches annotate digits with explicit place-value information, for instance, by prefixing digit counts (Schwartz et al., 2024), writing digit positions (Nogueira et al., 2021), or adding index hints (Zhou et al., 2024).

Another approach is to use a special `<NUM>` token as a placeholder for the numerical surface form and encode the numerical value directly into the embedding (Kreitner et al., 2025; Golkar et al., 2024; Zhou et al., 2026). We discuss this more in Section 14.2.2.

14.1.4 Tokenization in Other Domains

Tokenization has been adapted to a wide range of other domains and modalities. In sign language, Miyazaki et al. (2024) develop *gloss* pair encoding for sign sequences, which encodes gloss pairs as well as non-manual movements to capture meaning. In symbolic music, BPE has been applied in order to compress the input sequence and form more expressive tokens (Fradet et al., 2023). In biology, Suyunu et al. (2024) examine whether linguistic laws hold for protein-sequence tokenization, and Lindsey et al. (2025b) compare tokenization methods for genomic language models and find that “character-level” (in this case, nucleotide-level) modeling outperforms BPE for tasks that require nucleotide resolution, but BPE performs better on downstream classification tasks.

14.2 Embedding Layers

It is easy to consider the scope of tokenization research as ending at the discrete tokens produced by a tokenizer. However, the influence of tokenization reaches beyond this, as tokenizers directly contribute to the parameter count of a model through the token embedding layer, and information that is obscured by the tokenization process may not be captured by token embeddings, which impacts downstream model quality.

Here, we discuss some of the ways that embedding layers can be modified and used to augment subword tokenization.

14.2.1 Embedding Parameters

An immediate example of the influence of the tokenizer on the downstream model is the size of the embedding and output layers, which typically scale linearly with the number of tokens. Thus, a large vocabulary can cause parameter-allocation issues, since a large portion of the parameters are allocated to the embedding layer rather than the core language model.⁴¹

One mitigation strategy is to use **hashed embeddings**, decoupling the vocabulary size from the embedding table size using a hash function $h : \mathcal{V} \rightarrow \{1, 2, \dots, N\}^k$, where $N \ll |\mathcal{V}|$, that maps each token to k buckets in a shared embedding table. The embedding for a token t is then computed as

$$\text{emb}(t) = \sigma(\mathbf{e}_{h(t)_1}, \mathbf{e}_{h(t)_2}, \dots, \mathbf{e}_{h(t)_k}),$$

where $\mathbf{e}_i \in \mathbb{R}^d$ is the embedding of bucket i and $\sigma : \mathbb{R}^{d \times k} \rightarrow \mathbb{R}^d$ is a combining function (Svenstrup et al., 2017). The hash function can also be replaced with a learned, discrete assignment as in *codebook embeddings* (Chen et al., 2018; Shu & Nakayama, 2018). Hash embeddings have been used to construct token n -gram embeddings that are combined with the token-level embeddings to enhance local text representations and to artificially scale the vocabulary size. Often, this is done by using a hash embedding to decouple the n -gram order and vocabulary size from an n -gram embedding layer’s size (Roy et al., 2022; Jordan et al., 2024; Pagnoni et al., 2024; Huang et al., 2025; Cheng et al., 2026; Liu et al., 2026).

Another approach is to reduce the dimensionality of the token embeddings. Since large language models generate rich contextual representations, token embeddings can be kept small, for example by decoupling the embedding size from the model’s hidden dimension (Reid et al., 2021) or assigning different embedding dimensions to more or less frequent or semantically meaningful tokens (Chen et al., 2016), thereby allowing vocabulary scaling without an increase in embedding parameters.

14.2.2 Injecting Information Into Embeddings

Section 5 introduces many ways by which language-aware tokenizers can deliberately encode meaningful information such as script or glyph awareness into tokens. Alternatively, one could inject information that is lost or hidden by the tokenization process directly into the token embeddings. Character-level information has been used in sequence models since before the advent of transformers for task-specific training (Plank et al., 2016) and for post-embedding augmentation (Pinter et al., 2017; Stratos, 2017a; Zhao et al., 2018), adapted into various transformer-compliant architectures (El Boukkouri et al., 2020; Ma et al., 2020; Pinter et al., 2021). In CJK language modeling, Chinese character embeddings have been augmented with glyph or pinyin features to explicitly inject semantically meaningful orthographic or pronunciation information (Meng et al., 2019b; Sun et al., 2021). Korean language models have used Chinese character embeddings to improve the embeddings of Korean tokens with Chinese roots, both for homophone disambiguation and to take advantage of the higher-resource Chinese embedding representations (Kim et al., 2020), as well as subcharacter-level embeddings in character-level language models (Stratos, 2017b; Cognetta et al., 2023; Kim et al., 2024).

As discussed in Section 14.1.3, specialized embeddings have also been applied to numerical representations in language models. In order to properly encode fine-grained information about digit tokens, several works have encoded numerical information directly into embeddings by using scaling or rotation, sometimes injected into the embedding of a special `<NUM>` token (Thawani et al., 2021; Golkar et al., 2024; Kreitner et al., 2025; Zhou et al., 2026).

As an example, BitTokens (Kreitner et al., 2025) encodes numerical values into the `<NUM>` token’s embedding using the IEEE 754 double-precision binary floating point format. The binary encoding is placed into the first 64 positions of the embedding vector, which is zero-padded to match the model’s internal dimension.

⁴¹For the embedding and output layers, the Llama 3 models allocate 1B parameters (13%) of the 8B-parameter model, 2.1B parameters (3%) of the 70B-parameter model, and 4.2B parameters (1%) of the 405B-parameter model (Dubey et al., 2024).

During decoding, if the `<NUM>` token is predicted, then its last hidden representation is passed through a layer that recovers the floating point representation to produce the output value.

These augmented embeddings have been shown to help the models generalize, for example, by enabling scaling to arithmetic tasks with significantly longer digit sequences than those observed during training (McLeish et al., 2024).

14.3 Tokenizer Security

Using subword tokenizers introduces a number of security risks into models. One common source of tokenizer-related failure modes is exploiting the strong inductive bias of a language model towards its canonical tokenization. A wide range of attacks, designed to degrade a model’s performance, bypass guardrails, or elicit undesired behavior, work by corrupting the input tokenization to bring a model out of distribution (Eger & Benz, 2020; Pajola & Conti, 2021; Creo & Pudasaini, 2025; Gao et al., 2025; Geh et al., 2025). Some attacks can be viewed as *imperceptible*, where invisible characters like the zero-width joiner (U+200D) are inserted into the text, or where a character like the Latin script “a” (U+0061) is swapped for a visually identical character like the Cyrillic “a” (U+0430). These attacks are difficult for a human to spot, but severely perturb the input tokenization (Boucher et al., 2022; Goodside, 2023; Struppek et al., 2024), as shown in the example below using the GPT-5 tokenizer and with the Cyrillic character underlined as а for clarity. Text normalization as a pretokenization step (Section 4.1) can be used to mitigate these attacks, but makes the tokenization process lossy.

Example	
Input Text:	<code>attack_аttack</code>
Tokenization:	<code>attack_а tt а ck</code>

Since poor tokenizations resulting from the injection of variant characters or whitespace can be detected and mitigated, one could form adversarial tokens within the scope of the tokenizer without character-level artifacts. Non-canonical tokenizations (see Section 8) have been shown to evade alignment filters in modern LLMs (Geh et al., 2025), but this is limited to models with direct tokenizer access, since it requires supplying token IDs directly, so closed models that only accept text input are typically not vulnerable to this vector. However, even closed models, for which we do not have direct tokenizer access, can be susceptible to another type of attack that targets undertrained tokens. If a token appears rarely during training, perhaps because it contains rare or invalid Unicode, cannot normally be formed by the tokenization algorithm (e.g., unreachable tokens in GPT-4o’s tokenizer; Land, 2024b), or is an artifact of the tokenizer’s training corpus (as with the `SolidGoldMagikarp` token), then its embedding will be undertrained and can corrupt the contextual representation of the sentence (Land & Bartolo, 2024). These tokens can be used as an attack vector by identifying text that produces them and inserting it into the input to an LLM. Another technique is to provide text that will tokenize into adversarial token bigrams which are undertrained, such as bigrams containing partial-UTF-8-sequence tokens (Jang et al., 2025) or that form tokenizations that segment into semantically different morphemes than how a human would have parsed it (Wang et al., 2025a).

Large language models are also susceptible to attacks targeting a prompt template (see Section 11.2) and its interaction with the tokenizer. If a tokenizer does not properly sanitize user input, users can bypass some of a model’s instructions by injecting their own control tokens at specific points in the prompt or overrunning the prompt template boundaries (Jiang et al., 2025; Chang et al., 2026; Ye et al., 2026), allowing them to elicit behavior that was explicitly banned by the model’s system instructions. Preventing attacks of this sort requires developers to apply prompt templates carefully, so that users cannot break out of the intended prompt confines, as well as preventing the generation of control tokens from user-supplied text.

A different vein of attack tries to recover training corpus information from the tokenizer itself, since tokenizer training data typically mirrors the pretraining corpus of the language model it is used with. Hayase et al. (2024) infer the composition of the tokenizer’s training data mixture by solving a linear program formulated using the relative ordering of the tokens, and Zhang et al. (2025a) train a Zipf-inspired quantile

regression model to predict each token’s frequency in the training corpus directly from its token ID. One defense is to train tokenizers over private data via federated learning in order to mitigate private information leakage (Bagdasaryan et al., 2022).

Exfiltrating the tokenizer of closed models may also reveal proprietary information that vendors have chosen to keep hidden. Some researchers have attempted to extract information about the tokenizer from Claude, the most notable frontier LLM with a closed tokenizer, by inspecting network packet information (Rando & Tramèr., 2024) and by using Claude’s token count API (Land, 2026a).

14.4 Constrained Generation

Constrained generation is a language model decoding paradigm that restricts output to strings that satisfy a user-specified pattern, such as a JSON schema. This is done because the output softmax layer of a language model assigns non-zero probability to all tokens, and therefore all possible sequences, so syntactically invalid sequences can be sampled erroneously.

In theory, constrained generation is not related to tokenization, as it only constrains the *surface form* of the text. In practice, however, constrained generation must reconcile a mismatch in granularity between pattern matching and generation: constraints are typically specified at the character level (e.g., as regular expressions), while language models generate subwords.

Constrained generation enforces constraints by masking *invalid* logits (those that would lead to text that does not match the desired pattern) during decoding so that they receive zero probability during sampling.⁴² At each decoding step, a constraint mask $\mathbf{m}$ is applied element-wise to the logits ℓ produced by the language model so that the sampling distribution becomes:

$$\text{SOFTMAX}(\ell + \mathbf{m}) \quad \text{where} \quad \mathbf{m}_i = \begin{cases} 0 & \text{if } \exists \mathbf{t}' \in \mathcal{V}^* \text{ s.t. } \text{DETOKENIZE}(\mathbf{t} \circ t_i \circ \mathbf{t}') \in \mathcal{R} \\ -\infty & \text{otherwise,} \end{cases}$$

where $\mathbf{t}$ is the tokenized text generated so far, t_i is the i -th token in the vocabulary, and $\mathcal{V}^*$ is the set of all possible subword sequence continuations. Intuitively, a token is valid if there exists some continuation such that the resulting surface string satisfies the constraint $\mathcal{R}$. One way to efficiently compute $\mathbf{m}$ is to represent the constraint pattern as a subword-level finite automaton (Section 13.1), so that valid continuations correspond to outgoing arcs from the current automaton state (Willard & Louf, 2023; Koo et al., 2024).

Constrained generation must also account for other aspects of the tokenization pipeline, particularly pretokenization. Some pretokenizers, like simple text normalization, can be integrated directly into the subword lattice construction via finite-state transducers, while others, such as the GPT pretokenizer, which uses non-regular look-ahead semantics, are challenging to integrate (Cognetta et al., 2025; Vieira et al., 2025b).

Several libraries implement constrained generation with different constraint semantics and tokenization handling by building constraints at the token level or by constraining at the surface level and dynamically processing tokens during inference, including **Outlines** (Outlines), **Guidance** (Guidance AI, 2023), **XGrammar** (Dong et al., 2025), and **SGLang** (Zheng et al., 2024). Other work has exploited theoretical properties of tokenization and common constrained generation tasks to improve runtime performance (Kurt, 2024; Tran-Thien, 2024; Chen et al., 2025b).

Constrained generation can be used to enforce canonical tokenization generation, even in conjunction with other constraints (Section 13.1.2; Vieira et al. 2025b). Canonical constrained generation can be used to reduce perplexity, to ensure that the tokenization is always in-distribution, and to avoid ever having to pause generation, re-tokenize, and re-embed the entire sequence to put it back in canonical form. Constrained generation has also found use with marginalization (Section 8.2), where it is used to constrain a language model’s output to form a proxy distribution for importance sampling (Geh et al., 2024).

⁴²Since logits are unnormalized log probabilities, setting a logit to $-\infty$ gives it probability zero after SOFTMAX.

14.5 Token Healing

Consider generating the text *Attention is all you need* by providing a language model with most of the phrase and allowing it to complete the remainder, as with the following prompt:

Example	
Prompt:	<i>Attention is all you</i>
Tokenized:	Atten tion _is _all _you

The natural continuation is `_need`, which completes the phrase with the correct detokenized surface form.

Now, consider a slightly different prompt that includes a trailing space, but is still a prefix of the desired output:

Example	
Prompt:	<i>Attention is all you</i>
Tokenized:	Atten tion _is _all _you

The trailing space produces a `_` token at the end of the prompt, forcing the model to begin generation from a non-canonical boundary. The model must now produce a continuation that yields the correct surface form, but it is unclear whether it should generate `need` (the most direct answer), `_need` (which introduces another space and therefore does not match the desired surface form), `ne` `ed` (an unusual tokenization), or something else entirely.

This example is an instance of the **partial token problem** (Xu et al., 2026), when the final token in the prompt is a prefix of a longer token that the model would have produced if generation had started from an earlier position. In this case, the end of the prompt forces a token boundary after `_`, which is a prefix of what would have been generated (`_need`) if the provided context were instead `Atten tion _is _all _you`. The partial token problem appears frequently in prompting, templating, and constrained generation settings for LLM inference (Lundberg & Ribeiro, 2023). Despite empirical results demonstrating that language models can perform well even with non-canonical tokenizations in their context (see Section 8.2), other work has found that across diverse settings such as language and model size, language model performance degrades significantly when the partial token problem is present on tasks such as code completion (Athiwaratkun et al., 2024), or even simple tasks such as completing a repeated sentence or answering a multiple-choice query (Sanz-Guerrero et al., 2025; Xu et al., 2026).

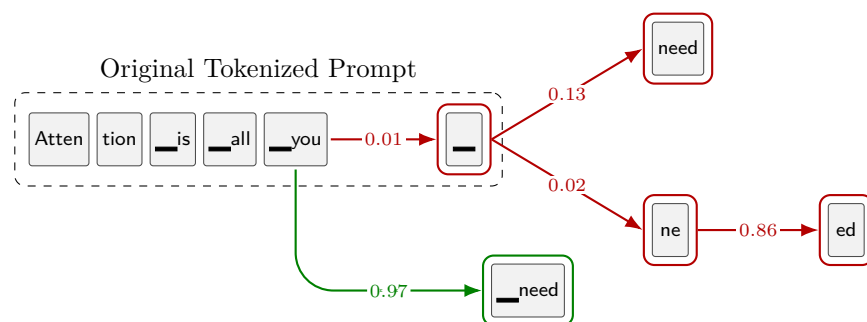


Figure 14.1 – An example of the decoding process with and without token healing. By forcing the tokenized prompt to end in `_`, the resulting autoregressive probabilities are extremely low. But by performing token healing and beginning generation from `_you` with the constraint that the next token must start with a `_` character, we generate a much higher probability sequence with the expected, canonical tokenization.

One heuristic for mitigating the partial token problem is **token healing**, where the final token of the prompt is removed and the model is constrained (as in Section 14.4) so that the next token contains the removed text as a prefix (Lundberg & Ribeiro, 2023; Athiwaratkun et al., 2024; Dagan et al., 2024). This aims to restore the tokenization boundary that would arise if the prompt and continuation were tokenized jointly, as shown in Figure 14.1.

However, token healing can be brittle, because in general it is not known in advance how many tokens to “back off”, or whether the ideal prompt boundary is even available as a token boundary in the truncated prompt. Consider, for instance, a prompt that ends with the token sequence `__natural__language__process__in__`. In this case, the ideal next-token `__processing__` covers *two* tokens at the end of the prompt (as `__processin__` is not a single token in the vocabulary). As a result, many exact methods have been proposed that preserve the sampling distribution of the original language model, often by constructing a *tree* of token sequences that are consistent with the surface form of the prompt (Phan et al., 2024; Hayase et al., 2025; Turaga, 2025; Vieira et al., 2025a).

14.6 Questions

How can we design tokenizers in conjunction with specialized embeddings in mind? That is, what properties of tokens make them amenable to domain-specific information injection?

For hashed embeddings, the parameter reduction is entirely on the *embedding* side, but the *output* layer must still be fully realized. What options are there to mitigate this?

Related to token healing, how should evaluations be designed, documented, and audited to account for minor variations in tokenization near prompt boundaries and structured outputs, and the non-negligible effect that these have on tokenization and model performance?

15 Conclusion

We have surveyed the field of subword tokenization, an oft-overlooked but impactful component of the modern language modeling pipeline. The influence of tokenization appears in many forms: inference speed and parameter count, parity across different languages and scripts, implicit constraints on families of large language models, and more. Many variants of the classical tokenization algorithms exist in order to mitigate some of their detrimental qualities or improve their performance on specific tasks. Entirely different paradigms, such as latent and visual tokenization, have also emerged, though these too involve trade-offs. Thus, our survey is intended to give as broad of a perspective of the field as possible to include the many facets of natural language processing that can be influenced by particular uses of subword tokenizers. With this, we introduce ways to measure and evaluate tokenizers in order to give researchers and practitioners the tools required to make informed decisions about their use of tokenizers and guide their future research.

We hope that this survey spurs an increase of research in tokenization, particularly in the context of modern language models. In this context, changes must be made carefully due to the large time and monetary costs involved in training and evaluation. So, while tokenization is an area where improvements could be realized, it may be passed over in favor of focusing on areas with perhaps better-understood dynamics with respect to downstream model quality. This necessitates a better understanding of how to evaluate tokenizers, where they improve models and where they cause problems, and how they interact with other aspects of scaling language models and the data they are trained on. To end with a few future directions, we look forward to new insights into tokenizer scaling laws, alternatives to subword tokenization to specifically address its shortcomings, and better benchmarks by which to evaluate tokenizers.

LLM Usage Disclosure

LLMs were used in the writing process for editing, revision, proof-reading, and readability review. They were also used to aid in the production of some figures in this document, for example the TikZ code used to produce Figure 14.1.

The use of LLMs was relatively minor and largely limited to the final polishing phase of the paper. The authors collectively accept responsibility for all contents of this manuscript.

Contribution Statement

All authors contributed to the preparation, organization, writing, editing, and reviewing of this manuscript. Below, we highlight the primary contribution areas of each author (in alphabetical order).

Section	Primary Contributors
Section 1 Introduction	Catherine Arnett, Marco Cagnetta
Section 2 Tokenization	Catherine Arnett, Marco Cagnetta, Clara Meister
Section 3 Tokenization Algorithms	Thomas Bauwens, Pavel Chizhov, Marco Cagnetta, Tatsuya Hiraoka, Sander Land, Yuval Pinter
Section 4 Pretokenization	Marco Cagnetta, Tatsuya Hiraoka, Sander Land, Alisa Liu, Yuval Pinter
Section 5 Language- and Script-Specific Considerations	Catherine Arnett, Marco Cagnetta, Eugene Jang, Sachin Kumar, Jindřich Libovický, Sangwhan Moon, Yuval Pinter, Xiulin Yang
Section 6 Multilingual Tokenization	Catherine Arnett, Tatsuya Hiraoka, Jindřich Libovický, Sangwhan Moon, Xiulin Yang
Section 7 Evaluation	Catherine Arnett, Marco Cagnetta, Omer Goldman, Jindřich Libovický, Clara Meister, Yuval Pinter, Gianluca Vico
Section 8 Non-Canonical Tokenizations	Thomas Bauwens, Marco Cagnetta, Alisa Liu, Clara Meister, David Pohl
Section 9 Byte and Character Representations	Pieter Delobelle, Valentin Hofmann, Julie Kallini, Sachin Kumar, Tomasz Limisiewicz, Sangwhan Moon
Section 10 Visual Text Representations	Desmond Elliott, Omer Goldman, Ilker Kesen, Pavel Stepachev
Section 11 Tokenizers in Modern LLMs	Christopher Akiki, Marco Cagnetta, Sander Land
Section 12 Tokenizer Modification and Cross-Tokenizer Knowledge Transfer	Thomas Bauwens, Pavel Chizhov, Pieter Delobelle, Konstantin Dobler, Benjamin Minixhofer, David Pohl
Section 13 Theoretical Aspects of Tokenization	Marco Cagnetta, Tiago Pimentel, Philip Whittington
Section 14 Additional Topics	Marco Cagnetta, Sander Land, Alisa Liu, Suchir Salhan, Tomohiro Sawada, Philip Whittington

Acknowledgments

The authors would like to thank: Jonathan Hayase for his help with early versions of this manuscript; Andy Yang from the FLaNN server and the Transformer Cookbook for his advice on running a large-scale writing

project; Stéphan Tulkens for the `skeletoken` package (Tulkens, 2026) and for his help with the technical aspects of the tokenizer census data curation; and Taku Kudo and Elizabeth Salesky for their comments and review of the manuscript.

Funding

Thomas Bauwens was supported by a Bijzonder Onderzoeksfonds (BOF) internal fund at KU Leuven, namely the C1 project fund with reference C14/23/096. *Pavel Chizhov* was supported by JetBrains via the ERIC project. *Pieter Delobelle* was supported by the Flemish Government under the “Onderzoeksprogramma Artificiële Intelligentie (AI) Vlaanderen” programme. *Desmond Elliott* was supported by research grant (VIL53122) from VILLUM FONDEN and by the Carlsberg Foundation, grant CF25 -1494. *Omer Goldman* was supported by the Blavatnik Family Foundation. *Tatsuya Hiraoka* was supported by JSPS KAKENHI Grant Number JP24K20852. *Ilker Kesen* was supported by the European Union’s Horizon 2020 research and innovation program under grant agreement No. 101135671 (TrustLLM). *Jindřich Libovický* and *Gianluca Vico* were supported by the Czech Science Foundation Project No. 25-16242S. *Clara Meister* was supported by the Swiss AI Initiative. *Yuval Pinter* was supported by the Israel Science Foundation, grant No. 1166/23. *Suchir Salhan* was supported by Cambridge University Press & Assessment.

Tokenization Terms

A

alphabet	13
ascii	34

B

bits-per-byte	48
bits-per-character	48
byte premiums	34
byte-level fallback	36
byte-tuple encoding	19

C

canonical tokenization	8
character encodings	34
characters	13
chat (prompt) templates	75
chunkability	54
code point	33
code switching	43
cognitive plausibility	54
constrained generation	94
contextual coherence	53
control tokens	75
corpus token count	42
cross-tokenizer distillation	82
crosslingual transfer	37

D

detokenization	8
----------------------	---

E

embedding layer	9
extrinsic evaluation	48

F

fertility	51
-----------------	----

G

glyphs	33
grapheme	33

H

hashed embeddings	92
-------------------------	----

I

inference	13
intermediate tokens	18
intrinsic evaluation	48

K

knowledge transfer	79
--------------------------	----

L

language confusion	44
language mixing	44
language modeling	8
language variation	44
latent tokens	60
lattice tokenizations	26

leaf tokens	80
logits	9

M

marginalize	9
morphemes	38
morphological alignment	53
morphologically aware tokenization	39
morphs	40
multi-word tokens	41
multiplexing tokenizer	26

N

non-canonical tokenization	55
normalization	13

O

orthographic granularity	33
out-of-vocabulary	5
output layer	9
oversegmentation	42

P

partial token problem	95
percentile frequency	53
pretokenization	13
pretokens	28

R

rare tokens	18
raw text	5
rényi efficiency	52

S

script-aware tokenizers	36
single token retention rate	51
stochastic tokenization	26
subword lattice	84
subword promotion	84
subword regularization	55
subword tokenization	2
subword tokenizer	8
subword tokens	8
subword transducer	84
supertokens	29

T

token healing	96
token parity	42
token tax	42
tokenization	2
tokenization inference algorithm	5
tokenization-aware subword promotion	85
tokenizer	5
tokenizer training	13
tokenizer transfer	79
tokenizer-agnostic subword lattice	84
tokenizer-agnostic subword promotion	84
tokens	2
top-down vocabulary construction	20

transliterated encoding 37

U

universal **bpe** automaton 86

unreachable tokens 18

utf-8 34

V

visual text representation 65

vocabulary 5

vocabulary overlap 43

vocabulary trimming 18

Z

zero-shot tokenizer transfer 81

zipfianness 53

Systems and Datasets

A

Acoustic BPE	90
ALM	83
AU-Nets	63
Autoregressive U-Nets	63

B

BLT	63
Bolmo	64
BoundlessBPE	19
BPE-dropout	26
BPE-knockout	19
ByT5	60
Byte Latent Transformer	63
Byte-pair encoding (BPE)	13

C

CAMeL Tools	30
CANINE	61
CharacterBERT	61
Charformer	61
CLIPPO	68
ConvexTok	23

D

DeepSeek-OCR	69
DiVRit	68
DSKD	82
Dynamic Programming Encoding (DPE)	25
Dynamic Token Pooling (DTP)	62

E

eByte	61
ElasticTok	91
EvaByte	60

F

Fast BLT	64
Few Longest Token Approximation (FLOTA)	25
FlexiTokens	63

G

Glyph	69
GOLD	82
GRaMPa	56
Guidance	94

H

H-Net	63
HAT	63
Hierarchical Autoregressive Transformer	63
Hourglass Transformer	61
HuBERT	90

I

IndicNLP	38
IndicSuperTokenizer	34

J

Juman++	30
---------	----

L

Language-Aware Token Boosting (LATB)	46
LBPE	25

M

MADAMIRA	30
MAGNET	62
MambaByte	60
MANTa	62
MaxMatch	24
MaxMatch-Dropout	26
MBLM	61
MeCab	30
MegaByte	61
MinED	82
MinGram	20
MIXAR	68
Morfessor	40
MorphBPE	40
MorphPiece	40
Moses	34
MrT5	62
MYTE	46

O

Orthographic Syllable Pair Encoding (OSPE)	45
Outlines	94
Overlap BPE (OBPE)	45

P

PA-BPE	19
Parity-aware BPE	45
PathPiece	25
PickyBPE	18
PIXAR	68
PIXEL	67
PixelGPT	69

Q

QTok	51
------	----

R

ReBPE	19
-------	----

S

S-BPE	19
SA-BPE	19
SaGe	23
ScaffoldBPE	18
SCRIPT	34
SGLang	94
SpaceByte	61
Sparse Transformer	60
SPLINTER	39
Sudachi	30
SuperBPE	19

T

TkTtT	51
TokCollate	51
Tokenizer Intrinsic Benchmark	51
tokenizer-scorer	51
TokEval	51
TokSuite	51
TrimmedBPE	18

U

ULD	82
Unigram	20
uroman	38

V

VIST	69
ViT-MAE	67
VOLT	23

W

WordPiece	20
-----------	----

X

X-Token	82
XGrammar	94
XLM-V	45
XRayEmb	61

References

- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9904–9923, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.emnlp-main.614](https://doi.org/10.18653/v1/2023.emnlp-main.614). URL <https://aclanthology.org/2023.emnlp-main.614/>.
- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Valentin Hofmann, Tomasz Limisiewicz, Yulia Tsvetkov, and Noah A Smith. Magnet: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. *Advances in Neural Information Processing Systems*, 37:47790–47814, 2024.
- Sina Ahmadi. A tokenization system for the Kurdish language. In Marcos Zampieri, Preslav Nakov, Nikola Ljubešić, Jörg Tiedemann, and Yves Scherrer (eds.), *Proceedings of the 7th Workshop on NLP for Similar Languages, Varieties and Dialects*, pp. 114–127, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics (ICCL). URL <https://aclanthology.org/2020.vardial-1.11/>.
- Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. *Commun. ACM*, 18(6):333–340, 1975. doi:[10.1145/360825.360855](https://doi.org/10.1145/360825.360855).
- Koichi Akabe, Shunsuke Kanda, Yusuke Oda, and Shinsuke Mori. Vaporetto: Efficient japanese tokenization based on improved pointwise linear classification, 2024. URL <https://arxiv.org/abs/2406.17185>.
- Christopher Akiki, Stéphan Tulkens, and Token ##ization Discord. tokenizer-census (revision 3e2611d), 2026. URL <https://huggingface.co/datasets/tokenizer-eval/tokenizer-census>.
- Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. Character-level language modeling with deeper self-attention. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI’19/IAAI’19/EAAI’19. AAAI Press, 2019. ISBN 978-1-57735-809-1. doi:[10.1609/aaai.v33i01.33013159](https://doi.org/10.1609/aaai.v33i01.33013159).
- Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Buschhoff, Charvi Jain, Alexander Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, et al. Tokenizer choice for LLM training: Negligible or crucial? In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 3907–3924, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-naacl.247](https://doi.org/10.18653/v1/2024.findings-naacl.247). URL <https://aclanthology.org/2024.findings-naacl.247/>.
- Allep Alpha, :, Adnen Abdessaied, Artur Baranowski, Lukas Balles, Michael Barlow, Fabien C. Y. Benureau, Felix Berkenkamp, Lukas Bluebaum, Bastian Boll, Thomas F. Burns, Björn Deiseroth, Constantin Eichenberg, David Friede, Pablo Iyu Guerrero, Ahmed Hammam, Bastian Harren, Johann Higl, Yasser Jadidi, et al. A family of llms liberated from static vocabularies, 2026. URL <https://arxiv.org/abs/2603.15953>.
- Gül Sena Altıntaş, Malikeh Ehghaghi, Brian Lester, Fengyuan Liu, Wanru Zhao, Marco Ciccone, and Colin Raffel. Toksuite: Measuring the impact of tokenizer choice on language model behavior, 2025. URL <https://arxiv.org/abs/2512.20757>.

-
- Catherine Arnett and Benjamin Bergen. Why do language models perform worse for morphologically complex languages? In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (eds.), *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 6607–6623, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-main.441/>.
- Catherine Arnett, Tyler A. Chang, and Benjamin Bergen. A bit of a problem: Measurement disparities in dataset sizes across languages. In Maite Melero, Sakriani Sakti, and Claudia Soria (eds.), *Proceedings of the 3rd Annual Meeting of the Special Interest Group on Under-resourced Languages @ LREC-COLING 2024*, pp. 1–9, Torino, Italia, May 2024a. ELRA and ICCL. URL <https://aclanthology.org/2024.sigul-1.1/>.
- Catherine Arnett, Pamela D. Rivière, Tyler A. Chang, and Sean Trott. Different tokenization schemes lead to comparable performance in Spanish number agreement. In Garrett Nicolai, Eleanor Chodroff, Frederic Mailhot, and Çağrı Çöltekin (eds.), *Proceedings of the 21st SIGMORPHON workshop on Computational Research in Phonetics, Phonology, and Morphology*, pp. 32–38, Mexico City, Mexico, June 2024b. Association for Computational Linguistics. doi:10.18653/v1/2024.sigmorphon-1.4. URL <https://aclanthology.org/2024.sigmorphon-1.4/>.
- Catherine Arnett, Tyler A. Chang, Stella Biderman, and Benjamin K. Bergen. Explaining and mitigating crosslingual tokenizer inequities. In *Proceedings of the Thirty-Ninth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- Ehsaneddin Asgari, Yassine El Kheir, MohammadAli SadraeiJavaheri, and Ali Nazari. MorphBPE: Morphology-aware tokenization for efficient LLM training. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 41610–41621, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi:10.18653/v1/2026.findings-acl.2068. URL <https://aclanthology.org/2026.findings-acl.2068/>.
- Gal Astrach and Yuval Pinter. Probing subphonemes in morphology models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 12954–12961, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:10.18653/v1/2025.findings-acl.672. URL <https://aclanthology.org/2025.findings-acl.672/>.
- Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Robert Kwiatkowski, Ramesh Nallapati, Parminder Bhatia, and Bing Xiang. Token alignment via character matching for subword completion. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 15725–15738, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-acl.929. URL <https://aclanthology.org/2024.findings-acl.929/>.
- Mohammed Attia. Arabic tokenization system. In Violetta Cavalli-Sforza and Imed Zitouni (eds.), *Proceedings of the 2007 Workshop on Computational Approaches to Semitic Languages: Common Issues and Resources*, pp. 65–72, Prague, Czech Republic, June 2007. Association for Computational Linguistics. URL <https://aclanthology.org/W07-0809/>.
- Eugene Bagdasaryan, Congzheng Song, Rogier van Dalen, Matt Seigel, and Áine Cahill. Training a tokenizer for free with private federated learning, 2022. URL <https://arxiv.org/abs/2203.09943>.
- Gunjan Balde, Soumyadeep Roy, Mainack Mondal, and Niloy Ganguly. Adaptive BPE tokenization for enhanced vocabulary adaptation in finetuning pretrained language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 14724–14733, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-emnlp.863. URL <https://aclanthology.org/2024.findings-emnlp.863/>.

-
- Gunjan Balde, Soumyadeep Roy, Mainack Mondal, and Niloy Ganguly. Evaluation of LLMs in medical text summarization: The role of vocabulary adaptation in high OOV settings. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 22989–23004, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.1179](https://doi.org/10.18653/v1/2025.findings-acl.1179). URL <https://aclanthology.org/2025.findings-acl.1179/>.
- Miguel Ballesteros, Chris Dyer, and Noah A. Smith. Improved transition-based parsing by modeling characters instead of words with LSTMs. In Lluís Màrquez, Chris Callison-Burch, and Jian Su (eds.), *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 349–359, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi:[10.18653/v1/D15-1041](https://doi.org/10.18653/v1/D15-1041). URL <https://aclanthology.org/D15-1041/>.
- David A Balota, Melvin J Yap, Keith A Hutchison, Michael J Cortese, Brett Kessler, Bjorn Loftis, James H Neely, Douglas L Nelson, Greg B Simpson, and Rebecca Treiman. The english lexicon project. *Behavior research methods*, 39(3):445–459, 2007.
- Zishuo Bao, Jiaqi Leng, Junxiong Wang, Bowen Peng, and Yucheng Lu. Distilling token-trained models into byte-level models, 2026. URL <https://arxiv.org/abs/2602.01007>.
- Khuyagbaatar Batsuren, Gábor Bella, Aryaman Arora, Viktor Martinovic, Kyle Gorman, Zdeněk Žabokrtský, Amarsanaa Ganbold, Šárka Dohnalová, Magda Ševčíková, Kateřina Pelegrinová, Fausto Giunchiglia, Ryan Cotterell, and Ekaterina Vylomova. The SIGMORPHON 2022 shared task on morpheme segmentation. In Garrett Nicolai and Eleanor Chodroff (eds.), *Proceedings of the 19th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pp. 103–116, Seattle, Washington, July 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.sigmorphon-1.11](https://doi.org/10.18653/v1/2022.sigmorphon-1.11). URL <https://aclanthology.org/2022.sigmorphon-1.11/>.
- Khuyagbaatar Batsuren, Ekaterina Vylomova, Verna Dankers, Tsetsuukhei Delgerbaatar, Omri Uzan, Yuval Pinter, and Gábor Bella. Evaluating subword tokenization: Alien subword composition and oov generalization challenge, 2024. URL <https://arxiv.org/abs/2404.13292>.
- Thomas Bauwens. BPE-knockout: Systematic review of BPE tokenisers and their flaws with application in Dutch morphology. Master’s thesis, KU Leuven, 2023. URL <https://bauwenst.github.io/cdn/doc/pdf/2023/masterthesis.pdf>.
- Thomas Bauwens. TkTkT: The Tokeniser Toolkit, 2024. URL <https://github.com/bauwenst/TkTkT>.
- Thomas Bauwens and Miryam de Lhoneux. ReBPE: Iteratively improving the internal structure of a structured tokeniser by mining its internal structure. In Vera Demberg, Kentaro Inui, and Lluís Màrquez (eds.), *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 4075–4090, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi:[10.18653/v1/2026.findings-eacl.211](https://doi.org/10.18653/v1/2026.findings-eacl.211). URL <https://aclanthology.org/2026.findings-eacl.211/>.
- Thomas Bauwens and Pieter Delobelle. BPE-knockout: Pruning pre-existing BPE tokenisers with backwards-compatible morphological semi-supervision. In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 5810–5832, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.naacl-long.324](https://doi.org/10.18653/v1/2024.naacl-long.324). URL <https://aclanthology.org/2024.naacl-long.324/>.
- Thomas Bauwens, David Kaczér, and Miryam de Lhoneux. GRaMPa: Subword regularisation by skewing uniform segmentation distributions with an efficient path-counting Markov model. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 24228–24257, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:[10.18653/v1/2025.acl-long.1180](https://doi.org/10.18653/v1/2025.acl-long.1180). URL <https://aclanthology.org/2025.acl-long.1180/>.

-
- Lisa Beinborn and Yuval Pinter. Analyzing cognitive plausibility of subword tokenization. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4478–4486, Singapore, December 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.emnlp-main.272. URL <https://aclanthology.org/2023.emnlp-main.272/>.
- Yonatan Belinkov and Yonatan Bisk. Synthetic and natural noise both break neural machine translation. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL <https://openreview.net/forum?id=BJ8vJebC->.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
- Martin Berglund and Brink van der Merwe. Formalizing bpe tokenization. *Electronic Proceedings in Theoretical Computer Science*, 388:16–27, September 2023. ISSN 2075-2180. doi:10.4204/eptcs.388.4.
- Martin Berglund, Willeke Martens, and Brink van der Merwe. *Constructing a BPE Tokenization DFA*, pp. 66–78. Springer Nature Switzerland, 2024. ISBN 9783031711121. doi:10.1007/978-3-031-71112-1_5.
- Martin Berglund, Anna Jonsson, Willeke Martens, and Brink van der Merwe. Constructing compact bpe token dfas. In *International Conference on Implementation and Application of Automata*, pp. 27–40. Springer, 2025.
- Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. Nougat: Neural optical understanding for academic documents, 2023.
- Nicolas Boizard, Kevin El Haddad, C’eline Hudelot, and Pierre Colombo. Towards cross-tokenizer distillation: the universal logit distillation loss for llms. *Trans. Mach. Learn. Res.*, 2025, 2024. URL <https://api.semanticscholar.org/CorpusID:267750227>.
- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146, 2017. doi:10.1162/tacl_a_00051. URL <https://aclanthology.org/Q17-1010/>.
- Nadav Borenstein, Phillip Rust, Desmond Elliott, and Isabelle Augenstein. PHD: Pixel-based language modeling of historical documents. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 87–107, Singapore, December 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.emnlp-main.7. URL <https://aclanthology.org/2023.emnlp-main.7/>.
- Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In Trevor Cohn, Yulan He, and Yang Liu (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4617–4624, Online, November 2020. Association for Computational Linguistics. doi:10.18653/v1/2020.findings-emnlp.414. URL <https://aclanthology.org/2020.findings-emnlp.414/>.
- Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. Bad characters: Imperceptible nlp attacks. In *2022 IEEE symposium on security and privacy (SP)*, pp. 1987–2004. IEEE, 2022.
- Maharaj Brahma, N J Karthika, Atul Singh, Devaraj Adiga, Smruti Bhate, Ganesh Ramakrishnan, Rohit Saluja, and Maunendra Sankar Desarkar. Morphtok: Morphologically grounded tokenization for indian languages, 2025. URL <https://arxiv.org/abs/2504.10335>.
- Rotem Brilliant and Yuval Pinter. Tokenizing crosslingual homographs, 2026. URL <https://arxiv.org/abs/2607.17689>.
- Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, Jennifer C. Lai, and Robert L. Mercer. An estimate of an upper bound for the entropy of English. *Computational Linguistics*, 18(1):31–40, 1992. URL <https://aclanthology.org/J92-1002/>.

-
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, et al. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Cristian Buciluă, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 535–541, 2006.
- Jacob Buckman and Graham Neubig. Neural lattice language models. *Transactions of the Association for Computational Linguistics*, 6:529–541, 2018. doi:10.1162/tacl_a_00036. URL <https://aclanthology.org/Q18-1036/>.
- Ben Burtenshaw, Joffrey Thomas, Thomas Simonini, and Sergio Paniego. The hugging face agents course. <https://github.com/huggingface/agents-course>, 2025. GitHub repository.
- Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokeni-sations. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Pro-ceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2104–2114, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:10.18653/v1/2021.emnlp-main.161. URL <https://aclanthology.org/2021.emnlp-main.161/>.
- Yekun Chai, Qingyi Liu, Jingwu Xiao, Shuohuan Wang, Yu Sun, and Hua Wu. Autoregressive pre-training on pixels and texts. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3106–3125, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.182. URL <https://aclanthology.org/2024.emnlp-main.182/>.
- Hwan Chang, Yonghyun Jun, and Hwanhee Lee. Chatinject: Abusing chat templates for prompt injection in LLM agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=WVhgFSKniL>.
- Andreas Chari, Iadh Ounis, and Sean MacAvaney. Lost in transliteration: Bridging the script gap in neural ir, 2025. URL <https://arxiv.org/abs/2505.08411>.
- Ivi Chatzi, Nina Corvelo Benz, Stratis Tsirtsis, and Manuel Gomez-Rodriguez. Canonical autoregressive generation, 2025. URL <https://arxiv.org/abs/2506.06446>.
- Ivi Chatzi, Nina Corvelo Benz, Stratis Tsirtsis, and Manuel Gomez-Rodriguez. Tokenization multiplicity leads to arbitrary price variation in llm-as-a-service, 2026. URL <https://arxiv.org/abs/2506.06446>.
- Iaroslav Chelombitko, Egor Safronov, and Aleksey Komissarov. Qtok: A comprehensive framework for evaluating multilingual tokenizer quality in large language models, 2024. URL <https://arxiv.org/abs/2410.12989>.
- Delong Chen, Samuel Cahyawijaya, Jianfeng Liu, Baoyuan Wang, and Pascale Fung. Subobject-level image tokenization. In *Forty-second International Conference on Machine Learning*, 2025a. URL <https://openreview.net/forum?id=imkFoKwFwd>.
- Junyi Chen, Shihao Bai, Zaijun Wang, Siyu Wu, Chuheng Du, Hailong Yang, Ruihao Gong, Shengzhong Liu, Fan Wu, and Guihai Chen. Pre³: Enabling deterministic pushdown automata for faster structured LLM generation. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11253–11267, Vienna, Austria, July 2025b. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:10.18653/v1/2025.acl-long.551. URL <https://aclanthology.org/2025.acl-long.551/>.

-
- Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International conference on machine learning*, pp. 1691–1703. PMLR, 2020.
- Ting Chen, Martin Renqiang Min, and Yizhou Sun. Learning k-way d-dimensional discrete codes for compact embedding representations. In Jennifer G. Dy and Andreas Krause (eds.), *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pp. 853–862. PMLR, 2018. URL <http://proceedings.mlr.press/v80/chen18g.html>.
- Wenlin Chen, David Grangier, and Michael Auli. Strategies for training large vocabulary neural language models. In Katrin Erk and Noah A. Smith (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1975–1985, Berlin, Germany, August 2016. Association for Computational Linguistics. doi:[10.18653/v1/P16-1186](https://doi.org/10.18653/v1/P16-1186). URL <https://aclanthology.org/P16-1186/>.
- Jiale Cheng, Yusen Liu, Xinyu Zhang, Yulin Fei, Wenyi Hong, Ruiliang Lyu, Weihang Wang, Zhe Su, Xiaotao Gu, Xiao Liu, Yushi Bai, Jie Tang, Hongning Wang, and Minlie Huang. Glyph: Scaling context windows via visual-text compression, 2025. URL <https://arxiv.org/abs/2510.17800>.
- Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL <https://arxiv.org/abs/2601.07372>.
- Colin Cherry, George Foster, Ankur Bapna, Orhan Firat, and Wolfgang Macherey. Revisiting character-based neural machine translation with capacity and compression. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 4295–4305, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi:[10.18653/v1/D18-1461](https://doi.org/10.18653/v1/D18-1461). URL <https://aclanthology.org/D18-1461/>.
- Jie Chi and Peter Bell. Analyzing the role of part-of-speech in code-switching: A corpus-based study. In Yvette Graham and Matthew Purver (eds.), *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 1712–1721, St. Julian’s, Malta, March 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-eacl.120](https://doi.org/10.18653/v1/2024.findings-eacl.120). URL <https://aclanthology.org/2024.findings-eacl.120/>.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019. URL <https://arxiv.org/abs/1904.10509>.
- Nadezhda Chirkova and Sergey Troshin. CodeBPE: Investigating subtokenization options for large language model pretraining on source code. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=htL4UZ344nF>.
- Nadezhda Chirkova, Germán Kruszewski, Jos Rozen, and Marc Dymetman. Should you marginalize over possible tokenizations? In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–12, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.acl-short.1](https://doi.org/10.18653/v1/2023.acl-short.1). URL <https://aclanthology.org/2023.acl-short.1/>.
- Pavel Chizhov, Catherine Arnett, Elizaveta Korotkova, and Ivan P. Yamshchikov. BPE gets picky: Efficient vocabulary refinement during tokenizer training. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16587–16604, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.925](https://doi.org/10.18653/v1/2024.emnlp-main.925). URL <https://aclanthology.org/2024.emnlp-main.925/>.
- Pavel Chizhov, Egor Bogomolov, and Ivan P. Yamshchikov. From where words come: Efficient regularization of code tokenizers through source attribution. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational*

-
- Linguistics (Volume 1: Long Papers)*, pp. 39053–39073, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi:[10.18653/v1/2026.acl-long.1812](https://doi.org/10.18653/v1/2026.acl-long.1812). URL <https://aclanthology.org/2026.acl-long.1812/>.
- Dokook Choe, Rami Al-Rfou, Mandy Guo, Heeyoung Lee, and Noah Constant. Bridging the gap for tokenizer-free language models. *arXiv preprint arXiv:1908.10322*, 2019. URL <https://arxiv.org/abs/1908.10322>.
- Sanghyun Choo and Wonjoon Kim. A study on the evaluation of tokenizer performance in natural language processing. *Applied Artificial Intelligence*, 37(1):2175112, 2023.
- Hyung Won Chung, Dan Garrette, Kiat Chuan Tan, and Jason Riesa. Improving multilingual models with language-clustered vocabularies. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4536–4546, Online, November 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.emnlp-main.367](https://doi.org/10.18653/v1/2020.emnlp-main.367). URL <https://aclanthology.org/2020.emnlp-main.367/>.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, et al. Scaling instruction-finetuned language models, 2022. URL <https://arxiv.org/abs/2210.11416>.
- Woojin Chung and Jeonghoon Kim. Exploiting vocabulary frequency imbalance in language model pre-training. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing Systems*, volume 38, pp. 137676–137706. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/c911506647555da03db9c5277cd2221d-Paper-Conference.pdf.
- Yu-An Chung, Yu Zhang, Wei Han, Chung-Cheng Chiu, James Qin, Ruoming Pang, and Yonghui Wu. W2v-bert: Combining contrastive learning and masked language modeling for self-supervised speech pre-training, 2021. URL <https://arxiv.org/abs/2108.06209>.
- Kenneth Ward Church and Patrick Hanks. Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1):22–29, 1990. URL <https://aclanthology.org/J90-1003/>.
- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022a. doi:[10.1162/tacl_a_00448](https://doi.org/10.1162/tacl_a_00448). URL <https://aclanthology.org/2022.tacl-1.5/>.
- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 01 2022b. ISSN 2307-387X. doi:[10.1162/tacl_a_00448](https://doi.org/10.1162/tacl_a_00448).
- Marco Cagnetta and Cyril Allauzen. Tutorial: φ -transductions in openfst via the gallic semiring, 2025. URL <https://arxiv.org/abs/2506.17942>.
- Marco Cagnetta and Naoaki Okazaki. Tokenization as finite-state transduction. *Computational Linguistics*, 51(4):1119–1149, December 2025. doi:[10.1162/coli.a.23](https://doi.org/10.1162/coli.a.23). URL <https://aclanthology.org/2025.cl-4.2/>.
- Marco Cagnetta, Sangwhan Moon, Lawrence Wolf-sonkin, and Naoaki Okazaki. Parameter-efficient Korean character-level language modeling. In Andreas Vlachos and Isabelle Augenstein (eds.), *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 2350–2356, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.eacl-main.172](https://doi.org/10.18653/v1/2023.eacl-main.172). URL <https://aclanthology.org/2023.eacl-main.172/>.
- Marco Cagnetta, Tatsuya Hiraoka, Rico Sennrich, Yuval Pinter, and Naoaki Okazaki. An analysis of BPE vocabulary trimming in neural machine translation. In Shabnam Tafreshi, Arjun Akula, João Sedoc, Aleksandr Drozd, Anna Rogers, and Anna Rumshisky (eds.), *Proceedings of the Fifth Workshop on Insights*

-
- from *Negative Results in NLP*, pp. 48–50, Mexico City, Mexico, June 2024a. Association for Computational Linguistics. doi:[10.18653/v1/2024.insights-1.7](https://doi.org/10.18653/v1/2024.insights-1.7). URL <https://aclanthology.org/2024.insights-1.7/>.
- Marco Cagnetta, Vilém Zouhar, Sangwhan Moon, and Naoaki Okazaki. Two counterexamples to tokenization and the noiseless channel. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (eds.), *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 16897–16906, Torino, Italia, May 2024b. ELRA and ICCL. URL <https://aclanthology.org/2024.lrec-main.1469/>.
- Marco Cagnetta, Vilém Zouhar, and Naoaki Okazaki. Distributional properties of subword regularization. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 10753–10763, Miami, Florida, USA, November 2024c. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.600](https://doi.org/10.18653/v1/2024.emnlp-main.600). URL <https://aclanthology.org/2024.emnlp-main.600/>.
- Marco Cagnetta, David Pohl, Junyoung Lee, and Naoaki Okazaki. Pitfalls, subtleties, and techniques in automata-based subword-level constrained generation. In *Tokenization Workshop*, 2025. URL <https://openreview.net/forum?id=DFyb0GeGDS>.
- Common Crawl. Character encoding of html pages. <https://commoncrawl.github.io/cc-crawl-statistics/plots/charsets>, 2026. URL <https://archive.ph/TXtkV>. Accessed: 2026-02-23.
- Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8440–8451, Online, July 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.acl-main.747](https://doi.org/10.18653/v1/2020.acl-main.747). URL <https://aclanthology.org/2020.acl-main.747/>.
- Adrian Cosma, Stefan Ruseti, Emilian Radoi, and Mihai Dascalu. The strawberry problem: Emergence of character-level understanding in tokenized language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 28252–28263, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.1434](https://doi.org/10.18653/v1/2025.emnlp-main.1434). URL <https://aclanthology.org/2025.emnlp-main.1434/>.
- Ryan Cotterell, Christo Kirov, Mans Hulden, and Jason Eisner. On the complexity and typology of inflectional morphological systems. *Transactions of the Association for Computational Linguistics*, 7:327–342, 2019. doi:[10.1162/tacl_a_00271](https://doi.org/10.1162/tacl_a_00271). URL <https://aclanthology.org/Q19-1021/>.
- Aldan Creo and Shushanta Pudasaini. SilverSpeak: Evading AI-generated text detectors using homoglyphs. In Firoj Alam, Preslav Nakov, Nizar Habash, Iryna Gurevych, Shammur Chowdhury, Artem Shelmanov, Yuxia Wang, Ekaterina Artemova, Mucahid Kutlu, and George Mikros (eds.), *Proceedings of the 1st Workshop on GenAI Content Detection (GenAIDetect)*, pp. 1–46, Abu Dhabi, UAE, January 2025. International Conference on Computational Linguistics. URL <https://aclanthology.org/2025.genaidetect-1.1/>.
- Mathias Creutz and Krista Lagus. Unsupervised discovery of morphemes. In *Proceedings of the ACL-02 Workshop on Morphological and Phonological Learning*, pp. 21–30. Association for Computational Linguistics, July 2002. doi:[10.3115/1118647.1118650](https://doi.org/10.3115/1118647.1118650). URL <https://aclanthology.org/W02-0603/>.
- Zoltan Csaki, Pian Pawakapan, Urmish Thakker, and Qiantong Xu. Efficiently adapting pretrained language models to new languages, 2023. URL <https://arxiv.org/abs/2311.05741>.
- Zoltan Csaki, Bo Li, Jonathan Lingjie Li, Qiantong Xu, Pian Pawakapan, Leon Zhang, Yun Du, Hengyu Zhao, Changran Hu, and Urmish Thakker. SambaLingo: Teaching large language models new languages. In Jonne Sällevä and Abraham Owodunni (eds.), *Proceedings of the Fourth Workshop on Multilingual Representation Learning (MRL 2024)*, pp. 1–21, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.mrl-1.1](https://doi.org/10.18653/v1/2024.mrl-1.1). URL <https://aclanthology.org/2024.mrl-1.1/>.

-
- Yiming Cui, Ziqing Yang, and Xin Yao. Efficient and effective text encoding for chinese llama and alpaca, 2024. URL <https://arxiv.org/abs/2304.08177>.
- Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation, 2024. URL <https://arxiv.org/abs/2402.01035>.
- Falcon Z. Dai and Zheng Cai. Glyph-aware embedding of Chinese characters. In Manaal Faruqui, Hinrich Schuetze, Isabel Trancoso, and Yadollah Yaghoobzadeh (eds.), *Proceedings of the First Workshop on Subword and Character Level Models in NLP*, pp. 64–69, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi:[10.18653/v1/W17-4109](https://doi.org/10.18653/v1/W17-4109). URL <https://aclanthology.org/W17-4109/>.
- Kareem Darwish and Hamdy Mubarak. Farasa: A new fast and accurate Arabic word segmenter. In Nicoletta Calzolari, Khalid Choukri, Thierry Declerck, Sara Goggi, Marko Grobelnik, Bente Maegaard, Joseph Mariani, Helene Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (eds.), *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pp. 1070–1074, Portorož, Slovenia, May 2016. European Language Resources Association (ELRA). URL <https://aclanthology.org/L16-1170/>.
- Sam Dauncey and Roger Wattenhofer. You can learn tokenization end-to-end with reinforcement learning. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=OrXBwsTWDB>.
- Mark Davis. Forms of Unicode. Originally published on IBM developerWorks, <http://www-106.ibm.com/developerworks/unicode/library/sun-ibm-java.html?dwzone=unicode>; archived by the ICU Project, September 1999. URL https://www.icu-project.org/docs/papers/forms_of_unicode/.
- Mark Davis, Martin Dürst, , and Ken Whistler. Unicode Standard Annex #15: Unicode Normalization Forms. Unicode Standard Annex 15, The Unicode Consortium, 2023. URL <https://www.unicode.org/reports/tr15/tr15-57.html>.
- Wietse de Vries and Malvina Nissim. As good as new. how to successfully recycle English GPT-2 to make models for other languages. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 836–846, Online, August 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.findings-acl.74](https://doi.org/10.18653/v1/2021.findings-acl.74). URL <https://aclanthology.org/2021.findings-acl.74/>.
- Gregoire Deletang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Marten, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=jznbginyus>.
- Pieter Delobelle, Thomas Winters, and Bettina Berendt. Robbert-2022: Updating a dutch language model to account for evolving language use. *arXiv preprint arXiv:2211.08192*, 2022.
- Aria Desires. Text Rendering Hates You. <https://faultlore.com/blah/text-hates-you/>, September 2019. URL <https://archive.ph/Qbbju>. Accessed: 2026-05-01.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Tamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi:[10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423). URL <https://aclanthology.org/N19-1423/>.
- Mona Diab, Kadri Hacioglu, and Daniel Jurafsky. *Automatic Processing of Modern Standard Arabic Text*, pp. 159–179. Springer Netherlands, Dordrecht, 2007. ISBN 978-1-4020-6046-5. doi:[10.1007/978-1-4020-6046-5_9](https://doi.org/10.1007/978-1-4020-6046-5_9).

-
- Shuoyang Ding, Adithya Renduchintala, and Kevin Duh. A call for prudent choice of subword merge operations in neural machine translation. In Mikel Forcada, Andy Way, Barry Haddow, and Rico Sennrich (eds.), *Proceedings of Machine Translation Summit XVII: Research Track*, pp. 204–213, Dublin, Ireland, August 2019. European Association for Machine Translation. URL <https://aclanthology.org/W19-6620/>.
- Konstantin Dobler and Gerard de Melo. FOCUS: Effective embedding initialization for monolingual specialization of multilingual models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13440–13454, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.emnlp-main.829](https://doi.org/10.18653/v1/2023.emnlp-main.829). URL <https://aclanthology.org/2023.emnlp-main.829/>.
- Konstantin Dobler, Desmond Elliott, and Gerard de Melo. Token distillation: Attention-aware input embeddings for new tokens. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=n20ml5nGEo>.
- Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models, 2025. URL <https://arxiv.org/abs/2411.15100>.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- C.M. Downey, Terra Blevins, Nora Goldfine, and Shane Steinert-Threlkeld. Embedding structure matters: Comparing methods to adapt multilingual vocabularies to new languages. In Duygu Ataman (ed.), *Proceedings of the 3rd Workshop on Multi-lingual Representation Learning (MRL)*, pp. 268–281, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.mrl-1.20](https://doi.org/10.18653/v1/2023.mrl-1.20). URL <https://aclanthology.org/2023.mrl-1.20/>.
- Dávid Držík and Frantisek Forgac. Slovak morphological tokenizer using the byte-pair encoding algorithm. *PeerJ Computer Science*, 10, 2024. URL <https://api.semanticscholar.org/CorpusID:274275647>.
- Dávid Držík and Jozef Kapusta. The importance of morphology-aware subword tokenization for nlp tasks in slovak language modeling. *Expert Systems with Applications*, pp. 131492, 2026.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, et al. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Shagun Dwivedi and Kaushik Gopalan. Comparative analysis of the intrinsic metrics for tokenizers and their effect on downstream tasks for Hindi and Marathi. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 22652–22663, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi:[10.18653/v1/2026.acl-long.1037](https://doi.org/10.18653/v1/2026.acl-long.1037). URL <https://aclanthology.org/2026.acl-long.1037/>.
- Chris Dyer, Victor Chahuneau, and Noah A. Smith. A simple, fast, and effective reparameterization of IBM model 2. In Lucy Vanderwende, Hal Daumé III, and Katrin Kirchhoff (eds.), *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 644–648, Atlanta, Georgia, June 2013. Association for Computational Linguistics. URL <https://aclanthology.org/N13-1073/>.
- Benedikt Ebing, Lennart Keller, and Goran Glavaš. One script instead of hundreds? on pretraining romanized encoder language models, 2026. URL <https://arxiv.org/abs/2601.05776>.
- Lukas Edman, Antonio Toral, and Gertjan van Noord. Subword-delimited downsampling for better character-level translation. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Findings of the Association*

-
- for *Computational Linguistics: EMNLP 2022*, pp. 981–992, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.findings-emnlp.69](https://doi.org/10.18653/v1/2022.findings-emnlp.69). URL <https://aclanthology.org/2022.findings-emnlp.69/>.
- Lukas Edman, Gabriele Sarti, Antonio Toral, Gertjan van Noord, and Arianna Bisazza. Are character-level translations worth the wait? comparing ByT5 and mT5 for machine translation. *Transactions of the Association for Computational Linguistics*, 12:392–410, 2024a. doi:[10.1162/tac1_a_00651](https://doi.org/10.1162/tac1_a_00651). URL <https://aclanthology.org/2024.tacl-1.22/>.
- Lukas Edman, Helmut Schmid, and Alexander Fraser. CUTE: Measuring LLMs’ understanding of their tokens. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3017–3026, Miami, Florida, USA, November 2024b. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.177](https://doi.org/10.18653/v1/2024.emnlp-main.177). URL <https://aclanthology.org/2024.emnlp-main.177/>.
- Lukas Edman, Helmut Schmid, and Alexander Fraser. EXECUTE: A multilingual benchmark for LLM token understanding. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 1878–1887, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.95](https://doi.org/10.18653/v1/2025.findings-acl.95). URL <https://aclanthology.org/2025.findings-acl.95/>.
- Avia Efrat, Or Honovich, and Omer Levy. LMentry: A language model benchmark of elementary language tasks. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 10476–10501, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-acl.666](https://doi.org/10.18653/v1/2023.findings-acl.666). URL <https://aclanthology.org/2023.findings-acl.666/>.
- Steffen Eger and Yannik Benz. From hero to zéro: A benchmark of low-level adversarial attacks. In Kam-Fai Wong, Kevin Knight, and Hua Wu (eds.), *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*, pp. 786–803, Suzhou, China, December 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.aacl-main.79](https://doi.org/10.18653/v1/2020.aacl-main.79). URL <https://aclanthology.org/2020.aacl-main.79/>.
- Eric Egli, Matteo Manica, and Jannis Born. Multiscale byte language models – a hierarchical architecture for causal million-length sequence modeling, 2025. URL <https://arxiv.org/abs/2502.14553>.
- Hicham El Boukkouri, Olivier Ferret, Thomas Lavergne, Hiroshi Noji, Pierre Zweigenbaum, and Jun’ichi Tsujii. CharacterBERT: Reconciling ELMo and BERT for word-level open-vocabulary representations from characters. In Donia Scott, Nuria Bel, and Chengqing Zong (eds.), *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6903–6915, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi:[10.18653/v1/2020.coling-main.609](https://doi.org/10.18653/v1/2020.coling-main.609). URL <https://aclanthology.org/2020.coling-main.609/>.
- Yair Elboher and Yuval Pinter. Hebrew diacritics restoration using visual representation. In Hansi Hetiarachchi, Tharindu Ranasinghe, Alistair Plum, Paul Rayson, Ruslan Mitkov, Mohamed Gaber, Damith Premasiri, Fiona Anting Tan, and Lasitha Uyangodage (eds.), *Proceedings of the Second Workshop on Language Models for Low-Resource Languages (LoResLM 2026)*, pp. 504–514, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-377-7. doi:[10.18653/v1/2026.loreslm-1.44](https://doi.org/10.18653/v1/2026.loreslm-1.44). URL <https://aclanthology.org/2026.loreslm-1.44/>.
- Khalid N. Elmadani, Francois Meyer, and Jan Buys. University of cape town’s WMT22 system: Multilingual machine translation for Southern African languages. In Philipp Koehn, Loïc Barrault, Ondřej Bojar, Fethi Bougares, Rajen Chatterjee, Marta R. Costa-jussà, Christian Federmann, Mark Fishel, Alexander Fraser, Markus Freitag, Yvette Graham, Roman Grundkiewicz, Paco Guzman, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Tom Kocmi, André Martins, Makoto Morishita, et al. (eds.), *Proceedings of the Seventh Conference on Machine Translation (WMT)*, pp. 1039–1048, Abu Dhabi, United Arab Emirates

-
- (Hybrid), December 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.wmt-1.101](https://doi.org/10.18653/v1/2022.wmt-1.101). URL <https://aclanthology.org/2022.wmt-1.101/>.
- Erenca Erkaya. A comprehensive analysis of subword tokenizers for morphologically rich languages. Master’s thesis, Boğaziçi University, 2022.
- Behdad Esfahbod. State of Text Rendering 2024. <https://behdad.org/doc/text2024/>, July 2024. URL <https://archive.ph/qAmcV>. Accessed: 2026-05-01.
- Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 12873–12883, 2021.
- Carlos Esteves, Mohammed Suhail, and Ameesh Makadia. Spectral image tokenizer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 17181–17190, 2025.
- Eunjeon Project. Mecab-ko: Korean morphological analyzer. <https://bitbucket.org/eunjeon/mecab-ko/src/master/>, 2025. Accessed: 2026-01-26.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23(1), January 2022. ISSN 1532-4435.
- Negar Foroutan, Clara Meister, Debjit Paul, Joel Niklaus, Sina Ahmadi, Antoine Bosselut, and Rico Senrich. Parity-aware byte-pair encoding: Improving cross-lingual fairness in tokenization. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7514–7538, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi:[10.18653/v1/2026.acl-long.342](https://doi.org/10.18653/v1/2026.acl-long.342). URL <https://aclanthology.org/2026.acl-long.342/>.
- Alasdair Forsythe. Tokenmonster, 2023. URL <https://github.com/alsadairforsythe/tokenmonster>.
- Nathan Fradet, Nicolas Gutowski, Fabien Chhel, and Jean-Pierre Briot. Byte pair encoding for symbolic music. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2001–2020, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.emnlp-main.123](https://doi.org/10.18653/v1/2023.emnlp-main.123). URL <https://aclanthology.org/2023.emnlp-main.123/>.
- Kazuki Fujii, Taishi Nakamura, Mengsay Loem, Hiroki Iida, Masanari Ohi, Kakeru Hattori, Hirai Shota, Sakae Mizuki, Rio Yokota, and Naoaki Okazaki. Continual pre-training for cross-lingual LLM adaptation: Enhancing japanese language capabilities. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=TQdd1VhWbe>.
- Takuro Fujii, Koki Shibata, Atsuki Yamaguchi, Terufumi Morishita, and Yasuhiro Sogawa. How do different tokenizers perform on downstream tasks in scriptio continua languages?: A case study in Japanese. In Vishakh Padmakumar, Gisela Vallejo, and Yao Fu (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pp. 39–49, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.acl-srw.5](https://doi.org/10.18653/v1/2023.acl-srw.5). URL <https://aclanthology.org/2023.acl-srw.5/>.
- Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, February 1994. ISSN 0898-9788.
- Matthias Gallé. Investigating the effectiveness of BPE: The power of shorter sequences. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 1375–1381, Hong Kong, China, November 2019. Association for Computational Linguistics. doi:[10.18653/v1/D19-1141](https://doi.org/10.18653/v1/D19-1141). URL <https://aclanthology.org/D19-1141/>.

-
- Isabel O Gallegos, Ryan A Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen K Ahmed. Bias and fairness in large language models: A survey. *Computational linguistics*, 50(3):1097–1179, 2024.
- Kuofeng Gao, Yiming Li, Chao Du, Xin Wang, Xingjun Ma, Shu-Tao Xia, and Tianyu Pang. Imperceptible jailbreaking against large language models, 2025. URL <https://arxiv.org/abs/2510.05025>.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2020a. URL <https://arxiv.org/abs/2101.00027>.
- Tianyu Gao, Zirui Wang, Adithya Bhaskar, and Danqi Chen. Improving language understanding from screenshots, 2024. URL <https://arxiv.org/abs/2402.14073>.
- Yingqiang Gao, Nikola I. Nikolov, Yuhuang Hu, and Richard H.R. Hahnloser. Character-level translation with self-attention. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1591–1604, Online, July 2020b. Association for Computational Linguistics. doi:10.18653/v1/2020.acl-main.145. URL <https://aclanthology.org/2020.acl-main.145/>.
- Alba Táboas García, Piotr Przybyła, and Leo Wanner. Exploring morphology-aware tokenization: A case study on Spanish language modeling. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 30505–30518, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:10.18653/v1/2025.emnlp-main.1552. URL <https://aclanthology.org/2025.emnlp-main.1552/>.
- Leandro von Werra Garreth Lee, Guilherme Penedo and Thomas Wolf. From digits to decisions: How tokenization impacts arithmetic in llms, 2024. URL <https://huggingface.co/spaces/huggingface/number-tokenization-blog>.
- Bar Gazit, Shaltiel Shmidman, Avi Shmidman, and Yuval Pinter. Splintering nonconcatenative languages for better tokenization. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 22405–22417, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:10.18653/v1/2025.findings-acl.1151. URL <https://aclanthology.org/2025.findings-acl.1151/>.
- Leonidas Gee, Andrea Zugarini, Leonardo Rigutini, and Paolo Torroni. Fast vocabulary transfer for language model compression. In Yunyao Li and Angeliki Lazaridou (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 409–416, Abu Dhabi, UAE, December 2022. Association for Computational Linguistics. doi:10.18653/v1/2022.emnlp-industry.41. URL <https://aclanthology.org/2022.emnlp-industry.41/>.
- Leonidas Gee, Leonardo Rigutini, Marco Ernandes, and Andrea Zugarini. Multi-word tokenization for sequence compression. In Mingxuan Wang and Imed Zitouni (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 612–621, Singapore, December 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.emnlp-industry.58. URL <https://aclanthology.org/2023.emnlp-industry.58/>.
- Renato Geh, Honghua Zhang, Kareem Ahmed, Benjie Wang, and Guy Van Den Broeck. Where is the signal in tokenization space? In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3966–3979, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.230. URL <https://aclanthology.org/2024.emnlp-main.230/>.

-
- Renato Geh, Zilei Shao, and Guy Van Den Broeck. Adversarial tokenization. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 20738–20765, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:[10.18653/v1/2025.acl-long.1012](https://doi.org/10.18653/v1/2025.acl-long.1012). URL <https://aclanthology.org/2025.acl-long.1012/>.
- Jonas Geiping, Alex Stein, Manli Shu, Khalid Saifullah, Yuxin Wen, and Tom Goldstein. Coercing llms to do and reveal (almost) anything, 2024. URL <https://arxiv.org/abs/2402.14020>.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivi re, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, et al. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Daniela Gerz, Ivan Vuli c, Edoardo Ponti, Jason Naradowsky, Roi Reichart, and Anna Korhonen. Language modeling for morphologically rich languages: Character-aware modeling for word-level prediction. *Transactions of the Association for Computational Linguistics*, 6:451–465, 2018. doi:[10.1162/tacl_a_00032](https://doi.org/10.1162/tacl_a_00032). URL <https://aclanthology.org/Q18-1032/>.
- Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. *arXiv preprint arXiv:2401.06102*, 2024.
- Dan Gillick, Cliff Brunk, Oriol Vinyals, and Amarnag Subramanya. Multilingual language processing from bytes. In Kevin Knight, Ani Nenkova, and Owen Rambow (eds.), *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1296–1306, San Diego, California, June 2016. Association for Computational Linguistics. doi:[10.18653/v1/N16-1155](https://doi.org/10.18653/v1/N16-1155). URL <https://aclanthology.org/N16-1155/>.
- Mario Giulianelli, Luca Malagutti, Juan Luis Gastaldi, Brian DuSell, Tim Vieira, and Ryan Cotterell. On the proper treatment of tokenization in psycholinguistics. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18556–18572, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.1032](https://doi.org/10.18653/v1/2024.emnlp-main.1032). URL <https://aclanthology.org/2024.emnlp-main.1032/>.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozi re, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024. URL <https://arxiv.org/abs/2404.19737>.
- Charles Goddard and Fernando Fernandes Neto. Training-free tokenizer transplantation via orthogonal matching pursuit, 2025. URL <https://arxiv.org/abs/2506.06607>.
- Nathan Godey, Roman Castagn ,  ric de la Clergerie, and Beno t Sagot. MANTa: Efficient gradient-based tokenization for end-to-end robust language modeling. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 2859–2870, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.findings-emnlp.207](https://doi.org/10.18653/v1/2022.findings-emnlp.207). URL <https://aclanthology.org/2022.findings-emnlp.207/>.
- Aaron Gokaslan and Vanya Cohen. Openwebtext corpus. <http://Skylion007.github.io/OpenWebTextCorpus>, 2019. URL <https://archive.ph/qPFUx>.
- Omer Goldman, Avi Caciularu, Matan Eyal, Kris Cao, Idan Szpektor, and Reut Tsarfaty. Unpacking tokenization: Evaluating text compression and its correlation with model performance. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 2274–2286, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-acl.134](https://doi.org/10.18653/v1/2024.findings-acl.134). URL <https://aclanthology.org/2024.findings-acl.134/>.

-
- Siavash Golkar, Mariel Pettee, Michael Eickenberg, Alberto Bietti, Miles Cranmer, Geraud Krawezik, Francois Lanusse, Michael McCabe, Ruben Ohana, Liam Parker, Bruno Régaldou-Saint Blancard, Tiberiu Tesileanu, Kyunghyun Cho, and Shirley Ho. xval: A continuous numerical tokenization for scientific language models, 2024. URL <https://arxiv.org/abs/2310.02989>.
- Riley Goodside. x.com, January 2023. URL <https://archive.ph/OMhip>.
- Kyle Gorman and Yuval Pinter. Don’t touch my diacritics. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pp. 285–291, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-190-2. doi:[10.18653/v1/2025.naacl-short.25](https://doi.org/10.18653/v1/2025.naacl-short.25). URL <https://aclanthology.org/2025.naacl-short.25/>.
- Edward Gow-Smith, Harish Tayyar Madabushi, Carolina Scarton, and Aline Villavicencio. Improving tokenisation by alternative treatment of spaces. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11430–11443, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.emnlp-main.786](https://doi.org/10.18653/v1/2022.emnlp-main.786). URL <https://aclanthology.org/2022.emnlp-main.786/>.
- Thamme Gowda and Jonathan May. Finding the optimal vocabulary size for neural machine translation. In Trevor Cohn, Yulan He, and Yang Liu (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3955–3964, Online, November 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.findings-emnlp.352](https://doi.org/10.18653/v1/2020.findings-emnlp.352). URL <https://aclanthology.org/2020.findings-emnlp.352/>.
- Édouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou. Efficient softmax approximation for GPUs. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1302–1310. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/grave17a.html>.
- Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Knowledge distillation of large language models. *ArXiv*, abs/2306.08543, 2023. URL <https://api.semanticscholar.org/CorpusID:259164722>.
- Guidance AI. Guidance: A language model programming framework, 2023. URL <https://github.com/guidance-ai/guidance>. [Online; accessed 18-Dec-2024].
- Barry Haddow, Rachel Bawden, Antonio Valerio Miceli Barone, Jindřich Helcl, and Alexandra Birch. Survey of low-resource machine translation. *Computational Linguistics*, 48(3):673–732, September 2022. doi:[10.1162/coli_a_00446](https://doi.org/10.1162/coli_a_00446). URL <https://aclanthology.org/2022.cl-3.6/>.
- Josh Hadley. Character to glyph: How Unicode® text makes it to your screen. Presentation slides, Unicode Technology Workshop (UTW) 2023, Nov. 7–8, Google, Sunnyvale, CA, USA, 2023. URL https://www.unicode.org/events/utw/2023/documents/slides/2023_10_HadleyJosh_UTW.pdf.
- Dilek Z Hakkani-Tür, Kemal Oflazer, and Gökhan Tür. Statistical morphological disambiguation for agglutinative languages. *Computers and the Humanities*, 36(4):381–410, 2002.
- Mykola Haliutuk and Aleksander Smywiński-Pohl. Model-aware tokenizer transfer, 2025. URL <https://arxiv.org/abs/2510.21954>.
- Katharina Hämmerl, Tomasz Limisiewicz, Jindřich Libovický, and Alexander Fraser. Beyond literal token overlap: Token alignability for multilinguality. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.),

-
- Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pp. 756–767, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-190-2. doi:[10.18653/v1/2025.naacl-short.63](https://doi.org/10.18653/v1/2025.naacl-short.63). URL <https://aclanthology.org/2025.naacl-short.63/>.
- HyoJung Han, Akiko Eriguchi, Haoran Xu, Hieu Hoang, Marine Carpuat, and Huda Khayrallah. Adapters for altering LLM vocabularies: What languages benefit the most? In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=KxQRH0re9D>.
- Martin Haspelmath. The european linguistic area: Standard average european. In Martin Haspelmath, Ekkehard König, Wulf Oesterreicher, and Wolfgang Raible (eds.), *Language Typology and Language Universals: An International Handbook*, volume 2, pp. 1492–1510. De Gruyter Mouton, Berlin, 2001.
- Martin Haspelmath. An empirical test of the agglutination hypothesis. In *Universals of language today*, pp. 13–29. Springer, 2009.
- Martin Haspelmath and Andrea Sims. *Understanding Morphology*. Routledge, 2013. ISBN 9781134645961.
- Jonathan Hayase, Alisa Liu, Yejin Choi, Sewoong Oh, and Noah A. Smith. Data mixture inference: What do bpe tokenizers reveal about their training data?, 2024. URL <https://arxiv.org/abs/2407.16607>.
- Jonathan Hayase, Alisa Liu, Noah A. Smith, and Sewoong Oh. Sampling from your language model one byte at a time, 2025. URL <https://arxiv.org/abs/2506.14123>.
- Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, June 2022.
- Xuanli He, Gholamreza Haffari, and Mohammad Norouzi. Dynamic programming encoding for subword segmentation in neural machine translation. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 3042–3051, Online, July 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.acl-main.275](https://doi.org/10.18653/v1/2020.acl-main.275). URL <https://aclanthology.org/2020.acl-main.275/>.
- Yanjin He, Qingkai Zeng, and Meng Jiang. Pre-trained models perform the best when token distributions follow Zipf’s law. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 28009–28021, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.1421](https://doi.org/10.18653/v1/2025.emnlp-main.1421). URL <https://aclanthology.org/2025.emnlp-main.1421/>.
- Ulf Hermjakob, Jonathan May, and Kevin Knight. Out-of-the-box universal Romanization tool uroman. In Fei Liu and Thamar Solorio (eds.), *Proceedings of ACL 2018, System Demonstrations*, pp. 13–18, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi:[10.18653/v1/P18-4003](https://doi.org/10.18653/v1/P18-4003). URL <https://aclanthology.org/P18-4003/>.
- John Hewitt. Initializing New Word Embeddings for Pretrained Language Models, 2021. URL <https://nlp.stanford.edu/~johnhew/vocab-expansion.html>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. URL <https://arxiv.org/abs/1503.02531>.
- Tatsuya Hiraoka. MaxMatch-dropout: Subword regularization for WordPiece. In Nicoletta Calzolari, Chu-Ren Huang, Hansaem Kim, James Pustejovsky, Leo Wanner, Key-Sun Choi, Pum-Mo Ryu, Hsin-Hsi Chen, Lucia Donatelli, Heng Ji, Sadao Kurohashi, Patrizia Paggio, Nianwen Xue, Seokhwan Kim, Younggyun Hahm, Zhong He, Tony Kyungil Lee, Enrico Santus, Francis Bond, and Seung-Hoon Na (eds.), *Proceedings of the 29th International Conference on Computational Linguistics*, pp. 4864–4872, Gyeongju, Republic of Korea, October 2022. International Committee on Computational Linguistics. URL <https://aclanthology.org/2022.coling-1.430/>.

-
- Tatsuya Hiraoka and Kentaro Inui. Spelling-out is not straightforward: LLMs’ capability of tokenization from token to characters. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 13340–13353, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi:[10.18653/v1/2025.findings-emnlp.719](https://doi.org/10.18653/v1/2025.findings-emnlp.719). URL <https://aclanthology.org/2025.findings-emnlp.719/>.
- Tatsuya Hiraoka and Tomoya Iwakura. Downstream task-oriented neural tokenizer optimization with vocabulary restriction as post processing, 2023. URL <https://arxiv.org/abs/2304.10808>.
- Tatsuya Hiraoka and Naoaki Okazaki. Knowledge of pretrained language models on surface information of tokens, 2024. URL <https://arxiv.org/abs/2402.09808>.
- Tatsuya Hiraoka, Hiroyuki Shindo, and Yuji Matsumoto. Stochastic tokenization with a language model for neural text classification. In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 1620–1629, Florence, Italy, July 2019. Association for Computational Linguistics. doi:[10.18653/v1/P19-1158](https://doi.org/10.18653/v1/P19-1158). URL <https://aclanthology.org/P19-1158/>.
- Tatsuya Hiraoka, Sho Takase, Kei Uchiumi, Atsushi Keyaki, and Naoaki Okazaki. Optimizing word segmentation for downstream task. In Trevor Cohn, Yulan He, and Yang Liu (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1341–1351, Online, November 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.findings-emnlp.120](https://doi.org/10.18653/v1/2020.findings-emnlp.120). URL <https://aclanthology.org/2020.findings-emnlp.120/>.
- Tatsuya Hiraoka, Sho Takase, Kei Uchiumi, Atsushi Keyaki, and Naoaki Okazaki. Joint optimization of tokenization and downstream model. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 244–255, Online, August 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.findings-acl.21](https://doi.org/10.18653/v1/2021.findings-acl.21). URL <https://aclanthology.org/2021.findings-acl.21/>.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, et al. Training compute-optimal large language models, 2022. URL <https://arxiv.org/abs/2203.15556>.
- Valentin Hofmann, Janet Pierrehumbert, and Hinrich Schütze. DagoBERT: Generating derivational morphology with a pretrained language model. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3848–3861, Online, November 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.emnlp-main.316](https://doi.org/10.18653/v1/2020.emnlp-main.316). URL <https://aclanthology.org/2020.emnlp-main.316/>.
- Valentin Hofmann, Janet Pierrehumbert, and Hinrich Schütze. Superbizarre is not superb: Derivational morphology improves BERT’s interpretation of complex words. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3594–3608, Online, August 2021a. Association for Computational Linguistics. doi:[10.18653/v1/2021.acl-long.279](https://doi.org/10.18653/v1/2021.acl-long.279). URL <https://aclanthology.org/2021.acl-long.279/>.
- Valentin Hofmann, Janet B. Pierrehumbert, and Hinrich Schütze. Superbizarre Is Not Superb: Derivational Morphology Improves BERT’s Interpretation of Complex Words, June 2021b. URL <http://arxiv.org/abs/2101.00403>. arXiv:2101.00403 [cs].
- Valentin Hofmann, Hinrich Schuetze, and Janet Pierrehumbert. An embarrassingly simple method to mitigate undesirable properties of pretrained language model tokenizers. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 385–393, Dublin, Ireland, May 2022. Association for

-
- Computational Linguistics. doi:10.18653/v1/2022.acl-short.43. URL <https://aclanthology.org/2022.acl-short.43/>.
- Jimin Hong, Gibbeum Lee, and Jaewoong Cho. Accelerating multilingual language model for excessively tokenized languages. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 11095–11111, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-acl.660. URL <https://aclanthology.org/2024.findings-acl.660/>.
- Shuyang Hou, Yi Hu, and Muhan Zhang. Subtokentest: A practical benchmark for real-world sub-token understanding, 2026. URL <https://arxiv.org/abs/2601.09089>.
- Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units, 2021. URL <https://arxiv.org/abs/2106.07447>.
- Chen Hu, Yintao Tai, Antonio Vergari, Frank Keller, and Alessandro Suglia. Mixar: Scaling autoregressive pixel-based language models to multiple languages and scripts, 2026. URL <https://arxiv.org/abs/2604.11575>.
- Yifan Hu, Frank Liang, Dachuan Zhao, Jonathan Geuter, Varshini Reddy, Craig W Schmidt, and Chris Tanner. Entropy-driven pre-tokenization for byte-pair encoding. *arXiv preprint arXiv:2506.15889*, 2025.
- Hongzhi Huang, Defa Zhu, Banggu Wu, Yutao Zeng, Ya Wang, Qiyang Min, and Zhou Xun. Over-tokenized transformer: Vocabulary is generally worth scaling. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=gbeZKej40m>.
- HuggingFace. Tokenizers: Fast state-of-the-art tokenizers optimized for research and production, 2019. URL <https://github.com/huggingface/tokenizers>.
- Jaavid Aktar Husain, Raj Dabre, Aswanth Kumar, Jay Gala, Thanmay Jayakumar, Ratish Puduppully, and Anoop Kunchukuttan. RomanSetu: Efficiently unlocking multilingual capabilities of large language models via Romanization. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15593–15615, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.acl-long.833. URL <https://aclanthology.org/2024.acl-long.833/>.
- Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling, 2025. URL <https://arxiv.org/abs/2507.07955>.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=6t0Kwf8-jrj>.
- Go Inoue, Bashar Alhafni, Nizar Habash, and Timothy Baldwin. Do diacritics matter? evaluating the impact of Arabic diacritics on tokenization and LLM benchmarks. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 426–442, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi:10.18653/v1/2026.findings-eacl.22. URL <https://aclanthology.org/2026.findings-eacl.22/>.
- International Labour Organization. Average hourly earnings of employees – ILO, 2025. URL <https://ourworldindata.org/grapher/average-hourly-earnings>. With major processing by Our World in Data. Original data: ILOSTAT.
- Oona Itkonen and Jörg Tiedemann. The impact of vocabulary overlaps on knowledge transfer in multilingual machine translation. *arXiv preprint arXiv:2605.04196*, 2026.

-
- Itay Itzhak and Omer Levy. Models in a spelling bee: Language models implicitly learn the character composition of tokens. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (eds.), *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5061–5068, Seattle, United States, July 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.naacl-main.373](https://doi.org/10.18653/v1/2022.naacl-main.373). URL <https://aclanthology.org/2022.naacl-main.373/>.
- Haris Jabbar. MorphPiece: A linguistic tokenizer for large language models. *arXiv*, 2024. URL <https://arxiv.org/pdf/2307.07262.pdf>.
- Cassandra L. Jacobs and Yuval Pinter. Lost in space marking, 2022. URL <https://arxiv.org/abs/2208.01561>.
- Neel Jain, Khalid Saifullah, Yuxin Wen, John Kirchenbauer, Manli Shu, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Bring your own data! self-supervised evaluation for large language models, 2023. URL <https://arxiv.org/abs/2306.13651>.
- Eugene Jang, Kimin Lee, Jin-Woo Chung, Keuntae Park, and Seungwon Shin. Improbable bigrams expose vulnerabilities of incomplete tokens in byte-level tokenizers. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 18209–18216, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.919](https://doi.org/10.18653/v1/2025.emnlp-main.919). URL <https://aclanthology.org/2025.emnlp-main.919/>.
- Fengqing Jiang, Zhangchen Xu, Luyao Niu, Bill Yuchen Lin, and Radha Poovendran. Chatbug: A common vulnerability of aligned llms induced by chat templates. In Toby Walsh, Julie Shah, and Zico Kolter (eds.), *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*, pp. 27347–27355. AAAI Press, 2025. doi:[10.1609/AAAI.V39I26.34945](https://doi.org/10.1609/AAAI.V39I26.34945).
- Shenghu Jiang and Ruihao Gong. Incremental BPE tokenization. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=ZbWgrDzCCQo>.
- Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, @fernbear.bsky.social, Boza Vlado, You Jiacheng, Franz Cesista, Braden Koszarsky, and @Grad62304977. modded-nanogpt: Speedrunning the nanogpt baseline, 2024. URL <https://github.com/KellerJordan/modded-nanogpt>.
- Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S. Weld, Luke Zettlemoyer, and Omer Levy. SpanBERT: Improving pre-training by representing and predicting spans. *Transactions of the Association for Computational Linguistics*, 8:64–77, 2020a. doi:[10.1162/tacl_a_00300](https://doi.org/10.1162/tacl_a_00300). URL <https://aclanthology.org/2020.tacl-1.5/>.
- Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the NLP world. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 6282–6293, Online, July 2020b. Association for Computational Linguistics. doi:[10.18653/v1/2020.acl-main.560](https://doi.org/10.18653/v1/2020.acl-main.560). URL <https://aclanthology.org/2020.acl-main.560/>.
- Haeji Jung, Jinju Kim, Kyungjin Kim, Youjeong Roh, and David R. Mortensen. Happiness is sharing a vocabulary: A study of transliteration methods. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7797–7816, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi:[10.18653/v1/2026.eacl-long.365](https://doi.org/10.18653/v1/2026.eacl-long.365). URL <https://aclanthology.org/2026.eacl-long.365/>.
- Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd edition, 2026. URL <https://web.stanford.edu/~jurafsky/slp3/>. Online manuscript released January 6, 2026.

-
- Karthikeyan K, Zihan Wang, Stephen Mayhew, and Dan Roth. Cross-lingual ability of multilingual bert: An empirical study. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=HJeT3yrtDr>.
- Manodyna K H and Luc De Nardi. Rethinking polarity detection: When BPE fails across scripts. In Mo El-Haj, Paul Rayson, Mustafa Jarrar, Ignatius Ezeani, Saad Ezzini, Sina Ahmadi, Amal Haddad Haddad, Cynthia Amol, Ahmad Abdelali, and Shadi Abudalfa (eds.), *Proceedings of the 2nd Workshop on NLP for Languages Using Arabic Script*, pp. 6–14, Rabat, Morocco, March 2026. Association for Computational Linguistics. doi:[10.18653/v1/2026.abjadnlp-1.2](https://doi.org/10.18653/v1/2026.abjadnlp-1.2). URL <https://aclanthology.org/2026.abjadnlp-1.2/>.
- Teruno Kajiura, Shiho Takano, Tatsuya Hiraoka, and Kimio Kuramitsu. Vocabulary replacement in SentencePiece for domain adaptation. In Chu-Ren Huang, Yasunari Harada, Jong-Bok Kim, Si Chen, Yu-Yin Hsu, Emmanuele Chersoni, Pranav A, Winnie Huiheng Zeng, Bo Peng, Yuxi Li, and Junlin Li (eds.), *Proceedings of the 37th Pacific Asia Conference on Language, Information and Computation*, pp. 645–652, Hong Kong, China, December 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.paclic-1.64/>.
- Kakao Corp. Khaiiii: Kakao hangul analyzer iii. <https://github.com/kakao/khaiiii>, 2018. Apache-2.0 licensed open-source Korean morphological analyzer. Accessed: 2026-01-26.
- Julie Kallini, Dan Jurafsky, Christopher Potts, and Martijn Bartelds. False Friends are not foes: Investigating vocabulary overlap in multilingual language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 21138–21154, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi:[10.18653/v1/2025.findings-emnlp.1153](https://doi.org/10.18653/v1/2025.findings-emnlp.1153). URL <https://aclanthology.org/2025.findings-emnlp.1153/>.
- Julie Kallini, Shikhar Murty, Christopher D Manning, Christopher Potts, and Róbert Csordás. Mrt5: Dynamic token merging for efficient byte-level language models. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=VYWBmq1L7H>.
- Julie Kallini, Artidoro Pagnoni, Tomasz Limisiewicz, Gargi Ghosh, Luke Zettlemoyer, Christopher Potts, Xiaochuang Han, and Srinu Iyer. Fast byte latent transformer. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=mo6rHb1z82>.
- Nishant Kambhatla, Logan Born, and Anoop Sarkar. Auxiliary subword segmentations as related languages for low resource multilingual translation. In Helena Moniz, Lieve Macken, Andrew Rufener, Loïc Barrault, Marta R. Costa-jussà, Christophe Declercq, Maarit Koponen, Ellie Kemp, Spyridon Pilos, Mikel L. Forcada, Carolina Scarton, Joachim Van den Bogaert, Joke Daems, Arda Tezcan, Bram Vanroy, and Margot Fonteyne (eds.), *Proceedings of the 23rd Annual Conference of the European Association for Machine Translation*, pp. 131–140, Ghent, Belgium, June 2022. European Association for Machine Translation. URL <https://aclanthology.org/2022.eamt-1.16/>.
- Vani Kanjirang, Tanja Samardzic, Ljiljana Dolamic, and Fabio Rinaldi. Tokenization and representation biases in multilingual models on dialectal NLP tasks. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 23992–24010, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.1224](https://doi.org/10.18653/v1/2025.emnlp-main.1224). URL <https://aclanthology.org/2025.emnlp-main.1224/>.
- Leonid V. Kantorovich. Mathematical methods of organizing and planning production. *Management Science*, 6:366–422, 1960. URL <https://api.semanticscholar.org/CorpusID:62611375>.
- Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz. From tokens to words: On the inner lexicon of LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=328vch6tRs>.

-
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Andrej Karpathy. minbpe: Minimal, clean code for the byte pair encoding (bpe) algorithm commonly used in llm tokenization, 2024. URL <https://github.com/karpathy/minbpe>. GitHub repository.
- Violeta Kastreva, Philip Whittington, Dennis Komm, and Tiago Pimentel. Tokenisation over bounded alphabets is hard, 2025. URL <https://arxiv.org/abs/2511.15709>.
- Ayush Kaushal and Kyle Mahowald. What do tokens know about their characters and how do they know it? In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (eds.), *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2487–2507, Seattle, United States, July 2022. Association for Computational Linguistics. doi:10.18653/v1/2022.naacl-main.179. URL <https://aclanthology.org/2022.naacl-main.179/>.
- Muhammad Dehan Al Kautsar and Fajri Koto. Parallel tokenizers: Rethinking vocabulary design for cross-lingual transfer, 2025. URL <https://arxiv.org/abs/2510.06128>.
- Daisuke Kawahara, Sadao Kurohashi, and Kôiti Hasida. Construction of a Japanese relevance-tagged corpus. In Manuel González Rodríguez and Carmen Paz Suarez Araujo (eds.), *Proceedings of the Third International Conference on Language Resources and Evaluation (LREC’02)*, Las Palmas, Canary Islands - Spain, May 2002. European Language Resources Association (ELRA). URL <https://aclanthology.org/L02-1302/>.
- Ilker Kesen, Jonas F. Lotz, Ingo Ziegler, Phillip Rust, and Desmond Elliott. Multilingual pretraining for pixel language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 29594–29611, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:10.18653/v1/2025.emnlp-main.1504. URL <https://aclanthology.org/2025.emnlp-main.1504/>.
- Mikkel Wildner Kildeberg, Emil Allerslev Schledermann, Nicolaj Larsen, and Rob van der Goot. From smørrebrød to subwords: Training llms on danish, one morpheme at a time, 2025. URL <https://arxiv.org/abs/2504.01540>.
- Gyeongmin Kim, Junyoung Son, Jinsung Kim, Hyunhee Lee, and Heuseok Lim. Enhancing korean named entity recognition with linguistic tokenization strategies. *IEEE Access*, 9:151814–151823, 2021. doi:10.1109/ACCESS.2021.3126882.
- Hwichan Kim, Toshio Hirasawa, and Mamoru Komachi. Korean-to-Japanese neural machine translation system using hanja information. In Toshiaki Nakazawa, Hideki Nakayama, Chenchen Ding, Raj Dabre, Anoop Kunchukuttan, Win Pa Pa, Ondřej Bojar, Shantipriya Parida, Isao Goto, Hidayat Mino, Hiroshi Manabe, Katsuhito Sudoh, Sadao Kurohashi, and Pushpak Bhattacharyya (eds.), *Proceedings of the 7th Workshop on Asian Translation*, pp. 127–134, Suzhou, China, December 2020. Association for Computational Linguistics. doi:10.18653/v1/2020.wat-1.15. URL <https://aclanthology.org/2020.wat-1.15/>.
- SungHo Kim, Juhyeong Park, Yeachan Kim, and SangKeun Lee. KOMBO: Korean character representations based on the combination rules of subcharacters. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 5102–5119, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-acl.302. URL <https://aclanthology.org/2024.findings-acl.302/>.
- Yoon Kim, Yacine Jernite, David Sontag, and Alexander M. Rush. Character-aware neural language models. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI’16, pp. 2741–2749. AAAI Press, 2016.

-
- Artur Kiulian, Anton Polishko, Mykola Khandoga, Yevhen Kostiuk, Guillermo Gabrielli, Łukasz Gagała, Fadi Zaraket, Qusai Abu Obaida, Hrishikesh Garud, Wendy Wing Yee Mak, Dmytro Chaplinskyi, Selma Amor, and Grigol Peradze. From English-centric to effective bilingual: LLMs with custom tokenizers for underrepresented languages. In Mariana Romanyshyn (ed.), *Proceedings of the Fourth Ukrainian Natural Language Processing Workshop (UNLP 2025)*, pp. 1–13, Vienna, Austria (online), July 2025. Association for Computational Linguistics. ISBN 979-8-89176-269-5. doi:[10.18653/v1/2025.unlp-1.1](https://doi.org/10.18653/v1/2025.unlp-1.1). URL <https://aclanthology.org/2025.unlp-1.1/>.
- Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, Chris Dyer, Ondřej Bojar, Alexandra Constantin, and Evan Herbst. Moses: Open source toolkit for statistical machine translation. In Sophia Ananiadou (ed.), *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics Companion Volume Proceedings of the Demo and Poster Sessions*, pp. 177–180, Prague, Czech Republic, June 2007. Association for Computational Linguistics. URL <https://aclanthology.org/P07-2045/>.
- Terry Koo, Frederick Liu, and Luheng He. Automata-based constraints for language model decoding. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BDBdblmzyY>.
- Fajri Koto, Jey Han Lau, and Timothy Baldwin. IndoBERTweet: A pretrained language model for Indonesian Twitter with effective domain-specific vocabulary initialization. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 10660–10668, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.emnlp-main.833](https://doi.org/10.18653/v1/2021.emnlp-main.833). URL <https://aclanthology.org/2021.emnlp-main.833/>.
- László Kozma and Johannes Voderholzer. Theoretical analysis of byte-pair encoding, 2024. URL <https://arxiv.org/abs/2411.08671>.
- Ben Krause, Iain Murray, Steve Renals, and Liang Lu. Multiplicative LSTM for sequence modelling. *ICLR Workshop track*, 2017. URL <https://openreview.net/forum?id=SJCS5rXF1>.
- Linus Kreitner, Paul Hager, Jonathan Mendedoht, Georgios Kaissis, Daniel Rueckert, and Martin J. Menten. Efficient numeracy in language models through single-token number embeddings, 2025. URL <https://arxiv.org/abs/2510.06824>.
- Taku Kudo. Mecab: Yet another part-of-speech and morphological analyzer. <https://mecab.sourceforge.net/>, 2005.
- Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In Iryna Gurevych and Yusuke Miyao (eds.), *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi:[10.18653/v1/P18-1007](https://doi.org/10.18653/v1/P18-1007). URL <https://aclanthology.org/P18-1007/>.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Eduardo Blanco and Wei Lu (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi:[10.18653/v1/D18-2012](https://doi.org/10.18653/v1/D18-2012). URL <https://aclanthology.org/D18-2012/>.
- Solomon Kullback and R. A. Leibler. On information and sufficiency. *Annals of Mathematical Statistics*, 22: 79–86, 1951. URL <https://api.semanticscholar.org/CorpusID:120349231>.
- Anoop Kunchukuttan. The IndicNLP Library. https://github.com/anoopkunchukuttan/indic_nlp_library/blob/master/docs/indicnlp.pdf, 2020.
- Mikko Kurimo, Sami Virpioja, Ville Turunen, and Krista Lagus. Morpho challenge 2005-2010: Evaluations and results. In Jeffrey Heinz, Lynne Cahill, and Richard Wicentowski (eds.), *Proceedings of the 11th*

-
- Meeting of the ACL Special Interest Group on Computational Morphology and Phonology*, pp. 87–95, Uppsala, Sweden, July 2010. Association for Computational Linguistics. URL <https://aclanthology.org/W10-2211/>.
- Sadao Kurohashi and Daisuke Kawahara. Juman: A user-extensible morphological analyzer for japanese. <https://nlp.ist.i.kyoto-u.ac.jp/EN/index.php?JUMAN>, 2007.
- Will Kurt. Coalescence: making llm inference 5x faster. <https://blog.dottxt.ai/coalescence.html>, 2024. URL <https://archive.ph/dlT3v>.
- Matthieu Labeau and Alexandre Allauzen. Character and subword-based word representation for neural language modeling prediction. In Manaal Faruqui, Hinrich Schuetze, Isabel Trancoso, and Yadollah Yaghoobzadeh (eds.), *Proceedings of the First Workshop on Subword and Character Level Models in NLP*, pp. 1–13, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi:10.18653/v1/W17-4101. URL <https://aclanthology.org/W17-4101/>.
- Yuxuan Lai, Yijia Liu, Yansong Feng, Songfang Huang, and Dongyan Zhao. Lattice-BERT: Leveraging multi-granularity representations in Chinese pre-trained language models. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1716–1731, Online, June 2021. Association for Computational Linguistics. doi:10.18653/v1/2021.naacl-main.137. URL <https://aclanthology.org/2021.naacl-main.137/>.
- Andrew K. Lampinen and James L. McClelland. One-shot and few-shot learning of word embeddings, January 2018. URL <http://arxiv.org/abs/1710.10280>. arXiv:1710.10280 [cs].
- Sander Land. Pre-tokenization on punctuation in GPT-4. Token Contributions (Substack), August 2024a. URL <https://tokencontributions.substack.com/p/pre-tokenization-on-punctuation-in>.
- Sander Land. Unreachable tokens in GPT-4o. Token Contributions (Substack), August 2024b. URL <https://tokencontributions.substack.com/p/unreachable-tokens-in-gpt-4o>.
- Sander Land. On the biology of Claude’s tokenizer. Token Contributions (Substack), July 2026a. URL <https://tokencontributions.substack.com/p/on-the-biology-of-claudes-tokenizer>.
- Sander Land. Mingram: A minimalist unigram tokenizer with high compression and competitive morphological alignment, 2026b. URL <https://arxiv.org/abs/2606.27019>.
- Sander Land and Catherine Arnett. BPE stays on SCRIPT: Structured encoding for robust multilingual pretokenization. In *ICML 2025 Tokenization Workshop*, 2025. URL <https://openreview.net/forum?id=A078CqwaU0>.
- Sander Land and Max Bartolo. Fishing for magikarp: Automatically detecting under-trained tokens in large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 11631–11646, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.649. URL <https://aclanthology.org/2024.emnlp-main.649/>.
- Sander Land and Yuval Pinter. Which pieces does Unigram tokenization really need?, July 2026. URL <https://aclanthology.org/2026.findings-acl.316/>.
- Junyoung Lee, Marco Cognetta, Sangwhan Moon, and Naoaki Okazaki. Jamo-level subword tokenization in low-resource Korean machine translation. In Atul Kr. Ojha, Chao-hong Liu, Ekaterina Vylomova, Flammie Pirinen, Jonathan Washington, Nathaniel Oco, and Xiaobing Zhao (eds.), *Proceedings of the Eighth Workshop on Technologies for Machine Translation of Low-Resource Languages (LoResMT 2025)*, pp. 66–80, Albuquerque, New Mexico, U.S.A., May 2025a. Association for Computational Linguistics. ISBN 979-8-89176-230-5. doi:10.18653/v1/2025.loresmt-1.8. URL <https://aclanthology.org/2025.loresmt-1.8/>.

-
- Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dsUB4bst9S>.
- Seungyoon Lee, Seongtae Hong, Hyeonseok Moon, and Heuiseok Lim. Semantic aware linear transfer by recycling pre-trained language models for cross-lingual transfer. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 16180–16193, Vienna, Austria, July 2025b. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:10.18653/v1/2025.findings-acl.832. URL <https://aclanthology.org/2025.findings-acl.832/>.
- Pietro Lesci, Clara Meister, Thomas Hofmann, Andreas Vlachos, and Tiago Pimentel. Causal estimation of tokenisation bias. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28325–28340, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:10.18653/v1/2025.acl-long.1374. URL <https://aclanthology.org/2025.acl-long.1374/>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. URL <https://arxiv.org/abs/2211.17192>.
- Raph Levien. Text layout is a loose hierarchy of segmentation. <https://raphlinus.github.io/text/2020/10/26/text-layout.html>, October 2020. URL <https://archive.ph/IcqXD>. Accessed: 2026-05-01.
- Yanhong Li, Zixuan Lan, and Jiawei Zhou. Text or pixels? evaluating efficiency and understanding of LLMs with visual text inputs. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 10564–10578, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi:10.18653/v1/2025.findings-emnlp.558. URL <https://aclanthology.org/2025.findings-emnlp.558/>.
- Yinxi Li, Yuntian Deng, and Pengyu Nie. Tokdrift: When llm speaks in subwords but code speaks in grammar, 2025b. URL <https://arxiv.org/abs/2510.14972>.
- Haoran Lian, Yizhe Xiong, Zijia Lin, Jianwei Niu, Shasha Mo, Hui Chen, Peng Liu, and Guiguang Ding. Lbpe: Long-token-first tokenization to improve large language models, 2024. URL <https://arxiv.org/abs/2411.05504>.
- Haoran Lian, Yizhe Xiong, Jianwei Niu, Shasha Mo, Zhenpeng Su, Zijia Lin, Hui Chen, Jungong Han, and Guiguang Ding. Scaffold-bpe: enhancing byte pair encoding for large language models with simple and effective scaffold token removal. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’25/IAAI’25/EAAI’25. AAAI Press, 2025. ISBN 978-1-57735-897-8. doi:10.1609/aaai.v39i23.34633.
- Davis Liang, Hila Gonen, Yuning Mao, Rui Hou, Naman Goyal, Marjan Ghazvininejad, Luke Zettlemoyer, and Madian Khabsa. XLM-V: Overcoming the vocabulary bottleneck in multilingual masked language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13142–13152, Singapore, December 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.emnlp-main.813. URL <https://aclanthology.org/2023.emnlp-main.813/>.
- Jindřich Libovický and Jindřich Helcl. Lexically grounded subword segmentation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 7403–7420, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.421. URL <https://aclanthology.org/2024.emnlp-main.421/>.

-
- Jindřich Libovický, Helmut Schmid, and Alexander Fraser. Why don't people use character-level machine translation? In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 2470–2485, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.findings-acl.194](https://doi.org/10.18653/v1/2022.findings-acl.194). URL <https://aclanthology.org/2022.findings-acl.194/>.
- Jia Peng Lim, Shawn Tan, Davin Choo, and Hady W. Lauw. A partition cover approach to tokenization. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=prGyR9id7X>.
- Tomasz Limisiewicz, Jiří Balhar, and David Mareček. Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 5661–5681, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-acl.350](https://doi.org/10.18653/v1/2023.findings-acl.350). URL <https://aclanthology.org/2023.findings-acl.350/>.
- Tomasz Limisiewicz, Terra Blevins, Hila Gonen, Orevaoghene Ahia, and Luke Zettlemoyer. MYTE: Morphology-driven byte encoding for better and fairer multilingual language modeling. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15059–15076, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.acl-long.804](https://doi.org/10.18653/v1/2024.acl-long.804). URL <https://aclanthology.org/2024.acl-long.804/>.
- Tomasz Limisiewicz, Artidoro Pagnoni, Srini Iyer, Mike Lewis, Sachin Mehta, Alisa Liu, Margaret Li, Gargi Ghosh, and Luke Zettlemoyer. Compute optimal tokenization. 2026. URL <https://arxiv.org/abs/2605.01188>.
- Peiqin Lin, Shaoxiong Ji, Jörg Tiedemann, André F. T. Martins, and Hinrich Schütze. Mala-500: Massive language adaptation of large language models, 2024. URL <https://arxiv.org/abs/2401.13303>.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025a. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- LeAnn M. Lindsey, Nicole L. Pershing, Anisa Habib, Keith Dufault-Thompson, W. Zac Stephens, Anne J. Blaschke, Xiaofang Jiang, and Hari Sundar. The impact of tokenizer selection in genomic language models. *bioRxiv*, 2025b. doi:[10.1101/2024.09.09.612081](https://doi.org/10.1101/2024.09.09.612081). URL <https://www.biorxiv.org/content/early/2025/07/26/2024.09.09.612081>.
- Alisa Liu, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A. Smith, and Yejin Choi. Superbpe: Space travel for language models, 2025. URL <https://arxiv.org/abs/2503.13423>.
- Hong Liu, Jiaqi Zhang, Chao Wang, Xing Hu, Linkun Lyu, Jiaqi Sun, Xurui Yang, Bo Wang, Fengcun Li, Yulei Qian, Lingtong Si, Yerui Sun, Rumei Li, Peng Pei, Yuchen Xie, and Xunliang Cai. Scaling embeddings outperforms scaling experts in language models, 2026. URL <https://arxiv.org/abs/2601.21204>.
- Tiedong Liu and Bryan Kian Hsiang Low. Goat: Fine-tuned llama outperforms gpt-4 on arithmetic tasks, 2023. URL <https://arxiv.org/abs/2305.14201>.
- Yihong Liu, Peiqin Lin, Mingyang Wang, and Hinrich Schuetze. OFA: A framework of initializing unseen subword embeddings for efficient large-scale multilingual continued pretraining. In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 1067–1097, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-naacl.68](https://doi.org/10.18653/v1/2024.findings-naacl.68). URL <https://aclanthology.org/2024.findings-naacl.68/>.

-
- Jonas Lotz, Elizabeth Salesky, Phillip Rust, and Desmond Elliott. Text rendering strategies for pixel language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10155–10172, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.emnlp-main.628](https://doi.org/10.18653/v1/2023.emnlp-main.628). URL <https://aclanthology.org/2023.emnlp-main.628/>.
- Jonas F. Lotz, António V. Lopes, Stephan Peitz, Hendra Setiawan, and Leonardo Emili. Beyond text compression: Evaluating tokenizers across scales. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 32155–32173, Vienna, Austria, July 2025a. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:[10.18653/v1/2025.acl-long.1546](https://doi.org/10.18653/v1/2025.acl-long.1546). URL <https://aclanthology.org/2025.acl-long.1546/>.
- Jonas F. Lotz, Hendra Setiawan, Stephan Peitz, and Yova Kementchedjheva. Overcoming vocabulary constraints with pixel-level fallback. In *Second Conference on Language Modeling*, 2025b. URL <https://openreview.net/forum?id=lEaHNS2qEv>.
- Nishant Luitel, Nirajan Bekoju, Anand Kumar Sah, and Subarna Shakya. Can perplexity predict finetuning performance? an investigation of tokenization effects on sequential language models for Nepali. In Ofir Arviv, Miruna Clinciu, Kaustubh Dhole, Rotem Dror, Sebastian Gehrmann, Eliya Habba, Itay Itzhak, Simon Mille, Yotam Perlitz, Enrico Santus, João Sedoc, Michal Shmueli Scheuer, Gabriel Stanovsky, and Oyvind Tafjord (eds.), *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*, pp. 239–248, Vienna, Austria and virtual meeting, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-261-9. URL <https://aclanthology.org/2025.gem-1.21/>.
- Scott Lundberg and Marco Tulio Ribeiro. The art of prompt design: Prompt boundaries and token healing. <https://medium.com/data-science/the-art-of-prompt-design-prompt-boundaries-and-token-healing-3b2448b0be38>, May 2023. URL <https://archive.ph/KMfN2>.
- Jessica M. Lundin, Ada Zhang, Nihal Karim, Hamza Louzan, Victor Wei, David Adelani, and Cody Carroll. The token tax: Systematic bias in multilingual tokenization, 2025. URL <https://arxiv.org/abs/2509.05486>.
- Wentao Ma, Yiming Cui, Chenglei Si, Ting Liu, Shijin Wang, and Guoping Hu. CharBERT: Character-aware pre-trained language model. In Donia Scott, Nuria Bel, and Chengqing Zong (eds.), *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 39–50, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi:[10.18653/v1/2020.coling-main.4](https://doi.org/10.18653/v1/2020.coling-main.4). URL <https://aclanthology.org/2020.coling-main.4/>.
- Dominik Macháček, Jonáš Vidra, and Ondřej Bojar. Morphological and language-agnostic word segmentation for NMT. In *International Conference on Text, Speech, and Dialogue*, pp. 277–284. Springer, 2018.
- Aviya Maimon, Amir DN Cohen, Gal Vishne, Shauli Ravfogel, and Reut Tsarfaty. Iq test for llms: An evaluation framework for uncovering core skills in llms, 2025. URL <https://arxiv.org/abs/2507.20208>.
- Konstantinos Mamouras, Angela W. Li, and Yudi Yang. An efficient algorithm for streaming bpe tokenization. *Proc. ACM Program. Lang.*, 10(PLDI), June 2026. doi:[10.1145/3808330](https://doi.org/10.1145/3808330).
- KH Manodnya and Animesh Giri. Orthographic syllable pair encoding for language modelling tasks in indic languages. In *2023 IEEE MIT Undergraduate Research Technology Conference (URTC)*, pp. 1–6. IEEE, 2023.
- Elman Mansimov, Mitchell Stern, Mia Chen, Orhan Firat, Jakob Uszkoreit, and Puneet Jain. Towards end-to-end in-image neural machine translation. In Giuseppe Castellucci, Simone Filice, Soujanya Poria, Erik Cambria, and Lucia Specia (eds.), *Proceedings of the First International Workshop on Natural Language Processing Beyond Text*, pp. 70–74, Online, November 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.nlpbt-1.8](https://doi.org/10.18653/v1/2020.nlpbt-1.8). URL <https://aclanthology.org/2020.nlpbt-1.8/>.

-
- Kelly Marchisio, Patrick Lewis, Yihong Chen, and Mikel Artetxe. Mini-Model Adaptation: Efficiently Extending Pretrained Models to New Languages via Aligned Shallow Training. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 5474–5490, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-acl.338](https://doi.org/10.18653/v1/2023.findings-acl.338). URL <https://aclanthology.org/2023.findings-acl.338/>.
- Kelly Marchisio, Wei-Yin Ko, Alexandre Berard, Théo Dehaze, and Sebastian Ruder. Understanding and mitigating language confusion in LLMs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 6653–6677, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.380](https://doi.org/10.18653/v1/2024.emnlp-main.380). URL <https://aclanthology.org/2024.emnlp-main.380/>.
- Costas Mavromatis, Petros Karypis, and George Karypis. Pack of LLMs: Model fusion at test-time via perplexity optimization. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=5Nsl0nlStc>.
- Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers can do arithmetic with the right embeddings. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 108012–108041. Curran Associates, Inc., 2024. doi:[10.52202/079017-3430](https://doi.org/10.52202/079017-3430). URL https://proceedings.neurips.cc/paper_files/paper/2024/file/c35986bc1ee29b31c1011481b77fe540-Paper-Conference.pdf.
- Clara Meister. UnigramLM: An attempt at writing the missing manual. Blog post, 2025. URL <https://archive.ph/jkyYy>.
- Clara Meister. TokEval: A tokenizer evaluation suite. In *Conference on Language Modeling*, 2026. URL <https://arxiv.org/abs/2608.18062>.
- Clara Meister, Suchir Salhan, Andrzej Szablewski, Pietro Lesci, Paula Buttery, and Tiago Pimentel. LangMAP: A language-adaptive approach to tokenization, 2026a. URL <https://arxiv.org/abs/2606.23566>.
- Clara Meister, Ahmetcan Yavuz, Pietro Lesci, and Tiago Pimentel. What language is this? Ask your tokenizer, 2026b. URL <https://arxiv.org/abs/2602.17655>.
- Asma Mekki, Inès Zribi, Mariem Ellouze, and Lamia Hadrich Belguith. Tokenization of tunisian arabic: A comparison between three machine learning models. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.*, 22(7), July 2023. ISSN 2375-4699. doi:[10.1145/3599234](https://doi.org/10.1145/3599234).
- Yuxian Meng, Wei Wu, Fei Wang, Xiaoya Li, Ping Nie, Fan Yin, Muyu Li, Qinghong Han, Xiaofei Sun, and Jiwei Li. Glyce: Glyph-vectors for chinese character representations. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 2742–2753, 2019a. URL <https://proceedings.neurips.cc/paper/2019/hash/452bf208bf901322968557227b8f6efe-Abstract.html>.
- Yuxian Meng, Wei Wu, Fei Wang, Xiaoya Li, Ping Nie, Fan Yin, Muyu Li, Qinghong Han, Xiaofei Sun, and Jiwei Li. Glyce: Glyph-vectors for chinese character representations. *Advances in Neural Information Processing Systems*, 32, 2019b.
- Merriam-Webster. How many words are there in english?, n.d. URL <https://www.merriam-webster.com/help/faq-how-many-english-words>. Accessed: 2026-03-02.
- Francois Meyer and Jan Buys. Subword segmental language modelling for nguni languages. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 6636–6649, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.findings-emnlp.494](https://doi.org/10.18653/v1/2022.findings-emnlp.494). URL <https://aclanthology.org/2022.findings-emnlp.494/>.

-
- Francois Meyer and Jan Buys. Subword segmental machine translation: Unifying segmentation and target sentence generation. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 2795–2809, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.findings-acl.175. URL <https://aclanthology.org/2023.findings-acl.175/>.
- Francois Meyer and Jan Buys. A systematic analysis of subwords and cross-lingual transfer in multilingual translation. In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 2194–2200, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-naacl.141. URL <https://aclanthology.org/2024.findings-naacl.141/>.
- Sabrina J. Mielke. Can you compare perplexity across different segmentations? <https://sjmielke.com/comparing-perplexities.htm>, Apr 2019. URL <https://archive.ph/D91Jk>.
- Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp, 2021. URL <https://arxiv.org/abs/2112.10508>.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- Milan Milić, Julie Kallini, and Ekaterina Shutova. Phonemes to the rescue: Multilingual tokenization based on international phonetic alphabet. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 40323–40349, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi:10.18653/v1/2026.acl-long.1872. URL <https://aclanthology.org/2026.acl-long.1872/>.
- Benjamin Minixhofer, Fabian Paischer, and Navid Rekabsaz. WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (eds.), *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 3992–4006, Seattle, United States, July 2022. Association for Computational Linguistics. doi:10.18653/v1/2022.naacl-main.293. URL <https://aclanthology.org/2022.naacl-main.293/>.
- Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulic. Zero-shot tokenizer transfer. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/532ce4fcf853023c4cf2ac38cbc5d002-Abstract-Conference.html.
- Benjamin Minixhofer, Tyler Murray, Tomasz Limisiewicz, Anna Korhonen, Luke Zettlemoyer, Noah A. Smith, Edoardo M. Ponti, Luca Soldaini, and Valentin Hofmann. Bolmo: Byteifying the next generation of language models, 2025a. URL <https://arxiv.org/abs/2512.15586>.
- Benjamin Minixhofer, Ivan Vulić, and Edoardo Maria Ponti. Universal cross-tokenizer distillation via approximate likelihood matching, 2025b. URL <https://arxiv.org/abs/2503.20083>.
- Taro Miyazaki, Sihan Tan, Tsubasa Uchida, and Hiroyuki Kaneko. Sign language translation with gloss pair encoding. In Eleni Efthimiou, Stavroula-Evita Fotinea, Thomas Hanke, Julie A. Hochgesang, Johanna Mesch, and Marc Schuler (eds.), *Proceedings of the LREC-COLING 2024 11th Workshop on the Representation and Processing of Sign Languages: Evaluation of Sign Language Resources*, pp. 262–268, Torino, Italia, May 2024. ELRA and ICCL. URL <https://aclanthology.org/2024.signlang-1.29/>.
- Daichi Mochihashi, Takeshi Yamada, and Naonori Ueda. Bayesian unsupervised word segmentation with nested Pitman-Yor language modeling. In Keh-Yih Su, Jian Su, Janyce Wiebe, and Haizhou Li (eds.),

-
- Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*, pp. 100–108, Suntec, Singapore, August 2009. Association for Computational Linguistics. URL <https://aclanthology.org/P09-1012/>.
- Sangwhan Moon and Naoaki Okazaki. Jamo pair encoding: Subcharacter representation-based extreme Korean vocabulary compression for efficient subword tokenization. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (eds.), *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pp. 3490–3497, Marseille, France, May 2020a. European Language Resources Association. ISBN 979-10-95546-34-4. URL <https://aclanthology.org/2020.lrec-1.429/>.
- Sangwhan Moon and Naoaki Okazaki. PatchBERT: Just-in-time, out-of-vocabulary patching. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7846–7852, Online, November 2020b. Association for Computational Linguistics. doi:10.18653/v1/2020.emnlp-main.631. URL <https://aclanthology.org/2020.emnlp-main.631/>.
- Sangwhan Moon, Won Ik Cho, Hye Joo Han, Naoaki Okazaki, and Nam Soo Kim. OpenKorPOS: Democratizing Korean tokenization with voting-based open corpus annotation. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Jan Odijk, and Stelios Piperidis (eds.), *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pp. 4975–4983, Marseille, France, June 2022. European Language Resources Association. URL <https://aclanthology.org/2022.lrec-1.531/>.
- Sangwhan Moon, Tatsuya Hiraoka, and Naoaki Okazaki. Bit-level bpe: Below the byte boundary, 2025. URL <https://arxiv.org/abs/2506.07541>.
- Ibraheem Muhammad Moosa, Mahmud Elahi Akhter, and Ashfia Binte Habib. Does transliteration help multilingual language modeling? In Andreas Vlachos and Isabelle Augenstein (eds.), *Findings of the Association for Computational Linguistics: EACL 2023*, pp. 670–685, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.findings-eacl.50. URL <https://aclanthology.org/2023.findings-eacl.50/>.
- Milad Moradi and Matthias Samwald. Evaluating the robustness of neural language models to input perturbations. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 1558–1570, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:10.18653/v1/2021.emnlp-main.117. URL <https://aclanthology.org/2021.emnlp-main.117/>.
- Hajime Morita, Daisuke Kawahara, and Sadao Kurohashi. Morphological analysis for unsegmented languages using recurrent neural network language model. In Lluís Màrquez, Chris Callison-Burch, and Jian Su (eds.), *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 2292–2297, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi:10.18653/v1/D15-1276. URL <https://aclanthology.org/D15-1276/>.
- Luca Moroni, Giovanni Puccetti, Pere-Lluís Huguet Cabot, Andrei Stefan Bejgu, Alessio Miaschi, Edoardo Barba, Felice Dell’Orletta, Andrea Esuli, and Roberto Navigli. Optimizing LLMs for Italian: Reducing token fertility and enhancing efficiency through vocabulary adaptation. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 6661–6675, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi:10.18653/v1/2025.findings-naacl.371. URL <https://aclanthology.org/2025.findings-naacl.371/>.
- Amit Moryossef, Clara Meister, Pavel Stepachev, and Desmond Elliott. Back to bytes: Revisiting tokenization through utf-8, 2025. URL <https://arxiv.org/abs/2510.16987>.

-
- Vladislav Mosin, Igor Samenko, Borislav Kozlovskii, Alexey Tikhonov, and Ivan P. Yamshchikov. Fine-tuning transformers: Vocabulary transfer. *Artificial Intelligence*, 317:103860, April 2023. ISSN 0004-3702. doi:[10.1016/j.artint.2023.103860](https://doi.org/10.1016/j.artint.2023.103860).
- Pooneh Mousavi, Gallil Maimon, Adel Moumen, Darius Petermann, Jiatong Shi, Haibin Wu, Haici Yang, Anastasia Kuznetsova, Artem Ploujnikov, Ricard Marxer, et al. Discrete audio tokens: More than a survey! *arXiv preprint arXiv:2506.10274*, 2025.
- Robert Mroczkowski, Piotr Rybak, Alina Wróblewska, and Ireneusz Gawlik. HerBERT: Efficiently pretrained transformer-based language model for Polish. In Bogdan Babych, Olga Kanishcheva, Preslav Nakov, Jakub Piskorski, Lidia Pivovarov, Vasyl Starko, Josef Steinberger, Roman Yangarber, Michał Marcińczuk, Senja Pollak, Pavel Přibán, and Marko Robnik-Šikonja (eds.), *Proceedings of the 8th Workshop on Balto-Slavic Natural Language Processing*, pp. 1–10, Kiyv, Ukraine, April 2021. Association for Computational Linguistics. URL <https://aclanthology.org/2021.bsnlp-1.1/>.
- Nandini Mundra, Aditya Nanda Kishore Khandavally, Raj Dabre, Ratish Puduppully, Anoop Kunchukuttan, and Mitesh M Khapra. An empirical comparison of vocabulary expansion and initialization approaches for language models. In Libby Barak and Malihe Alikhani (eds.), *Proceedings of the 28th Conference on Computational Natural Language Learning*, pp. 84–104, Miami, FL, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.conll-1.8](https://doi.org/10.18653/v1/2024.conll-1.8). URL <https://aclanthology.org/2024.conll-1.8/>.
- Alberto Muñoz-Ortiz, Verena Blaschke, and Barbara Plank. Evaluating pixel language models on non-standardized languages. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (eds.), *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 6412–6419, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-main.427/>.
- Sathvik Nair and Philip Resnik. Words, subwords, and morphemes: What really matters in the surprisal-reading time relationship? In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 11251–11260, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-emnlp.752](https://doi.org/10.18653/v1/2023.findings-emnlp.752). URL <https://aclanthology.org/2023.findings-emnlp.752/>.
- Itay Nakash, Nitay Calderon, Eyal Ben-David, Elad Hoffer, and Roi Reichart. Adaptivocab: Enhancing LLM efficiency in focused domains through lightweight vocabulary adaptation. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=TyXf9dwpZP>.
- Piotr Nawrot, Szymon Tworowski, Michał Tyrolski, Lukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. Hierarchical transformers are more efficient language models. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (eds.), *Findings of the Association for Computational Linguistics: NAACL 2022*, pp. 1559–1571, Seattle, United States, July 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.findings-naacl.117](https://doi.org/10.18653/v1/2022.findings-naacl.117). URL <https://aclanthology.org/2022.findings-naacl.117/>.
- Piotr Nawrot, Jan Chorowski, Adrian Lancucki, and Edoardo Maria Ponti. Efficient transformers with dynamic token pooling. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6403–6417, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.acl-long.353](https://doi.org/10.18653/v1/2023.acl-long.353). URL <https://aclanthology.org/2023.acl-long.353/>.
- Mir Tafseer Nayeem, Sawsan Alqahtani, Md Tahmid Rahman Laskar, Tasnim Mohiuddin, and M Saiful Bari. Beyond fertility: STRR as a metric for multilingual tokenization evaluation. In *NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*, 2025. URL <https://openreview.net/forum?id=QphCVdMCyG>.

-
- Pit Neitemeier, Björn Deiseroth, Constantin Eichenberg, and Lukas Balles. Hierarchical autoregressive transformers: Combining byte- and word-level processing for robust, adaptable language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tU074jg2vS>.
- Graham Neubig and Junjie Hu. Rapid adaptation of neural machine translation to new languages. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 875–880, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi:[10.18653/v1/D18-1103](https://doi.org/10.18653/v1/D18-1103). URL <https://aclanthology.org/D18-1103/>.
- Graham Neubig and Shinsuke Mori. Word-based partial annotation for efficient corpus construction. In Nicoletta Calzolari, Khalid Choukri, Bente Maegaard, Joseph Mariani, Jan Odijk, Stelios Piperidis, Mike Rosner, and Daniel Tapias (eds.), *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10)*, Valletta, Malta, May 2010. European Language Resources Association (ELRA). URL <https://aclanthology.org/L10-1281/>.
- Graham Neubig, Yosuke Nakata, and Shinsuke Mori. Pointwise prediction for robust, adaptable Japanese morphological analysis. In Dekang Lin, Yuji Matsumoto, and Rada Mihalcea (eds.), *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pp. 529–533, Portland, Oregon, USA, June 2011. Association for Computational Linguistics. URL <https://aclanthology.org/P11-2093/>.
- Duy-Kien Nguyen, Mahmoud Assran, Unnat Jain, Martin R Oswald, Cees GM Snoek, and Xinlei Chen. An image is worth more than 16x16 patches: Exploring transformers on individual pixels. *arXiv preprint arXiv:2406.09415*, 2024a.
- Viet Nguyen, Julian Brooke, and Timothy Baldwin. Sub-character neural language modelling in Japanese. In Manaal Faruqui, Hinrich Schuetze, Isabel Trancoso, and Yadollah Yaghoobzadeh (eds.), *Proceedings of the First Workshop on Subword and Character Level Models in NLP*, pp. 148–153, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi:[10.18653/v1/W17-4122](https://doi.org/10.18653/v1/W17-4122). URL <https://aclanthology.org/W17-4122/>.
- Xuan-Phi Nguyen, Wenxuan Zhang, Xin Li, Mahani Aljunied, Zhiqiang Hu, Chenhui Shen, Yew Ken Chia, Xingxuan Li, Jianyu Wang, Qingyu Tan, Liying Cheng, Guanzheng Chen, Yue Deng, Sen Yang, Chaoqun Liu, Hang Zhang, and Lidong Bing. SeaLLMs - large language models for Southeast Asia. In Yixin Cao, Yang Feng, and Deyi Xiong (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 294–304, Bangkok, Thailand, August 2024b. Association for Computational Linguistics. doi:[10.18653/v1/2024.acl-demos.28](https://doi.org/10.18653/v1/2024.acl-demos.28). URL <https://aclanthology.org/2024.acl-demos.28/>.
- Alex Nichol. Finding optimal tokenizers. <https://blog.aqnichol.com/2026/06/10/optimal-tokenizers/>, June 2026. URL <https://archive.ph/2XZo1>. Accessed: 2026-06-18.
- NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Hefernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loic Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, et al. Scaling neural machine translation to 200 languages. *Nature*, 630(8018):841–846, 2024. ISSN 1476-4687. doi:[10.1038/s41586-024-07335-x](https://doi.org/10.1038/s41586-024-07335-x).
- Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. Investigating the limitations of transformers with simple arithmetic tasks, 2021. URL <https://arxiv.org/abs/2102.13019>.
- Antoine Nzeyimana and Andre Niyongabo Rubungo. KinyabERT: a morphology-aware Kinyarwanda language model. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5347–5363, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.acl-long.367](https://doi.org/10.18653/v1/2022.acl-long.367). URL <https://aclanthology.org/2022.acl-long.367/>.

-
- Ossama Obeid, Nasser Zalmout, Salam Khalifa, Dima Taji, Mai Oudah, Bashar Alhafni, Go Inoue, Fadhli Eryani, Alexander Erdmann, and Nizar Habash. CAMEL tools: An open source python toolkit for Arabic natural language processing. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (eds.), *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pp. 7022–7032, Marseille, France, May 2020. European Language Resources Association. ISBN 979-10-95546-34-4. URL <https://aclanthology.org/2020.lrec-1.868/>.
- Byung-Doh Oh and William Schuler. Leading whitespaces of language models’ subword vocabulary pose a confound for calculating word probabilities. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3464–3472, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.202. URL <https://aclanthology.org/2024.emnlp-main.202/>.
- Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, et al. Olmo 3, 2026. URL <https://arxiv.org/abs/2512.13961>.
- OpenAI. tiktoken: Fast BPE tokenizer, 2022. URL <https://github.com/openai/tiktoken>.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, et al. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- Malte Ostendorff and Georg Rehm. Efficient language model training through cross-lingual and progressive transfer learning, 2023. URL <https://arxiv.org/abs/2301.09626>.
- Robert Östling and Jörg Tiedemann. Efficient word alignment with Markov Chain Monte Carlo. *Prague Bulletin of Mathematical Linguistics*, 106:125–146, October 2016. URL <http://ufal.mff.cuni.cz/pbml/106/art-ostling-tiedemann.pdf>.
- Outlines. dottxt-ai/outlines. <https://github.com/dottxt-ai/outlines>. URL <https://github.com/dottxt-ai/outlines>.
- Abraham Toluase Owodunni, Orevaoghene Ahia, and Sachin Kumar. FLEXITOKENS: Flexible tokenization for evolving language models. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 17170–17190, San Diego, California, United States, July 2026a. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi:10.18653/v1/2026.findings-acl.848. URL <https://aclanthology.org/2026.findings-acl.848/>.
- Abraham Toluase Owodunni, Chibuzor Okocha, Christan Grant, Tomasz Limisiewicz, and Sachin Kumar. Dynamic multi-byte prediction with hierarchical language models, 2026b. URL <https://arxiv.org/abs/2608.15454>.
- Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens, 2024. URL <https://arxiv.org/abs/2412.09871>.
- Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinini Iyer. Byte latent transformer: Patches scale better than tokens. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9238–9258, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:10.18653/v1/2025.acl-long.453. URL <https://aclanthology.org/2025.acl-long.453/>.

-
- Luca Pajola and Mauro Conti. Fall of giants: How popular text-based mlaas fall against a simple evasion attack, 2021. URL <https://arxiv.org/abs/2104.05996>.
- David D. Palmer. Tokenisation and sentence segmentation. In Robert Dale, Hermann Moisl, and Harold Somers (eds.), *Handbook of Natural Language Processing*, pp. 11–35. Marcel Dekker, 2000.
- Hyunji Hayley Park, Katherine J. Zhang, Coleman Haley, Kenneth Steimel, Han Liu, and Lane Schwartz. Morphology matters: A multilingual language modeling analysis. *Transactions of the Association for Computational Linguistics*, 9:261–276, 2021. doi:[10.1162/tacl_a_00365](https://doi.org/10.1162/tacl_a_00365). URL <https://aclanthology.org/2021.tacl-1.16/>.
- Kyubyong Park, Joohong Lee, Seongbo Jang, and Dawoon Jung. An empirical study of tokenization strategies for various Korean NLP tasks. In Kam-Fai Wong, Kevin Knight, and Hua Wu (eds.), *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*, pp. 133–142, Suzhou, China, December 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.aacl-main.17](https://doi.org/10.18653/v1/2020.aacl-main.17). URL <https://aclanthology.org/2020.aacl-main.17/>.
- Arfath Pasha, Mohamed Al-Badrashiny, Mona Diab, Ahmed El Kholly, Ramy Eskander, Nizar Habash, Manoj Pooleery, Owen Rambow, and Ryan Roth. MADAMIRA: A fast, comprehensive tool for morphological analysis and disambiguation of Arabic. In Nicoletta Calzolari, Khalid Choukri, Thierry Declerck, Hrafn Loftsson, Bente Maegaard, Joseph Mariani, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (eds.), *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC’14)*, pp. 1094–1101, Reykjavik, Iceland, May 2014. European Language Resources Association (ELRA). URL <https://aclanthology.org/L14-1479/>.
- Vaidehi Patil, Partha Talukdar, and Sunita Sarawagi. Overlap-based vocabulary generation improves cross-lingual transfer among related languages. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 219–233, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.acl-long.18](https://doi.org/10.18653/v1/2022.acl-long.18). URL <https://aclanthology.org/2022.acl-long.18/>.
- Carlos Miguel Patiño, Kashif Rasul, Quentin Gallouédec, Ben Burtenshaw, Sergio Paniego, Vaibhav Srivastav, Thibaud Frere, Ed Beeching, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. Unlocking on-policy distillation for any model family, 2025. URL <https://huggingface.co/spaces/HuggingFaceH4/on-policy-distillation>.
- Tim Pearce and Jinyeop Song. Reconciling kaplan and chinchilla scaling laws. In *TMLR*, 2024.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=n6SCkn2QaG>.
- Guilherme Penedo, Hynek Kydlíček, Vinko Sabolcec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro von Werra, and Thomas Wolf. Fineweb2: One pipeline to scale them all - adapting pre-training data processing to every language. *CoRR*, abs/2506.20920, 2025. doi:[10.48550/ARXIV.2506.20920](https://doi.org/10.48550/ARXIV.2506.20920).
- Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, et al. Rwkv: Reinventing rnns for the transformer era, 2023. URL <https://arxiv.org/abs/2305.13048>.
- Jing Peng, Yucheng Wang, Bohan Li, Yiwei Guo, Hankun Wang, YanGui Fang, Yu Xi, Haoyu Li, Xu Li, Ke Zhang, Shuai Wang, and Kai Yu. A survey on speech large language models for understanding. *IEEE Journal of Selected Topics in Signal Processing*, 20(1):2–31, 2026. doi:[10.1109/JSTSP.2025.3640535](https://doi.org/10.1109/JSTSP.2025.3640535).

-
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html.
- Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, and Sebastian Ruder. UNKs everywhere: Adapting multilingual language models to new scripts. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 10186–10203, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:10.18653/v1/2021.emnlp-main.800. URL <https://aclanthology.org/2021.emnlp-main.800/>.
- Buu Phan, Marton Havasi, Matthew Muckley, and Karen Ullrich. Understanding and mitigating tokenization bias in language models, 2024. URL <https://arxiv.org/abs/2406.16829>.
- Buu Phan, Brandon Amos, Itai Gat, Marton Havasi, Matthew Muckley, and Karen Ullrich. Exact byte-level probabilities from tokenized language models for fim-tasks and model ensembles, 2025. URL <https://arxiv.org/abs/2410.09303>.
- Buu Phan, Ashish Khisti, and Karen Ullrich. Cross-tokenizer likelihood scoring algorithms for language model distillation, 2026. URL <https://arxiv.org/abs/2512.14954>.
- Tiago Pimentel and Clara Meister. How to compute the probability of a word. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18358–18375, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.1020. URL <https://aclanthology.org/2024.emnlp-main.1020/>.
- Yuval Pinter, Robert Guthrie, and Jacob Eisenstein. Mimicking word embeddings using subword RNNs. In Martha Palmer, Rebecca Hwa, and Sebastian Riedel (eds.), *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 102–112, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi:10.18653/v1/D17-1010. URL <https://aclanthology.org/D17-1010/>.
- Yuval Pinter, Amanda Stent, Mark Dredze, and Jacob Eisenstein. Learning to look inside: Augmenting token-based encoders with character-level information, 2021. URL <https://arxiv.org/abs/2108.00391>.
- Telmo Pires, Eva Schlinger, and Dan Garrette. How multilingual is multilingual BERT? In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4996–5001, Florence, Italy, July 2019. Association for Computational Linguistics. doi:10.18653/v1/P19-1493. URL <https://aclanthology.org/P19-1493/>.
- Barbara Plank, Anders Søgaard, and Yoav Goldberg. Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss. In Katrin Erk and Noah A. Smith (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 412–418, Berlin, Germany, August 2016. Association for Computational Linguistics. doi:10.18653/v1/P16-2067. URL <https://aclanthology.org/P16-2067/>.
- Wessel Poelman and Miryam de Lhoneux. Form and meaning in intrinsic multilingual evaluations. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2503–2521, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi:10.18653/v1/2026.eacl-long.113. URL <https://aclanthology.org/2026.eacl-long.113/>.
- Wessel Poelman, Thomas Bauwens, and Miryam de Lhoneux. Confounding factors in relating model performance to morphology. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet

-
- Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 7262–7287, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.369](https://doi.org/10.18653/v1/2025.emnlp-main.369). URL <https://aclanthology.org/2025.emnlp-main.369/>.
- Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 100535–100570. Curran Associates, Inc., 2024. doi:[10.52202/079017-3189](https://doi.org/10.52202/079017-3189). URL https://proceedings.neurips.cc/paper_files/paper/2024/file/b6341525cd84f3be0ef203e4d7cd8556-Paper-Conference.pdf.
- Matt Post. A call for clarity in reporting BLEU scores. In Ondřej Bojar, Rajen Chatterjee, Christian Federmann, Mark Fishel, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Christof Monz, Matteo Negri, Aurélie Nèveol, Mariana Neves, Matt Post, Lucia Specia, Marco Turchi, and Karin Verspoor (eds.), *Proceedings of the Third Conference on Machine Translation: Research Papers*, pp. 186–191, Brussels, Belgium, October 2018. Association for Computational Linguistics. doi:[10.18653/v1/W18-6319](https://doi.org/10.18653/v1/W18-6319). URL <https://aclanthology.org/W18-6319/>.
- Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1882–1892, Online, July 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.acl-main.170](https://doi.org/10.18653/v1/2020.acl-main.170). URL <https://aclanthology.org/2020.acl-main.170/>.
- Taido Purason, Pavel Chizhov, Ivan P. Yamshchikov, and Mark Fishel. Teaching old tokenizers new words: Efficient tokenizer adaptation for pretrained models. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 6492–6516, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi:[10.18653/v1/2026.findings-eacl.341](https://doi.org/10.18653/v1/2026.findings-eacl.341). URL <https://aclanthology.org/2026.findings-eacl.341/>.
- Sukannya Purkayastha, Sebastian Ruder, Jonas Pfeiffer, Iryna Gurevych, and Ivan Vulić. Romanization-based large-scale adaptation of multilingual language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 7996–8005, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-emnlp.538](https://doi.org/10.18653/v1/2023.findings-emnlp.538). URL <https://aclanthology.org/2023.findings-emnlp.538/>.
- Alec Radford, Rafal Józefowicz, and Ilya Sutskever. Learning to generate reviews and discovering sentiment. *CoRR*, abs/1704.01444, 2017. URL <http://arxiv.org/abs/1704.01444>.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI*, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. Accessed: 2024-11-15.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PmLR, 2021.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Dinesh Ramoo. *Psychology of language*. BCcampus, BC Open Textbook Project, 2021.

-
- Souvik Rana, Arul Menezes, Ashish Kulkarni, Chandra Khatri, and Shubham Agarwal. Indicsupertokenizer: An optimized tokenizer for indic multilingual llms, 2025. URL <https://arxiv.org/abs/2511.03237>.
- Javier Rando and Florian Tramèr. The worst (but only) claude 3 tokenizer. <https://javirando.com/blog/2024/claude-tokenizer/>, 2024. URL <https://archive.ph/VaKT1>.
- Varshini Reddy, Craig W. Schmidt, Yuval Pinter, and Chris Tanner. How much is enough? the diminishing returns of tokenization training data, 2025. URL <https://arxiv.org/abs/2502.20273>.
- Varshini Reddy, Craig W Schmidt, Seth Ebner, Adam Wiemerslage, Yuval Pinter, and Chris Tanner. 1,729 vs. 1729: The effect of scripts and formats on LLM numeracy. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 10679–10696, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi:[10.18653/v1/2026.findings-acl.518](https://doi.org/10.18653/v1/2026.findings-acl.518). URL <https://aclanthology.org/2026.findings-acl.518/>.
- Machel Reid, Edison Marrese-Taylor, and Yutaka Matsuo. Subformer: Exploring weight sharing for parameter efficiency in generative transformers. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 4081–4090, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.findings-emnlp.344](https://doi.org/10.18653/v1/2021.findings-emnlp.344). URL <https://aclanthology.org/2021.findings-emnlp.344/>.
- François Remy, Pieter Delobelle, Hayastan Avetisyan, Alfiya Khabibullina, Miryam de Lhoneux, and Thomas Demeester. Trans-tokenization and cross-lingual vocabulary transfers: Language adaptation of llms for low-resource nlp, 2024. URL <https://arxiv.org/abs/2408.04303>.
- Richard H Richens and A Donald Booth. Some methods of mechanized translation. In *Proceedings of the Conference on Mechanical Translation*, 1952.
- Jihyeon Roh, Sang-Hoon Oh, and Soo-Young Lee. Unigram-normalized perplexity as a language model performance measure with different vocabulary sizes, 2020. URL <https://arxiv.org/abs/2011.13220>.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. URL <https://arxiv.org/abs/1505.04597>.
- Nicolas P. Rougier and Behdad Esfahbod. Digital typography: 25 years of text rendering in computer graphics. In *ACM SIGGRAPH 2018 Courses*, SIGGRAPH ’18, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450358095. doi:[10.1145/3214834.3214837](https://doi.org/10.1145/3214834.3214837).
- Aurko Roy, Rohan Anil, Guangda Lai, Benjamin Lee, Jeffrey Zhao, Shuyuan Zhang, Shibo Wang, Ye Zhang, Shen Wu, Rigel Swavelly, Tao, Yu, Phuong Dao, Christopher Fifty, Zhifeng Chen, and Yonghui Wu. N-grammer: Augmenting transformers with latent n-grams, 2022. URL <https://arxiv.org/abs/2207.06366>.
- Sebastian Ruder, Matthew E. Peters, Swabha Swayamdipta, and Thomas Wolf. Transfer learning in natural language processing. In Anoop Sarkar and Michael Strube (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Tutorials*, pp. 15–18, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi:[10.18653/v1/N19-5004](https://doi.org/10.18653/v1/N19-5004). URL <https://aclanthology.org/N19-5004/>.
- Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3118–3135, Online, August 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.acl-long.243](https://doi.org/10.18653/v1/2021.acl-long.243). URL <https://aclanthology.org/2021.acl-long.243/>.

-
- Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. Language modelling with pixels. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=FkSp8VW8RjH>.
- Vin Sachidananda, Jason Kessler, and Yi-An Lai. Efficient domain adaptation of language models via adaptive tokenization. In Nafise Sadat Moosavi, Iryna Gurevych, Angela Fan, Thomas Wolf, Yufang Hou, Ana Marasović, and Sujith Ravi (eds.), *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pp. 155–165, Virtual, November 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.sustainlp-1.16](https://doi.org/10.18653/v1/2021.sustainlp-1.16). URL <https://aclanthology.org/2021.sustainlp-1.16/>.
- Gözde Gül Şahin, Clara Vania, Ilia Kuznetsov, and Iryna Gurevych. LINSPECTOR: Multilingual probing tasks for word representations. *Computational Linguistics*, 46(2):335–385, June 2020. doi:[10.1162/coli_a_00376](https://doi.org/10.1162/coli_a_00376). URL <https://aclanthology.org/2020.cl-2.4/>.
- Elizabeth Salesky, David Etter, and Matt Post. Robust open-vocabulary translation from visual text representations. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 7235–7252, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.emnlp-main.576](https://doi.org/10.18653/v1/2021.emnlp-main.576). URL <https://aclanthology.org/2021.emnlp-main.576/>.
- Elizabeth Salesky, Neha Verma, Philipp Koehn, and Matt Post. Multilingual pixel representations for translation and effective cross-lingual transfer. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13845–13861, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.emnlp-main.854](https://doi.org/10.18653/v1/2023.emnlp-main.854). URL <https://aclanthology.org/2023.emnlp-main.854/>.
- Jonne Saleva and Constantine Lignos. The effectiveness of morphology-aware segmentation in low-resource neural machine translation. In Ionut-Teodor Sorodoc, Madhumita Sushil, Ece Takmaz, and Eneko Agirre (eds.), *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, pp. 164–174, Online, April 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.eacl-srw.22](https://doi.org/10.18653/v1/2021.eacl-srw.22). URL <https://aclanthology.org/2021.eacl-srw.22/>.
- Jonne Saleva and Constantine Lignos. What changes when you randomly choose BPE merge operations? not much. In Shabnam Tafreshi, Arjun Akula, João Sedoc, Aleksandr Drozd, Anna Rogers, and Anna Rumshisky (eds.), *Proceedings of the Fourth Workshop on Insights from Negative Results in NLP*, pp. 59–66, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.insights-1.7](https://doi.org/10.18653/v1/2023.insights-1.7). URL <https://aclanthology.org/2023.insights-1.7/>.
- Mario Sanz-Guerrero, Minh Duc Bui, and Katharina von der Wense. Mind the gap: A closer look at tokenization for multiple-choice question answering with LLMs. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 19573–19583, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.988](https://doi.org/10.18653/v1/2025.emnlp-main.988). URL <https://aclanthology.org/2025.emnlp-main.988/>.
- Ashutosh Sathe, Divyanshu Aggarwal, and Sunayana Sitaram. Improving consistency in LLM inference using probabilistic tokenization. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 4781–4793, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi:[10.18653/v1/2025.findings-naacl.268](https://doi.org/10.18653/v1/2025.findings-naacl.268). URL <https://aclanthology.org/2025.findings-naacl.268/>.
- Danielle Saunders, Weston Feely, and Bill Byrne. Inference-only sub-character decomposition improves translation of unseen logographic characters. In Toshiaki Nakazawa, Hideki Nakayama, Chenchen Ding, Raj Dabre, Anoop Kunchukuttan, Win Pa Pa, Ondřej Bojar, Shantipriya Parida, Isao Goto, Hidaya Mino, Hiroshi Manabe, Katsuhito Sudoh, Sadao Kurohashi, and Pushpak Bhattacharyya (eds.), *Proceedings of the 7th Workshop on Asian Translation*, pp. 170–177, Suzhou, China, December 2020. Association for Computational Linguistics. doi:[10.18653/v1/2020.wat-1.21](https://doi.org/10.18653/v1/2020.wat-1.21). URL <https://aclanthology.org/2020.wat-1.21/>.

-
- Tomohiro Sawada and Kartik Goyal. Train it and forget it: Merge lists are unnecessary for BPE inference in language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 35045–35058, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.1775](https://doi.org/10.18653/v1/2025.emnlp-main.1775). URL <https://aclanthology.org/2025.emnlp-main.1775/>.
- Craig W Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. Tokenization is more than compression. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 678–702, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.40](https://doi.org/10.18653/v1/2024.emnlp-main.40). URL <https://aclanthology.org/2024.emnlp-main.40/>.
- Craig W Schmidt, Varshini Reddy, Chris Tanner, and Yuval Pinter. Boundless byte pair encoding: Breaking the pre-tokenization barrier. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=oPAjXGV8qQ>.
- Craig W. Schmidt, Chris Tanner, and Yuval Pinter. Faster superword tokenization, 2026. URL <https://arxiv.org/abs/2604.05192>.
- Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5149–5152, 2012. doi:[10.1109/ICASSP.2012.6289079](https://doi.org/10.1109/ICASSP.2012.6289079).
- Eli Schwartz, Leshem Choshen, Joseph Shtok, Sivan Doveh, Leonid Karlinsky, and Assaf Arbelle. Numero-Logic: Number encoding for enhanced LLMs’ numerical reasoning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 206–212, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.12](https://doi.org/10.18653/v1/2024.emnlp-main.12). URL <https://aclanthology.org/2024.emnlp-main.12/>.
- Rico Sennrich. subword-nmt. <https://github.com/rsennrich/subword-nmt/>, 2015.
- Rico Sennrich. How grammatical is character-level neural machine translation? assessing MT quality with contrastive translation pairs. In Mirella Lapata, Phil Blunsom, and Alexander Koller (eds.), *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pp. 376–382, Valencia, Spain, April 2017. Association for Computational Linguistics. URL <https://aclanthology.org/E17-2060/>.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In Katrin Erk and Noah A. Smith (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi:[10.18653/v1/P16-1162](https://doi.org/10.18653/v1/P16-1162). URL <https://aclanthology.org/P16-1162/>.
- Rico Sennrich, Alexandra Birch, Anna Currey, Ulrich Germann, Barry Haddow, Kenneth Heafield, Antonio Valerio Miceli Barone, and Philip Williams. The University of Edinburgh’s neural MT systems for WMT17. In Ondřej Bojar, Christian Buck, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, and Julia Kreutzer (eds.), *Proceedings of the Second Conference on Machine Translation*, pp. 389–399, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi:[10.18653/v1/W17-4739](https://doi.org/10.18653/v1/W17-4739). URL <https://aclanthology.org/W17-4739/>.
- Uri Shaham and Omer Levy. Neural machine translation without embeddings. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 181–186, Online, June 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.naacl-main.17](https://doi.org/10.18653/v1/2021.naacl-main.17). URL <https://aclanthology.org/2021.naacl-main.17/>.

-
- Chen Shani, Yuval Reif, Nathan Roll, Dan Jurafsky, and Ekaterina Shutova. The roots of performance disparity in multilingual language models: Intrinsic modeling difficulty or design choices?, 2026. URL <https://arxiv.org/abs/2601.07220>.
- Feiyu Shen, Yiwei Guo, Chenpeng Du, Xie Chen, and Kai Yu. Acoustic bpe for speech generation with discrete tokens. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 11746–11750, 2024. doi:[10.1109/ICASSP48485.2024.10446063](https://doi.org/10.1109/ICASSP48485.2024.10446063).
- Xinlei Shi, Junjie Zhai, Xudong Yang, Zehua Xie, and Chao Liu. Radical embedding: Delving deeper to Chinese radicals. In Chengqing Zong and Michael Strube (eds.), *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pp. 594–598, Beijing, China, July 2015. Association for Computational Linguistics. doi:[10.3115/v1/P15-2098](https://doi.org/10.3115/v1/P15-2098). URL <https://aclanthology.org/P15-2098/>.
- Connor Shorten and Taghi M. Khoshgohar. A survey on image data augmentation for deep learning. *J. Big Data*, 6:60, 2019. doi:[10.1186/S40537-019-0197-0](https://doi.org/10.1186/S40537-019-0197-0).
- Raphael Shu and Hideki Nakayama. Compressing word embeddings via deep compositional code learning. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=BJRZzF1Rb>.
- Chenglei Si, Zhengyan Zhang, Yingfa Chen, Fanchao Qi, Xiaozhi Wang, Zhiyuan Liu, Yasheng Wang, Qun Liu, and Maosong Sun. Sub-character tokenization for Chinese pretrained language models. *Transactions of the Association for Computational Linguistics*, 11:469–487, 2023. doi:[10.1162/tacl_a_00560](https://doi.org/10.1162/tacl_a_00560). URL <https://aclanthology.org/2023.tacl-1.28/>.
- Carlo Siebensschuh, Kyle Hippe, Ozan Gokdemir, Alexander Brace, Arham Mushtaq Khan, Khalid Hos-sain, Yadu Babuji, Nicholas Chia, Venkatram Vishwanath, Arvind Ramanathan, Rick L. Stevens, Ian Foster, and Robert Underwood. Adaparse: An adaptive parallel pdf parsing and resource scaling engine. In M. Zaharia, G. Joshi, and Y. Lin (eds.), *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/file/678773d96b5822e93348aeb5c80d4dc5-Paper-Conference.pdf.
- Amanda Silberling. Why ai can’t spell ‘strawberry’. Online article on TechCrunch, August 2024. URL <https://techcrunch.com/2024/08/27/why-ai-cant-spell-strawberry/>. Accessed: 2025-01-25.
- Anya Sims, Cong Lu, Klara Kaleb, Jakob Nicolaus Foerster, and Yee Whye Teh. Stochastok: Improving fine-grained subword understanding in LLMs. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025. URL <https://openreview.net/forum?id=PZnDZdkGsE>.
- Aaditya K. Singh and DJ Strouse. Tokenization counts: the impact of tokenization on arithmetic in frontier llms, 2024. URL <https://arxiv.org/abs/2402.14903>.
- Pranaydeep Singh, Eneko Agirre, Gorka Azkune, Orphee De Clercq, and Els Lefever. EnerGIZAr: Leveraging GIZA++ for effective tokenizer initialization. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 2124–2137, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.109](https://doi.org/10.18653/v1/2025.findings-acl.109). URL <https://aclanthology.org/2025.findings-acl.109/>.
- Kevin Slagle. Spacebyte: Towards deleting tokenization from large language modeling, 2024. URL <https://arxiv.org/abs/2404.14408>.
- Peter Smit, Sami Virpioja, Stig-Arne Grönroos, and Mikko Kurimo. Morfessor 2.0: Toolkit for statistical morphological segmentation. In Shuly Wintner, Marko Tadić, and Bogdan Babych (eds.), *Proceedings of the Demonstrations at the 14th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 21–24, Gothenburg, Sweden, April 2014. Association for Computational Linguistics. doi:[10.3115/v1/E14-2006](https://doi.org/10.3115/v1/E14-2006). URL <https://aclanthology.org/E14-2006/>.

-
- R. Smith. An overview of the tesseract ocr engine. In *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, volume 2, pp. 629–633, 2007. doi:[10.1109/ICDAR.2007.4376991](https://doi.org/10.1109/ICDAR.2007.4376991).
- Anders Søgaard, Sebastian Ruder, and Ivan Vulić. On the limitations of unsupervised bilingual dictionary induction. In Iryna Gurevych and Yusuke Miyao (eds.), *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 778–788, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi:[10.18653/v1/P18-1072](https://doi.org/10.18653/v1/P18-1072). URL <https://aclanthology.org/P18-1072/>.
- Haiyue Song, Francois Meyer, Raj Dabre, Hideki Tanaka, Chenhui Chu, and Sadao Kurohashi. SubMerge: Merging equivalent subword tokenizations for subword regularized models in neural machine translation. In Carolina Scarton, Charlotte Prescott, Chris Bayliss, Chris Oakley, Joanna Wright, Stuart Wrigley, Xingyi Song, Edward Gow-Smith, Rachel Bawden, Víctor M Sánchez-Cartagena, Patrick Cadwell, Ekaterina Lapshinova-Koltunski, Vera Cabarrão, Konstantinos Chatzitheodorou, Mary Nurminen, Diptesh Kanojia, and Helena Moniz (eds.), *Proceedings of the 25th Annual Conference of the European Association for Machine Translation (Volume 1)*, pp. 147–163, Sheffield, UK, June 2024. European Association for Machine Translation (EAMT). URL <https://aclanthology.org/2024.eamt-1.15/>.
- Xinying Song, Alex Salcianu, Yang Song, Dave Dopson, and Denny Zhou. Fast WordPiece tokenization. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2089–2103, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.emnlp-main.160](https://doi.org/10.18653/v1/2021.emnlp-main.160). URL <https://aclanthology.org/2021.emnlp-main.160/>.
- Sharath Turuvekere Sreenivas, Adithyakrishna Venkatesh Hanasoge, Mingyu Yang, Ali Taghibakhshi, Saurav Muralidharan, Ashwath Aithal, and Pavlo Molchanov. X-token: Projection-guided cross-tokenizer knowledge distillation, 2026. URL <https://arxiv.org/abs/2605.21699>.
- Tejas Srinivasan, Ramon Sanabria, and Florian Metze. Multitask learning for different subword segmentations in neural machine translation. In Jan Niehues, Rolando Cattoni, Sebastian Stüker, Matteo Negri, Marco Turchi, Thanh-Le Ha, Elizabeth Salesky, Ramon Sanabria, Loic Barrault, Lucia Specia, and Marcello Federico (eds.), *Proceedings of the 16th International Conference on Spoken Language Translation*, Hong Kong, November 2–3 2019. Association for Computational Linguistics. URL <https://aclanthology.org/2019.iwslt-1.25/>.
- Beat Stamm. The Raster Tragedy at Low-Resolution Revisited: Opportunities and Challenges beyond “Delta-Hinting”. <http://rastertragedy.com/>, 2011. Accessed: 2026-05-01.
- David Stap, Vlad Niculae, and Christof Monz. Viewing knowledge transfer in multilingual machine translation through a representational lens. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 14973–14987, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-emnlp.998](https://doi.org/10.18653/v1/2023.findings-emnlp.998). URL <https://aclanthology.org/2023.findings-emnlp.998/>.
- Abishek Stephen and Jindřich Libovický. Evaluating morphological plausibility of subword tokenization via statistical alignment with morpho-syntactic features. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 3783–3791, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-386-9. doi:[10.18653/v1/2026.findings-eacl.196](https://doi.org/10.18653/v1/2026.findings-eacl.196). URL <https://aclanthology.org/2026.findings-eacl.196/>.
- Igor Sterner and Simone Teufel. Code-switching and syntax: A large-scale experiment. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 11526–11533, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.600](https://doi.org/10.18653/v1/2025.findings-acl.600). URL <https://aclanthology.org/2025.findings-acl.600/>.

-
- Karl Stratos. Reconstruction of word embeddings from sub-word parameters. In Manaal Faruqui, Hinrich Schuetze, Isabel Trancoso, and Yadollah Yaghoobzadeh (eds.), *Proceedings of the First Workshop on Subword and Character Level Models in NLP*, pp. 130–135, Copenhagen, Denmark, September 2017a. Association for Computational Linguistics. doi:[10.18653/v1/W17-4119](https://doi.org/10.18653/v1/W17-4119). URL <https://aclanthology.org/W17-4119/>.
- Karl Stratos. A sub-character architecture for Korean language processing. In Martha Palmer, Rebecca Hwa, and Sebastian Riedel (eds.), *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 721–726, Copenhagen, Denmark, September 2017b. Association for Computational Linguistics. doi:[10.18653/v1/D17-1075](https://doi.org/10.18653/v1/D17-1075). URL <https://aclanthology.org/D17-1075/>.
- Lukas Struppek, Dom Hintersdorf, Felix Friedrich, Manuel br, Patrick Schramowski, and Kristian Kersting. Exploiting cultural biases via homoglyphs in text-to-image synthesis. *J. Artif. Int. Res.*, 78, January 2024. ISSN 1076-9757. doi:[10.1613/jair.1.15388](https://doi.org/10.1613/jair.1.15388).
- Zijun Sun, Xiaoya Li, Xiaofei Sun, Yuxian Meng, Xiang Ao, Qing He, Fei Wu, and Jiwei Li. Chinese-BERT: Chinese pretraining enhanced by glyph and Pinyin information. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 2065–2075, Online, August 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.acl-long.161](https://doi.org/10.18653/v1/2021.acl-long.161). URL <https://aclanthology.org/2021.acl-long.161/>.
- Ilya Sutskever, James Martens, and Geoffrey Hinton. Generating text with recurrent neural networks. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML’11*, pp. 1017–1024, Madison, WI, USA, 2011a. Omnipress. ISBN 9781450306195.
- Ilya Sutskever, James Martens, and Geoffrey Hinton. Generating text with recurrent neural networks. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML’11*, pp. 1017–1024, Madison, WI, USA, 2011b. Omnipress. ISBN 9781450306195.
- Richard Sutton. The bitter lesson, 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- Burak Suyunu, Enes Taylan, and Arzucan Özgür. Linguistic laws meet protein sequences: A comparative analysis of subword tokenization methods, 2024. URL <https://arxiv.org/abs/2411.17669>.
- Dan Svenstrup, Jonas Meinertz Hansen, and Ole Winther. Hash embeddings for efficient word representations, 2017. URL <https://arxiv.org/abs/1709.03933>.
- Yintao Tai, Xiyang Liao, Alessandro Suglia, and Antonio Vergari. PIXAR: Auto-regressive language modeling in pixel space. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 14673–14695, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-acl.874](https://doi.org/10.18653/v1/2024.findings-acl.874). URL <https://aclanthology.org/2024.findings-acl.874/>.
- Kazuma Takaoka, Sorami Hisamoto, Noriko Kawahara, Miho Sakamoto, Yoshitaka Uchida, and Yuji Matsumoto. Sudachi: a Japanese tokenizer for business. In Nicoletta Calzolari, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Koiti Hasida, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, Stelios Piperidis, and Takenobu Tokunaga (eds.), *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan, May 2018. European Language Resources Association (ELRA). URL <https://aclanthology.org/L18-1355/>.
- Sho Takase, Tatsuya Hiraoka, and Naoaki Okazaki. Single model ensemble for subword regularized models in low-resource machine translation. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 2536–2541, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.findings-acl.199](https://doi.org/10.18653/v1/2022.findings-acl.199). URL <https://aclanthology.org/2022.findings-acl.199/>.

-
- Sho Takase, Ryokan Ri, Shun Kiyono, and Takuya Kato. Large vocabulary size improves large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 1015–1026, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.57](https://doi.org/10.18653/v1/2025.findings-acl.57). URL <https://aclanthology.org/2025.findings-acl.57/>.
- Chaofan Tao, Qian Liu, Longxu Dou, Niklas Muennighoff, Zhongwei Wan, Ping Luo, Min Lin, and Ngai Wong. Scaling laws with vocabulary: Larger models deserve larger vocabularies. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=sCKKPr8cRL>.
- Kushal Tatariya, Vladimir Araujo, Thomas Bauwens, and Miryam de Lhoneux. Pixology: Probing the linguistic and visual capabilities of pixel-based language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3307–3320, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.emnlp-main.194](https://doi.org/10.18653/v1/2024.emnlp-main.194). URL <https://aclanthology.org/2024.emnlp-main.194/>.
- Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=JtBRnr10EFN>.
- Atula Tejaswi, Nilesch Gupta, and Eunsol Choi. Exploring design choices for building language-specific LLMs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10485–10500, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-emnlp.614](https://doi.org/10.18653/v1/2024.findings-emnlp.614). URL <https://aclanthology.org/2024.findings-emnlp.614/>.
- Jan Tempus, Philip Whittington, Craig W. Schmidt, Dennis Komm, and Tiago Pimentel. Tokenisation via convex relaxations, 2026. URL <https://arxiv.org/abs/2605.22821>.
- Avijit Thawani, Jay Pujara, Filip Ilievski, and Pedro Szekely. Representing numbers in NLP: a survey and a vision. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 644–656, Online, June 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.naacl-main.53](https://doi.org/10.18653/v1/2021.naacl-main.53). URL <https://aclanthology.org/2021.naacl-main.53/>.
- Avijit Thawani, Saurabh Ghanekar, Xiaoyuan Zhu, and Jay Pujara. Learn your tokens: Word-pooled tokenization for language modeling. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 9883–9893, Singapore, December 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-emnlp.662](https://doi.org/10.18653/v1/2023.findings-emnlp.662). URL <https://aclanthology.org/2023.findings-emnlp.662/>.
- Arseny Tolmachev, Daisuke Kawahara, and Sadao Kurohashi. Juman++: A morphological analysis toolkit for scriptio continua. In Eduardo Blanco and Wei Lu (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 54–59, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi:[10.18653/v1/D18-2010](https://doi.org/10.18653/v1/D18-2010). URL <https://aclanthology.org/D18-2010/>.
- Cagri Toraman, Eyup Halit Yilmaz, Furkan Şahinuç, and Oguzhan Ozelik. Impact of tokenization on language models: An analysis for Turkish. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 22(4):1–21, 2023. URL <https://dl.acm.org/doi/10.1145/3578707>.
- Taiichi Hashimoto Toshinori Sato and Manabu Okumura. Implementation of a word segmentation dictionary called mecab-ipadic-neologd and study on how to use it effectively for information retrieval (in japanese). In *Proceedings of the Twenty-three Annual Meeting of the Association for Natural Language Processing*, pp. NLP2017–B6–1. The Association for Natural Language Processing, 2017.

-
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, et al. Llama 2: Open foundation and fine-tuned chat models, 2023. URL <https://arxiv.org/abs/2307.09288>.
- Ke Tran. From english to foreign languages: Transferring pre-trained language models, 2020. URL <https://arxiv.org/abs/2002.07306>.
- Vivien Tran-Thien. Fast, high-fidelity llm decoding with regex constraints. <https://vivien000.github.io/blog/journal/llm-decoding-with-regex-constraints.html>, 2024. URL <https://archive.ph/aeDQX>. [Online; accessed 30-May-2025].
- Michael Tschannen, Basil Mustafa, and Neil Houlsby. Clippo: Image-and-language understanding from pixels only. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11006–11017, 2023.
- Huihsin Tseng and Keh-Jiann Chen. Design of Chinese morphological analyzer. In *COLING-02: The First SIGHAN Workshop on Chinese Language Processing*, 2002. URL <https://aclanthology.org/W02-1811/>.
- Kohei Tsuji, Tatsuya Hiraoka, Yuchang Cheng, and Tomoya Iwakura. SubRegWeigh: Effective and efficient annotation weighing with subword regularization. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (eds.), *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 1908–1921, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-main.130/>.
- Alexander Tsvetkov and Alon Kipnis. Information parity: Measuring and predicting the multilingual capabilities of language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 7971–7989, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-emnlp.468. URL <https://aclanthology.org/2024.findings-emnlp.468/>.
- Stephan Tulkens. skeletoken, February 2026. URL <https://zenodo.org/records/19401859>.
- Anil Turaga. Character prefix conditioning with back tokenization. <https://anilturaga.github.io/cpc>, 2025. URL <https://archive.ph/ECiYH>.
- UAX #24. Unicode script property. Unicode Standard Annex 24, Unicode Consortium, July 2025. URL <https://www.unicode.org/reports/tr24/>. Revision 39, Version Unicode 17.0.0.
- UAX #29. Unicode text segmentation. Unicode Standard Annex 29, Unicode Consortium, August 2025. URL <https://www.unicode.org/reports/tr29/>. Revision 47, Version Unicode 17.0.0.
- UAX #9. Unicode bidirectional algorithm. Unicode Standard Annex 9, Unicode Consortium, August 2025. URL <https://www.unicode.org/reports/tr9/>. Revision 51, Version Unicode 17.0.0.
- Trapoom Ukarapol, Pakhapoom Sarapat, and Nut Chukamphaeng. Language-aware token boosting: LLM language confusion reduction without tuning. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 481–489, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-391-3. doi:10.18653/v1/2026.acl-short.40. URL <https://aclanthology.org/2026.acl-short.40/>.
- Omri Uzan and Yuval Pinter. Charbench: Evaluating the role of tokenization in character-level tasks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(39):33296–33304, Mar. 2026. doi:10.1609/aaai.v40i39.40615. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40615>.

-
- Omri Uzan, Craig W. Schmidt, Chris Tanner, and Yuval Pinter. Greed is all you need: An evaluation of tokenizer inference methods. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 813–822, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.acl-short.73](https://doi.org/10.18653/v1/2024.acl-short.73). URL <https://aclanthology.org/2024.acl-short.73/>.
- Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 6306–6315, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html>.
- Dušan Variš, Abishek Stephen, and Jindřich Libovický. TokCollate: A comprehensive tool for tokenizer evaluation and visualization across languages. In Greg Durrett and Ping Jian (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 418–427, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-392-0. doi:[10.18653/v1/2026.acl-demo.41](https://doi.org/10.18653/v1/2026.acl-demo.41). URL <https://aclanthology.org/2026.acl-demo.41/>.
- Mihaela Alexandra VasIU and Rodica Potolea. Enhancing tokenization by embedding romanian language specific morphology. *2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing (ICCP)*, pp. 243–250, 2020. URL <https://api.semanticscholar.org/CorpusID:227232820>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Raúl Vázquez, Umut Sulubacak, and Jörg Tiedemann. The University of Helsinki submission to the WMT19 parallel corpus filtering task. In Ondřej Bojar, Rajen Chatterjee, Christian Federmann, Mark Fishel, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, André Martins, Christof Monz, Matteo Negri, Aurélie Névél, Mariana Neves, Matt Post, Marco Turchi, and Karin Verspoor (eds.), *Proceedings of the Fourth Conference on Machine Translation (Volume 3: Shared Task Papers, Day 2)*, pp. 294–300, Florence, Italy, August 2019. Association for Computational Linguistics. doi:[10.18653/v1/W19-5441](https://doi.org/10.18653/v1/W19-5441). URL <https://aclanthology.org/W19-5441/>.
- Ander Artola Velasco, Stratis Tsirtsis, and Manuel Gomez-Rodriguez. Auditing pay-per-token in large language models, 2025. URL <https://arxiv.org/abs/2510.05181>.
- Menan Velayuthan and Kengatharaiyer Sarveswaran. Egalitarian language representation in language models: It all begins with tokenizers. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (eds.), *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 5987–5996, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-main.400/>.
- Saketh Reddy Vemula, Sandipan Dandapat, Dipti Sharma, and Parameswari Krishnamurthy. Rethinking tokenization for rich morphology: The dominance of unigram over BPE and morphological alignment. In Santosh T.Y.S.S, Shuichiro Shimizu, and Yifan Gong (eds.), *The 14th International Joint Conference on Natural Language Processing and The 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pp. 232–252, Mumbai, India, December 2025a. Association for Computational Linguistics. ISBN 979-8-89176-304-3. doi:[10.18653/v1/2025.ijcnlp-srw.20](https://doi.org/10.18653/v1/2025.ijcnlp-srw.20). URL <https://aclanthology.org/2025.ijcnlp-srw.20/>.

-
- Saketh Reddy Vemula, Sandipan Dandapat, Dipti Misra Sharma, and Parameswari Krishnamurthy. Rethinking tokenization for rich morphology: The dominance of unigram over bpe and morphological alignment, 2025b. URL <https://arxiv.org/abs/2508.08424>.
- Gianluca Vico and Jindřich Libovický. Conditional unigram tokenization with parallel data. In *Tokenization Workshop*, 2025. URL <https://openreview.net/forum?id=lnWJWNA8YW>.
- Mathurin Videau, Badr Youbi Idrissi, Alessandro Leite, Marc Schoenauer, Olivier Teytaud, and David Lopez-Paz. From bytes to ideas: Language modeling with autoregressive u-nets, 2025. URL <https://arxiv.org/abs/2506.14761>.
- Tim Vieira, Benjamin Lebrun, Mario Giulianelli, Juan Luis Gastaldi, Brian Dusell, John Terilla, Timothy J. O’Donnell, and Ryan Cotterell. From language models over tokens to language models over characters. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 61391–61412. PMLR, 13–19 Jul 2025a. URL <https://proceedings.mlr.press/v267/vieira25a.html>.
- Tim Vieira, Tianyu Liu, Clemente Pasti, Yahya Emara, Brian DuSell, Benjamin LeBrun, Mario Giulianelli, Juan Luis Gastaldi, John Terilla, Timothy J. O’Donnell, and Ryan Cotterell. Language models over canonical byte-pair encodings. In *Forty-second International Conference on Machine Learning*, 2025b. URL <https://openreview.net/forum?id=eCVrfVDNSY>.
- David Vilar and Marcello Federico. A statistical extension of byte-pair encoding. In Marcello Federico, Alex Waibel, Marta R. Costa-jussà, Jan Niehues, Sebastian Stuker, and Elizabeth Salesky (eds.), *Proceedings of the 18th International Conference on Spoken Language Translation (IWSLT 2021)*, pp. 263–275, Bangkok, Thailand (online), August 2021. Association for Computational Linguistics. doi:10.18653/v1/2021.iwslt-1.31. URL <https://aclanthology.org/2021.iwslt-1.31/>.
- A. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967. doi:10.1109/TIT.1967.1054010.
- Eric Wallace, Yizhong Wang, Sujian Li, Sameer Singh, and Matt Gardner. Do NLP models know numbers? probing numeracy in embeddings. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 5307–5315, Hong Kong, China, November 2019. Association for Computational Linguistics. doi:10.18653/v1/D19-1534. URL <https://aclanthology.org/D19-1534/>.
- Ada Wan. Fairness in representation for multilingual nlp: Insights from controlled experiments on conditional language modeling. In *International Conference on Learning Representations*, 2022.
- Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. *ArXiv*, abs/2401.10491, 2024. URL <https://api.semanticscholar.org/CorpusID:267061245>.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019a. URL <https://openreview.net/forum?id=rJ4km2R5t7>.
- Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):9154–9160, Apr. 2020a. doi:10.1609/aaai.v34i05.6451. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6451>.
- Dixuan Wang, Yanda Li, Junyuan Jiang, Zepeng Ding, Ziqin Luo, Guochao Jiang, Jiaqing Liang, and Deqing Yang. Tokenization matters! degrading large language models through challenging their tokenization, 2025a. URL <https://arxiv.org/abs/2405.17067>.

-
- Hai Wang, Dian Yu, Kai Sun, Jianshu Chen, and Dong Yu. Improving pre-trained multilingual model with vocabulary expansion. In Mohit Bansal and Aline Villavicencio (eds.), *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, pp. 316–327, Hong Kong, China, November 2019b. Association for Computational Linguistics. doi:[10.18653/v1/K19-1030](https://doi.org/10.18653/v1/K19-1030). URL <https://aclanthology.org/K19-1030/>.
- Haifeng Wang, Hua Wu, Tian Wu, Yu Sun, Jing Liu, Dianhai Yu, Yanjun Ma, Jingzhou He, Zhongjun He, Dou Hong, Qiwen Liu, Shuohuan Wang, Junyuan Shang, Zhenyu Zhang, Yuchen Ding, Jinle Zeng, Jiabin Yang, Liang Shen, Ruibiao Chen, et al. Ernie 5.0 technical report, 2026. URL <https://arxiv.org/abs/2602.04705>.
- Hongyi Wang, Felipe Maia Polo, Yuekai Sun, Souvik Kundu, Eric Xing, and Mikhail Yurochkin. Fusing models with complementary expertise. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=PhMrGCMIRL>.
- Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. Mambabyte: Token-free selective state space model, 2024b. URL <https://arxiv.org/abs/2401.13660>.
- Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application, 2024c. URL <https://arxiv.org/abs/2302.00487>.
- Luting Wang, Yang Zhao, Zijian Zhang, Jiashi Feng, Si Liu, and Bingyi Kang. Image understanding makes for a good tokenizer for image generation. *Advances in Neural Information Processing Systems*, 37:31015–31035, 2024d.
- Mingyang Wang, Lukas Lange, Heike Adel, Yunpu Ma, Jannik Strötgen, and Hinrich Schuetze. Language mixing in reasoning language models: Patterns, impact, and internal causes. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 2637–2665, Suzhou, China, November 2025b. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.132](https://doi.org/10.18653/v1/2025.emnlp-main.132). URL <https://aclanthology.org/2025.emnlp-main.132/>.
- Shirui Wang, Wenan Zhou, and Qiang Zhou. Radical and stroke-enhanced chinese word embeddings based on neural networks. *Neural Processing Letters*, 52(2):1109–1121, 2020b.
- Xilong Wang, Hao Fu, Jindong Wang, and Neil Zhenqiang Gong. StringLLM: Understanding the string processing capability of large language models. In *The Thirteenth International Conference on Learning Representations*, 2025c. URL <https://openreview.net/forum?id=kTXChtaaN0>.
- Xinyi Wang, Sebastian Ruder, and Graham Neubig. Multi-view subword regularization. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 473–482, Online, June 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.naacl-main.40](https://doi.org/10.18653/v1/2021.naacl-main.40). URL <https://aclanthology.org/2021.naacl-main.40/>.
- Zhijun Wang, Xuebo Liu, and Min Zhang. Breaking the representation bottleneck of Chinese characters: Neural machine translation with stroke sequence modeling. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 6473–6484, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi:[10.18653/v1/2022.emnlp-main.434](https://doi.org/10.18653/v1/2022.emnlp-main.434). URL <https://aclanthology.org/2022.emnlp-main.434/>.
- Zihan Wang, Karthikeyan K, Stephen Mayhew, and Dan Roth. Extending multilingual BERT to low-resource languages. In Trevor Cohn, Yulan He, and Yang Liu (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2649–2656, Online, November 2020c. Association for Computational Linguistics. doi:[10.18653/v1/2020.findings-emnlp.240](https://doi.org/10.18653/v1/2020.findings-emnlp.240). URL <https://aclanthology.org/2020.findings-emnlp.240/>.

-
- Anna Wegmann, Dong Nguyen, and David Jurgens. Tokenization is sensitive to language variation. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 10958–10983, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.572](https://doi.org/10.18653/v1/2025.findings-acl.572). URL <https://aclanthology.org/2025.findings-acl.572/>.
- Haoran Wei, Yaofeng Sun, and Yukun Li. Deepseek-ocr: Contexts optical compression, 2025. URL <https://arxiv.org/abs/2510.18234>.
- Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=gEZrGCozdqR>.
- Philip Whittington, Gregor Bachmann, and Tiago Pimentel. Tokenisation is NP-complete. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28133–28153, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:[10.18653/v1/2025.acl-long.1365](https://doi.org/10.18653/v1/2025.acl-long.1365). URL <https://aclanthology.org/2025.acl-long.1365/>.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023. URL <https://arxiv.org/abs/2307.09702>.
- Christin Wilson and Vedrana Mihalicek. *Language files: Materials for an introduction to language and linguistics*. Ohio State University Press, Columbus, OH, USA, 2011.
- Genta Winata, Alham Fikri Aji, Zheng Xin Yong, and Tamar Solorio. The decades progress on code-switching research in NLP: A systematic survey on trends and challenges. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 2936–2978, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-acl.185](https://doi.org/10.18653/v1/2023.findings-acl.185). URL <https://aclanthology.org/2023.findings-acl.185/>.
- Terry Winograd. Procedures as a representation for data in a computer program for understanding natural language. Technical report, MASSACHUSETTS INST OF TECH CAMBRIDGE PROJECT MAC, 1971.
- BigScience Workshop, :, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, et al. Bloom: A 176b-parameter open-access multilingual language model, 2023. URL <https://arxiv.org/abs/2211.05100>.
- Lijun Wu, Shufang Xie, Yingce Xia, Yang Fan, Jian-Huang Lai, Tao Qin, and Tieyan Liu. Sequence generation with mixed representations. In Hal Daumé III and Aarti Singh (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 10388–10398. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/wu20e.html>.
- Shijie Wu and Mark Dredze. Beto, bentz, becas: The surprising cross-lingual effectiveness of BERT. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 833–844, Hong Kong, China, November 2019. Association for Computational Linguistics. doi:[10.18653/v1/D19-1077](https://doi.org/10.18653/v1/D19-1077). URL <https://aclanthology.org/D19-1077/>.
- Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation, 2016. URL <https://arxiv.org/abs/1609.08144>.

-
- Orgest Xhelili, Yihong Liu, and Hinrich Schuetze. Breaking the script barrier in multilingual pre-trained language models with transliteration-based post-training alignment. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 11283–11296, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-emnlp.659](https://doi.org/10.18653/v1/2024.findings-emnlp.659). URL <https://aclanthology.org/2024.findings-emnlp.659/>.
- Junyi Xiang and Maofu Liu. Exploiting phonetics and glyph representation at radical-level for classical Chinese understanding. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 22850–22871, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi:[10.18653/v1/2025.findings-acl.1173](https://doi.org/10.18653/v1/2025.findings-acl.1173). URL <https://aclanthology.org/2025.findings-acl.1173/>.
- Fengshun Xiao, Jiangtong Li, Hai Zhao, Rui Wang, and Kehai Chen. Lattice-based transformer encoder for neural machine translation. In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3090–3097, Florence, Italy, July 2019. Association for Computational Linguistics. doi:[10.18653/v1/P19-1298](https://doi.org/10.18653/v1/P19-1298). URL <https://aclanthology.org/P19-1298/>.
- Ling Xing, Alex Jinpeng Wang, Rui Yan, Xiangbo Shu, and Jinhui Tang. Vision-centric token compression in large language model. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=YdggdEL41C>.
- Hao Xu, Alisa Liu, Jonathan Hayase, Yejin Choi, and Noah A. Smith. Are you going to finish that? a practical study of the partial token problem, 2026. URL <https://arxiv.org/abs/2601.23223>.
- Jingjing Xu, Hao Zhou, Chun Gan, Zaixiang Zheng, and Lei Li. Vocabulary learning via optimal transport for neural machine translation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 7361–7373, Online, August 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.acl-long.571](https://doi.org/10.18653/v1/2021.acl-long.571). URL <https://aclanthology.org/2021.acl-long.571/>.
- Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. mT5: A massively multilingual pre-trained text-to-text transformer. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 483–498, Online, June 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.naacl-main.41](https://doi.org/10.18653/v1/2021.naacl-main.41). URL <https://aclanthology.org/2021.naacl-main.41/>.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. doi:[10.1162/tacl_a_00461](https://doi.org/10.1162/tacl_a_00461). URL <https://aclanthology.org/2022.tacl-1.17/>.
- Atsuki Yamaguchi, Aline Villavicencio, and Nikolaos Aletras. An empirical study on cross-lingual vocabulary adaptation for efficient language model inference. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 6760–6785, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:[10.18653/v1/2024.findings-emnlp.396](https://doi.org/10.18653/v1/2024.findings-emnlp.396). URL <https://aclanthology.org/2024.findings-emnlp.396/>.
- Atsuki Yamaguchi, Aline Villavicencio, and Nikolaos Aletras. How can we effectively expand the vocabulary of llms with 0.01gb of target language text? *Computational Linguistics*, pp. 1–36, February 2026. ISSN 1530-9312. doi:[10.1162/coli.a.581](https://doi.org/10.1162/coli.a.581).
- Wilson Yan, Volodymyr Mnih, Aleksandra Faust, Matei Zaharia, Pieter Abbeel, and Hao Liu. Elastictok: Adaptive tokenization for image and video. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tFV5GrWOGm>.

-
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, et al. Qwen3 technical report, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- Haotong Yang, Yi Hu, Shijia Kang, Zhouchen Lin, and Muhan Zhang. Number cookbook: Number understanding of language models and how to improve it. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 3946–3984, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/0b8ccc9229328bdcedf54b989e7cc330-Paper-Conference.pdf.
- Xiulin Yang, Ethan Gotlieb Wilcox, and Catherine Arnett. Apples to apples? towards comparable crosslingual language model evaluation. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, 2026. URL <https://arxiv.org/abs/2608.25089>.
- Yunzhi Yao, Shaohan Huang, Wenhui Wang, Li Dong, and Furu Wei. Adapt-and-distill: Developing small, fast and effective pretrained language models for domains. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 460–470, Online, August 2021. Association for Computational Linguistics. doi:[10.18653/v1/2021.findings-acl.40](https://doi.org/10.18653/v1/2021.findings-acl.40). URL <https://aclanthology.org/2021.findings-acl.40/>.
- Charles Ye, Jasmine Cui, and Dylan Hadfield-Menell. Prompt injection as role confusion, 2026. URL <https://arxiv.org/abs/2603.12277>.
- Shaked Yehezkel and Yuval Pinter. Incorporating context into subword vocabularies. In Andreas Vlachos and Isabelle Augenstein (eds.), *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 623–635, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.eacl-main.45](https://doi.org/10.18653/v1/2023.eacl-main.45). URL <https://aclanthology.org/2023.eacl-main.45/>.
- Naoki Yoshinaga. Back to patterns: Efficient Japanese morphological analysis with feature-sequence trie. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 13–23, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:[10.18653/v1/2023.acl-short.2](https://doi.org/10.18653/v1/2023.acl-short.2). URL <https://aclanthology.org/2023.acl-short.2/>.
- Lijun Yu, Yong Cheng, Kihyuk Sohn, José Lezama, Han Zhang, Huiwen Chang, Alexander G Hauptmann, Ming-Hsuan Yang, Yuan Hao, Irfan Essa, et al. Magvit: Masked generative video transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10459–10469, 2023a.
- Lili Yu, Daniel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. Megabyte: Predicting million-byte sequences with multiscale transformers. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 78808–78823. Curran Associates, Inc., 2023b. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/f8f78f8043f35890181a824e53a57134-Paper-Conference.pdf.
- Qingcheng Zeng, Lucas Garay, Peilin Zhou, Dading Chong, Yining Hua, Jiageng Wu, Yikang Pan, Han Zhou, Rob Voigt, and Jie Yang. GreenPLM: Cross-Lingual Transfer of Monolingual Pre-Trained Language Models at Almost No Cost, May 2023. URL <http://arxiv.org/abs/2211.06993>. arXiv:2211.06993 [cs].
- Muge Zhang, Aaron Jencks, Krishna Badikela, Yulia Tsvetkov, and Sachin Kumar. One form to transfer them all: Pretraining multilingual language models beyond native orthography. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2026a. URL <https://arxiv.org/abs/2608.25904>.
- Qingjie Zhang, Di Wang, Haoting Qian, Liu Yan, Tianwei Zhang, Ke Xu, Qi Li, Minlie Huang, Hewu Li, and Han Qiu. Speculating LLMs’ Chinese training data pollution from their tokens. In

-
- Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 26113–26133, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:[10.18653/v1/2025.emnlp-main.1327](https://doi.org/10.18653/v1/2025.emnlp-main.1327). URL <https://aclanthology.org/2025.emnlp-main.1327/>.
- Ran Zhang, Miryam de Lhoneux, and Wessel Poelman. Seeing the unseen: Visual similarity for pixel language model adaptation. In *2026 Conference on Empirical Methods in Natural Language Processing*, 2026b. URL <https://arxiv.org/abs/2608.30541>.
- Shiyue Zhang, Vishrav Chaudhary, Naman Goyal, James Cross, Guillaume Wenzek, Mohit Bansal, and Francisco Guzman. How robust is neural machine translation to language imbalance in multilingual tokenizer training? In Kevin Duh and Francisco Guzmán (eds.), *Proceedings of the 15th biennial conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*, pp. 97–116, Orlando, USA, September 2022. Association for Machine Translation in the Americas. URL <https://aclanthology.org/2022.amta-research.8/>.
- Songming Zhang, Xue Zhang, Zengkui Sun, Yufeng Chen, and Jinan Xu. Dual-space knowledge distillation for large language models. *ArXiv*, abs/2406.17328, 2024. URL <https://api.semanticscholar.org/CorpusID:270711539>.
- Wanpeng Zhang, Zilong Xie, Yicheng Feng, Yijiang Li, Xingrun Xing, Sipeng Zheng, and Zongqing Lu. From pixels to tokens: Byte-pair encoding on quantized visual modalities. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=3TnLGgHhNx>.
- Wei Zhang, Feifei Lin, Xiaodong Wang, Zhenshuang Liang, and Zhen Huang. Subcharacter chinese-english neural machine translation with wubi encoding, 2019. URL <https://arxiv.org/abs/1911.02737>.
- Xiang Zhang and Yann LeCun. Which encoding is the best for text classification in chinese, english, japanese and korean?, 2017. URL <https://arxiv.org/abs/1708.02657>.
- Xiaoyi Zhang, Lilian de Greef, Amanda Swearngin, Samuel White, Kyle Murray, Lisa Yu, Qi Shan, Jeffrey Nichols, Jason Wu, Chris Fleizach, Aaron Everitt, and Jeffrey P Bigham. Screen recognition: Creating accessibility metadata for mobile applications from pixels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI ’21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450380966. doi:[10.1145/3411764.3445186](https://doi.org/10.1145/3411764.3445186).
- Daniel Zhang-Li, Nianyi Lin, Jifan Yu, Zheyuan Zhang, Zijun Yao, Xiaokang Zhang, Lei Hou, Jing Zhang, and Juanzi Li. Reverse that number! decoding order matters in arithmetic learning, 2024. URL <https://arxiv.org/abs/2403.05845>.
- Jinman Zhao, Sidharth Mudgal, and Yingyu Liang. Generalizing word embeddings using bag of subwords. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 601–606, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi:[10.18653/v1/D18-1059](https://doi.org/10.18653/v1/D18-1059). URL <https://aclanthology.org/D18-1059/>.
- Bo Zheng, Li Dong, Shaohan Huang, Saksham Singhal, Wanxiang Che, Ting Liu, Xia Song, and Furu Wei. Allocating large vocabulary capacity for cross-lingual language model pre-training. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3203–3215, Online and Punta Cana, Dominican Republic, November 2021a. Association for Computational Linguistics. doi:[10.18653/v1/2021.emnlp-main.257](https://doi.org/10.18653/v1/2021.emnlp-main.257). URL <https://aclanthology.org/2021.emnlp-main.257/>.
- Bo Zheng, Li Dong, Shaohan Huang, Wenhui Wang, Zewen Chi, Saksham Singhal, Wanxiang Che, Ting Liu, Xia Song, and Furu Wei. Consistency regularization for cross-lingual fine-tuning. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3403–3417, Online, August 2021b. Association for Computational Linguistics. doi:[10.18653/v1/2021.acl-long.264](https://doi.org/10.18653/v1/2021.acl-long.264). URL <https://aclanthology.org/2021.acl-long.264/>.

-
- Brian Siyuan Zheng, Alisa Liu, Orevaoghene Ahia, Jonathan Hayase, Yejin Choi, and Noah A. Smith. Broken tokens? your language model can secretly handle non-canonical tokenizations. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a. URL <https://openreview.net/forum?id=WryWolqKh3>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.
- Lin Zheng, Xueliang Zhao, Guangtao Wang, Chen Wu, David Dong, Angela Wang, Mingran Wang, Yun Du, Haige Bo, Amol Sharma, Bo Li, Kejie Zhang, Changran Hu, Urmish Thakker, and Lingpeng Kong. Evabyte: Efficient byte-level language models at scale. <https://hkunlp.github.io/blog/2025/evabyte>, 2025b. URL <https://archive.ph/P4AKa>.
- Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Joshua M. Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? a study in length generalization. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=AssIuHnmHX>.
- Tianyi Zhou, Deqing Fu, Mahdi Soltanolkotabi, Robin Jia, and Vatsal Sharan. FoNE: Precise single-token number embeddings via fourier features. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=gOvtWmwDDh>.
- Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5184–5207, Toronto, Canada, July 2023a. Association for Computational Linguistics. doi:[10.18653/v1/2023.acl-long.284](https://doi.org/10.18653/v1/2023.acl-long.284). URL <https://aclanthology.org/2023.acl-long.284/>.
- Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. A formal perspective on byte-pair encoding. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 598–614, Toronto, Canada, July 2023b. Association for Computational Linguistics. doi:[10.18653/v1/2023.findings-acl.38](https://doi.org/10.18653/v1/2023.findings-acl.38). URL <https://aclanthology.org/2023.findings-acl.38/>.